Київський столичний університет імені Бориса Грінченка

Володимир Соколов

ТЕХНОЛОГІЇ РОЗРОБКИ І ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ МЕРЕЖЕВОЇ БЕЗПЕКИ

Навчальний посібник

для здобувачів другого (освітнього) рівня вищої освіти
спеціальності 125/F5 «Кібербезпека та захист інформації»
галузі знань 12/F «Інформаційні технології»

Київ — 2027

УДК 004.056.5:004.415.5(075.8)
ББК 32.973.26-018
С594

Препринт

Соколов В. Ю. Технології розробки і тестування програмного забезпечення мережевої безпеки : навчальний посібник для здобувачів другого (освітнього) рівня вищої освіти. Київ, КСУБГ, 2027, 257 с. — 17 розділів.

Навчальний посібник охоплює теоретичні основи та практичні аспекти технологій розроблення і тестування програмного забезпечення мережевої безпеки: міжмережевих екранів, систем виявлення та запобігання вторгненням, VPN-шлюзів, засобів контролю доступу до мережі, криптографічних компонентів і реалізацій захищених протоколів. Розглянуто програмне забезпечення мережевої безпеки як об'єкт розроблення і тестування; нормативно-правове й стандартизаційне забезпечення в Україні та Європейському Союзі, зокрема вимоги Регламенту про кіберстійкість; процесні моделі та метрики безпечного життєвого циклу розроблення; вимоги безпеки, моделювання загроз і безпечне проєктування; безпечне програмування, мови з безпечною пам'яттю й безпечний розбір ворожого вводу; розроблення криптографічних компонентів, засобів фільтрації трафіку, систем виявлення вторгнень, засобів захищених комунікацій; безпеку ланцюга постачання і конвеєра CI/CD; методологію тестування безпеки, статичний аналіз і формальну верифікацію, динамічний аналіз і фазинг; тестування продуктивності й стійкості, оцінювання ефективності виявлення й кіберполігони; управління вразливостями й координоване розкриття; кейси гарантування, застосування штучного інтелекту та напрями наукових досліджень.

Кожен розділ завершується висновками та питаннями для самоперевірки, серед яких — дослідницькі завдання, придатні як відправна точка для наукової статті чи фрагмента дисертаційного дослідження. Посібник призначено для магістрантів галузі знань 12/F «Інформаційні технології» спеціальності 125/F5 «Кібербезпека та захист інформації», а також буде корисним науковим і науково-педагогічним працівникам, фахівцям із захисту інформації та кібербезпеки.

© Володимир Соколов, 2027
© Київський столичний університет імені Бориса Грінченка, 2027

Зміст

Основні терміни та скорочення **1**

Вступ **5**

1 Програмне забезпечення мережевої безпеки як об'єкт розроблення і тестування **9**

1.1 Поняття та класифікація програмного забезпечення мережевої безпеки . . . 9
1.2 Функціональні архітектури та форми виконання . . . 11
1.3 Специфіка ПЗМБ як об'єкта розроблення . . . 12
1.4 Парадокс засобу захисту як поверхні атаки . . . 13
1.5 Модель порушника для ПЗМБ . . . 16
1.6 Формальна модель «дефект → вразливість → експлуатація» . . . 17
1.7 Життєвий цикл ПЗМБ і зацікавлені сторони . . . 19
1.8 Український контекст . . . 20
1.9 Наскрізні приклади посібника . . . 21
Висновки до розділу . . . 21
Питання для самоперевірки . . . 22

2 Нормативно-правове та стандартизаційне забезпечення розроблення і тестування програмного забезпечення мережевої безпеки **25**

2.1 Структура нормативного поля . . . 25
2.2 Законодавство України . . . 26
2.3 Право Європейського Союзу . . . 28
2.4 Міжнародні стандарти . . . 32
2.5 Рамкові документи і настанови . . . 33
2.6 Оцінювання відповідності та сертифікація . . . 34
2.7 Порівняльний аналіз вимог України та ЄС . . . 34
2.8 Кейс: вихід українського VPN-шлюзу на ринок ЄС . . . 36
Висновки до розділу . . . 37
Питання для самоперевірки . . . 38

3 Процесні моделі та метрики безпечного життєвого циклу розроблення **41**

3.1 Моделі життєвого циклу і точки вбудовування безпеки . . . 41

3.2 Microsoft SDL і точки дотику McGraw 42
3.3 NIST Secure Software Development Framework 43
3.4 ISO/IEC 27034 та IEC 62443-4-1: процесна суть 44
3.5 Моделі зрілості: OWASP SAMM і BSIMM 45
3.6 DevSecOps: ворота якості, безпека як код і зворотний зв'язок 46
3.7 Кількісні моделі безпечного життєвого циклу 47
3.8 Метрики процесу безпеки 50
3.9 Кейс: оцінювання зрілості команди-розробника ПЗМБ за SAMM 52
Висновки до розділу 54
Питання для самоперевірки 54

4 Вимоги безпеки, моделювання загроз і безпечне проєктування програмного забезпечення мережевої безпеки 57
4.1 Інженерія вимог безпеки 57
4.2 Моделювання загроз ПЗМБ 59
4.3 Принципи безпечного проєктування 62
4.4 Архітектурні патерни ПЗМБ 63
4.5 Метрики поверхні атаки 65
4.6 Кількісне ранжування заходів 67
4.7 Огляд проєкту безпеки й архітектурні рішення 67
4.8 Кейс: модель загроз і архітектура IDS-сенсора 68
Висновки до розділу 70
Питання для самоперевірки 70

5 Безпечне програмування мережевого програмного забезпечення 73
5.1 Класи дефектів мережевого коду 73
5.2 Безпека пам'яті та мови з безпечною пам'яттю 74
5.3 Безпечний розбір ворожого вводу 77
5.4 Цілочислова арифметика 79
5.5 Конкурентність, обробка помилок і керування ресурсами . . 81
5.6 Стандарти кодування й огляд коду 82
5.7 Засоби посилення під час компіляції та виконання 83
5.8 Кейс: переписування парсерів протоколів Suricata мовою Rust 85
Висновки до розділу 86
Питання для самоперевірки 86

6 Розроблення криптографічних компонентів і реалізація захищених протоколів 89
6.1 Вибір криптографічної бібліотеки 89
6.2 Неправильне використання криптографічних API 90

6.3 Реалізація зі сталим часом і побічні канали 92
6.4 Випадкові числа та керування ключами 94
6.5 Реалізація протоколів як кінцевих автоматів 96
6.6 Криптографічна гнучкість і постквантова міграція 98
6.7 Національні криптографічні алгоритми в програмному забезпеченні 100
6.8 Тестування криптографічних реалізацій 100
6.9 Кейс: міграція VPN-шлюзу на гібридну постквантову криптографію 101
Висновки до розділу 103
Питання для самоперевірки 103

7 Розроблення засобів фільтрації та контролю мережевого трафіку 105
7.1 Архітектури міжмережевих екранів 105
7.2 Підсистеми ОС і програмовані площини даних 106
7.3 Алгоритми класифікації пакетів 108
7.4 Політики та їхні аномалії 109
7.5 Таблиця станів і стійкість до виснаження 110
7.6 Тестування коректності фільтрів 111
7.7 Кейс: XDP-фільтр для захисту сервісу оператора зв'язку . . . 112
Висновки до розділу 113
Питання для самоперевірки 113

8 Розроблення систем виявлення та запобігання вторгненням 115
8.1 Архітектура NIDS/NIPS як програмного конвеєра 115
8.2 Snort 3, Suricata і Zeek 116
8.3 Алгоритми зіставлення шаблонів 116
8.4 Неоднозначність трафіку і нормалізація 118
8.5 Розроблення й супровід правил як коду 119
8.6 Аналітичні компоненти і шифрований трафік 120
8.7 Продуктивність і масштабування 121
8.8 Кейс: набір правил за звітом CERT-UA#4435 122
Висновки до розділу 122
Питання для самоперевірки 123

9 Розроблення засобів захищених комунікацій і контролю доступу до мережі 125
9.1 Інженерія VPN: протоколи та реалізації 125
9.2 Ядро чи простір користувача: продуктивність, ізоляція, накладні витрати 127
9.3 Засоби ZTNA і NAC 128
9.4 Площина менеджменту як головне джерело вразливостей . . 130

9.5 Безпечне оновлення прошивок і програмного забезпечення 131
9.6 Керування конфігурацією і секретами 133
9.7 Кейс: WireGuard як зразок мінімізації коду і безпеки за замовчуванням 133
Висновки до розділу 133
Питання для самоперевірки 134

10 Безпека ланцюга постачання програмного забезпечення та конвеєра CI/CD 135
10.1 Модель загроз ланцюга постачання 135
10.2 Перелік компонентів ПЗ (SBOM) і VEX 137
10.3 Походження і цілісність збирань 138
10.4 Керування залежностями та ризиком відкритих компонентів 140
10.5 Захист конвеєра CI/CD 141
10.6 Метрики і зрілість ланцюга постачання 142
10.7 Кейс: захищений конвеєр для ПЗМБ українського виробника 142
Висновки до розділу 144
Питання для самоперевірки 144

11 Методологія тестування безпеки програмного забезпечення 147
11.1 Тестування безпеки серед методів верифікації 147
11.2 ISO/IEC/IEEE 29119 і ризик-орієнтоване тестування 148
11.3 Рівні тестування ПЗМБ 149
11.4 Техніки проєктування тестів 150
11.5 Проблема оракула і метаморфне тестування 151
11.6 Адекватність тестів: покриття і мутації 152
11.7 Тестування в оцінюванні відповідності 153
11.8 Кейс: план тестування безпеки VPN-шлюзу 154
Висновки до розділу 155
Питання для самоперевірки 156

12 Статичний аналіз і формальна верифікація програмного забезпечення мережевої безпеки 157
12.1 Теоретичні межі статичного аналізу 157
12.2 Статичний аналіз безпеки коду 158
12.3 Символьне та конколічне виконання 161
12.4 Перевірка моделей і обмежена перевірка коду 161
12.5 Формальна верифікація протоколів 162
12.6 Верифіковані реалізації 163
12.7 Інтеграція в конвеєр розроблення 164
12.8 Кейс: SAST і обмежена перевірка моделі для парсера VPN-шлюзу 164

Висновки до розділу 165
Питання для самоперевірки 165

13 Динамічний аналіз і фазинг мережевого програмного забезпечення 167
13.1 Динамічний аналіз і санітайзери 167
13.2 Фазинг, керований покриттям 168
13.3 Обв'язки і безперервний фазинг 169
13.4 Фазинг мережевих протоколів зі станом 171
13.5 Статистичні моделі фазингу 172
13.6 Оцінювання фазерів 173
13.7 DAST і тестування API площини менеджменту 174
13.8 Кейс: фазинг парсера IKEv2 VPN-шлюзу 174
Висновки до розділу 175
Питання для самоперевірки 175

14 Тестування продуктивності, стійкості та відмовостійкості засобів мережевої безпеки 177
14.1 Продуктивність як властивість безпеки 177
14.2 Методології бенчмаркінгу 177
14.3 Генерація навантаження і тестові стенди 179
14.4 Моделі продуктивності 180
14.5 Стійкість до DoS і стрес-тестування 181
14.6 Відмовостійкість 182
14.7 Кейс: порівняльне тестування NGFW з увімкненими функціями безпеки 184
Висновки до розділу 185
Питання для самоперевірки 185

15 Оцінювання ефективності виявлення, тестові середовища та кіберполігони 187
15.1 Метрики якості виявлення 187
15.2 Набори даних і їхні вади 189
15.3 Методологічні пастки ML-IDS 190
15.4 Змагальна стійкість і тестування ухилення 190
15.5 Тестові середовища і кіберполігони 191
15.6 Емуляція противника і покриття ATT&CK 192
15.7 Відтворюваність і статистика 193
15.8 Кейс: порівняльне оцінювання сигнатурної та ML-NIDS . . . 193
Висновки до розділу 194
Питання для самоперевірки 195

16 Управління вразливостями, розкриття та супровід програмного забезпечення мережевої безпеки 197
16.1 Процес обробки вразливостей виробником 197
16.2 Координоване розкриття вразливостей 198
16.3 Ідентифікація та оцінювання вразливостей 199
16.4 Обов'язки виробника за CRA протягом періоду підтримки . . 201
16.5 Виправлення для граничних пристроїв і вікно вразливості . 201
16.6 Аналіз першопричин і варіантів 202
16.7 Оптимальний час випуску виправлення 203
16.8 Кейс: критична вразливість у VPN-шлюзі українського виробника 204
Висновки до розділу 205
Питання для самоперевірки 205

17 Гарантування безпеки, штучний інтелект у розробленні й тестуванні та напрями наукових досліджень 207
17.1 Кейси гарантування 207
17.2 Організація робіт розробника для сертифікації 208
17.3 Метрики і програма безпеки продукту 210
17.4 Штучний інтелект у розробленні 210
17.5 Штучний інтелект у тестуванні 211
17.6 Методологія наукових досліджень 211
17.7 Відкриті проблеми і напрями дисертаційних досліджень . . 212
17.8 Приклад: кейс гарантування для парсера IKEv2 213
Висновки до розділу 214
Питання для самоперевірки 215

Список використаних джерел 217

Додаток. Завдання для самостійної роботи 247

Основні терміни та скорочення

ДСТУ	національний стандарт України
ІКС	інформаційно-комунікаційна система
КЗІ	криптографічний захист інформації
КМУ	Кабінет Міністрів України
КСЗІ	комплексна система захисту інформації
МЕ	міжмережевий екран
НД ТЗІ	нормативний документ системи технічного захисту інформації
ОС	операційна система
ПЗ	програмне забезпечення
ПЗМБ	програмне забезпечення мережевої безпеки
ТЗІ	технічний захист інформації
ШІ	штучний інтелект

API	Application Programming Interface ‘прикладний програмний інтерфейс’
ASVS	Application Security Verification Standard ‘стандарт верифікації безпеки застосунків’ (OWASP)
CC	Common Criteria ‘Загальні критерії’ (ISO/IEC 15408)
CERT-UA	Урядова команда реагування на комп’ютерні надзвичайні події України
CI/CD	Continuous Integration / Continuous Delivery ‘безперервна інтеграція / безперервне постачання’
CRA	Cyber Resilience Act ‘Регламент (ЄС) 2024/2847 про кіберстійкість’
CSIRT	Computer Security Incident Response Team ‘команда реагування на інциденти кібербезпеки’
CVD	Coordinated Vulnerability Disclosure ‘координоване розкриття вразливостей’
CVE	Common Vulnerabilities and Exposures ‘перелік загальновідомих вразливостей’
CVSS	Common Vulnerability Scoring System ‘загальна система оцінювання вразливостей’

CWE	Common Weakness Enumeration ‘перелік типових слабкостей’
DAST	Dynamic Application Security Testing ‘динамічне тестування безпеки застосунків’
DFD	Data Flow Diagram ‘діаграма потоків даних’
DPDK	Data Plane Development Kit ‘набір для розроблення площини даних’
EDR	Endpoint Detection and Response ‘виявлення й реагування на кінцевих пристроях’
ENISA	European Union Agency for Cybersecurity ‘Агентство ЄС з кібербезпеки’
EPSS	Exploit Prediction Scoring System ‘система прогнозування експлуатації’
EUCC	European Common Criteria-based certification scheme ‘європейська схема сертифікації на основі CC’
FIPS	Federal Information Processing Standard ‘федеральний стандарт обробки інформації’ (США)
FPR	False Positive Rate ‘частка хибних спрацювань серед легітимних об’єктів’
GSN	Goal Structuring Notation ‘нотація структурування цілей’
HSM	Hardware Security Module ‘апаратний модуль безпеки’
IDS/IPS	Intrusion Detection / Prevention System ‘система виявлення / запобігання вторгненням’
IKEv2	Internet Key Exchange version 2 ‘протокол обміну ключами в Інтернеті, версія 2’
KEV	Known Exploited Vulnerabilities ‘каталог вразливостей, що активно експлуатуються’ (CISA)
KLOC	Kilo Lines of Code ‘тисяча рядків коду’
LLM	Large Language Model ‘велика мовна модель’
MFA	Multi-Factor Authentication ‘багатофакторна автентифікація’
ML	Machine Learning ‘машинне навчання’
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism ‘механізм інкапсуляції ключа на модульних ґратках’
MTTR	Mean Time to Remediate ‘середній час усунення’
MTU	Maximum Transmission Unit ‘максимальний розмір блока передавання’
NAC	Network Access Control ‘контроль доступу до мережі’

NDR	Network Detection and Response ‘мережеве виявлення й реагування’
NGFW	Next-Generation Firewall ‘міжмережевий екран нового покоління’
NIS2	Directive (EU) 2022/2555 ‘Директива ЄС про заходи для високого спільного рівня кібербезпеки’
PCAP	Packet Capture ‘формат файлу перехоплених пакетів’
PQC	Post-Quantum Cryptography ‘постквантова криптографія’
PSIRT	Product Security Incident Response Team ‘команда реагування на інциденти безпеки продукту’
SAMM	Software Assurance Maturity Model ‘модель зрілості гарантування ПЗ’ (OWASP)
SAST	Static Application Security Testing ‘статичне тестування безпеки застосунків’
SBOM	Software Bill of Materials ‘перелік компонентів програмного забезпечення’
SDL	Security Development Lifecycle ‘життєвий цикл безпечного розроблення’
SFR / SAR	Security Functional / Assurance Requirements ‘функціональні вимоги безпеки / вимоги гарантій’
SIEM	Security Information and Event Management ‘керування інформацією та подіями безпеки’
SLSA	Supply-chain Levels for Software Artifacts ‘рівні захищеності ланцюга постачання програмних артефактів’
SMT	Satisfiability Modulo Theories ‘виконуваність за модулем теорій’
SSDF	Secure Software Development Framework ‘рамка безпечного розроблення ПЗ’ (NIST)
SSVC	Stakeholder-Specific Vulnerability Categorization ‘категоризація вразливостей з урахуванням зацікавленої сторони’
TCB	Trusted Computing Base ‘довірена обчислювальна база’
TLS	Transport Layer Security ‘протокол захисту транспортного рівня’
TPM	Trusted Platform Module ‘довірений платформний модуль’
TPR	True Positive Rate ‘частка виявлених серед атакувальних об’єктів’
TUF	The Update Framework ‘рамка захищеного оновлення’

VEX	Vulnerability Exploitability eXchange ‘обмін даними про експлуатованість вразливостей’
VPN	Virtual Private Network ‘віртуальна приватна мережа’
WAF	Web Application Firewall ‘міжмережевий екран вебзастосунків’
XDP	eXpress Data Path ‘швидкий шлях обробки даних’ (ядро Linux)
ZTNA	Zero Trust Network Access ‘мережевий доступ за принципом нульової довіри’

Вступ

Міжмережеві екрани, системи виявлення та запобігання вторгненням, VPN-шлюзи, засоби контролю доступу до мережі, криптографічні бібліотеки й реалізації захищених протоколів утворюють перший рубіж захисту будь-якої інформаційно-комунікаційної системи. Водночас саме ці продукти, розміщені на межі довіри й наділені найвищими привілеями, у 2023–2025 рр. стали однією з головних цілей зловмисників: вразливості граничних пристроїв посідають помітне місце в каталогах активно експлуатованих вразливостей, а компрометація засобу захисту відкриває порушникові шлях до всієї мережі в обхід периметра. Причини цих інцидентів здебільшого лежать не в криптографічних примітивах, а в інженерії: помилках роботи з пам'яттю, розбору ворожого вводу, кінцевих автоматах протоколів, незахищеній площині керування, слабкому ланцюгу постачання й недостатньому тестуванні.

Тому якість процесів розроблення і тестування програмного забезпечення мережевої безпеки стала предметом не лише інженерної практики, а й державного регулювання. Регламент ЄС про кіберстійкість (Cyber Resilience Act) покладає на виробників обов'язки щодо безпечного проєктування, обробки вразливостей і звітування; NIST оновлює рамку безпечного розроблення SSDF; ініціатива Secure by Design вимагає усувати цілі класи вразливостей, зокрема через перехід на мови з безпечною пам'яттю. В Україні оновлено законодавство про кібербезпеку, порядок державної експертизи засобів технічного та криптографічного захисту інформації, запроваджено механізм пошуку вразливостей із залученням дослідників. Досвід війни показав, що стійкість вітчизняних засобів захисту й здатність виробника швидко виправляти вразливості є питанням національної безпеки.

Мета і завдання навчальної дисципліни

Мета курсу — формування в магістрантів знань, умінь і навичок із застосування сучасних технологій, методів і стандартів розроблення та тестування програмного забезпечення мережевої безпеки, а також здатності до наукового дослідження в цій галузі.

Завдання курсу — отримання теоретичних знань і практичних умінь з проєктування, реалізації, верифікації, тестування й супроводу програмного забезпечення мережевої безпеки та набуття таких компетентностей:

- здатність застосовувати процесні моделі, стандарти й технології безпечного життєвого циклу розроблення під час створення засобів мережевої безпеки;
- здатність обґрунтовано обирати й застосовувати методи статичного й динамічного аналізу, фазингу, формальної верифікації та тестування продуктивності й ефективності виявлення;
- здатність оцінювати відповідність програмного забезпечення мережевої безпеки національним і європейським вимогам та організовувати управління його вразливостями;
- здатність до наукового дослідження й удосконалення методів розроблення і тестування програмного забезпечення мережевої безпеки.

Структура посібника

Матеріал згруповано у чотири логічні блоки.

Блок I. Засади (розділи 1–4) формує понятійний апарат: програмне забезпечення мережевої безпеки як об'єкт розроблення і тестування; нормативно-правове та стандартизаційне поле України, Європейського Союзу й міжнародних організацій; процесні моделі та метрики безпечного життєвого циклу розроблення; вимоги безпеки, моделювання загроз і безпечне проєктування.

Блок II. Технології розроблення (розділи 5–10) присвячено безпечному програмуванню мережевого коду, розробленню криптографічних компонентів і реалізації захищених протоколів, засобів фільтрації трафіку, систем виявлення та запобігання вторгненням, засобів захищених комунікацій і контролю доступу, а також безпеці ланцюга постачання програмного забезпечення та конвеєра CI/CD.

Блок III. Технології тестування (розділи 11–15) охоплює методологію тестування безпеки, статичний аналіз і формальну верифікацію, динамічний аналіз і фазинг, тестування продуктивності й стійкості, оцінювання ефективності виявлення та використання тестових середовищ і кіберполігонів.

Блок IV. Супровід, гарантування та дослідження (розділи 16–17) розглядає управління вразливостями й координоване розкриття, кейси гарантування безпеки, організацію сертифікації, застосування штучного інтелекту в розробленні й тестуванні та напрями наукових досліджень.

Як працювати з посібником

Посібник розраховано на здобувачів другого (освітнього) рівня вищої освіти, які вже володіють базовими знаннями з програмування, комп'ютерних мереж, інформаційної безпеки й криптографії. Виклад побудовано як систе-

матизація на рівні моделей, стандартів, кількісних оцінок і критичного аналізу чинних рішень. Формули супроводжуються поясненням позначень і числовими прикладами, які варто відтворити самостійно; фрагменти коду ілюструють захисні практики й придатні для експериментів на власному стенді.

Питання для самоперевірки наприкінці кожного розділу неоднорідні: частина перевіряє відтворення матеріалу, частина потребує аналізу й розрахунків, а завершальні питання сформульовано як дослідницькі завдання — вони можуть слугувати відправною точкою для наукової статті або фрагмента дисертаційного дослідження.

Посилання на джерела наведено в квадратних дужках; повний список подано наприкінці видання. Перевагу віддано чинним нормативним документам, національним і міжнародним стандартам, специфікаціям IETF і рецензованим науковим працям. Усі нормативні акти, стандарти й версії інструментів слід перевіряти на чинність на момент використання: ця сфера змінюється дуже швидко.

Етичне застереження

Матеріал посібника призначено для розроблення, верифікації та захисту *власного* програмного забезпечення і систем, на тестування яких є дозвіл власника. Посібник свідомо не містить робочих експлойтів, корисних навантажень чи технік обходу засобів захисту: вразливості й атаки розглядаються на рівні класів, першопричин і контрзаходів. Використання описаних методів щодо чужих систем без дозволу є кримінально караним.

Термінологічне зауваження

У виданні вживається скорочення **ПЗМБ** — програмне забезпечення мережевої безпеки. Термін «розроблення» позначає процес створення програмного забезпечення; у назві посібника збережено усталене в освітніх програмах слово «розробка». Англомовні терміни наводяться в дужках мовою оригіналу при першому вживанні; для понять, що не мають усталеного українського відповідника, використано транслітерацію (фазинг, санітайзер), а для решти — український термін (обв'язка — harness, перелік компонентів ПЗ — SBOM). Терміни «хибне спрацювання» і «хибна тривога» вживаються як синоніми. Регламент (ЄС) 2024/2847 про кіберстійкість позначається скороченням CRA.

Наскрізні умовні приклади посібника (VPN-шлюз, IDS-сенсор, NGFW), до яких повертаються різні розділи, описано в підрозділі 1.9.

РОЗДІЛ 1

Програмне забезпечення мережевої безпеки як об'єкт розроблення і тестування

Міжмережеві екрани, VPN-шлюзи, системи виявлення вторгнень і криптографічні бібліотеки традиційно оцінюють за функціональною ефективністю. Проте саме ці продукти, розміщені на межі довіри, стали однією з головних точок початкового проникнення в мережі державних установ і критичної інфраструктури, тож їх потрібно розглядати як окремий інженерний об'єкт зі своїми вимогами до розроблення, верифікації та супроводу. Розділ вводить поняття й класифікацію такого ПЗ, описує його функціональні архітектури та специфіку як об'єкта розроблення, аналізує «парадокс засобу захисту» на кейсах 2014–2025 рр., формулює модель порушника й імовірнісну модель «дефект → вразливість → експлуатація», окреслює життєвий цикл, зацікавлені сторони, український контекст і наскрізні приклади посібника. Нормативну базу розглянуто в розділі 2, процесні моделі й метрики — у розділі 3, моделювання загроз — у розділі 4.

1.1 Поняття та класифікація програмного забезпечення мережевої безпеки

1.1.1 Базові означення

Програмне забезпечення мережевої безпеки (ПЗМБ) — програмні засоби, що реалізують функції захисту інформації в мережах і на їхніх межах: контроль і фільтрацію трафіку, виявлення та запобігання вторгненням, криптографічний захист каналів, автентифікацію й контроль доступу до мережевих ресурсів, а також моніторинг мережевої активності. До ПЗМБ належать і компоненти, на які ці функції спираються: криптографічні бібліотеки, реалізації захищених протоколів, мережеві стеки й прошивки мережевого обладнання. ПЗМБ є окремим випадком *засобу захисту інформації* в розумінні законодавства України [42, 37], а в термінології ЄС — продуктом із цифровими елементами з функціями, критичними для кібербезпеки [301].

Слід розрізняти *функцію безпеки*, яку продукт надає захищуваній системі, і *власну захищеність* продукту — його стійкість до атак на нього самого. Класичні критерії оцінювання (ISO/IEC 15408 [201], НД ТЗІ 2.5-004 [29]) зосереджуються передусім на першій; посібник наголошує на другій.

1.1.2 Класи ПЗМБ

Класифікацію за основною функцією й точкою розміщення, що визначають поверхню атаки і профіль вимог, наведено в табл. 1.1 (детально — у розділах 6–9).

Таблиця 1.1 — Класи ПЗМБ, їхні функції, розміщення і типові вимоги

Клас	Основні функції	Розміщення	Типові вимоги
ME, NGFW	фільтрація зі станом, NAT, ідентифікація застосунків, інспекція TLS	периметр, межі сегментів	пропускна здатність, коректність політик, відмова в закритий стан
WAF	розбір HTTP, захист від ін'єкцій, віртуальне виправлення	перед вебзастосунками	повнота розбору, низька частка хибних спрацювань
IDS/IPS, NDR	декодування протоколів, сигнатурний і поведінковий аналіз	дзеркало трафіку або в розриві	стійкість до ухилення, відсутність втрат пакетів
VPN-шлюзи й клієнти	IPsec/IKEv2, TLS-VPN, WireGuard, автентифікація користувачів	межа мережі, кінцеві пристрої	криптостійкість, захищений портал, керування ключами
NAC, ZTNA	перевірка стану пристрою, політики доступу, посередництво	вхід до мережі, хмара	нульова довіра, безперервна автентифікація
Проксі, шлюзи безпеки	SWG, поштові шлюзи, DNS-фільтрація	вихід в Інтернет, хмара	масштабованість, приватність даних
Сенсори й агенти	збирання телеметрії (EDR/NDR), локальне реагування	хости, сегменти	мінімальні привілеї, цілісність телеметрії
Криптобібліотеки	TLS, IPsec, SSH, QUIC, примітиви й протоколи	усередині інших продуктів	сталий час, коректність автомата станів
Мережеві стеки, прошивки	TCP/IP, маршрутизація, керування пристроєм	маршрутизатори, комутатори, IoT	безпечне завантаження й оновлення

Межі між класами розмиваються: сучасний NGFW поєднує фільтрацію, IPS, VPN-шлюз, проксі й елементи ZTNA в одному образі прошивки. Для інженерії це означає, що поверхня атаки продукту дорівнює *об'єднанню* поверхонь усіх увімкнених функцій, а вразливість у другорядному модулі (наприклад, у порталі SSL-VPN) компрометує весь пристрій разом із функціями фільтрації. Рекомендації NIST щодо ME, IDPS і VPN [316, 317, 83] описують ці класи з погляду експлуатанта; у посібнику вони розглядаються з погляду розробника.

1.2 Функціональні архітектури та форми виконання

1.2.1 Площини даних, керування і менеджменту

Мережеві засоби захисту, як і мережеве обладнання загалом, зручно декомпозувати на три **площини** (рис. 1.1). **Площина даних** (data plane) обробляє кожен пакет або потік: розбирає заголовки, класифікує, застосовує правила, шифрує чи розшифровує, пересилає. Вона працює на лінійній швидкості й безпосередньо отримує ворожий вхід. **Площина керування** (control plane) встановлює стан, необхідний площині даних: виконує рукостискання IKE або TLS, обмінюється маршрутною інформацією, компілює політики в таблиці класифікації, керує сеансами. **Площина менеджменту** (management plane) забезпечує взаємодію з адміністратором і зовнішніми системами: вебінтерфейс, REST API, CLI через SSH, SNMP, журналювання, отримання оновлень сигнатур і прошивок, а також портали автентифікації користувачів VPN.

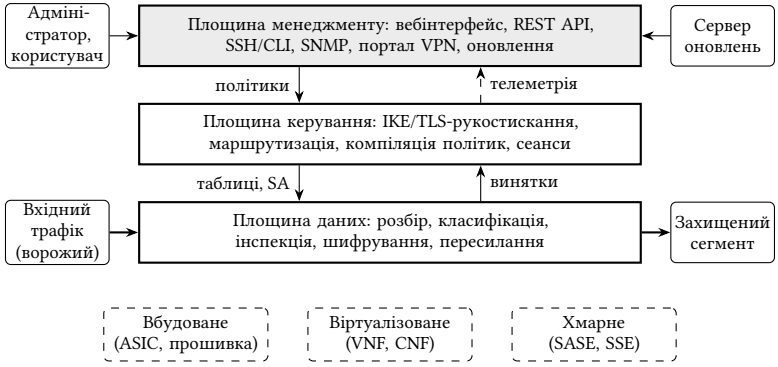


Рис. 1.1 — Площини ПЗМБ і форми їх виконання

Площини мають різні профілі ризику. Площина даних обробляє найбільший обсяг ворожого входу, але її код компактний і часто реалізований у спеціалізованих підсистемах (eBPF/XDP, DPDK, мережеві процесори). Площина менеджменту обробляє мало запитів, проте містить найбільший обсяг коду (вебсервери, інтерпретатори, бібліотеки розбору форматів), нерідко успадкованого. Саме на неї та на портали автентифікації припадає більшість критичних вразливостей граничних пристроїв 2023–2025 рр. (підрозділ 1.4). Розділення площин за привілеями, процесами й інтерфейсами — основний архітектурний принцип, розглянутий у розділах 4 і 9.

1.2.2 Форми виконання

Вбудоване (апаратне) виконання — спеціалізований пристрій із власною прошивкою, часто з апаратним прискоренням: передбачувана продуктивність ціною закритості для моніторингу й криміналістики та тривалих циклів оновлення.

Віртуалізоване виконання реалізує функції безпеки як віртуальні мережеві функції (VNF) на загальній інфраструктурі або як контейнерні мережеві функції (CNF) в оркестрованому середовищі. Архітектурну рамку NFV задає ETSI [167]; віртуалізація додає до поверхні атаки гіпервізор, оркестратор і їхні API.

Хмарне виконання надає функції ME, проксі, ZTNA і DLP як сервіс постачальника (моделі SASE і SSE). Виробник стає оператором багатокористувацької платформи, і вразливість у його ПЗ — вразливістю тисяч клієнтів; концептуальну основу цих моделей дає архітектура нульової довіри [309].

1.3 Специфіка ПЗМБ як об'єкта розроблення

1.3.1 Сім визначальних властивостей

Безпекові вимоги до ПЗМБ суворіші, ніж до більшості прикладного ПЗ, через поєднання таких властивостей.

1. *Ворожий вхід на лінійній швидкості.* Кожен байт, що надходить на інтерфейс, потенційно сформований порушником, а обробити його потрібно за наносекунди.
2. *Розташування на межі довіри.* Для граничних пристроїв це неавтентифікований доступ з Інтернету до порталів, рукостискань і вебінтерфейсів.
3. *Привілейоване виконання.* Обробка пакетів у ядрі, доступ до ключів, керування маршрутизацією вимагають високих привілеїв; помилка в будь-якому компоненті часто дає порушнику права root на пристрої.
4. *Складні протоколи й формати.* TLS, IKEv2, QUIC, HTTP/2, DNS містять сотні полів і станів; специфікації неоднозначні, а реалізації мусять обробляти й некоректні, але поширені варіанти [304, 220, 213].
5. *Вимоги сумісності.* Підтримка застарілих протоколів і шифрів створює можливість атак зниження версії.
6. *Довгий життєвий цикл.* Пристрої експлуатуються 7–15 років, а підтримка кількох гілок прошивки множить обсяг коду, який треба виправляти.
7. *Непрозорість для захисника.* На граничний пристрій не встановити агент EDR, а його журнали контролює потенційно скомпрометова-

ний код; виявлення компрометації вимагає вбудованих механізмів перевірки цілісності та криміналістичного журналювання [105].

1.3.2 Бюджет часу на пакет

Перша властивість має просте кількісне вираження. Мінімальний кадр Ethernet займає в лінії $L + 20$ байтів ($L = 64$ байти кадру, 8 байтів преамбули, 12 байтів міжкадрового інтервалу), тому за пропускної здатності інтерфейсу C (біт/с) часовий бюджет на обробку одного пакета дорівнює

$$t_{\text{пак}} = \frac{8\,(L + 20)}{C}, \qquad N_{\text{такт}} = t_{\text{пак}} \cdot f_{\text{ЦП}} \cdot n_{\text{ядер}}, \tag{1.1}$$

де $N_{\text{такт}}$ — кількість тактів процесора, доступних на пакет; $f_{\text{ЦП}}$ — тактова частота; $n_{\text{ядер}}$ — кількість ядер, що паралельно обробляють трафік. Для інтерфейсу 10 Гбіт/с і пакетів по 64 байти маємо $t_{\text{пак}} = 8 \cdot 84/10^{10} = 67{,}2$ нс (14,88 млн пакетів за секунду). За частоти 3 ГГц одне ядро має лише ≈ 200 тактів на пакет — порівнянно з одним промахом кешу останнього рівня. Навіть вісім ядер дають ≈ 1600 тактів, чого недостатньо для повної інспекції прикладного рівня. Звідси розбір у площині даних має бути мінімальним і однопрохідним, а складна інспекція виноситься на окремі потоки. Методологію вимірювання продуктивності засобів мережевої безпеки задають RFC 2544 і RFC 9411 [98, 82] (розділ 14).

1.3.3 Компроміси «безпека–продуктивність–сумісність»

Три групи вимог утворюють трикутник, у якому покращення однієї вершини, як правило, погіршує інші. Сувора валідація полів і відкидання неоднозначних пакетів підвищує стійкість до ухилення (розділ 8), але порушує сумісність із некоректними, проте легітимними реалізаціями. Ізоляція розбору в окремому процесі зменшує наслідки помилок, але додає копіювання й перемикання контекстів. Рішення вимагає кількісних моделей і вимірювань (розділи 3, 14).

1.4 Парадокс засобу захисту як поверхні атаки

1.4.1 Від Heartbleed до граничних пристроїв

Парадокс засобу захисту полягає в тому, що продукт, установлений для зменшення ризику, сам додає до системи поверхню атаки, причому розташовану в найбільш експонованій точці і з найвищими привілеями. Якщо ймовірність і наслідки компрометації засобу перевищують ефект від виконуваної ним функції, сумарний ризик зростає (кількісну умову сформульовано в підрозділі 1.6).

Класичним прикладом стала вразливість Heartbleed (CVE-2014-0160) у бібліотеці OpenSSL. Обробник розширення heartbeat протоколу TLS копіював

у відповідь стільки байтів, скільки вказувало поле довжини в запиті, не звіряючи його з фактичною довжиною запису, тож порушник без автентифікації отримував до 64 КіБ пам'яті процесу, зокрема закриті ключі й сесійні маркери. Дефект існував близько двох років; за оцінкою Дурумеріка та співавторів, на момент розкриття вразливими були 24–55 % HTTPS-серверів зі списку Alexa Top 1 Million, а повідомлення власників збільшило частку виправлених хостів майже на 50 % [156]. Урок двоїстий: дефект розбору довжини (CWE-125 [136]) зруйнував саме ту властивість — конфіденційність, — яку бібліотека мала забезпечувати, а повторне використання компонента перетворило одну помилку на системну подію для всього Інтернету.

Клас помилки ілюструє лістинг 1.1: оголошена в повідомленні довжина є *недовіреними даними* і має перевірятися щодо фактично отриманого обсягу ще до будь-якого доступу до пам'яті. Систематично методи безпечного розбору розглянуто в розділі 5.

Лістинг 1.1 — Перевірка оголошеної довжини під час розбору запису

```
/* Запис: тип (1 байт) | довжина (2 байти) | дані */
int parse_rec(const uint8_t *buf, size_t avail,
              struct rec *out)
{
    if (avail < 3)               /* немає повного заголовка */
        return -1;
    uint16_t len = (uint16_t)((buf[1] << 8) | buf[2]);
    /* оголошена довжина -- недовірені дані: вона не може
       перевищувати фактично отриманий обсяг */
    if (len > avail - 3)
        return -1;
    out->type = buf[0];
    out->len  = len;
    out->data = buf + 3;         /* вказівник у межах buf */
    return 0;
}
```

Упродовж 2023–2025 рр. центр ваги атак остаточно змістився на граничні пристрої. Аналіз Google Threat Intelligence Group показав, що з 75 вразливостей нульового дня, експлуатованих у реальних атаках 2024 р., 33 (44 %) стосувалися корпоративних технологій, і понад 60 % із них (20 вразливостей) — засобів безпеки й мережевих продуктів [108]. За даними M-Trends 2025, експлуатація вразливостей є найпоширенішим вектором початкового проникнення (33 % інцидентів 2024 р.), а чотири найчастіше експлуатовані вразливості належали граничним і мережевим засобам PAN-OS, Ivanti і Fortinet [239]. За підрахунком автора за каталогом CISA KEV [123] (версія 2026.10.04, евристична класифікація за виробником і продуктом) близько 19 % з 1734 записів стосуються мережевого й граничного обладнання, а серед доданих у 2024–2025 рр. — близько 22 %; Cisco (100 записів) по-

ступається лише Microsoft, а Ivanti (35), Fortinet (31) і Citrix (27) входять до першої дюжини виробників.

1.4.2 Кейс: вразливості граничних пристроїв 2023–2025 рр.

Табл. 1.2 узагальнює найбільш показові випадки. Ідентифікатори CWE подано за каталогом KEV і бюлетенями виробників.

Таблиця 1.2 — Вразливості ПЗМБ: клас дефекту, першопричина, урок

Продукт, CVE	CWE	Першопричина	Урок для розроблення
OpenSSL, CVE-2014-0160	125	довжина з повідомлення heartbeat не звірялася з довжиною запису	перевірка довжин, фазинг парсерів, безпечна пам'ять
Citrix NetScaler, CVE-2023-4966	119	запит зі спеціально сформованим заголовком Host повертав вміст пам'яті з сесійними маркерами	обробка буферів у порталі; анулювання сеансів після виправлення
Ivanti Connect Secure, CVE-2023-46805, CVE-2024-21887	287, 77	обхід автентифікації у вебкомпоненті й ін'єкція команд в інтерфейсі адміністрування	ланцюг двох дефектів площини менеджменту; не формувати команди оболонки з вводу
FortiOS SSL-VPN, CVE-2024-21762	787	запис за межами буфера в процесі `sslvpnd` під час обробки HTTP-запитів	неавтентифікований розбір HTTP у привілейованому процесі; ізоляція
PAN-OS GlobalProtect, CVE-2024-3400	20, 77	довільне створення файлу, що веде до ін'єкції команд ОС із правами root	недовірені дані в шляхах файлів; найменші привілеї
Cisco ASA/FTD (ArcaneDoor), CVE-2024-20353, CVE-2024-20359	835, 94	неповна перевірка помилок під час розбору HTTP-заголовка; виконання коду з неперевіреного файлу у флеш-пам'яті	цілісність завантаження й файлів; криміналістична прозорість

CitrixBleed використовували афіліати LockBit 3.0 для захоплення сеансів в обхід багатофакторної автентифікації; CISA рекомендувало також анулювати всі активні сеанси, бо викрадені маркери залишалися дійсними [116]. Через вразливості Ivanti CISA видало екстрену директиву ED 24-01, що зобов'язала федеральні органи США тимчасово від'єднати пристрої від мереж [122]. Вразливість PAN-OS отримала оцінку CVSS 4.0 10,0 [284, 173], FortiOS — 9,6 за CVSS 3 [177]. Кампанія ArcaneDoor державного порушника поєднувала вразливості Cisco ASA зі спеціальними імплантами для прихованої присутності на граничних пристроях державних мереж [125].

1.4.3 Спільні першопричини

Аналіз табл. 1.2 дає змогу виділити три групи першопричин. *Безпека пам'яті*: CWE-125, 119, 787 — класичні наслідки ручного керування пам'яттю в C/C++ у коді, що розбирає ворожий вхід. *Розбір і нейтраліза-*

ція вводу: CWE-20 і CWE-77 — дані з мережі потрапляють у шляхи файлів і команди оболонки без нейтралізації. *Архітектура площини менеджменту*: вебпортали й інтерфейси адміністрування доступні з Інтернету, виконуються з високими привілеями і мають слабке розмежування між неавтентифікованою і автентифікованою частинами. Більшість цих класів (CWE-787, 125, 77, 78, 20, 94) входить до CWE Top 25 2025 [143], решта — їхні батьківські або суміжні слабкості: проблема не в невідомих вразливостях, а в недостатньому застосуванні відомих практик. Цей висновок покладено в основу ініціативи Secure by Design [114] і спільних настанов агенцій «П'яти очей» щодо граничних пристроїв, які вимагають від виробників безпечних налаштувань за замовчуванням, журналювання для криміналістичного аналізу і переходу на мови з безпечною пам'яттю [105].

1.5 Модель порушника для ПЗМБ

1.5.1 Профілі порушників

Модель порушника для ПЗМБ охоплює не лише атаки *через* засіб захисту (обхід фільтрації, ухилення від виявлення), а й атаки *на* сам засіб і процес його створення. Типові профілі наведено в табл. 1.3; техніки подано за базою знань MITRE ATT&CK [253].

Таблиця 1.3 — Профілі порушника для ПЗМБ

Профіль	Можливості й точки входу	Техніки ATT&CK
Дистанційний неавтентифікований	мережевий доступ до площини даних, порталів VPN, вебінтерфейсу; сканування Інтернету	T1190, T1133
Автентифікований користувач	дійсні облікові дані (зокрема викрадені), доступ до функцій після входу	T1078, T1068
Привілейований інсайдер оператора	доступ до площини менеджменту, зміна конфігурацій і модулів автентифікації	T1556, T1562
Порушник у ланцюгу постачання	компрометація залежностей, засобів збирання, каналів оновлення	T1195
Зловмисний або недбалий розробник	доступ до вихідного коду і конвеєра CI/CD, можливість внесення закладок	T1195.002
Державний порушник (APT)	поєднання наведених можливостей, вразливості нульового дня, модифікація образу пристрою	T1190, T1601, T1542

1.5.2 Цілі та можливості

Цілі порушника щодо ПЗМБ зводяться до чотирьох груп: *початковий доступ* до мережі через граничний пристрій (домінує техніка T1190 «Експлуатація застосунку, доступного ззовні»); *тривала прихована присутність* на пристрої, недоступному для EDR, з модифікацією образу системи (T1601)

або модулів автентифікації (T1556); *перехоплення* трафіку й облікових даних; *руйнування* мережевого обладнання. Державним порушникам властиві всі чотири цілі, кримінальним угрупованням — переважно перша, з масовою експлуатацією відомих невиправлених вразливостей.

Можливості порушника параметризують позицією щодо продукту, доступними інтерфейсами, знаннями й ресурсами. Важлива особливість ПЗМБ — *доступність образу продукту*: прошивку зазвичай можна завантажити, тож порушник має двійковий код і може порівнювати версії до й після виправлення для швидкого відтворення вразливості. Звідси вимога, щоб безпека не ґрунтувалася на неопублікованості коду, а вікно між випуском виправлення і його встановленням було мінімальним (розділ 16). Формалізацію моделей загроз розглянуто в розділі 4, загальні вимоги до моделі порушника в національній нормативній базі — у [28].

1.6 Формальна модель «дефект → вразливість → експлуатація»

1.6.1 Ланцюг перетворень

Розглянемо, як дефекти коду перетворюються на експлуатовані вразливості (рис. 1.2). **Дефект** — відхилення реалізації від специфікації або очікуваної поведінки. Лише частина дефектів є **вразливостями**, що дають змогу порушити властивість безпеки; лише частина вразливостей *досяжна* з ворожого входу, і лише частина досяжних буде *експлуатована* в реальних атаках.

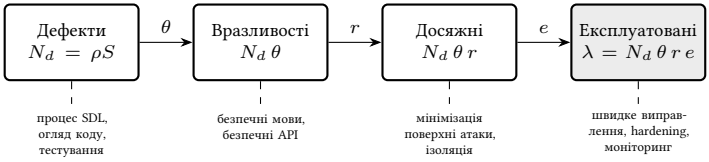


Рис. 1.2 — Ланцюг «дефект → вразливість → експлуатація» і точки впливу інженерних заходів

1.6.2 Імовірнісна модель

Нехай продукт складається з K компонентів обсягом S_k тисяч рядків коду (KLOC) із залишковою щільністю дефектів ρ_k (дефектів на KLOC у випущеній версії). Позначимо θ_k частку дефектів, що є вразливостями; r_k — імовірність досяжності вразливості з ворожого входу для заданої моделі порушника; e_k — імовірність того, що досяжна вразливість буде експлуатована в реальних атаках протягом періоду підтримки версії. Очікувана кількість

латентних вразливостей

$$N_v = \sum_{k=1}^{K} \rho_k \, S_k \, \theta_k, \tag{1.2}$$

а очікувана кількість експлуатованих вразливостей

$$\lambda = \sum_{k=1}^{K} \rho_k \, S_k \, \theta_k \, r_k \, e_k. \tag{1.3}$$

Якщо експлуатовані вразливості виникають незалежно і їх кількість описується розподілом Пуассона з параметром λ, імовірність того, що версія продукту матиме хоча б одну експлуатовану вразливість, дорівнює

$$P_x = 1 - e^{-\lambda}. \tag{1.4}$$

Параметри моделі мають емпіричні оцінки. Для зрілих процесів залишкова щільність дефектів у випущеному продукті становить близько 0,5–1 дефекту на KLOC, тоді як під час внутрішнього тестування фіксують десятки дефектів на KLOC [244]. Частка дефектів, що є вразливостями, та їхня досяжність суттєво залежать від мови й типу компонента; у моделі їх слід розглядати як параметри, що оцінюються для конкретної кодової бази (розділ 3). Для e_k орієнтиром є модель EPSS, яка оцінює ймовірність експлуатації вразливості в найближчі 30 днів за ознаками з відкритих джерел [174, 214]. Дослідження Озмента і Шехтера на прикладі OpenBSD показало, що з віком кодової бази темп виявлення успадкованих вразливостей знижується, проте повільно [283] — отже, довгий життєвий цикл ПЗМБ сам по собі не гарантує «дозрівання» коду.

Числовий приклад (параметри ілюстративні). Розглянемо прошивку NGFW із чотирма групами компонентів (усі $\rho_k = 0{,}7$ дефекту/KLOC, $\theta_k = 0{,}03$, $e_k = 0{,}05$): площина менеджменту з вебінтерфейсом, доступним з Інтернету ($S = 500$, $r = 0{,}2$), площина даних ($S = 300$, $r = 0{,}3$), портал і демон VPN ($S = 150$, $r = 0{,}5$), решта системи ($S = 1050$, $r = 0{,}01$). За (1.2) $N_v = 0{,}7 \cdot 2000 \cdot 0{,}03 = 42$ латентні вразливості. За (1.3) внески груп становлять 0,105; 0,095; 0,079 і 0,011, тобто $\lambda \approx 0{,}29$, а за (1.4) $P_x = 1 - e^{-0{,}29} \approx 0{,}25$. Якщо закрити доступ до вебінтерфейсу з Інтернету (виділений інтерфейс менеджменту, $r = 0{,}02$), внесок першої групи падає до 0,011, $\lambda \approx 0{,}20$ і $P_x \approx 0{,}18$. Переписування VPN-демона мовою з безпечною пам’яттю, що зменшує θ для нього втричі, дає ще $-0{,}053$: $\lambda \approx 0{,}14$, $P_x \approx 0{,}13$. Отже, два архітектурні рішення майже вдвічі зменшують імовірність появи експлуатованої вразливості, тоді як зменшення щільності дефектів на 20 % лише пропорційно знизило б λ. Вплив на r і θ часто ефективніший за загальне «підвищення якості».

1.6.3 Умова доцільності засобу захисту

Сформулюємо парадокс підрозділу 1.4 кількісно. Нехай R_0 — імовірність компрометації захищуваної мережі за рік без засобу захисту, η — частка атак, які засіб відвертає, q — імовірність того, що за наявності експлуатованої вразливості в засобі конкретну інсталяцію буде скомпрометовано (вона залежить від експонованості та швидкості встановлення виправлень). Тоді ризик із засобом

$$R_1 = R_0\,(1-\eta) + q\,P_x, \qquad R_1 < R_0 \iff \eta\,R_0 > q\,P_x. \tag{1.5}$$

За $R_0 = 0{,}3$, $\eta = 0{,}7$ і $P_x = 0{,}25$ засіб доцільний, доки $q < 0{,}84$: при оперативному виправленні ($q = 0{,}2$) $R_1 = 0{,}09 + 0{,}05 = 0{,}14$, тобто ризик зменшується більш ніж удвічі. Проте якщо інтерфейс менеджменту відкритий в Інтернет, а оновлення встановлюють із запізненням на місяці ($q \to 0{,}9$), то $R_1 = 0{,}09 + 0{,}225 = 0{,}315 > R_0$ — засіб захисту *збільшує* ризик. Умова (1.5) пояснює, чому відповідальність за безпеку ПЗМБ розподілена між виробником, який впливає на P_x, і оператором, який впливає на q (підрозділ 1.7). Умова (1.5) — модель-евристика з ілюстративними параметрами; їх оцінювання для реальних продуктів — відкрита задача.

1.7 Життєвий цикл ПЗМБ і зацікавлені сторони

1.7.1 Етапи життєвого циклу

Життєвий цикл ПЗМБ охоплює звичні етапи — вимоги, проєктування, реалізацію, збирання, верифікацію, випуск, супровід, виведення з експлуатації, — але кожен має безпекову специфіку: модель порушника й профілі захисту у вимогах, вибір мови й бібліотек розбору в реалізації, захищеність конвеєра збирання, поєднання статичного аналізу, фазингу й навантажувального тестування у верифікації. Супровід для ПЗМБ особливо тривалий: виробник випускає виправлення для кількох гілок прошивки роками після припинення продажів (розділ 16). Практики для всіх етапів описують NIST SSDF [328], IEC 62443-4-1 [197] і ISO/IEC 27034 [205] (розділ 3); українські дослідники пропонують методи інтегрування вимог кібербезпеки у фази життєвого циклу в межах підходу DevSecOps [4].

1.7.2 Зацікавлені сторони та розподіл відповідальності

Безпека ПЗМБ залежить від шести груп учасників.

Виробник визначає архітектуру, процеси розроблення, налаштування за замовчуванням, політику підтримки й випуску виправлень; впливає на параметри ρ, θ, r моделі (1.3).

Постачальники компонентів — спільноти відкритого коду і комерційні постачальники бібліотек, ОС, апаратних платформ; вразливість у їхньому компоненті успадковують усі продукти (Heartbleed).

Інтегратор розгортає й налаштовує продукт у складі системи захисту, визначає експонованість інтерфейсів.

Оператор експлуатує продукт, установлює оновлення, контролює журнали; впливає на параметр q умови (1.5).

Регулятор і орган оцінювання відповідності встановлюють обов'язкові вимоги, проводять сертифікацію чи експертизу.

Дослідник безпеки виявляє вразливості й повідомляє про них за процедурами координованого розкриття [206, 207].

Сучасна регуляторна тенденція — зміщення відповідальності від оператора до виробника: ініціатива CISA Secure by Design закликає виробників відповідати за безпекові результати клієнтів [114]. Регламент (ЄС) 2024/2847 про кіберстійкість (CRA) [301] робить це юридичним обов'язком виробника протягом усього періоду підтримки; класи важливих продуктів, до яких віднесено мережеві засоби безпеки, і строки звітування про активно експлуатовані вразливості розглянуто в розділі 2.

1.8 Український контекст

Повномасштабна війна зробила мережеве обладнання і постачальників ПЗ пріоритетною ціллю державних порушників. Показовим є аналіз CERT-UA (CERT-UA#7627): з 11.05.2023 по 27.09.2023 угруповання, що відстежується як UAC-0165 і пов'язується з Sandworm, втрутилося в інформаційно-комунікаційні системи щонайменше 11 українських провайдерів телекомунікацій, що призвело до перебоїв у наданні послуг [31]. Серед інструментів — шкідливий модуль автентифікації PAM, який дає змогу входити за статичним паролем і накопичувати облікові дані адміністраторів, а метою на завершальній стадії було виведення з ладу мережевого й серверного обладнання та систем зберігання даних. Цей випадок поєднує кілька профілів табл. 1.3: державного порушника, модифікацію механізмів автентифікації (T1556) і руйнування інфраструктури, — і показує, що мережеве обладнання є не лише точкою входу, а й об'єктом деструктивного впливу. Координацію реагування на такі інциденти здійснюють CERT-UA [107] і Держспецзв'язку [10] відповідно до законодавства про кібербезпеку [45].

Для вітчизняних розробників засобів КЗІ і ТЗІ війна має подвійний наслідок. З одного боку, зросла потреба в довірених засобах власної розробки — VPN-шлюзах із підтримкою національних криптографічних стандартів [13, 14, 15], IDS для мереж державних органів; українські дослідження розглядають, зокрема, інтеграцію систем виявлення вторгнень у корпора-

тивні мережі з урахуванням моделі загроз [35]. З іншого боку, вітчизняні виробники стають ціллю тих самих порушників, тож вимоги до їхніх процесів розроблення й ланцюга постачання не можуть бути нижчими за міжнародні.

Засоби захисту для державних інформаційних систем проходять державну експертизу в сфері ТЗІ та КЗІ (порядок — [40], критерії — НД ТЗІ [29]). Вона зосереджена на функціональних послугах і рівні гарантій, тоді як вимоги до процесів розроблення, керування вразливостями й прозорості ланцюга постачання, закладені в CRA і IEC 62443-4-1, у національному полі лише формуються (розділ 2).

1.9 Наскрізні приклади посібника

Кейси посібника спираються на три умовні продукти українських виробників з модельними параметрами (табл. 1.4).

Таблиця 1.4 — Паспорт наскрізних прикладів

Продукт	Параметри	Розділи
VPN-шлюз	гілки прошивки 7.2 і 7.4; модуль IKEv2 ≈9 KLOC на C, з них парсер навантажень ≈6 KLOC; ≈1200 експонованих інсталяцій; ≈40 розробників	2, 11, 12, 13, 16, 17
IDS-сенсор	дзеркальний порт, декодери промислових протоколів, REST API, оновлення правил	4, 8, 15
NGFW	прошивка на Linux; інспекція TLS, IPS	10, 14

Висновки до розділу

ПЗМБ є специфічним об’єктом розроблення: ворожий вхід на лінійній швидкості, межа довіри, привілейоване виконання, довгий життєвий цикл; бюджет у кілька сотень тактів на пакет (1.1) робить компроміс між повнотою перевірок і продуктивністю постійним. Декомпозиція на площини даних, керування й менеджменту показує, що найбільший ризик зосереджено не в оптимізованій площині даних, а в об’ємній і часто успадкованій площині менеджменту та порталах автентифікації. Кейси 2014–2025 рр. демонструють парадокс засобу захисту: граничні пристрої стали одним з основних векторів початкового проникнення, а їхні першопричини — безпека пам’яті, нейтралізація вводу, архітектура площини менеджменту — добре відомі. Імовірнісна модель (1.3)–(1.4) показує, що зменшення досяжності вразливостей і перехід на мови з безпечною пам’яттю часто ефективніші за загальне зниження щільності дефектів, а умова (1.5) формалізує розподіл відповідальності між виробником і оператором. Модель порушника для ПЗМБ має охоплювати атаки на сам продукт і процес його створення, включно з ланцюгом постачання. Український досвід атак на

провайдерів і мережеве обладнання підтверджує, що вимоги до процесів розроблення вітчизняних засобів захисту мають відповідати міжнародному рівню, закріпленому, зокрема, в CRA. Наступні розділи відтворюють життєвий цикл ПЗМБ у чотирьох блоках — засади (розділи 2–4), технології розроблення (5–10), технології тестування (11–15), супровід і гарантування (16–17) — на методологічній основі СуВОК [291, 344] і класичних праць [245, 195, 74, 154].

Питання для самоперевірки

1. Дайте означення ПЗМБ. Чим відрізняються функція безпеки, яку надає продукт, і його власна захищеність?
2. Наведіть основні класи ПЗМБ (табл. 1.1). Чому поверхня атаки сучасного NGFW дорівнює об'єднанню поверхонь усіх увімкнених функцій?
3. Опишіть площини даних, керування й менеджменту (рис. 1.1). Яка з них має найбільший обсяг коду і чому саме в ній зосереджено більшість критичних вразливостей граничних пристроїв?
4. Порівняйте вбудоване, віртуалізоване та хмарне виконання ПЗМБ з погляду поверхні атаки й моделі довіри.
5. Перелічіть визначальні властивості ПЗМБ як об'єкта розроблення. Які з них відсутні у звичайного вебзастосунку?
6. Користуючись (1.1), обчисліть бюджет часу й тактів на пакет для інтерфейсу 25 Гбіт/с, пакетів по 64 і по 512 байтів та 16 ядер з частотою 2,5 ГГц.
7. Поясніть механізм вразливості Heartbleed і роль перевірки, показаної в лістингу 1.1. Чому вплив вразливості був глобальним?
8. Які спільні першопричини мають вразливості, наведені в табл. 1.2? Наведіть для кожної групи інженерний захід, що усуває її на рівні класу.
9. Що таке парадокс засобу захисту? Сформулюйте умову (1.5) і поясніть, які параметри контролює виробник, а які — оператор.
10. Чим модель порушника для ПЗМБ відрізняється від моделі для експлуатанта мережі? Наведіть техніки ATT&CK, характерні для атак на граничні пристрої.
11. Для прикладу підрозділу 1.6 обчисліть λ і P_x, якщо площину даних переписано мовою з безпечною пам'яттю (θ зменшено вчетверо), а вебінтерфейс залишився доступним з Інтернету.
12. Як CRA та ініціатива Secure by Design змінюють розподіл відповідальності між виробником і оператором ПЗМБ?

13. Які уроки для розробників ПЗМБ випливають з атак UAC-0165 на українських провайдерів телекомунікацій?
14. Дослідницьке: запропонуйте методику емпіричного оцінювання параметрів θ_k і r_k моделі (1.3) для відкритого VPN-шлюзу з використанням історії виправлень, баз CVE і даних фазингу; оцініть джерела зміщення таких оцінок.
15. Дослідницьке: сформулюйте задачу оптимального розподілу бюджету розроблення між зменшенням щільності дефектів, мінімізацією поверхні атаки й переходом на безпечні мови для мінімізації ризику (1.5) та визначте, які дані потрібні для її розв'язання.

РОЗДІЛ 2

Нормативно-правове та стандартизаційне забезпечення розроблення і тестування програмного забезпечення мережевої безпеки

Вимоги до ПЗМБ надходять з кількох незалежних джерел: законодавства про кібербезпеку й захист інформації, технічних регламентів, державної експертизи, права ЄС, міжнародних стандартів, профілів захисту й добровільних рамок. Одна інженерна практика (наприклад, перелік компонентів ПЗ чи координоване розкриття вразливостей) може водночас задовольняти Регламент (ЄС) 2024/2847 про кіберстійкість (CRA), IEC 62443-4-1 і вимоги державного замовника — або не задовольняти жодної з них через відсутність потрібного доказу. Розділ систематизує нормативне поле України, ЄС і міжнародних організацій зі стандартизації з погляду *розробника* ПЗМБ, порівнює схеми оцінювання відповідності, вводить кількісні показники покриття вимог і розриву відповідності та завершується кейсом виходу українського виробника VPN-шлюзу на ринок ЄС. Предмет розроблення описано в розділі 1, процесні моделі — у розділі 3, організацію робіт під час сертифікації — у розділі 17.

2.1 Структура нормативного поля

2.1.1 Рівні регулювання

Нормативне поле розробника ПЗМБ доцільно подати як ієрархію з семи рівнів (рис. 2.1). *Закони* (і регламенти ЄС прямої дії) визначають суб'єктів, обов'язки й відповідальність; *підзаконні акти* деталізують процедури; *технічні регламенти* встановлюють суттєві вимоги до продукції та процедури оцінювання відповідності. *Стандарти* задають технічний зміст вимог; вони добровільні, але стають обов'язковими через посилання в регламентах чи договорах, а гармонізовані європейські стандарти дають **презумпцію відповідності**. *Профілі захисту* (ISO/IEC 15408) і *профілі безпеки* (у законодавстві України) конкретизують вимоги для класу продуктів чи систем; *галузеві настанови* і *добровільні рамки* описують кращі практики, що часто стають вимогами закупівель.



Рис. 2.1 — Рівні нормативного забезпечення розроблення ПЗМБ (деталізація вимог зростає згори донизу)

2.1.2 Обов'язкові й добровільні вимоги; продукт і процес

Друга вісь класифікації — юридична сила. Обов'язкові вимоги випливають із закону або технічного регламенту й підтверджуються процедурою оцінювання відповідності; добровільні стають обов'язковими через договір (технічне завдання державного замовника), умову закупівлі або презумпцію відповідності. Третя вісь — *об'єкт* вимоги. Вимоги до **продукту** описують властивості, які можна перевірити на зразку (налаштування за замовчуванням, автентифікацію, криптографію); вимоги до **процесу** — діяльність виробника (моделювання загроз, тестування, обробку вразливостей, випуск оновлень). Традиційні національні процедури (НД ТЗІ 2.5-004 [29], державна експертиза) зосереджені переважно на продукті; CRA [301], IEC 62443-4-1 [197], NIST SSDF [328] дедалі більше регулюють процес і тривалість підтримки. Тому доказом відповідності стає не лише протокол випробувань зразка, а й артефакти конвеєра: модель загроз, звіти аналізаторів, SBOM, журнал обробки вразливостей.

Четверта вісь — *суб'єкт зобов'язань*: виробник продукту, власник системи, у якій продукт експлуатується, і орган оцінювання. CRA зобов'язує виробника постачати оновлення безпеки, NIS2 [151] — оператора встановлювати їх і оцінювати безпеку ланцюга постачання.

2.2 Законодавство України

2.2.1 Кібербезпека та захист інформації

Закон № 4336-IX від 27.03.2025 [38] суттєво оновив Закон «Про основні засади забезпечення кібербезпеки України» [45] і Закон «Про захист ін-

формації в інформаційно-комунікаційних системах» [42]: запроваджено національну систему реагування на кіберінциденти, закріплено ризик-орієнтований підхід і гармонізацію з практиками ЄС і НАТО. Ключова для розробника новація — перехід від створення комплексної системи захисту інформації (КСЗІ) з її державною експертизою до **авторизації з безпеки** на основі **профілів безпеки**: *базового* (мінімальні вимоги щодо заходів захисту, встановлювані Кабінетом Міністрів), *галузевого* (затверджується органом влади за погодженням із Держспецзв'язку) і *цільового* (для конкретної системи з урахуванням аналізу ризиків). Авторизація ґрунтується на повідомному (декларативному) принципі; раніше створені КСЗІ продовжують застосовуватися. Порядок авторизації з безпеки і порядок розроблення та затвердження профілів безпеки затверджено постановою КМУ від 18.06.2025 № 712 [11], а мінімальні вимоги (базовий профіль) оновлено змінами до постанови № 373 [37], внесеними постановою від 26.11.2025 № 1531 [39].

Для виробника ПЗМБ важливі ще два положення оновленого закону. По-перше, Держспецзв'язку отримала повноваження встановлювати вимоги до *постачальників* товарів, робіт і послуг для державних систем і об'єктів критичної інформаційної інфраструктури відповідно до рівня ризику: визначати критерії критичності продукції, порядок оцінювання ризику постачання і порядок підтвердження постачальником упроваджених заходів безпеки [38]. Це національний аналог вимог NIS2 щодо безпеки ланцюга постачання. По-друге, цільовий профіль безпеки системи замовника, що містить VPN-шлюз чи МЕ, транслюється у вимоги до функцій і налаштувань цього продукту, тобто стає джерелом вимог для розробника (розділ 4).

Закон «Про критичну інфраструктуру» [44] і Загальні вимоги до кібербезпеки об'єктів КІ [18] зобов'язують оператора, зокрема, керувати вразливостями й безпекою ланцюга постачання. Постанова КМУ від 03.12.2025 № 1580 [12] затвердила Порядок пошуку та виявлення потенційних вразливостей у державних ІКС і на об'єктах критичної інформаційної інфраструктури: поряд із діяльністю CERT-UA [107], галузевих CSIRT і скануванням державних ресурсів передбачено програми пошуку вразливостей за винагороду та узгодженого розкриття із залученням зовнішніх дослідників. Дослідник має повідомити власника системи і CERT-UA (або галузеву CSIRT) про вразливість не пізніше 24 год, власник — ухвалити рішення щодо виправлення протягом 30 робочих днів; виробник ПЗМБ фактично стає учасником процесу (розділ 16).

2.2.2 Технічне регулювання і державна експертиза

Закон «Про технічні регламенти та оцінку відповідності» [46] установлює загальну модель: технічний регламент визначає суттєві вимоги і модулі

оцінювання відповідності, а відповідність стандартам з опублікованого переліку дає презумпцію відповідності. Для засобів КЗІ діє окремий Технічний регламент [41].

Державна експертиза у сфері ТЗІ та КЗІ є головною національною процедурою оцінювання ПЗМБ, призначеного для державних систем. Експертизу засобів ТЗІ проводять згідно з Положенням, затвердженим наказом № 93 [40], до якого наказом від 17.04.2026 № 287 внесено зміни, що узгоджують його з Законом № 4336-IX і постановою № 712. Положення про державну експертизу в сфері КЗІ [34] викладено в новій редакції наказом від 24.04.2026 № 302. Воно встановлює строки дії висновків: до 5 років для серійних і програмних засобів (з фіксацією геш-значення), алгоритмів і протоколів, до 3 років для одиничного засобу чи партії. Принципова новація обох актів — *визнання міжнародного оцінювання*: для засобів, що мають чинний сертифікат за ISO/IEC 15408 (для КЗІ — з рівнем гарантії не нижче EAL4) або внесені до каталогу NIAPC НАТО, експертиза може проводитися шляхом зіставлення результатів наявної сертифікації з національними вимогами, а строк дії висновку не перевищує строку дії іноземного сертифіката [34]. Сертифікація за CC стає корисною і на внутрішньому ринку.

Критерії оцінювання захищеності інформації від несанкціонованого доступу встановлюють НД ТЗІ 1.1-002-99 [28] і 2.5-004-99 [29]: функціональний профіль формується з послуг конфіденційності, цілісності, доступності й спостережності, гарантії — з рівнів Г1–Г7; вимог до обробки вразливостей після випуску ці документи не містять.

2.2.3 Національні криптографічні стандарти

VPN-шлюзи й засоби КЗІ для державних систем мають реалізовувати національні алгоритми: цифровий підпис на еліптичних кривих ДСТУ 4145-2002 [13], блоковий шифр «Калина» ДСТУ 7624:2014 [14], функцію гешування «Купина» ДСТУ 7564:2014 [15] і потоковий шифр «Струмок» ДСТУ 8845:2019 [16]. Продукт для обох ринків має підтримувати два криптографічні профілі — національний і міжнародний (AES, SHA-2, ECDSA, у перспективі ML-KEM і ML-DSA [266, 265]) — з узгодженими ідентифікаторами алгоритмів в IKEv2 і TLS (розділ 6).

2.3 Право Європейського Союзу

2.3.1 CRA: сфера дії і класифікація продуктів

CRA [301] уперше встановлює горизонтальні обов'язкові вимоги кібербезпеки до всіх **продуктів із цифровими елементами**, що надаються на ринку ЄС: апаратних і програмних продуктів та рішень віддаленої обробки даних, без яких продукт не може виконувати свої функції. З-під дії виключено продукти, урегульовані секторальним правом, продукти для національ-

ної безпеки й оборони і відкрите ПЗ поза комерційною діяльністю; для «розпорядників відкритого ПЗ» (open-source stewards) установлено полегшений режим, суттєво змінений порівняно з проєктом [111].

Продукти поділено на три групи. *Звичайні* (за замовчуванням) оцінюються самим виробником. **Важливі продукти** (Додаток III) поділено на клас I і клас II, **критичні продукти** перелічено в Додатку IV. Технічний опис кожної категорії містить Імплементаційний регламент (ЄС) 2025/2392 [133]; вирішальним критерієм віднесення є *основна функціональність* продукту, а не наявність окремої функції. Майже всі класи ПЗМБ потрапляють до важливих продуктів (табл. 2.1).

Таблиця 2.1 — Класифікація ПЗМБ за Додатками III–IV CRA (за [301, 133])

Клас	Категорії, релевантні для ПЗМБ	Мінімальна процедура
Клас I	продукти з функцією VPN (клієнти, сервери, шлюзи); системи керування мережею; SIEM; маршрутизатори, модеми, комутатори; фізичні й віртуальні мережеві інтерфейси; системи керування ідентифікацією й доступом; ПЗ PKI; менеджери завантаження; ОС	модуль A лише за повного застосування гармонізованих стандартів або сертифікації; інакше B+C або H
Клас II	ME; системи виявлення та запобігання вторгненням; гіпервізори й середовища виконання контейнерів; захищені від втручання мікропроцесори й мікроконтролери	B+C, H або EUCC рівня «суттєвий»
Критичні	апаратні пристрої з «захищеними блоками» (security boxes), шлюзи інтелектуальних лічильників, смарт-картки й захищені елементи	європейська сертифікація (якщо вимагається), інакше як для класу II

Отже, VPN-шлюз належить до класу I, а NGFW, IDS/IPS — до класу II; комбінований пристрій класифікують за основною функціональністю, тож класифікація є першим інженерно-юридичним рішенням проєкту (підрозділ 2.8).

2.3.2 Суттєві вимоги CRA

Додаток I CRA містить дві частини. *Частина I* — вимоги до властивостей продукту: продукт проєктується з рівнем кібербезпеки, відповідним ризикам, і на основі оцінки ризиків має (a) надаватися без відомих експлуатованих вразливостей; (b) мати безпечну конфігурацію за замовчуванням із можливістю повернення до початкового стану; (c) забезпечувати усунення вразливостей оновленнями, зокрема автоматичними за замовчуванням із можливістю відмови; (d) захищати від несанкціонованого доступу механізмами автентифікації та керування доступом; (e) захищати конфіденційність даних, зокрема шифруванням за сучасним рівнем техніки; (f) захища-

ти цілісність даних, команд, програм і конфігурації; (g) мінімізувати обсяг оброблюваних даних; (h) зберігати доступність основних функцій, зокрема стійкість до атак на відмову в обслуговуванні; (i) мінімізувати негативний вплив на інші пристрої й мережі; (j) обмежувати поверхню атаки, зокрема зовнішні інтерфейси; (k) зменшувати наслідки інцидентів механізмами протидії експлуатації; (l) реєструвати й відстежувати внутрішню активність, що стосується безпеки; (m) дозволяти безпечно й повністю видаляти дані й налаштування.

Частина II — вимоги до процесу обробки вразливостей: (1) ідентифікувати й документувати компоненти, зокрема складати машинозчитуваний SBOM щонайменше для залежностей верхнього рівня; (2) усувати вразливості без зволікання, по можливості окремими оновленнями безпеки; (3) регулярно тестувати безпеку продукту; (4) після випуску оновлення публічно розкривати інформацію про усунені вразливості; (5) застосовувати політику координованого розкриття; (6) сприяти обміну інформацією про вразливості, зокрема надати контактну адресу; (7) захищено розповсюджувати оновлення; (8) поширювати оновлення безпеки без зволікання і безоплатно. Для ПЗМБ вимоги частини I мають пряму інженерну проєкцію: пункт (j) — відокремлення площини менеджменту від Інтернету, пункт (k) — ізоляція розбору й захисні механізми компілятора (розділи 4, 5), пункт (h) — тестування стійкості (розділ 14); частина II реалізується процесами SBOM і PSIRT (розділи 10, 16). Виробник також визначає *період підтримки* (як правило, не менше п'яти років), веде технічну документацію (Додаток VII), складає декларацію ЄС і наносить маркування CE.

2.3.3 Звітування і строки застосування

Стаття 14 CRA зобов'язує виробника повідомляти через єдину платформу звітування ENISA координатора CSIRT про *активно експлуатовані вразливості* та *серйозні інциденти*, що впливають на безпеку продукту. Нехай t_0 — момент, коли виробнику стало відомо про активно експлуатовану вразливість, t_1, t_2, t_3 — моменти подання раннього попередження, повідомлення про вразливість і остаточного звіту, t_f — момент, коли стали доступні коригувальні або пом'якшувальні заходи. Тоді

$$t_1 - t_0 \le 24 \text{ год}, \qquad t_2 - t_0 \le 72 \text{ год}, \qquad t_3 - t_f \le 14 \text{ діб}. \tag{2.1}$$

Для серйозного інциденту остаточний звіт подається протягом місяця. Обмеження (2.1) задають цільові показники PSIRT. *Числовий приклад.* Повідомлення CERT-UA про експлуатацію вразливості порталу VPN надійшло в п'ятницю о 18:00. Якщо звернення реєструють із затримкою до 12 год, а перевірка експлуатації триває до 8 год, на погодження раннього попередження залишається $24 - 12 - 8 = 4$ год; без цілодобового чергування PSIRT

перший строк систематично порушуватиметься. Якщо виправлення випущено на 9-ту добу після t_0, остаточний звіт має бути подано не пізніше 23-ї доби. Обов'язки звітування застосовуються з 11.09.2026 до *всіх* продуктів на ринку, зокрема випущених до повного застосування регламенту.

Хронологію застосування CRA подано на рис. 2.2.

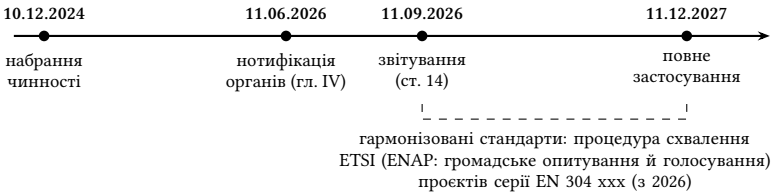


Рис. 2.2 — Хронологія застосування CRA (за [301, 168])

2.3.4 Процедури оцінювання відповідності та гармонізовані стандарти

Стаття 32 CRA пропонує чотири процедури. *Модуль A* — внутрішній контроль виробництва (самооцінювання). *Модуль B+C* — експертиза типу ЄС нотифікованим органом, який оцінює технічну документацію, зразок і процеси обробки вразливостей, плюс відповідність типу на основі внутрішнього контролю виробництва. *Модуль H* — оцінювання системи повного забезпечення якості виробника нотифікованим органом. *Європейська сертифікація кібербезпеки* — зокрема схема EUCC з рівнем довіри не нижче «суттєвого». Для продуктів класу I модуль A допустимий, лише якщо виробник повністю застосував гармонізовані стандарти, загальні специфікації або європейську сертифікацію; для класу II самооцінювання не допускається.

Тут виникає практична проблема: станом на жовтень 2026 р. жоден гармонізований стандарт на підтримку CRA ще не опубліковано в Офіційному віснику ЄС. 13.08.2026 розпочато процедуру схвалення ETSI (ENAP: громадське опитування й голосування) 17 проєктів вертикальних (продуктових) стандартів серії EN 304 xxx, серед яких стандарти для VPN, ME з функціями IDS/IPS, маршрутизаторів і комутаторів, систем керування мережею; опитування триває до вересня–листопада 2026 р. залежно від категорії, а метою є можливість презумпції відповідності до кінця 2027 р. [168]. Горизонтальні стандарти розробляють CEN-CENELEC. Отже, виробник VPN-шлюзу не може без ризику розраховувати на модуль A.

2.3.5 NIS2, Регламент про кібербезпеку і EUCC, RED

Директива NIS2 [151] регулює не виробника, а суттєві й важливі суб'єкти — споживачів ПЗМБ. Проте стаття 21 зобов'язує їх дбати про безпеку ланцюга постачання (п. 2(d)) і обробку вразливостей (п. 2(e)), ураховуючи пра-

ктики безпечного розроблення постачальників (п. 3); для постачальників цифрової інфраструктури їх деталізовано Імплементаційним регламентом (ЄС) 2024/2690 [131]. Стаття 24 дозволяє вимагати використання сертифікованих продуктів. Отже, NIS2 перетворює безпечне розроблення на *ринкову* вимогу: клієнти в ЄС запитуватимуть його докази незалежно від CRA.

Регламент (ЄС) 2019/881 про кібербезпеку [300] заснував європейську рамку сертифікації з рівнями довіри «базовий», «суттєвий», «високий». Перша схема — EUCC [132], застосовна з 27.02.2025, — ґрунтується на ISO/IEC 15408 і 18045: рівню «суттєвий» відповідають AVA_VAN.1–2, рівню «високий» — AVA_VAN.3–5; сертифікат видається на строк до п'яти років, а його власник зобов'язаний обробляти вразливості й повідомляти про них орган сертифікації. EUCC поступово замінює національні схеми держав-членів, що діяли за угодою SOG-IS.

Делегований регламент (ЄС) 2022/30 до Директиви про радіообладнання (RED) [129] застосовується з 01.08.2025 до радіообладнання, під'єднаного до Інтернету. Гармонізовані стандарти EN 18031-1/2/3 [163] внесено до переліку Рішенням (ЄС) 2025/138 з обмеженнями [130]. Для ПЗМБ це стосується пристроїв із вбудованим Wi-Fi чи LTE-модулем (SOHO-маршрутизатори з ME, мобільні VPN-шлюзи).

2.4 Міжнародні стандарти

2.4.1 Загальні критерії та колективні профілі захисту

ISO/IEC 15408:2022 (CC:2022) [201] і методологія оцінювання ISO/IEC 18045:2022 [203] залишаються основою оцінювання ПЗМБ для державних замовників. Вимоги поділено на функціональні (SFR) і вимоги гарантій (SAR). Для розробника найважливіші три класи гарантій: *ALC* (підтримка життєвого циклу: керування конфігурацією, постачання, безпека середовища розроблення ALC_DVS, усунення недоліків ALC_FLR), *ATE* (тестування) і *AVA_VAN* (аналіз вразливостей і тестування на проникнення оцінювачем). Оціночні рівні довіри EAL1–EAL7 є наперед визначеними пакетами SAR; угода CCRA [78] забезпечує взаємне визнання сертифікатів до EAL2 і за колективними профілями захисту.

Колективні профілі захисту (cPP) розробляють міжнародні технічні спільноти (iTC). Для мережевих пристроїв базовим є NDcPP: версія 4.0 від 25.11.2025 ґрунтується на CC:2022 і вимагає точної відповідності (exact conformance) [128]; у 2025–2026 рр. широко застосовується також конфігурація NDcPP v3.0e з PP-модулями для ME з фільтрацією зі станом (v1.4e), VPN-шлюзів (v1.3) та IPS (v1.0) [293]. Особливість cPP — замість EAL використовується мінімальний набір SAR (ADV_FSP.1, ALC_CMC.1, ALC_CMS.1, ATE_IND.1, AVA_VAN.1, за бажання ALC_FLR), а глибину перевірок задають *оціночні дії* допоміжного документа: конкретні тести протоколів TLS, SSH,

IPsec, захищеного оновлення, самотестування криптографії. Отже, cPP — детальне джерело вимог до продукту (розділ 4), але не до процесу.

2.4.2 Стандарти процесу, тестування і криптографічних модулів

ISO/IEC 27034 [205] вводить організаційну нормативну рамку застосунків; IEC 62443-4-1 [197] визначає вісім практик безпечного життєвого циклу продукту і чотири рівні зрілості, IEC 62443-4-2 [198] — технічні вимоги до компонентів із рівнями безпеки SL 1–4. ISO/IEC/IEEE 29119 [210] задає процеси, документацію й техніки тестування (розділ 11), ISO/IEC 29147 [206] і 30111 [207] — розкриття й обробку вразливостей, на які прямо спирається частина II Додатка I CRA (розділ 16). FIPS 140-3 [268], що посилається на ISO/IEC 19790 [204], встановлює чотири рівні безпеки криптографічних модулів; валідацію CMVP/CAVP криптографічних ядер VPN-шлюзів для федеральних органів США, як правило, вимагають поряд із сертифікацією за cPP. Процесний зміст цих документів розглянуто в розділі 3.

2.5 Рамкові документи і настанови

Добровільні рамки — джерело формулювань вимог і мова договорів: NIST SSDF [328], ініціатива CISA Secure by Design [114] з переліком небезпечних практик [117], OWASP ASVS 5.0 [277] для вебінтерфейсів і API площини менеджменту; їхній процесний зміст розглянуто в розділі 3.

Документи перекриваються, тож постає задача вибору мінімального набору стандартів, що покриває застосовні вимоги. Нехай $\mathcal{R} = \{r_1, \dots, r_N\}$ — вимоги з вагами w_i, $\mathcal{S}$ — множина стандартів, $m_{is} \in [0;1]$ — ступінь, з яким стандарт s покриває вимогу r_i (1 — повністю, 0,5 — частково). **Покриття вимог набором стандартів** $S \subseteq \mathcal{S}$:

$$C(S) = \frac{\sum_{i=1}^{N} w_i \max_{s \in S} m_{is}}{\sum_{i=1}^{N} w_i}. \tag{2.2}$$

Числовий приклад. Нехай $\mathcal{R}$ — 21 суттєва вимога Додатка I CRA (13 вимог частини I і 8 частини II, $w_i = 1$). NDcPP покриває повністю 9 і частково 2 вимоги частини I та 1 повністю й 1 частково — частини II: $C = (10 + 1{,}5)/21 \approx 0{,}55$. IEC 62443-4-1 покриває 3 повністю і 4 частково вимоги частини I та 6 повністю і 2 частково вимоги частини II: $C = (5 + 7)/21 \approx 0{,}57$. Якщо об'єднання дає 11 «зважених» вимог частини I і 7,5 частини II, то $C(\{\text{NDcPP}, \text{62443-4-1}\}) = 18{,}5/21 \approx 0{,}88$. Стандарти продукту й процесу комплементарні. Функція $C(S)$ монотонна й субмодулярна, тому жадібний вибір стандартів за приростом покриття за обмеження на їхню кількість гарантує не менше ніж $(1 - 1/e) \approx 0{,}63$ від оптимального покриття. Показник (2.2), як і моделі (2.3)–(2.5) нижче, — модель-евристика з ілюстративними параметрами.

2.6 Оцінювання відповідності та сертифікація

Схеми оцінювання відрізняються об’єктом, суб’єктом, тривалістю й доказами, які готує розробник (табл. 2.2; тривалість — орієнтовна, за типовою практикою; організацію робіт розглянуто в розділі 17).

Таблиця 2.2 — Порівняння схем оцінювання відповідності ПЗМБ

Схема	Об’єкт оцінювання	Хто оцінює	Строк, міс.	Артефакти розробника
Експертиза ТЗІ/КЗІ	засіб, алгоритм, протокол на відповідність ТЗ і НД ТЗІ	експертна організація, Держспецзв’язку	3–12	ТЗ, програма й методика, вихідні тексти (для КЗІ), зразки
CC / cPP	продукт (ST, SFR, SAR)	лабораторія, орган сертифікації	6–12	завдання безпеки, настанови, тестові свідчення
EUCC	продукт за CC, рівні «суттєвий»/«високий»	ITSEF, орган сертифікації	6–18	як для CC, плюс процес обробки вразливостей
FIPS 140-3	криптографічний модуль	лабораторія, CMVP	12–24	політика безпеки і межа модуля, результати CAVP
CRA, модуль A	продукт і процеси виробника	сам виробник	—	технічна документація, оцінка ризиків, SBOM, декларація ЄС
CRA, B+C або H	тип продукту / система якості	нотифікований орган	3–9	те саме, плюс зразки й аудит процесів
IEC 62443-4-1	процес розроблення	орган сертифікації	3–6	описи процесів, записи виконання практик

Артефакти схем значною мірою спільні: модель загроз, звіти тестування, SBOM і журнал обробки вразливостей використовують повторно, що знижує вартість відповідності (підрозділ 2.7).

2.7 Порівняльний аналіз вимог України та ЄС

2.7.1 Відповідність підходів

Порівняння ключових аспектів наведено в табл. 2.3. Після Закону № 4336-IX національне законодавство рухається в напрямку ЄС (ризик-орієнтованість, профілі, вимоги до постачальників), проте механізми різняться: в Україні центр ваги — експертиза засобів для державних систем, у ЄС — відповідальність виробника за весь період підтримки на всьому ринку.

Таблиця 2.3 — Порівняння вимог до ПЗМБ в Україні та ЄС

Аспект	Україна	ЄС
Сфера дії	засоби для державних ІКС та об'єктів КІ	усі продукти з цифровими елементами на ринку
Суб'єкт	власник системи; розробник — через експертизу й вимоги до постачальників	виробник, імпортер, дистриб'ютор
Вимоги до продукту	НД ТЗІ, ТЗ, профілі безпеки, ДСТУ-криптографія	Додаток I ч. I CRA, гармонізовані стандарти
Вимоги до процесу	фрагментарні (вимоги до постачальників за ризиком)	Додаток I ч. II CRA, NIS2 ст. 21
Обробка вразливостей	ПКМУ № 1580: пошук, CERT-UA, 24 год для дослідника	звітування виробника 24 год / 72 год / 14 діб
Оцінювання	державна експертиза ТЗІ/КЗІ, визнання CC EAL4+ і NIAPC	модулі A, B+C, H, EUCC; маркування CE
Строк підтримки	строк дії висновку (до 5 років)	період підтримки, як правило не менше 5 років

2.7.2 Індекс розриву відповідності

Для кількісного аналізу стану продукту щодо множини вимог використаємо зважений індекс розриву. Нехай $c_i \in [0;1]$ — ступінь виконання вимоги r_i (0 — не виконано, 0,5 — частково, 1 — повністю), w_i — вага, що відображає юридичну силу й критичність (наприклад, 3 — обов'язкова суттєва вимога, 2 — обов'язкова документальна, 1 — добровільна). Тоді

$$G = 1 - \frac{\sum_{i=1}^{N} w_i\, c_i}{\sum_{i=1}^{N} w_i}, \qquad G \in [0;1], \tag{2.3}$$

з додатковою блокувальною умовою для множини обов'язкових вимог $\mathcal{R}_{\text{обов}}$

$$\min_{i:\, r_i \in \mathcal{R}_{\text{обов}}} c_i = 1. \tag{2.4}$$

Індекс (2.3) компенсаторний, тому лише разом з умовою (2.4) він придатний для рішення «допускати продукт на ринок чи ні»; сам по собі він слугує для пріоритизації. Ваги доцільно визначати експертно методом аналізу ієрархій, як це запропоновано для оцінювання рівня кіберзахисту об'єктів критичної інформаційної інфраструктури [60]. Числовий приклад наведено в кейсі (підрозділ 2.8).

2.7.3 Модель витрат на відповідність

Нехай k_i — вартість реалізації вимоги r_i «з нуля» (тоді залишкова вартість дорівнює $k_i(1 - c_i)$), K_s — вартість проходження схеми оцінювання s з набору S, $\rho_s \in [0;1)$ — частка вартості, що заощаджується повторним вико-

ристанням доказів з інших схем. Тоді витрати на відповідність

$$K(S) = \sum_{i=1}^{N} k_i\,(1 - c_i) + \sum_{s \in S} K_s\,(1 - \rho_s), \tag{2.5}$$

Повне виконання вимоги r_i зменшує G на $w_i(1 - c_i)/\sum_j w_j$, тому зменшення індексу на одиницю витрат $\pi_i = w_i/\bigl(k_i \sum_j w_j\bigr)$ не залежить від c_i і є природним критерієм пріоритету серед добровільних вимог; обов'язкові вимоги, що порушують умову (2.4), усуваються першочергово незалежно від π_i. *Числовий приклад.* Нехай доопрацювання продукту коштує $\sum k_i(1 - c_i) = 14$ люд.-міс., експертиза КЗІ — 4 люд.-міс. (вже пройдена, у розрахунок не входить), оцінювання за модулем B+C — 6 люд.-міс. внутрішніх витрат, оцінювання за NDcPP — 8 люд.-міс. Без повторного використання $K = 14 + 6 + 8 = 28$ люд.-міс.; якщо спільний комплект доказів (модель загроз, звіти тестів, SBOM) дає $\rho = 0{,}3$ для обох схем, то $K = 14 + 0{,}7\,(6 + 8) = 23{,}8$ люд.-міс., тобто економія становить 15 %.

2.8 Кейс: вихід українського VPN-шлюзу на ринок ЄС

Український виробник постачає державним органам VPN-шлюз із підтримкою IKEv2/IPsec і національних алгоритмів, що має чинний експертний висновок у сфері КЗІ, і планує продажі в ЄС з 2027 р. Визначення застосовних вимог виконується в п'ять кроків.

1. *Класифікація.* Основна функція — зашифрований тунель, тож продукт належить до класу I CRA [133]. Якщо ж його позиціонують як NGFW із VPN, основною стає функція ME і продукт переходить до класу II без права самооцінювання; тому призначення фіксується в технічній документації й узгоджується з маркетинговими матеріалами.
2. *Вибір процедури.* Гармонізованого стандарту для VPN на момент планування немає [168], тож базовий сценарій — модуль B+C; модуль A стане можливим, якщо гармонізований стандарт для VPN буде опубліковано в OJEU до виходу на ринок.
3. *Реєстр вимог*: CRA (Додатки I, II, VII), NDcPP із модулем VPN-шлюзу (продукт, майбутня EUCC), IEC 62443-4-1 і SSDF (процес), національні вимоги (експертиза КЗІ, ДСТУ-криптографія, вимоги до постачальників).
4. *Оцінювання розриву* за (2.3) і (2.4).
5. *План робіт* з пріоритизацією за π_i і розрахунком витрат за (2.5).

Результати оцінювання розриву для ринку ЄС подано в табл. 2.4.

Таблиця 2.4 — Реєстр вимог VPN-шлюзу для ринку ЄС і стан їх виконання

Група вимог	N	w	Виконано: повн./част./ні	$\sum wc$
CRA, Додаток I ч. I (властивості)	13	3	8 / 4 / 1	30
CRA, Додаток I ч. II (вразливості)	8	3	3 / 3 / 2	13,5
CRA, документація, декларація, CE	6	2	1 / 2 / 3	4
Добровільні (NDcPP, 62443-4-1, SSDF)	10	1	6 / 2 / 2	7
Разом	37	—	сума ваг 85	54,5

За (2.3) $G = 1 - 54{,}5/85 \approx 0{,}36$, а умова (2.4) порушена для 15 обов'язкових вимог. Картина типова для вітчизняних виробників: криптографія, автентифікація й цілісність (пункти (d)–(f) частини I) виконані, а розриви зосереджено в процесі: немає SBOM, публічної політики координованого розкриття, автоматичних оновлень; не відокремлено інтерфейс менеджменту від зовнішніх інтерфейсів (пункт (j)); не задокументовано оцінку ризиків і період підтримки. Для порівняння, щодо національного реєстру (14 вимог) розрив становить лише $G_{\text{UA}} \approx 0{,}05$.

План робіт містить три етапи: (1) процесні вимоги частини II — генерація SBOM у CycloneDX у конвеєрі CI/CD, політика CVD відповідно до ISO/IEC 29147, процедура звітування з цілодобовим чергуванням PSIRT для виконання (2.1); (2) вимоги частини I — виділений інтерфейс менеджменту, автоматичне оновлення з підписом і захистом від відкату, протидія експлуатації; (3) технічна документація й оцінювання нотифікованим органом. Після першого й другого етапів $\sum w_i c_i = 39 + 24 + 4 + 7 = 74$, $G = 1 - 74/85 \approx 0{,}13$, проте умова (2.4) ще порушена для документальних вимог; її задоволено лише після етапу 3, коли $\sum w_i c_i = 82$ і $G \approx 0{,}035$, а розрив, що залишився, стосується лише добровільних вимог і визначає дорожню карту до сертифікації за NDcPP чи EUCC. Жоден із цих заходів не суперечить національним вимогам.

Висновки до розділу

Нормативне поле розроблення ПЗМБ багатовимірне: вимоги розрізняються юридичною силою, об'єктом (продукт чи процес) і суб'єктом зобов'язань. В Україні Закон № 4336-IX і постанова КМУ № 712 замінили модель КСЗІ авторизацією з безпеки за профілями й запровадили ризик-орієнтовані вимоги до постачальників; державна експертиза ТЗІ і КЗІ в редакціях 2026 р. визнає сертифікацію за ISO/IEC 15408 і каталог НАТО, а постанова № 1580 упорядкувала пошук і узгоджене розкриття вразливостей. CRA відносить VPN до класу I, ME та IDS/IPS — до класу II і встановлює

вимоги до властивостей продукту й обробки вразливостей; без опублікованих гармонізованих стандартів для більшості ПЗМБ реалістичним є залучення нотифікованого органу. Міжнародні стандарти комплементарні: cPP описують продукт, IEC 62443-4-1, ISO/IEC 29147/30111 і SSDF — процес. Показники (2.2)–(2.5) дають змогу вибирати набір стандартів, пріоритизувати доопрацювання й оцінювати економію від повторного використання доказів. Кейс VPN-шлюзу показує, що основний розрив вітчизняних виробників щодо ринку ЄС зосереджений не в криптографії, а в процесах SBOM, координованого розкриття й звітування.

Питання для самоперевірки

1. Назвіть рівні нормативного забезпечення розроблення ПЗМБ (рис. 2.1). Яким чином добровільний стандарт стає обов'язковим?
2. Чим відрізняються вимоги до продукту і до процесу розроблення? Наведіть по три приклади для VPN-шлюзу.
3. Які зміни в систему захисту інформації в Україні вніс Закон № 4336-IX? Що таке базовий, галузевий і цільовий профілі безпеки?
4. Які новації містять редакції 2026 р. положень про державну експертизу у сферах ТЗІ та КЗІ? Як вони впливають на стратегію сертифікації розробника?
5. Які обов'язки дослідника, власника системи й CERT-UA встановлює постанова КМУ № 1580? Яку роль у цьому процесі відіграє виробник ПЗМБ?
6. До яких класів CRA належать VPN-шлюз, NGFW, IDS/IPS і система керування мережею (табл. 2.1)? Яку роль відіграє Імплементаційний регламент (ЄС) 2025/2392?
7. Перелічіть суттєві вимоги частини II Додатка I CRA і вкажіть, які процеси розробника їх реалізують.
8. Користуючись (2.1), визначте максимальну тривалість первинної перевірки повідомлення про експлуатацію, якщо реєстрація звернення займає до 6 год, а погодження раннього попередження — 3 год.
9. Чому відсутність гармонізованих стандартів CRA впливає на вибір процедури оцінювання для продуктів класу I? Які процедури доступні для класу II?
10. Як NIS2 впливає на виробника ПЗМБ, хоча регулює операторів? Що змінює для нього схема EUCC?
11. Порівняйте за табл. 2.2 державну експертизу КЗІ, оцінювання за cPP і CRA-модуль B+C за об'єктом, суб'єктом і артефактами розробника.

12. Обчисліть покриття (2.2) для прикладу підрозділу 2.5, якщо додати SSDF, що повністю покриває ще дві вимоги частини II і частково — одну вимогу частини I.
13. Для кейсу підрозділу 2.8 обчисліть G і перевірте умову (2.4), якщо виконано лише перший етап плану (усі вимоги частини II). Які вимоги блокують допуск на ринок і на якому етапі їх буде знято?
14. Дослідницьке: запропонуйте некомпенсаторну модель агрегування показників відповідності, яка враховує залежності між вимогами (наприклад, звітування неможливе без процесу обробки вразливостей), і порівняйте її з (2.3).
15. Дослідницьке: розробіть методику автоматизованого відображення вимог Додатка I CRA на вимоги NDcPP, IEC 62443-4-1 і НД ТЗІ (наприклад, з використанням онтологій або мовних моделей) та критерії оцінювання якості такого відображення.

РОЗДІЛ 3

Процесні моделі та метрики безпечного життєвого циклу розроблення

Нормативні документи, розглянуті в розділі 2, вимагають від виробника ПЗМБ не лише безпечного продукту, а й *відтворюваного процесу*, що його створює, і доказів виконання цього процесу. Розділ розглядає вбудовування безпеки в моделі життєвого циклу, процесні рамки (Microsoft SDL, NIST SSDF із змінами проєкту версії 1.2, ISO/IEC 27034, IEC 62443-4-1), моделі зрілості OWASP SAMM і BSIMM та практику DevSecOps. Друга половина присвячена кількісному апарату (економіка дефектів, моделі надійності й виявлення вразливостей, метрики процесу) і кейсу оцінювання зрілості за SAMM. Модель «дефект → вразливість → експлуатація» наведено в розділі 1, конвеєр CI/CD — у розділі 10, управління вразливостями — у розділі 16.

3.1 Моделі життєвого циклу і точки вбудовування безпеки

3.1.1 Класичні та гнучкі моделі

Модель життєвого циклу визначає етапи створення ПЗ, їхню послідовність і правила переходу. Для безпеки важливі *точки прийняття рішень*, де можна зупинити небезпечний випуск, *тривалість зворотного зв'язку* від внесення дефекту до виявлення і *артефакти*, що стають доказами для оцінювання відповідності.

Каскадна модель дає явні рубежі (огляд вимог, огляд проєкту, остаточний огляд безпеки) і повну документацію, але довгий зворотний зв'язок: архітектурну ваду може бути виявлено лише під час системного тестування через місяці. *V-модель* ставить кожному рівню деталізації у відповідність рівень верифікації (вимоги — приймальне тестування, архітектура — системне, код — модульне) і відображає зв'язок вимог безпеки з тестами (розділ 11), тому поширена в розробленні сертифікованих засобів. *Гнучкі моделі* скорочують цикл до тижнів і переносять безпекові дії у визначення готовності, ризикуючи витісненням з беклогу оновлення моделі загроз. У *DevOps* безпека реалізується переважно як автоматизовані ворота якості (підрозділ 3.6). Процесні рамки від моделі не залежать [328, 249]; узагальнену схему етапів і безпекових дій наведено на рис. 3.1.



Рис. 3.1 — Життєвий цикл ПЗМБ і безпекові дії на етапах (узагальнено за [328, 195, 245])

3.1.2 «Зсув уліво» та його межі

Зсув уліво (shift left) — перенесення безпекових дій на ранні етапи, щоб виявляти дефекти ближче до моменту їх внесення; обґрунтування — *економічне* (вада ще не обросла залежними артефактами) і *когнітивне* (автор щойно написаного коду виправляє його швидше). Межі принципу: кількісні оцінки виграшу спираються на дані сумнівної достовірності (підрозділ 3.7); помилки кінцевих автоматів протоколів і гонитви під навантаженням проявляються лише під час фазингу, навантаження чи в експлуатації (розділи 13, 14); неналаштовані аналізатори породжують «втому від сповіщень». Тому сучасне формулювання — «зсув усюди» (shift everywhere): кожну дію розміщують там, де відношення ефективності до вартості максимальне, а «правий» край циклу (моніторинг, PSIRT, аналіз першопричин) замикає зворотний зв'язок на ранні етапи [88].

3.2 Microsoft SDL і точки дотику McGraw

Microsoft SDL — перша промислова процесна модель безпечного розроблення: після масових інцидентів початку 2000-х (Code Red, Slammer) з 2004 р. його зроблено обов'язковим для продуктів, доступних з Інтернету. Ліпнер [235] описав початкову фазову модель: навчання, вимоги безпеки й моделювання загроз, інструменти аналізу й заборонені функції API, *остаточний огляд безпеки* як умова випуску, план реагування; детально процес викладено в [195]. Сучасна редакція [249] сформульована як десять пра-

ктик, інтегрованих у DevOps; нові з них — захист ланцюга постачання й середовища розроблення (відповідь на атаки типу SolarWinds, розділ 10) і «перевірені мови», що охоплюють перехід на мови з безпечною пам'яттю (розділ 5).

Мак-Гроу [245] запропонував легшу альтернативу — сім **точок дотику** (touchpoints), які вбудовують у будь-який наявний процес: огляд коду з інструментами; архітектурний аналіз ризиків; тестування на проникнення; тестування безпеки на основі ризиків; випадки зловживання; вимоги безпеки; безпека експлуатації. Ключова ідея — розрізняти *помилки реалізації* (bugs; для ПЗМБ — переповнення буфера в розборі заголовка) і *вади проєкту* (flaws; відсутність розділення привілеїв між площинами даних і менеджменту), які виявляє лише архітектурний аналіз (розділ 4).

3.3 NIST Secure Software Development Framework

3.3.1 Структура SSDF

NIST SSDF (SP 800-218) [328] описує *результати*, яких має досягти виробник, а не конкретні інструменти. Кожна практика складається із задач, прикладів реалізації й посилань на інші документи (SP 800-53, IEC 62443-4-1, SAMM, BSIMM, ISO/IEC 29147 тощо); практики згруповано в чотири групи (табл. 3.1). Нормативний статус SSDF розглянуто в підрозділі 2.5.

Таблиця 3.1 — Групи практик NIST SSDF і зміни проєкту версії 1.2 (за [328, 267])

Група	Практики	Приклади задач і зміни v1.2
PO — підготовка організації	PO.1–PO.5; **PO.6** (нова)	вимоги до розроблення (PO.1), ролі й навчання (PO.2), ланцюжки інструментів (PO.3), критерії перевірок (PO.4), захист середовищ розроблення (PO.5); PO.6 — план безперервного вдосконалення
PS — захист ПЗ	PS.1–PS.3; **PS.4** (нова)	захист коду від змін (PS.1), цілісність випуску (PS.2), архівування й походження (PS.3); PS.4 — надійні оновлення: тестування, поетапне розгортання, відкат
PW — створення захищеного ПЗ	PW.1, PW.2, PW.4–PW.9	моделювання ризиків (PW.1), огляд проєкту (PW.2), повторне використання компонентів (PW.4), безпечне кодування (PW.5), налаштування компілятора (PW.6), аналіз коду (PW.7), тестування (PW.8), безпечні налаштування за замовчуванням (PW.9)
RV — реагування на вразливості	RV.1–RV.3	виявлення (RV.1), оцінювання й усунення (RV.2), аналіз першопричин (RV.3)

Версія 1.1 містить 19 практик і 42 задачі; у проєкті версії 1.2 за підрахунком автора за текстом проєкту — 21 практика і 49 задач. Ідентифікатори вилучених практик (PW.3) повторно не використовують.

3.3.2 Зміни у версії 1.2 і профілі

Проєкт SP 800-218 Rev. 1 (SSDF v1.2) [267] оприлюднено 17.12.2025 на виконання Указу Президента США 14306. Журнал змін фіксує дві нові практики. *PO.6* — план безперервного вдосконалення: оновлення середовищ розроблення, пошук технік і мов, що усувають цілі класи вразливостей, перегляд рішень щодо реагування (PO.6.1–PO.6.3). *PS.4* — надійність оновлень: тестування в середовищах, що відтворюють реальні інсталяції, «канаркові» розгортання, механізми відкату, надійна інфраструктура доставлення (PS.4.1–PS.4.4). Для ПЗМБ PS.4 особливо важлива: невдале оновлення ME порушує доступність мережі. Чинною залишається версія 1.1.

Профіль спільноти SP 800-218A [93] додає задачі для розроблення моделей генеративного ШІ (розділ 17). Колонка посилань є відображенням на інші рамки (задача PW.7.2 відповідає огляду коду в SAMM і BSIMM, вимозі SI-1 IEC 62443-4-1 і контролю SA-11 SP 800-53), що дає змогу обчислити покриття за (2.2).

3.4 ISO/IEC 27034 та IEC 62443-4-1: процесна суть

3.4.1 ISO/IEC 27034: безпека застосунків як система контролів

ISO/IEC 27034 [205] розглядає безпеку застосунку як *систему контролів*, вибір яких обґрунтовано ризиками:

ONF — **організаційна нормативна рамка**: репозиторій практик організації (контексти, ролі, процеси, бібліотека контролів, еталонна модель життєвого циклу);

ANF — **нормативна рамка застосунку**: підмножина ONF для конкретного проєкту з урахуванням його контексту й цільового рівня довіри;

ASC — **контроль безпеки застосунку**: опис дії безпеки з ціллю, етапом, відповідальною роллю і *вимірюванням верифікації*;

Рівень довіри — мітка, що пов'язує ризик застосунку з набором ASC.

Процес замкнений: аналіз ризиків визначає цільовий рівень довіри і набір ASC; їх верифікація дає *фактичний* рівень, а розбіжність живить удосконалення ONF. Стандарт концептуально найстрогіший, але через абстрактність поширений менше.

3.4.2 IEC 62443-4-1: вісім практик і рівні зрілості

IEC 62443-4-1 [197] визначає вимоги до процесу розроблення продуктів промислової автоматизації, але завдяки чіткості й схемі сертифікації його широко застосовують виробники мережевого обладнання. Вісім практик: **SM** — управління безпекою (ролі, середовище розроблення, сторонні компоненти); **SR** — специфікація вимог і модель загроз; **SD** — безпека за проєктом; **SI** — безпечна реалізація; **SVV** — верифікація й валідація (тестування вимог, вра-

зливостей і фазинг, тестування на проникнення, незалежність тестувальників SVV-5); **DM** — управління проблемами безпеки (повідомлення, розкриття, першопричини); **SUM** — управління оновленнями; **SG** — настанови з безпеки для користувача.

Кожну практику оцінюють за чотирма **рівнями зрілості**, узгодженими з CMMI: 1 — початковий (ситуативне виконання), 2 — керований (процес задокументовано), 3 — визначений (процес відтворюваний в організації, є записи виконання), 4 — удосконалюваний (процес вимірюється й поліпшується). Для сертифікації процесу зазвичай вимагають рівня 3, тобто *доказів* виконання; явні вимоги до незалежності тестування й строків оновлень роблять стандарт легше перевірюваним, ніж SSDF.

3.5 Моделі зрілості: OWASP SAMM і BSIMM

3.5.1 OWASP SAMM 2

OWASP SAMM [279] — *прескриптивна* модель зрілості: п'ять бізнес-функцій (управління, проєктування, реалізація, верифікація, експлуатація) по три практики (перелік — у табл. 3.4). Кожна практика має два *потоки* (A і B) і три рівні зрілості в кожному; рівень 0 означає відсутність діяльності.

Для кожного рівня кожного потоку ставлять одне запитання з відповідями «ні», «так, для деяких», «так, для половини», «так, для більшості», яким відповідають бали $a \in \{0;\ 0{,}25;\ 0{,}5;\ 1\}$. Бал потоку — сума балів трьох рівнів, бал практики — середнє двох потоків:

$$S_p = \frac{1}{2} \sum_{s \in \{A,B\}} \sum_{l=1}^{3} a_{p,s,l}, \qquad S_f = \frac{1}{3} \sum_{p \in f} S_p, \qquad S = \frac{1}{15} \sum_{p=1}^{15} S_p, \tag{3.1}$$

де $a_{p,s,l}$ — бал для практики p, потоку s і рівня l; S_f — бал бізнес-функції f; $S \in [0;3]$ — загальний бал. Наприклад, якщо в практиці «тестування безпеки» потік A має відповіді $1, 0{,}5, 0$, а потік B — $0{,}5, 0{,}5, 0$, то бали потоків $1{,}5$ і $1{,}0$, а $S_p = 1{,}25$. Дробові бали чутливі до часткового впровадження, але вимагають узгодженого тлумачення відповідей; різниця поточного й цільового профілів дає дорожню карту (підрозділ 3.9).

3.5.2 BSIMM і порівняння рамок

BSIMM — *дескриптивна* модель: вона описує, що фактично роблять організації-учасники. Рамка має чотири домени (управління, розвідка, точки дотику SSDL, розгортання), 12 практик і 128 дій, віднесених до трьох рівнів за частотою спостереження. BSIMM16 [88] (початок 2026 р.) охоплює 111 організацій; головними чинниками змін названо ШІ і регуляторні вимоги. BSIMM — інструмент *порівняння з галуззю*, а не нормативна ціль: поширеність дії не доводить її ефективності, а вибірка складається перева-

жно з великих компаній. Порівняння всіх розглянутих моделей подано в табл. 3.2.

Таблиця 3.2 — Порівняння процесних моделей безпечного розроблення

Модель	Тип	Структура	Оцінювання
MS SDL	корпоративна, прескриптивна	10 практик (з 2004 р. — фазова модель)	внутрішні ворота випуску
NIST SSDF	рамка результатів	4 групи, 19 практик, 42 задачі (v1.1); v1.2 — PO.6, PS.4	самодекларування, відображення
ISO/IEC 27034	система контролів	ONF, ANF, ASC, рівні довіри	вимірювання верифікації
IEC 62443-4-1	вимоги до процесу	8 практик: SM, SR, SD, SI, SVV, DM, SUM, SG	рівні зрілості 1–4, сертифікація
OWASP SAMM 2	прескриптивна модель зрілості	5 функцій, 15 практик, 2 потоки, рівні 0–3	самооцінювання, бали (3.1)
BSIMM16	дескриптивна модель	4 домени, 12 практик, 128 дій	порівняння з галуззю

Моделі доповнюють одна одну: SSDF і 62443-4-1 задають, *що* має бути досягнуто, SAMM — *траєкторію* впровадження, BSIMM — галузевий *орієнтир*. Типова стратегія виробника ПЗМБ для ринку ЄС — будувати процес за 62443-4-1, вимірювати прогрес за SAMM і підтримувати відображення на SSDF.

3.6 DevSecOps: ворота якості, безпека як код і зворотний зв'язок

DevSecOps вбудовує безпекові дії в конвеєр CI/CD як автоматизовані кроки й розподіляє відповідальність за безпеку між розробниками, експлуатацією й командою безпеки. Механізмів три. *Автоматизовані ворота якості* (quality gates) зупиняють зміну, що порушує політику (нова критична знахідка SAST, вразлива залежність, секрет, збій фазинг-тесту, відсутній SBOM), — на кожній зміні за хвилини. *Безпека як код*: політики воріт, правила аналізаторів, IDS і ME, конфігурації інфраструктури версіонуються й проходять огляд, а історія змін стає доказом для аудиту (PO.3, PO.4 SSDF). *Зворотний зв'язок*: результати воріт, інциденти й звернення до PSIRT повертаються розробникам як дефекти з першопричиною, а агреговані метрики — власникам процесу (практика PO.6). Схему показано на рис. 3.2.

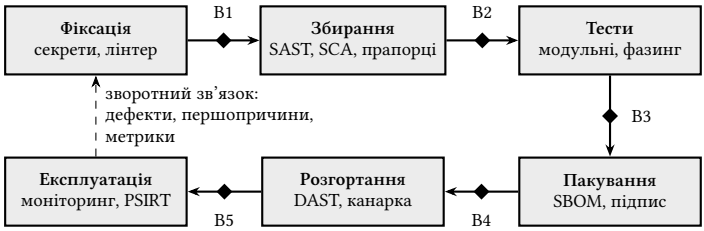


Рис. 3.2 — Конвеєр DevSecOps з воротами якості B1–B5 і контуром зворотного зв'язку

Політику воріт доцільно описувати декларативно, окремо від сценаріїв конвеєра (лістинг 3.1): ворота блокують лише *нові* критичні знахідки (щоб накопичений технічний борг не паралізував випуски), допускають винятки лише з обґрунтуванням у форматі VEX (розділ 16) і експортують метрики.

Лістинг 3.1 — Декларативна політика воріт якості (ілюстрація)

```
# Політика воріт якості для гілки release/*
gates:
  - {id: secrets, fail_on: {any: true}}                # B1
  - {id: sast, fail_on: {severity: high, new_only: true}} # B2
  - {id: sca, fail_on: {cvss_min: 7.0, kev: true},
     exceptions: vex/accepted.json}   # винятки з обґрунтуванням
  - {id: fuzz_smoke, minutes: 15, fail_on: {new_crash: true}}
  - {id: sbom, format: cyclonedx, require_signature: true} # B4
metrics: [gate_pass_rate, gate_override_rate, mttr_vuln]
```

Ефективність доставлення вимірюють метриками DORA [146]: частотою розгортань, часом виконання змін, часткою невдалих змін і часом відновлення після невдалого розгортання (у 2024 р. додано частку переробних розгортань). Для безпеки короткий час виконання змін визначає, як швидко можна розгорнути виправлення критичної вразливості. Їх доповнюють безпековими: часткою ручних обходів воріт і часткою артефактів із SBOM і перевіреним підписом. Українські дослідження показують, що інтеграція контролів у конвеєр контейнерних застосунків скорочує час розроблення [51], пропонують моделі інтеграції безпеки в життєвий цикл [26, 4] і аналізують застосування ШІ в DevSecOps [2].

3.7 Кількісні моделі безпечного життєвого циклу

3.7.1 Економіка виправлення дефектів

Класичне твердження: вартість виправлення дефекту зростає на порядки з кожним наступним етапом. Бем і Базілі [92] наводять співвідношення близько 100:1 між виправленням після постачання і на етапі вимог для

великих критичних систем, але лише близько 5:1 для невеликих некритичних. Популярні коефіцієнти 1–6–15–100, приписувані «Інституту системних наук IBM», Боссаві [94] простежив до слайдів і переказів без первинного дослідження. Отже, зростання вартості *правдоподібне*, але його *масштаб* не є універсальною константою.

Нехай у продукт внесено N дефектів, на етапі $k = 1, \dots, K$ дефект, що дожив до нього, виявляється з імовірністю e_k, а виправлення коштує c_k; етап $K+1$ — експлуатація. Очікувані кількість виявлених на етапі дефектів і сумарна вартість:

$$D_k = N\, e_k \prod_{j<k} (1 - e_j), \qquad D_{K+1} = N \prod_{j=1}^{K} (1 - e_j), \qquad C = \sum_{k=1}^{K+1} c_k D_k. \tag{3.2}$$

Числовий приклад. Нехай $N = 100$, етапи: моделювання загроз ($e_1 = 0{,}25$), огляд коду й SAST ($e_2 = 0{,}35$), тестування й фазинг ($e_3 = 0{,}45$). Тоді $D = (25;\ 26{,}25;\ 21{,}94;\ 26{,}81)$. За «класичних» вартостей $c = (1; 5; 10; 100)$ $C = 3057$ умовних одиниць, за помірних $c = (1; 2; 4; 10) - 433$. Посилення моделювання загроз до $e_1 = 0{,}45$ дає $D = (45;\ 19{,}25;\ 16{,}09;\ 19{,}66)$ і $C = 2268$ (економія 26 %) чи 345 (економія 21 %). Якщо посилення коштує 150 одиниць, то за класичних коефіцієнтів воно окупається вп'ятеро, а за помірних — *не окупається*. Відносний ефект стійкий до вибору коефіцієнтів, абсолютне обґрунтування — ні; для ПЗМБ основну частину c_{K+1} становлять наслідки експлуатації у клієнтів (модель (1.5)).

3.7.2 Моделі зростання надійності

Моделі зростання надійності ПЗ (SRGM) описують виявлення й усунення дефектів під час тестування як неоднорідний пуассонівський процес (NHPP) з функцією середнього значення $m(t)$. Модель Goel–Okumoto [183] припускає скінченну кількість дефектів a і сталу інтенсивність виявлення кожного залишкового дефекту b:

$$m(t) = a\left(1 - e^{-bt}\right), \qquad \lambda(t) = ab\, e^{-bt}, \qquad \bar{N}(t) = a\, e^{-bt}, \tag{3.3}$$

де $\lambda(t) = m'(t)$ — інтенсивність виявлення, $\bar{N}(t)$ — очікуваний залишок. Імовірність відсутності знахідок за інтервал x дорівнює $R(x \mid t) = \exp\{-[m(t+x) - m(t)]\}$. Параметри зазвичай оцінюють методом максимальної правдоподібності; для навчальних цілей зручна оцінка за двома рівновіддаленими точками: якщо $m(t_1) = y_1$, $m(2t_1) = y_2$, то $y_2/y_1 = 1 + e^{-bt_1}$, звідки $b = -\ln(y_2/y_1 - 1)/t_1$ і $a = y_1/(1 - e^{-bt_1})$.

Числовий приклад. Під час верифікації VPN-шлюзу за 10 тижнів виявлено 62 унікальні безпекові дефекти, за 20 тижнів — 92. Тоді $e^{-10b} = 0{,}484$, $b = 0{,}0726$ тиж.$^{-1}$, $a = 120{,}1$; після 20 тижнів $\bar{N} = 28{,}1$, $\lambda = 2{,}04$ тиж.$^{-1}$,

$R(1\,|\,20) = 0{,}14$. Для зниження залишку до 10 дефектів потрібно $t = \ln(a/10)/b = 34{,}2$ тижні (рис. 3.3, *а*).

Коли кожен наступний дефект знайти дедалі важче (типово для фазингу), краща логарифмічна модель Musa–Okumoto [258, 259] з часом виконання τ:

$$\mu(\tau) = \frac{1}{\theta}\ln\left(\lambda_0\theta\tau + 1\right), \qquad \lambda(\tau) = \frac{\lambda_0}{\lambda_0\theta\tau + 1}, \tag{3.4}$$

де λ_0 — початкова інтенсивність, θ — її спадання на один дефект. Для фазингу з $\lambda_0 = 2$ год$^{-1}$ і $\theta = 0{,}05$ за 1000 год очікується $\mu = 20\ln 101 = 92{,}3$ унікальні збої з інтенсивністю $0{,}0198$ год$^{-1}$, а її зниження вдвічі потребує ще 990 год («спадна віддача», розділ 13).

3.7.3 Моделі виявлення вразливостей

Після випуску вразливості шукають зовнішні дослідники й порушники, зусилля яких залежать від популярності продукту. Алхазмі й Малайя [69, 68] запропонували логістичну модель з трьома фазами: повільний старт, лінійне зростання й насичення:

$$\Omega(t) = \frac{B}{BC\,e^{-ABt} + 1}, \qquad \frac{d\Omega}{dt} = A\,\Omega\,(B - \Omega), \qquad t^* = \frac{\ln(BC)}{AB}, \tag{3.5}$$

де $\Omega(t)$ — кумулятивна кількість виявлених вразливостей, B — загальна їх кількість, A — параметр інтенсивності, C — параметр початкових умов, t^* — точка перегину з максимальною інтенсивністю $AB^2/4$. *Числовий приклад.* Для прошивки ME з $B = 120$, $A = 0{,}01$, $C = 0{,}2$ (час у роках) $\Omega(1) = 14{,}6$, $\Omega(3) = 72{,}5$, $\Omega(5) = 113{,}3$; пік — $t^* = \ln 24/1{,}2 = 2{,}65$ року з інтенсивністю 36 вразливостей на рік. Через три роки залишок $B - \Omega(3) = 47{,}5$, тож ресурси PSIRT слід планувати саме на цей період (рис. 3.3, *б*).

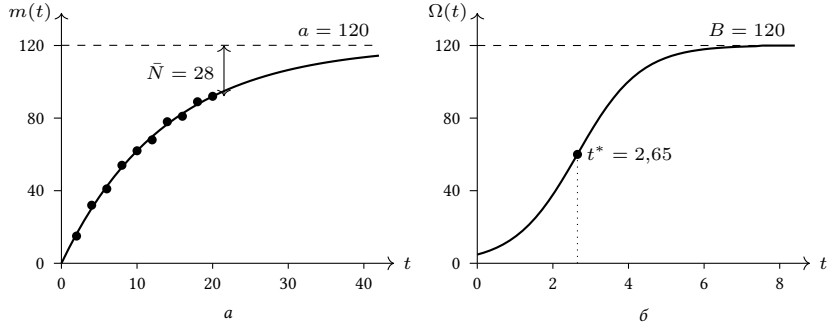


Рис. 3.3 — Моделі виявлення: *а* — Goel–Okumoto (3.3), підігнана до даних верифікації (t у тижнях); *б* — логістична модель Alhazmi–Malaiya (3.5) (t у роках)

Массаччі й Нгуєн [242] показали, що якість прогнозу моделей суттєво залежить від визначення «вразливості» (запис CVE, бюлетень, виправлення в коді) і від урахування версій; логістична модель була серед кращих, але не універсальною. Рескорла [303] не знайшов свідчень, що пошук помітно вичерпує запас вразливостей; разом із даними [283] це аргумент на користь *запобігання* (безпечні мови, архітектура, розбір вводу), а не лише виявлення.

3.7.4 Щільність і оцінка залишкових вразливостей

Щільність вразливостей $V_D = V/S$ (на тисячу рядків коду) і відношення вразливостей до дефектів $V_{KD} = V/D$ дають апріорну оцінку латентних вразливостей за обсягом коду (модель (1.2)); для операційних систем Алхазмі й Малайя отримали V_{KD} на рівні одиниць відсотків [69]. Апостеріорну оцінку дає метод *захоплення–повторного захоплення* (capture–recapture), застосований до інспекцій ПЗ [161]. Якщо два незалежні методи верифікації виявили n_1 і n_2 дефектів, з яких m спільні, то оцінка Лінкольна–Петерсена загальної кількості й залишку:

$$\hat{N} = \frac{n_1 n_2}{m}, \qquad \hat{N}_{\text{зал}} = \hat{N} - (n_1 + n_2 - m). \tag{3.6}$$

Числовий приклад. Фазинг парсерів IDS виявив $n_1 = 40$ безпекових дефектів, ручний огляд коду — $n_2 = 30$, спільних $m = 12$. Тоді $\hat{N} = 100$, знайдено 58 різних дефектів, залишок $\hat{N}_{\text{зал}} = 42$ (виправлена оцінка Чапмена $(n_1+1)(n_2+1)/(m+1) - 1 = 96{,}8$ дає близько 39). Обидва методи краще знаходять «легкі» дефекти, тобто незалежність порушується й оцінка зміщується донизу; її слід трактувати як нижню межу.

3.8 Метрики процесу безпеки

Джонс і Бонсіньйор [217] вважають найважливішою метрикою якості процесу **ефективність виявлення дефектів** (DRE) — частку дефектів, усунених до випуску; для етапу k визначають ефективність етапу:

$$\text{DRE} = \frac{E}{E + F}, \qquad \text{DRE}_k = \frac{D_k}{D_k + \sum_{j>k} D_j^{(\le k)}}, \tag{3.7}$$

де E — дефекти, виявлені до випуску; F — виявлені після випуску за фіксований період (традиційно 90 днів); $D_j^{(\le k)}$ — дефекти, внесені до етапу k включно, але виявлені на пізнішому етапі j. За даними авторів, середня DRE — близько 85 %, а понад 95 % досягають лише поєднанням інспекцій, статичного аналізу й тестування. *Приклад:* до випуску прошивки виявлено $E = 180$ безпекових дефектів, за 90 днів після — $F = 12$, отже $\text{DRE} = 0{,}94$;

для моделі (3.2) $\mathrm{DRE} = 1 - D_4/N = 0{,}73$, а після посилення моделювання загроз — $0{,}80$.

Час виправлення вразливостей характеризують середнім (MTTR) і часткою виправлень у строк:

$$\mathrm{MTTR} = \frac{1}{n}\sum_{i=1}^{n} \Delta_i, \qquad P_{\mathrm{SLA}} = \frac{1}{n}\bigl|\{i : \ \Delta_i \le T_{s(i)}\}\bigr|, \tag{3.8}$$

де Δ_i — час від отримання повідомлення до випуску виправлення i-ї вразливості; $T_{s(i)}$ — цільовий строк для рівня критичності $s(i)$. Розподіл Δ_i має важкий хвіст: для п'яти критичних вразливостей з $\Delta = 3, 7, 12, 30, 90$ днів $\mathrm{MTTR} = 28{,}4$ дня, медіана — 12 днів, а за строку 30 днів $P_{\mathrm{SLA}} = 0{,}8$; одна «застрягла» вразливість подвоює середнє, тому керівними є P_{SLA}, медіана й максимальний вік відкритої вразливості. Вікно вразливості ПЗМБ закривається лише після *встановлення* виправлення клієнтами (розділ 16).

Покриття контролями $\mathrm{Cov} = \sum_i w_i x_i / \sum_i w_i$ — зважена за критичністю частка компонентів, до яких застосовано обов'язковий контроль ($x_i \in \{0;1\}$; структурно збігається з (2.2)). **Частка зовнішнього виявлення** $F_{\text{зовн}}/(E+F)$ (вразливості, знайдені клієнтами, дослідниками чи порушниками) — найчутливіший індикатор прогалин верифікації.

Для обґрунтування бюджету використовують показник повернення інвестицій у безпеку (ROSI):

$$\mathrm{ROSI} = \frac{\mathrm{ALE}_0 - \mathrm{ALE}_1 - C_{\text{зах}}}{C_{\text{зах}}}, \qquad \mathrm{ALE} = \mathrm{ARO}\cdot\mathrm{SLE}, \tag{3.9}$$

де ALE_0, ALE_1 — очікувані річні втрати без заходу і з ним; ARO — річна частота інцидентів; SLE — втрати від одного інциденту; $C_{\text{зах}}$ — річна вартість заходу. Наприклад, за $\mathrm{SLE} = 500$ тис. ум. од. і зниження ARO з $0{,}4$ до $0{,}15$ програмою фазингу вартістю 60 тис./рік $\mathrm{ROSI} = (200 - 75 - 60)/60 = 1{,}08$; через невизначеність ARO результат подають інтервалом. Зведення метрик — у табл. 3.3.

Таблиця 3.3 — Метрики процесу безпечного розроблення

Метрика	Обчислення	Призначення і застереження
DRE, DRE етапу	(3.7)	якість верифікації; залежить від періоду спостереження F
Частка зовнішнього виявлення	$F_{\text{зовн}}/(E+F)$	прогалини верифікації; чутлива до популярності продукту
MTTR, P_{SLA}	(3.8)	швидкість реагування; доповнювати медіаною
Покриття контролями	$\sum w_i x_i / \sum w_i$	повнота впровадження, але не якість контролю
Залишкові вразливості	(3.3), (3.6)	рішення про випуск; припущення моделей
Частка обходу воріт	обходи / спрацювання	дієвість DevSecOps; зростання — сигнал «втоми»
Бал SAMM	(3.1)	зрілість процесу; суб'єктивність оцінювача
ROSI	(3.9)	обґрунтування бюджету; невизначеність ARO

За законом Гудгарта метрика, що стає ціллю, перестає бути гарною метрикою (команда, оцінювана за MTTR, може знижувати критичність вразливостей), тому метрики застосовують наборами, що взаємно контролюють одна одну.

3.9 Кейс: оцінювання зрілості команди-розробника ПЗМБ за SAMM

Українська компанія розробляє VPN-шлюз і NGFW (близько 40 розробників, дві гілки прошивки на підтримці) і готується до виходу на ринок ЄС, де ці продукти належать до важливих продуктів класів I і II CRA (розділ 2). Мета — за 18 місяців побудувати процес, придатний для сертифікації за IEC 62443-4-1 на рівні зрілості 3 і для обробки вразливостей та звітування за CRA. Оцінювання за SAMM 2 проведено інтерв'ю з ключовими ролями; бали обчислено за (3.1). Цільовий профіль T_p визначено з вимог CRA і 62443-4-1: найвищі цілі $(2{,}5)$ — безпечне збирання (SBOM, підпис, походження), тестування безпеки (фазинг протоколів) і управління інцидентами (PSIRT, звітування за 24/72 год). Кожній практиці приписано вагу релевантності $w_p \in \{1;2;3\}$ і обчислено пріоритет $P_p = w_p\,(T_p - S_p)$ (табл. 3.4).

Таблиця 3.4 — Поточний і цільовий профілі зрілості за SAMM

Практика	A	B	S_p	T_p	w_p	P_p
Стратегія і метрики	0,5	0,25	0,38	1,5	1	1,13
Політики й відповідність	1,0	0,5	0,75	2,0	2	2,50
Навчання й настанови	1,25	0,5	0,88	1,5	1	0,63
Оцінювання загроз	0,5	0,25	0,38	2,0	3	4,88
Вимоги безпеки	1,0	0,25	0,63	2,0	2	2,75
Безпечна архітектура	1,5	1,0	1,25	2,0	2	1,50
Безпечне збирання	1,5	0,5	1,00	2,5	3	4,50
Безпечне розгортання	1,0	0,5	0,75	2,0	2	2,50
Управління дефектами	1,0	0,5	0,75	2,0	2	2,50
Оцінювання архітектури	0,25	0,25	0,25	1,5	1	1,25
Тестування на основі вимог	1,0	0,5	0,75	2,0	2	2,50
Тестування безпеки	1,5	1,0	1,25	2,5	3	3,75
Управління інцидентами	1,0	0,5	0,75	2,5	3	5,25
Управління середовищем	1,25	1,0	1,13	2,0	1	0,88
Експлуатаційне управління	0,5	0,5	0,50	1,5	1	1,00
Загальний бал S			0,76	1,97		

Бали бізнес-функцій: управління — $0{,}67$, проєктування — $0{,}75$, реалізація — $0{,}83$, верифікація — $0{,}75$, експлуатація — $0{,}79$; загальний бал $0{,}76$ проти цільового $1{,}97$. Технічні практики сильніші за управлінські та «ліві». Найвищий пріоритет мають управління інцидентами ($5{,}25$: немає формалізованого PSIRT, а звітування за CRA діє з 11.09.2026), оцінювання загроз ($4{,}88$) і безпечне збирання ($4{,}50$). Дорожня карта має три фази, кожна підвищує бал практики не більш ніж на рівень:

1. *0–6 місяців:* PSIRT, приймання повідомлень і звітування (управління інцидентами $0{,}75 \to 1{,}75$); моделі загроз для площини менеджменту й демона VPN ($0{,}38 \to 1{,}25$); SBOM і підпис артефактів у конвеєрі ($1{,}0 \to 1{,}75$); очікуваний $S \approx 0{,}76 + 2{,}625/15 \approx 0{,}93$.
2. *6–12 місяців:* постійний фазинг протоколів і ворота якості (лістинг 3.1), вимоги до постачальників компонентів, політика відповідності CRA/62443-4-1, управління дефектами з SLA; очікуваний $S \approx 1{,}45$.
3. *12–18 місяців:* доведення решти практик до цілей, запровадження метрик (DRE, P_{SLA}, частка зовнішнього виявлення), повторне оцінювання й передсертифікаційний аудит.

Відображення практик SAMM на 62443-4-1 (оцінювання загроз — SR, тестування безпеки — SVV, управління інцидентами — DM) показує готовність до аудиту; бали самооцінювання слід підкріплювати артефактами.

Висновки до розділу

Безпекові дії однакові для каскадної, ітеративної й DevOps-моделей, що відрізняються частотою циклу й автоматизацією. «Зсув уліво» обґрунтований, але обмежений: частина вразливостей ПЗМБ проявляється лише під час фазингу, навантаження й експлуатації. Процесні рамки комплементарні: SDL еволюціював до десяти практик, інтегрованих у DevOps; SSDF описує результати й служить мовою відображень, а проєкт версії 1.2 додає безперервне вдосконалення (PO.6) і надійні оновлення (PS.4); ISO/IEC 27034 дає апарат контролів і рівнів довіри; IEC 62443-4-1 придатний для сертифікації; SAMM задає траєкторію впровадження, BSIMM — галузевий орієнтир. Моделі надійності, виявлення вразливостей і захоплення–повторного захоплення прогнозують залишок дефектів, тривалість верифікації й навантаження на PSIRT, але їхні припущення слід перевіряти на власних даних, а економічні аргументи — будувати на виміряних коефіцієнтах вартості. Метрики процесу ефективні лише в наборах, що взаємно контролюють одна одну. Кейс показав перетворення оцінювання за SAMM на дорожню карту, узгоджену з CRA та IEC 62443-4-1.

Питання для самоперевірки

1. Порівняйте каскадну, V-модель і DevOps за розташуванням точок контролю безпеки й тривалістю циклу зворотного зв'язку.
2. Сформулюйте принцип «зсуву вліво» і наведіть три аргументи щодо його меж для ПЗМБ.
3. Як змінився Microsoft SDL від фазової моделі 2004 р. до десяти сучасних практик і чим це зумовлено?
4. Розрізніть помилки реалізації й вади проєкту за Мак-Гроу. Наведіть приклади кожного класу для VPN-шлюзу.
5. Опишіть групи практик NIST SSDF. Які практики додає проєкт версії 1.2 і чому PS.4 важлива для ПЗМБ?
6. Поясніть поняття ONF, ANF, ASC і рівня довіри в ISO/IEC 27034 та зв'язок між ними.
7. Перелічіть практики IEC 62443-4-1 і охарактеризуйте рівні зрілості 1–4. Чому для сертифікації потрібен рівень 3?
8. Чим прескриптивна модель зрілості відрізняється від дескриптивної? Які обмеження має BSIMM для невеликого виробника?
9. Обчисліть за (3.1) бал практики, якщо потік A має відповіді 1, 1, 0,25, а потік B — 0,5, 0, 0.

10. За моделлю (3.2) з $N = 100$ і $e = (0{,}3; 0{,}4; 0{,}5)$ обчисліть кількість дефектів, що потрапляють в експлуатацію, і DRE. Як зміниться вартість за $c = (1; 3; 6; 30)$, якщо e_2 зросте до $0{,}6$?
11. За 8 тижнів тестування виявлено 40 дефектів, за 16 тижнів — 64. Оцініть параметри моделі (3.3) і залишок дефектів після 16 тижнів.
12. Для моделі (3.5) з $B = 80$, $A = 0{,}02$, $C = 0{,}5$ визначте точку перегину й максимальну інтенсивність виявлення.
13. Чому MTTR слід доповнювати часткою виправлень у строк і медіаною? Наведіть приклад маніпулювання цією метрикою.
14. Дослідницьке: розробіть методику перевірки адекватності моделей виявлення вразливостей для VPN-шлюзів за відкритими даними (CVE, бюлетені виробників) з урахуванням гілок прошивки.
15. Дослідницьке: запропонуйте модель розподілу бюджету верифікації між моделюванням загроз, статичним аналізом і фазингом, що мінімізує вартість (3.2) за невизначеності c_k, і обґрунтуйте вимоги до даних для калібрування.

РОЗДІЛ 4

Вимоги безпеки, моделювання загроз і безпечне проєктування програмного забезпечення мережевої безпеки

Модель «дефект → вразливість → експлуатація» (розділ 1) показала, що найсильніші важелі зменшення ризику ПЗМБ — досяжність вразливостей r_k і наслідки їх експлуатації — визначаються ще до кодування: у вимогах і архітектурі. Нормативні джерела (розділ 2) і процесні рамки (розділ 3) задають, *що* має бути зроблено; цей розділ розглядає, *як* перетворити їх на перевірювані вимоги, модель загроз і архітектуру, метрики поверхні атаки й вибір заходів за обмеженого бюджету на кейсі IDS-сенсора (підрозділ 1.9). Реалізаційні аспекти патернів розглянуто в розділах 5–9, їхню верифікацію — у розділах 11–14.

4.1 Інженерія вимог безпеки

4.1.1 Функціональні вимоги безпеки й вимоги гарантій

Вимоги безпеки до ПЗМБ поділяють на два типи, закріплені в ISO/IEC 15408 [201]. **Функціональні вимоги безпеки** (SFR) описують поведінку продукту: механізми автентифікації, криптографічного захисту, журналювання, контролю цілісності. **Вимоги гарантій** (SAR) описують дії розробника й оцінювача, що забезпечують довіру до коректності реалізації (розділ 2). Для ПЗМБ варто виділяти й третій тип — *вимоги стійкості*: поведінка за ворожого входу, ізоляція компонентів, відмова в безпечний стан. Саме вони найчастіше залишаються неявними.

Добра вимога безпеки є *перевірюваною*, *трасованою* (пов'язаною із загрозою чи джерелом) і *сформульованою щодо властивості, а не механізму*. «Вебінтерфейс має бути захищеним» не відповідає жодному з критеріїв; «усі запити до REST API, крім `/login`, відхиляються з кодом 401 без дійсного маркера сеансу; маркер має не менше 128 біт ентропії й анулюється після 15 хв неактивності» — відповідає всім трьом.

4.1.2 Джерела вимог

Для ПЗМБ склалися три взаємодоповнювальні каталоги вимог (табл. 4.1). Профіль NDcPP [128] задає SFR у класах FCS (криптографічна підтримка), FIA (ідентифікація й автентифікація), FPT (захист функцій безпеки), FAU (аудит), FTP (довірені канали) і FTA (доступ до сеансів), а PP-модулі доповнюють їх для ME, VPN-шлюзів та IPS [293]. OWASP ASVS 5.0 [277] (близько

350 вимог у 17 розділах, три рівні) доречний для вебінтерфейсів керування, порталів VPN і REST API — площини менеджменту, на яку припадає більшість критичних вразливостей граничних пристроїв (розділ 1). IEC 62443-4-2 [198] групує вимоги за базовими вимогами FR 1–7: загальні для компонентів (CR) і специфічні для вбудованих пристроїв (EDR), хостів (HDR), застосунків (SAR) і мережевих пристроїв (NDR) з рівнями спроможності SL-C 1–4.

Таблиця 4.1 — Джерела вимог безпеки до ПЗМБ

Джерело	Структура	Приклади вимог для IDS-сенсора чи VPN-шлюзу
NDcPP і PP-модулі	SFR за класами FCS, FIA, FPT, FAU, FTP, FTA; оціночні дії допоміжного документа	захищене оновлення з перевіркою підпису (FPT_TUD_EXT.1); самотестування (FPT_TST_EXT.1); захист ключів від читання (FPT_SKP_EXT.1); передавання журналів довіреним каналом (FTP_ITC.1)
OWASP ASVS 5.0	17 розділів, рівні L1–L3	автентифікація адміністратора, керування сеансами, авторизація кожного виклику API, кодування виводу, журналювання без секретів
IEC 62443-4-2	FR 1–7; CR, EDR, HDR, SAR, NDR; SL-C 1–4	цілісність ПЗ й інформації (CR 3.4), захист від DoS (CR 7.1), цілісність процесу завантаження (EDR 3.14), захист межі зони (NDR 5.2)

Каталоги містять спільне для класу продуктів, але не специфічне для архітектури, тому їх доповнюють вимогами з моделі загроз (підрозділ 4.2) і суттєвими вимогами CRA [301]. Для пріоритизації вимог і розподілу ресурсів українські дослідники запропонували поєднання методу аналізу ієрархій, нечіткої логіки й байєсових мереж [5] (пор. підрозділ 4.6).

4.1.3 Вимоги-зловживання

Випадок зловживання (misuse case), за Сіндре й Опдалем [325], — варіант використання, який виконує *порушник*; він «загрожує» легітимному варіанту, а захисний варіант його «пом'якшує». Для ПЗМБ випадки зловживання виводяться з профілів порушника (підрозділ 1.5): неавтентифікований порушник формує пакети, що спричиняють розбіжність інтерпретації між IDS і кінцевим вузлом; користувач VPN звертається до функцій адміністратора; порушник у ланцюгу постачання підміняє пакет правил. Кожен сценарій породжує *вимогу-зловживання* — негативну вимогу («система не по-

винна…») з критерієм перевірки — і специфікацію негативного тесту (розділ 11).

4.1.4 Трасованість вимог

Те, що кожна істотна загроза покрита вимогою, а кожна вимога — перевіркою, фіксує **матриця трасованості** «загроза → вимога → тест». Нехай T — множина загроз моделі, R — вимог, V — верифікаційних дій (тестів, аналізів, оглядів); відношення $\tau \subseteq T \times R$ і $\nu \subseteq R \times V$ задають матрицю. *Повнота трасування* дорівнює

$$C_{\text{тр}} = \frac{\left|\{t \in T : \exists r \in R,\ \exists v \in V :\ (t,r) \in \tau,\ (r,v) \in \nu\}\right|}{|T|}, \tag{4.1}$$

тобто частка загроз, що мають хоча б один ланцюжок до перевірки. Якщо модель IDS-сенсора містить 40 загроз, 34 з них покрито вимогами, а для 29 існують тести, то $C_{\text{тр}} = 29/40 = 0{,}725$: чверть загроз не адресована або адресована вимогами, виконання яких ніхто не перевіряє. Фрагмент матриці наведено в табл. 4.2.

Таблиця 4.2 — Фрагмент матриці трасованості для IDS-сенсора

Загроза	Вимога	Джерело	Перевірка
T03 (E)	декодер виконується в окремому процесі без привілеїв під фільтром seccomp	модель загроз; CR 3.4	тест виклику забороненого системного виклику; огляд проєкту
T07 (T)	пакет правил приймається лише з дійсним підписом	FPT_TUD_EXT.1	негативні тести з пошкодженим і простроченим підписом
T11 (D)	пам’ять на відновлення потоків обмежена квотою	вимога-зловживання; CR 7.1	навантажувальний тест (розділ 14)
T15 (S)	API менеджменту доступний лише на виділеному інтерфейсі	CISA SbD; NDR 5.2	сканування портів з інтерфейсів даних

Якщо матрицю вести в репозиторії як мітки тестів з ідентифікаторами вимог, (4.1) обчислюється в конвеєрі CI/CD і стає доказом для кейсу гарантування (розділ 17).

4.2 Моделювання загроз ПЗМБ

4.2.1 Діаграми потоків даних і межі довіри

Моделювання загроз відповідає на питання: що ми будуємо, що може піти не так, що ми з цим робимо і чи добре ми це зробили [324]; цінність

має не разовий документ, а безперервний процес [99]. Відповіддю на перше питання є **діаграма потоків даних** (DFD) з елементами чотирьох типів (зовнішня сутність, процес, сховище, потік) і **межами довіри**, де дані переходять між суб'єктами з різними привілеями. Для ПЗМБ межі проводять і *усередині* продукту: між площинами даних, керування й менеджменту (розділ 1) і між процесами. На рис. 4.1 наведено DFD IDS-сенсора — наскрізного прикладу розділу.

4.2.2 STRIDE для елементів і взаємодій

Категорії STRIDE [324] — підміна (S), модифікація (T), відмова від авторства (R), розкриття (I), відмова в обслуговуванні (D), підвищення привілеїв (E) — відповідають порушенню автентичності, цілісності, неспростовності, конфіденційності, доступності й авторизації. У варіанті **STRIDE-per-element** зовнішнім сутностям відповідають S і R, процесам — усі шість, сховищам і потокам — T, I, D (журналам аудиту також R). Варіант **STRIDE-per-interaction** аналізує кожну взаємодію «джерело–потік–приймач», що перетинає межу довіри; він дає менше дублікатів і краще відповідає ПЗМБ, де більшість загроз виникає на межах. Для 10 вузлів рис. 4.1 STRIDE-per-element дає близько 40 пар «елемент–категорія» (9 потоків аналізують окремо), а аналіз п'яти перетинів меж зосереджує увагу на розборі ворожого трафіку (МД1, МД2), командах і оновленнях (МД3, МД5) і телеметрії (МД4). Для специфічних застосувань категорії STRIDE доцільно розширювати, наприклад шкодою третім особам і фізичною компрометацією вузлів [33].

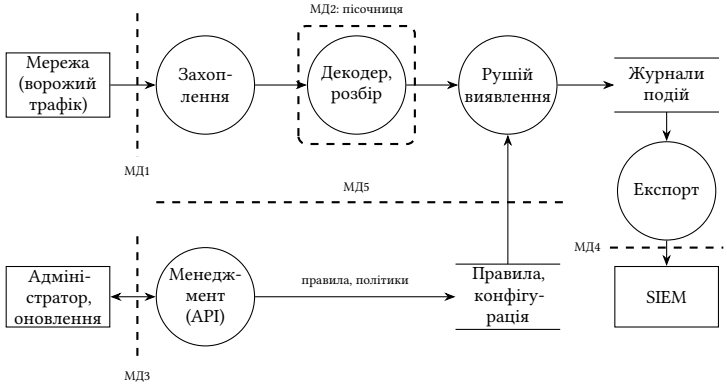


Рис. 4.1 — DFD мережевого IDS-сенсора з межами довіри: МД1 — мережа/сенсор, МД2 — пісочниця декодера, МД3 — адміністратор/менеджмент, МД4 — сенсор/SIEM, МД5 — менеджмент/площина даних

4.2.3 Кількісні дерева атак

Дерево атак [319] подає мету порушника коренем, підцілі — вузлами І (потрібні всі дочірні) чи АБО (достатньо однієї), елементарні дії — листям з атрибутами. Для ймовірності успіху (за незалежності листків) і мінімальної вартості атаки

$$p_{\text{І}} = \prod_k p_k, \quad p_{\text{АБО}} = 1 - \prod_k (1 - p_k); \qquad c_{\text{І}} = \sum_k c_k, \quad c_{\text{АБО}} = \min_k c_k. \tag{4.2}$$

Огляд розширень — дерев атак і захисту, графів атак, байєсових мереж — наведено в [226].

Числовий приклад. Мета G — виконання коду з привілеями на IDS-сенсорі. Шлях A1 (площина даних) = І(доставити пакет до сенсора: $p = 0{,}9$, $c = 0{,}1$; вразливість у декодері: $0{,}2$, $c = 5$); A2 (менеджмент) = І(досяжність API: $0{,}3$, $c = 0{,}5$; обхід автентифікації: $0{,}25$, $c = 3$); A3 (оновлення правил) = І(компрометація сервера оновлень: $0{,}05$, $c = 8$; відсутність перевірки підпису: $0{,}5$, $c = 0$); вартість — у людино-тижнях порушника. За (4.2) $p_{A1} = 0{,}18$, $p_{A2} = 0{,}075$, $p_{A3} = 0{,}025$, $p_G = 1 - 0{,}82 \cdot 0{,}925 \cdot 0{,}975 \approx 0{,}260$; $c_{A1} = 5{,}1$, $c_{A2} = 3{,}5$, $c_{A3} = 8$, $c_G = 3{,}5$. Тепер уведемо три архітектурні заходи: ізоляцію декодера в пісочниці (до A1 додається лист «вихід із пісочниці», $p = 0{,}1$, $c = 6$), виділений інтерфейс менеджменту ($0{,}3 \to 0{,}02$) і обов'язкову перевірку підпису правил ($0{,}5 \to 0{,}01$). Тоді $p_{A1} = 0{,}018$, $p_{A2} = 0{,}005$, $p_{A3} = 0{,}0005$ і $p_G \approx 0{,}023$ — у 11 разів менше. Мінімальна ж вартість атаки не змінилася ($c_G = c_{A2} = 3{,}5$): виділений інтерфейс зменшує *досяжність*, але не *трудомісткість* для порушника, що вже перебуває в мережі менеджменту. Отже, ймовірнісний і вартісний атрибути відповідають різним порушникам (опортуністичному й цілеспрямованому); проти другого потрібен захід, що підвищує c, — наприклад, багатофакторна автентифікація адміністратора.

4.2.4 PASTA, LINDDUN і моделювання загроз як код

PASTA [337] — ризик-орієнтований процес із семи етапів: бізнес-цілі, технічний обсяг, декомпозиція, аналіз загроз, аналіз вразливостей, моделювання атак (дерева атак, CAPEC [134]), аналіз ризику й впливу; він пов'язує модель загроз із реєстром ризиків.

Телеметрія IDS і NDR (IP-адреси, імена з DNS і TLS SNI) містить персональні дані [299, 43]. LINDDUN [148] доповнює STRIDE категоріями зв'язуваності, ідентифіковуваності, неспростовності, виявлюваності, розкриття, необізнаності суб'єкта й невідповідності; для потоку МД4 вона породжує вимоги мінімізації полів, псевдонімізації адрес із ключем поза SIEM і обмеження строку зберігання.

Ручна модель загроз швидко застаріває, тому застосовують *моделювання загроз як код*: опис системи зберігається в репозиторії, а DFD і перелік загроз генеруються в конвеєрі (OWASP pytm [278], OWASP Threat Dragon); новий потік через межу довіри видно в огляді коду.

4.3 Принципи безпечного проєктування

4.3.1 Принципи Зальцера–Шредера

Вісім принципів Зальцера й Шредера (1975) [313] залишаються основою безпечної архітектури; більшість сучасних патернів ПЗМБ є їх прямою реалізацією (табл. 4.3).

Таблиця 4.3 — Принципи Зальцера–Шредера в сучасній інтерпретації для ПЗМБ

Принцип	Інтерпретація для ПЗМБ
Економія механізму	мінімальна довірена обчислювальна база (TCB); компактні протоколи (WireGuard, розділ 9)
Безпечні налаштування за замовчуванням	заборона без явного дозволу; вимкнені застарілі протоколи; немає заводських паролів
Повне посередництво	авторизація кожного виклику API, а не лише входу; немає «внутрішніх» необлікованих кінцевих точок
Відкритий проєкт	безпека не ґрунтується на закритості прошивки, доступної порушнику
Розділення привілеїв	кілька незалежних умов для критичних дій (підпис оновлення і підтвердження адміністратора)
Найменші привілеї	декодер протоколів без прав root, у пісочниці; окремі облікові записи для служб
Найменше спільного механізму	окремі процеси й пам'ять для площин; відсутність спільних кешів між орендарями хмарного ME
Психологічна прийнятність	безпечна конфігурація має бути найпростішою; зрозумілі повідомлення про помилки політик

4.3.2 Принципи NIST SP 800-160

NIST SP 800-160 Vol. 1 Rev. 1 [310] розвиває класичні принципи в системну інженерію довіреності. Для ПЗМБ найпродуктивніші: *розділення доменів* і *опосередкований доступ*; *мінімум довірених елементів*; *захисна відмова* і *захисне відновлення*; *найменша тривалість зберігання* ключів; *співмірна строгість* — глибина верифікації пропорційна критичності елемента, що обґрунтовує зосередження формальних методів і фазингу на парсерах і криптографії (розділи 12, 13).

4.3.3 Secure by Design і Secure by Default

З ініціативи CISA Secure by Design [114] (розділ 1) для проєктування випливають дві вимоги. *Безпечно за замовчуванням*: без заводських паролів, з уві-

мкненою MFA адміністраторів і журналюванням, з менеджментом, недоступним з Інтернету [117, 301]. *Усунення класів вразливостей* замість екземплярів: параметризовані запити, заборона формування команд оболонки з вводу, мови з безпечною пам'яттю для нового коду розбору (розділ 5).

4.4 Архітектурні патерни ПЗМБ

4.4.1 Розділення привілеїв

Класичну реалізацію розділення привілеїв описали Провос, Фрідль і Ханімен для OpenSSH [295]. Демон розділяється на привілейований *монітор* і непривілейований *дочірній процес*, що виконує розбір протоколу й автентифікацію від окремого користувача в порожньому `chroot`, а привілейовані дії (перевірка пароля, підпис ключем хоста) запитує в монітора через вузький інтерфейс IPC. За оцінкою авторів, з привілеями виконується лише близько 32 % коду, а низка раніше виявлених вразливостей за такої архітектури не давала б прав root. Для ПЗМБ патерн узагальнюється так (рис. 4.2): компоненти, що отримують ворожий вхід, виконуються без привілеїв у пісочниці, а ключі й таблиці ядра доступні лише малому монітору, що перевіряє кожен запит за явною політикою.

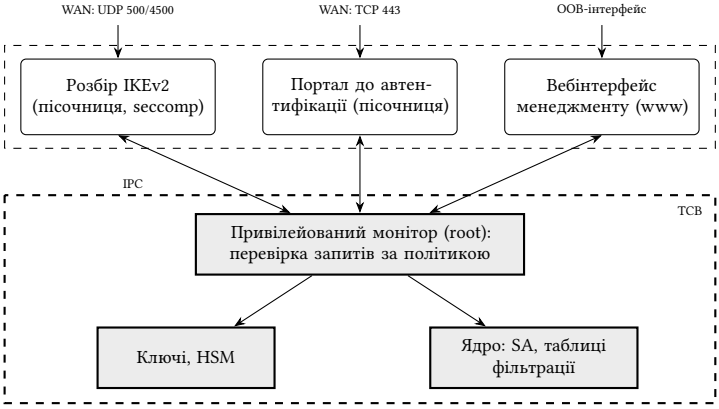


Рис. 4.2 — Архітектура VPN-шлюзу з розділенням привілеїв: непривілейовані процеси (вгорі) і мінімальна довірена база (внизу)

4.4.2 Ізоляція розбору і пісочниці

Ізоляція розбору — окремий випадок розділення привілеїв: найризикованіший код (декодери протоколів, розбір сертифікатів X.509, розпакування архівів правил) виноситься в окремий процес, що отримує байти й повертає структуровані, повторно перевірені дані. У Linux пісочницю будують філь-

трами *seccomp-bpf*, що для кожного системного виклику й аргументів повертають дозвіл, помилку чи завершення процесу [321], і модулем *Landlock*, що дає непривілейованому процесу змогу обмежити собі доступ до файлів і (з ABI 4) до TCP-портів [232]. В OpenBSD для цього є виклики `pledge` (дозволені групи операцій) і `unveil` (видимі шляхи) [292], у FreeBSD — режим можливостей Capsicum: після `cap_enter` процес працює лише з дескрипторами з обмеженими правами [342]. Мінімальну політику для процесу-парсера наведено в лістингу 4.1.

Лістинг 4.1 — Пісочниця seccomp-bpf для процесу-парсера (libseccomp)

```
/* Парсер після ініціалізації лише обмінюється даними з
   монітором через ipc_fd; решта викликів -> SIGSYS     */
#include <seccomp.h>
#include <sys/prctl.h>

int sandbox_parser(int ipc_fd)
{
    if (prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) != 0)
        return -1;                 /* без нових привілеїв */
    scmp_filter_ctx c = seccomp_init(SCMP_ACT_KILL_PROCESS);
    if (c == NULL)
        return -1;
    int rc = 0;
    rc |= seccomp_rule_add(c, SCMP_ACT_ALLOW, SCMP_SYS(read),
              1, SCMP_A0(SCMP_CMP_EQ, (scmp_datum_t)ipc_fd));
    rc |= seccomp_rule_add(c, SCMP_ACT_ALLOW, SCMP_SYS(write),
              1, SCMP_A0(SCMP_CMP_EQ, (scmp_datum_t)ipc_fd));
    rc |= seccomp_rule_add(c, SCMP_ACT_ALLOW,
              SCMP_SYS(exit_group), 0);
    if (rc == 0)
        rc = seccomp_load(c);      /* фільтр не можна зняти */
    seccomp_release(c);
    return rc;                     /* 0 -- пісочницю ввімкнено */
}
```

Політика побудована за принципом «заборонено все, що не дозволено»; аналог для OpenBSD — `pledge("stdio", NULL)`. Ціна ізоляції — копіювання й перемикання контекстів, тому в площині даних її застосовують до складних, але рідкісних об'єктів (прикладний рівень, сертифікати), а не до кожного пакета (див. (1.1)).

4.4.3 Мінімізація TCB, мікроядра й мікросервіси

Довірена обчислювальна база (TCB) — сукупність компонентів, від коректності яких залежить політика безпеки. Її мінімізація робить реальною поглиблену верифікацію. Граничний випадок — мікроядро seL4 з формально доведеною відповідністю реалізації специфікації [225]; у масових продуктах ідею реалізують розкладанням монолітного демона на процеси або мі-

кросервіси. Мікросервіси додають власну поверхню атаки (внутрішні API, оркестратор, секрети), тому їх межі мають бути межами довіри на DFD.

4.4.4 Корінь довіри, оновлення, відмова і площина менеджменту

Ланцюг довіри починається з апаратного *кореня довіри* (незмінний завантажувач, TPM), що перевіряє підпис наступного етапу завантаження, — так виконують вимоги до цілісності завантаження й оновлень (табл. 4.1); архітектуру безпечного оновлення розглянуто в розділі 9. Поведінка за відмови має бути явною вимогою: *fail-secure* блокує трафік, якщо інспекція недоступна, *fail-open* пропускає його; для ME й VPN-шлюзів типовий перший варіант, для IPS у розриві лінії — часто другий із сигналізацією (розділ 14). *Позасмугова (out-of-band) площина менеджменту* — найдешевший захід проти найпоширенішого класу атак на граничні пристрої: у моделі (1.3) вона зменшує r найбільшого компонента.

4.5 Метрики поверхні атаки

4.5.1 Модель Манадгати–Вінг

Поверхня атаки — сукупність ресурсів системи, які порушник може використати для атаки. Манадгата й Вінг [240] формалізували її через три множини ресурсів: *методи* M (точки входу й виходу), *канали* C (сокети, іменовані канали) і *недовірені елементи даних* I (файли, записи конфігурації, доступні порушнику). Внесок ресурсу — відношення *потенціалу шкоди* до *зусиль порушника*:

$$AS = \Big\langle \sum_{m \in M} \frac{dp(m)}{ac(m)},\ \sum_{c \in C} \frac{dp(c)}{ac(c)},\ \sum_{d \in I} \frac{dp(d)}{ac(d)} \Big\rangle, \tag{4.3}$$

де $dp(\cdot)$ — оцінка привілеїв, з якими виконується метод (для каналу — протоколу, для даних — типу), $ac(\cdot)$ — оцінка прав доступу, які порушник має здобути. Версія B має меншу поверхню, ніж A, якщо жодна з трьох сум не більша, а хоча б одна менша. Шкали порядкові й експертні, тому метрика придатна для порівняння варіантів, а не для абсолютних оцінок.

Числовий приклад. Порівняємо VPN-шлюз A (монолітний, усі служби від root, вебінтерфейс і SNMPv2c доступні з WAN) і B (розділення привілеїв за рис. 4.2, позасмуговий менеджмент). Шкала dp: root — 5, непривілейований користувач — 3, пісочниця — 1. Шкала ac: неавтентифікований доступ із WAN — 1, користувач VPN — 3, доступ до мережі менеджменту — 4, адміністратор у ній — 5 (адміністратор на WAN у варіанті A — 4). Результати — у табл. 4.4.

Таблиця 4.4 — Методи поверхні атаки двох варіантів VPN-шлюзу

Точка входу	A: dp/ac	A: внесок	B: dp/ac	B: внесок
IKEv2, UDP 500/4500	5/1	5,00	1/1	1,00
Портал до автентифікації	5/1	5,00	1/1	1,00
Портал після автентифікації	5/3	1,67	3/3	1,00
Вебінтерфейс адміністратора до автентифікації	5/1	5,00	3/4	0,75
Вебінтерфейс і API після автентифікації	5/4	1,25	3/5	0,60
SSH CLI	5/4	1,25	5/5	1,00
SNMPv2c	5/1	5,00	вимкнено	0
Сума за методами		24,17		5,35

SSH CLI у варіанті B зберігає $dp = 5$: CLI адміністратора лишається привілейованим, і зменшення досягають лише підвищенням ac. Для каналів приймемо $dp = 1$ для кожного відкритого сокета: у варіанті A п'ять сокетів на WAN ($ac = 1$) дають 5,0; у варіанті B три сокети на WAN і два на інтерфейсі менеджменту ($ac = 4$) дають $3 + 0{,}5 = 3{,}5$. Отже, $AS_A = \langle 24{,}17;\ 5{,}0;\ \cdot\rangle$, $AS_B = \langle 5{,}35;\ 3{,}5;\ \cdot\rangle$: за методами поверхня зменшилася в 4,5 раза, головно через зниження привілеїв (dp) і підвищення бар'єра доступу (ac), а не через скорочення кількості точок входу.

4.5.2 Зв'язок із моделлю ризику

Поверхня атаки природно пов'язується з досяжністю r_k моделі (1.3). Оскільки атаку визначає найслабша з точок входу E_k компонента k, приймемо евристику

$$r_k = r_0 \cdot \max_{x \in E_k} \frac{dp(x)/ac(x)}{d_{\max}}, \tag{4.4}$$

де r_0 — досяжність для найгіршого випадку (привілейований метод без автентифікації), $d_{\max} = 5$ — максимум відношення на обраних шкалах. Візьмемо параметри прикладу розділу 1 ($\rho\theta e = 0{,}7 \cdot 0{,}03 \cdot 0{,}05 = 1{,}05 \cdot 10^{-3}$ на KLOC), компоненти менеджменту ($S = 500$) і VPN ($S = 150$), $r_0 = 0{,}5$. У варіанті A обидва максимуми дорівнюють 5, $r = 0{,}5$, $\lambda_A = 1{,}05 \cdot 10^{-3} \cdot 0{,}5 \cdot 650 \approx 0{,}341$ і за (1.4) $P_x \approx 0{,}29$. У варіанті B максимуми дорівнюють 1, $r = 0{,}1$, $\lambda_B \approx 0{,}068$, $P_x \approx 0{,}066$ — у 4,4 раза менше. Це модель-евристика з ілюстративними параметрами; вона груба (пісочниця не абсолютна, точки входу можна поєднувати), але переводить архітектурне рішення в ту саму шкалу, що й рішення щодо процесу (розділ 3).

4.5.3 Метрики складності

Поверхню атаки доповнюють метрики складності, що корелюють зі щільністю дефектів ρ. Цикломатична складність Мак-Кейба [243] $V(G) = E -$

$N + 2P$ (ребра, вершини й компоненти зв'язності графа керування) дорівнює кількості лінійно незалежних шляхів; функції розбору з $V(G) > 20$–30 — кандидати на рефакторинг або генерацію парсера з граматики (розділ 5). Корисні також обсяг коду в ТСВ і кількість станів автомата протоколу (розділ 6); у конвеєрі CI/CD їх відстежують як ворота якості.

4.6 Кількісне ранжування заходів

Модель загроз зазвичай породжує більше заходів, ніж дозволяє бюджет випуску. Нехай J — множина кандидатних заходів, c_j — вартість заходу (людино-тижні), ΔR_j — зменшення очікуваних втрат від нього окремо, B — бюджет. Вибір формулюється як **задача про рюкзак** з булевими змінними x_j:

$$\max_{x \in \{0,1\}^{|J|}} \sum_{j \in J} \Delta R_j \, x_j \qquad \text{за умови} \qquad \sum_{j \in J} c_j \, x_j \le B. \tag{4.5}$$

Для десятків заходів задача точно розв'язується динамічним програмуванням за $O(|J|\,B)$; жадібний алгоритм за спаданням $\Delta R_j / c_j$ дає наближення, а багатокритеріальне узагальнення на деревах атак запропонували Деврі та співавтори [149]. Головна пастка (4.5) — *неадитивність*: заходи на одній гілці дерева перекривають один одного, а на різних гілках можуть давати ефект, більший за суму. Тому лінійну модель використовують для відбору, а кращі набори перевіряють за деревом атак.

Числовий приклад. Для дерева атак підрозділу 4.2 приймемо втрати від компрометації сенсора $L = 2000$ тис. грн; тоді $R_0 = p_G L \approx 521$. Кандидатні заходи: M1 — пісочниця декодера ($c = 6$, $\Delta R = 292$); M2 — позасмуговий менеджмент ($c = 3$, $\Delta R = 112$); M3 — перевірка підпису правил ($c = 2$, $\Delta R = 37$); M4 — фазинг декодера, що вдвічі знижує ймовірність вразливості ($c = 5$, $\Delta R = 162$); M5 — багатофакторна автентифікація адміністратора, $0{,}25 \to 0{,}05$ ($c = 2$, $\Delta R = 96$). Кожне ΔR_j обчислено за (4.2) за зміни одного листка. За $B = 10$ розв'язок (4.5) — набір {M1, M3, M5} з адитивною оцінкою 425; точне обчислення за деревом дає 455 ($p_G = 0{,}033$), тобто ефект на різних гілках є нададитивним. Натомість набір {M2, M4, M5} з адитивною оцінкою 370 насправді дає лише 294: M2 і M5 діють на одну гілку A2 і дублюють одне одного. Жадібний алгоритм ($\Delta R/c$: 48,7; 48,0; 37,3; 32,5; 18,6) тут знаходить той самий оптимум. Обов'язкові заходи (NDcPP, CRA) входять до розв'язку безумовно й лише зменшують бюджет.

4.7 Огляд проєкту безпеки й архітектурні рішення

Огляд проєкту безпеки (security design review) — формальна перевірка архітектури незалежними від авторів фахівцями до масової реалізації. Вхідні артефакти — DFD, модель загроз, матриця трасованості, розрахунок поверх-

ні атаки; вихідні — зауваження за критичністю й рішення «прийнято / з умовами / повернуто». Огляд відповідає практиці SD-3 IEC 62443-4-1 [197] і задачі PW.2 SSDF [328], у CC — сімейству ADV (розділ 2), і зосереджується на перетинах меж довіри, поведінці за відмов і припущеннях щодо середовища.

Рішення фіксують як **записи архітектурних рішень** (ADR) за шаблоном Найгарда [272]: назва, статус, контекст, рішення, наслідки. Для безпекових рішень додають посилання на загрози, розглянуті альтернативи з оцінками за (4.3) чи (4.5) і умови перегляду (наприклад, «переглянути fail-open для IPS, якщо сенсор розгортатимуть на межі мережі»). ADR у репозиторії разом із моделлю загроз як кодом стають доказами гарантування (розділ 17).

4.8 Кейс: модель загроз і архітектура IDS-сенсора

4.8.1 Постановка

Український виробник розробляє IDS-сенсор для мереж державних органів і операторів критичної інфраструктури на основі відкритого рушія виявлення з власними декодерами промислових протоколів. Сенсор отримує копію трафіку з дзеркального порту, передає сповіщення в SIEM замовника, оновлює правила з сервера виробника і керується через REST API. Модель порушника охоплює неавтентифікованого порушника в мережі, порушника в ланцюгу постачання правил і державного порушника (розділ 1). Продукт має відповідати NDcPP з модулем IPS і вимогам до кіберзахисту об'єктів критичної інфраструктури [18].

4.8.2 Модель загроз

DFD сенсора з п'ятьма межами довіри наведено на рис. 4.1. Аналіз STRIDE-per-interaction для перетинів меж і STRIDE-per-element для процесів дав 40 загроз; найвагоміші з них і обрані заходи подано в табл. 4.5.

Таблиця 4.5 — Основні загрози IDS-сенсора за STRIDE і заходи

Елемент	Кат.	Загроза	Захід
Мережа → захоплення (МД1)	D	переповнення кільцевого буфера й «сліпе вікно»	лічильники втрат із сигналізацією, балансування за потоками (розділ 8)
Декодер	E, T	помилка пам'яті в розборі протоколу → виконання коду	окремий процес у пісочниці (лістинг 4.1), фазинг (розділ 13), Rust для нових декодерів
Декодер, рушій	D	алгоритмічна складність відновлення потоків і регулярних виразів	квоти пам'яті й часу на потік, рушії зіставлення з лінійним часом
Менеджмент (МД3)	S, E	обхід автентифікації, ін'єкція команд	позасмуговий інтерфейс, MFA, жодних викликів оболонки, ASVS рівня 2
Правила (МД5)	T	підміна пакета правил	підпис правил, захист від відкату версії
Журнали	R, T	знищення слідів після компрометації	потокове передавання в SIEM, ланцюжок гешів записів
Експорт (МД4)	I, S	розкриття персональних даних телеметрії; підміна SIEM	TLS із взаємною автентифікацією (FTP_ITC.1), псевдонімізація (LINDDUN)

4.8.3 Архітектура і поверхня атаки

Цільова архітектура реалізує патерни підрозділу 4.4: захоплення й рушій виявлення працюють від окремого користувача (`CAP_NET_RAW` лише на етапі ініціалізації); декодери складних протоколів винесено в пул процесів-парсерів під фільтром seccomp; API менеджменту доступне лише на позасмуговому інтерфейсі й виконується без привілеїв; підпис правил перевіряє малий монітор, а розбирає їх непривілейований процес. Методи поверхні атаки за (4.3) на шкалах підрозділу 4.5 *до* змін: декодер від root (5/1), API до автентифікації на всіх інтерфейсах (5/1) і після неї (5/4), отримання правил HTTP без підпису, доступне посереднику (5/1), — разом 16,25; *після*: $1/1 + 3/4 + 3/5 + 3/5 = 2{,}95$, тобто в 5,5 раза менше. За деревом атак імовірність досягнення мети знижується з 0,26 до 0,02–0,03.

4.8.4 План впровадження

Бюджет випуску — 10 людино-тижнів. Захід М5 (MFA адміністратора) обов'язковий за Secure by Design, тож вільний бюджет дорівнює 8, і розв'язок (4.5) — М1 і М3, тобто оптимальний набір {М1, М3, М5} ($p_G \approx 0{,}033$). М2 і М4 переносяться на наступний випуск; з М2 $p_G \approx 0{,}020$. Рішення оформлено як ADR із посиланнями на табл. 4.5; модель загроз описано мовою pytm і перевіряють у конвеєрі (новий потік через межу довіри без вимоги блокує злиття), а повнота трасування (4.1) зросла з 0,725 до 0,95

завдяки негативним тестам. Залишкові ризики — вихід із пісочниці через вразливість ядра і компрометацію ключа підпису правил — явно прийнято з умовою перегляду після впровадження апаратного кореня довіри.

Висновки до розділу

Вимоги безпеки до ПЗМБ формуються з NDcPP, OWASP ASVS 5.0, IEC 62443-4-2 і вимог-зловживань; їхня цінність визначається перевірюваністю й трасованістю, яку характеризує повнота (4.1). Моделювання загроз будують на DFD з межами довіри й між площинами та процесами; STRIDE-per-interaction зосереджує аналіз на цих межах, LINDDUN покриває приватність телеметрії, моделювання як код утримує модель актуальною. Кількісні дерева атак (4.2) показали, що архітектурні заходи можуть на порядок зменшити ймовірність досягнення мети, не змінюючи мінімальної вартості атаки, тож опортуністичного й цілеспрямованого порушника слід аналізувати окремо. Принципи безпечного проєктування втілюються в розділенні привілеїв, ізоляції розбору, мінімізації TCB, корені довіри та позасмуговій площині менеджменту. Метрика Манадгати–Вінг (4.3) з евристикою (4.4) переводить ці рішення в шкалу ризику розділу 1 (для VPN-шлюзу — зменшення в 4,5 і 4,4 раза). Вибір заходів зводиться до задачі про рюкзак (4.5), розв'язки якої через неадитивність ефектів слід перевіряти на моделі загроз; огляд проєкту й ADR перетворюють результати на докази гарантування.

Питання для самоперевірки

1. Чим функціональні вимоги безпеки відрізняються від вимог гарантій? Чому для ПЗМБ доцільно виділяти вимоги стійкості?
2. Сформулюйте три перевірювані вимоги до вебінтерфейсу керування VPN-шлюзу на основі OWASP ASVS 5.0 і вкажіть, як кожну буде перевірено.
3. Як структуровано вимоги IEC 62443-4-2? Що означають CR, EDR, NDR і рівні SL-C?
4. Побудуйте випадок зловживання для порталу SSL-VPN і виведіть з нього вимогу-зловживання та негативний тест.
5. Обчисліть повноту трасування (4.1) для моделі з 60 загрозами, з яких 50 покрито вимогами, а для 42 існують тести. Як урахувати різну критичність загроз?
6. Чим STRIDE-per-element відрізняється від STRIDE-per-interaction? Застосуйте обидва до межі МДЗ (рис. 4.1).
7. Для дерева атак підрозділу 4.2 обчисліть p_G і c_G, якщо впроваджено лише MFA адміністратора ($0{,}25 \to 0{,}05$, вартість листка $3 \to 7$).

8. Які категорії LINDDUN застосовні до телеметрії NDR-сенсора? Запропонуйте заходи для кожної.
9. Опишіть архітектуру розділення привілеїв OpenSSH. Які дії виконує монітор і чому інтерфейс IPC має бути вузьким?
10. Порівняйте seccomp-bpf, Landlock, pledge/unveil і Capsicum за моделлю обмежень і зручністю для розробника. Які виклики треба додати до лістингу 4.1 для парсера з динамічним виділенням пам'яті?
11. Перерахуйте табл. 4.4 для варіанта B з увімкненим SNMPv3 на OOB-інтерфейсі (root, $ac = 5$). Як зміниться (4.4)?
12. Розв'яжіть задачу (4.5) для прикладу підрозділу 4.6 за бюджету $B = 8$ і перевірте результат точним обчисленням за деревом атак.
13. Дослідницьке: запропонуйте методику калібрування шкал dp і ac метрики (4.3) за історичними даними про вразливості граничних пристроїв (CVE, CWE, KEV) і перевірте гіпотезу про кореляцію поверхні атаки з кількістю експлуатованих вразливостей.
14. Дослідницьке: сформулюйте задачу вибору контрзаходів на дереві атак із неадитивними ефектами та двома моделями порушника (ймовірнісною й вартісною) як задачу багатокритеріальної оптимізації; запропонуйте алгоритм і оцініть його складність.

РОЗДІЛ 5

Безпечне програмування мережевого програмного забезпечення

Вимоги й архітектура (розділ 4) визначають, *де* у ПЗМБ проходять межі довіри; цей розділ розглядає, *як* писати код компонентів, що отримують ворожий вхід, щоб дефекти не ставали вразливостями. Розглянуто класи дефектів за CWE Top 25, безпеку пам'яті й модель переходу на мови з безпечною пам'яттю, LangSec, цілочислову арифметику, конкурентність, обробку помилок, стандарти кодування, огляд коду, посилення збирання і кейс Suricata. Криптографічні аспекти реалізації розглянуто в розділі 6, статичний аналіз — у розділі 12, санітайзери й фазинг — у розділі 13.

5.1 Класи дефектів мережевого коду

Рейтинг CWE Top 25 2025 р. (опубліковано 15.12.2025) [143] очолюють слабкості вебзастосунків: міжсайтовий скриптинг (CWE-79), SQL-ін'єкція (CWE-89), підробка міжсайтових запитів (CWE-352), відсутня авторизація (CWE-862). Далі йдуть класи, типові для коду на C/C++: запис за межами буфера (CWE-787, 5-те місце), використання після звільнення (use-after-free, CWE-416, 7-ме), читання за межами (CWE-125, 8-ме), класичне, стекове й купове переповнення (CWE-120, 121, 122), розіменування нульового вказівника (CWE-476, 13-те). Для ПЗМБ актуальні обидві половини рейтингу: площина даних написана переважно мовами без безпечної пам'яті, а площина менеджменту є вебзастосунком із привілеями root, де трапляються впровадження команд (CWE-78, 9-те; CWE-77, 23-тє) і обхід шляху (CWE-22, 6-те).

Цілочислове переповнення (CWE-190) до рейтингу 2025 р. не увійшло, хоча в мережевому коді воно є типовою *першопричиною* запису чи читання за межами: CVE класифікують за кінцевим наслідком, а не за першою помилкою ланцюга. Для аналізу першопричин (розділ 16) це означає, що виправлення лише перевірки межі без виправлення арифметики залишає варіанти вразливості. Натомість до рейтингу потрапило неконтрольоване виділення ресурсів (CWE-770, 25-те), що для IDS і ME означає виснаження таблиць станів (розділ 14). OWASP Top 10:2025 виділив категорію A10 «Неправильна обробка виняткових умов» [281]: необроблені помилки й «відкрита» поведінка за збою (відповідні слабкості — CWE-703, 754, 755, 636), що для засобу захисту рівнозначно вимкненню його функції. Узагальнення наведено в табл. 5.1.

Таблиця 5.1 — Класи дефектів мережевого коду (ранг за CWE Top 25 2025 р.)

Клас	CWE (ранг)	Типовий прояв у ПЗМБ	Основний захід
Запис/читання за межами	787 (5), 125 (8), 120 (11)	довжина поля TLV чи розширення не звірена з отриманим обсягом	розпізнавач до обробки; перевірка меж мовою
Використання після звільнення	416 (7), 415	стан сеансу звільнено, а таймер чи асинхронний колбек його використовує	модель власності; єдиний шлях очищення
Цілочислові помилки	190, 191, 197	переповнення суми зміщення й довжини; звуження `u32`→`u16`	перевірена арифметика (5.3)
Нульовий вказівник	476 (13)	необов'язкове поле відсутнє	типи-опції
Гонитви, TOCTOU	362, 367	перевірка правила і його застосування розділені в часі	атомарна заміна, RCU
Обробка винятків	703, 755; OWASP A10	помилка декодера → пакет пропускається без інспекції	fail-secure
Впровадження команд	78 (9), 77 (23)	параметр вебінтерфейсу потрапляє в командний рядок оболонки	виклик без оболонки, білий список
Виснаження ресурсів	770 (25)	таблиця станів, черга фрагментів без лімітів	квоти, тайм-аути, витіснення

За класифікацією Секереша та співавторів [334] атака на пам'ять починається з *просторового* (вихід за межі об'єкта) або *часового* (доступ до звільненого чи неініціалізованого об'єкта) порушення, після чого зіпсований вказівник або дані використовуються для зміни потоку керування, витоку чи підміни даних, від яких залежить рішення безпеки. Мова з безпечною пам'яттю розриває цей ланцюг на першому етапі детерміновано, засоби посилення (підрозділ 5.7) — на пізніших і ймовірнісно; у моделі (1.3) перші знижують імовірність появи вразливості, другі — досяжність і наслідки. Українські дослідники на прикладі переповнення буфера також обґрунтовують вбудовування заходів на етапах проєктування й кодування [8].

5.2 Безпека пам'яті та мови з безпечною пам'яттю

5.2.1 Масштаб проблеми й державні настанови

За даними Microsoft, близько 70 % вразливостей, яким компанія щороку присвоює CVE, є помилками безпеки пам'яті [250]; таку саму частку дає Chromium, і Google називає її «галузевою нормою» [339]. В Android, де з 2019 р. новий системний код пишуть переважно мовами з безпечною пам'яттю, частка вразливостей пам'яті знизилася з 76 % у 2019 р. до 24 %

у 2024 р. [339] і нижче 20 % у 2025 р. Щільність вразливостей пам'яті в коді на Rust приблизно в 1000 разів нижча, ніж у C/C++, а зміни на Rust мають у 4 рази менше відкочувань [338].

Спільний документ CISA, NSA, FBI і агенцій «П'яти очей» «The Case for Memory Safe Roadmaps» (грудень 2023 р.) закликає виробників публікувати *дорожні карти безпеки пам'яті*: які компоненти й у які строки переводяться на мови з безпечною пам'яттю і які компенсаційні заходи діють для решти [119]. Перелік небезпечних практик CISA (версія 2) вважає небезпечною відсутність такої карти до 01.01.2026 для наявних продуктів [117]. Інформаційний лист NSA і CISA (червень 2025 р.) аналізує перешкоди переходу (продуктивність, сумісність, кадри) і рекомендує поступову інтеграцію замість повного переписування [271].

5.2.2 Rust: власність, запозичення, час життя

Rust забезпечує безпеку пам'яті без збирача сміття трьома механізмами, які перевіряє компілятор. **Власність** (ownership): кожне значення має одного власника і звільняється детерміновано, коли той виходить з області видимості; передавання переміщує власність, тож повторне звільнення неможливе. **Запозичення** (borrowing): посилання підкоряються правилу «або одне змінюване, або будь-яка кількість незмінних» (aliasing XOR mutability), що усуває використання після звільнення й гонитви даних. **Час життя** (lifetimes): компілятор доводить, що посилання не переживає об'єкт; парсер може повертати зрізи вхідного буфера без копіювання (лістинг 5.1). Індексування перевіряє межі, а `Option` і `Result` замінюють нульові вказівники й коди повернення, які можна проігнорувати.

Межі гарантій такі. По-перше, блоки `unsafe` (сирі вказівники, FFI до бібліотек на C, апаратура) вимикають частину перевірок. RustBelt формально довів обґрунтованість ядра системи типів і низки бібліотек з `unsafe` усередині (`Arc`, `Rc`, `RefCell`, `Mutex`) [218], але для власного `unsafe`-коду доведення — обов'язок автора; Google описала перший «майже інцидент» Android саме в `unsafe`-коді [338]. Тому `unsafe` мінімізують і ізолюють за безпечним інтерфейсом. По-друге, Rust не захищає від логічних помилок і помилок кінцевих автоматів протоколу. По-третє, у збиранні випуску переповнення за замовчуванням не перевіряється (вмикають `overflow-checks` або `checked_*`). По-четверте, паніка для мережевого демона означає відмову в обслуговуванні (підрозділ 5.5).

5.2.3 Go, сучасний C++ і CHERI

Go гарантує безпеку пам'яті збирачем сміття й перевіркою меж і поширений у площині менеджменту, проксі й агентах. Обмеження: паузи збирача сміття, гонитви на багатослівних значеннях, що можуть поруши-

ти безпеку пам'яті, неперевірене переповнення. *Сучасний C++* не примушує використовувати розумні вказівники й `std::span`; найдієвіший захід — *посилені режими* стандартних бібліотек (`_LIBCPP_HARDENING_MODE`, `_GLIBCXX_ASSERTIONS`), що перевіряють межі в `operator[]`, `span` тощо за малих витрат [276]; *профілі безпеки* C++ до C++26 не увійшли, тож на них не варто розраховувати. Для об'єктів сеансу корисні *арени* — звільнення всіх об'єктів разом, що зменшує кількість місць можливої часової помилки [59]. *CHERI* замінює вказівники апаратними *можливостями* (capabilities) з межами й правами, що перевіряються під час кожного доступу [343]; після перекомпіляції наявний код на C/C++ отримує просторову безпеку й дрібнозернисту компартменталізацію, що перспективно для прошивок, які неможливо переписати. Порівняння — у табл. 5.2.

Таблиця 5.2 — Гарантії підходів до безпеки пам'яті

Підхід	Просторова	Часова	Гонитви даних	Ціна
C + посилення	частково, ймовірнісно	ні (MTE — ймовірнісно)	ні	мала
C++, посилена бібліотека	контейнери, `span`	частково (RAII)	ні	мала
Rust (безпечна частина)	так	так	так	навчання, FFI
Go	так	так (GC)	детектор у тестах	GC-паузи
CHERI	так (апаратно)	частково	ні	апаратура

5.2.4 Кількісна модель поступової міграції

Google [339], спираючись на дослідження Алексопулоса та співавторів [67], показала, що час життя вразливостей розподілений приблизно експоненційно: вони зосереджені в новому коді, а щільність латентних вразливостей у коді віку a спадає як

$$\rho(a) = \rho_0 \, e^{-a/\tau}, \tag{5.1}$$

де ρ_0 — щільність у новому коді (на KLOC); τ — середній час життя вразливості. Код віком 5 років має в 3,4 (за даними [67]) — 7,4 раза (за даними Android і Chromium) нижчу щільність, ніж новий. З (5.1) $e^{5/\tau} = 3{,}4$ дає $\tau = 5/\ln 3{,}4 \approx 4{,}1$ року, що збігається з виміряним середнім часом життя близько 4 років; $e^{5/\tau} = 7{,}4$ дає $\tau \approx 2{,}5$ року.

Нехай щороку додається L KLOC коду, частка якого мовами без безпечної пам'яті дорівнює $u(s)$, а відносна щільність вразливостей пам'яті в коді безпечними мовами — $\varepsilon \ll 1$. Інтенсивність виявлення вразливостей пам'яті в рік t складається з внесків усіх «когорт» коду; для різкого перехо-

ду ($u = 1$ до $t = 0$, $u = 0$ після)

$$V(t) = \frac{\rho_0 L}{\tau} \int_{-\infty}^{t} \big[u(s) + \varepsilon(1 - u(s))\big] e^{-\frac{t-s}{\tau}}\, ds = \rho_0 L \big[e^{-t/\tau} + \varepsilon(1 - e^{-t/\tau})\big]. \tag{5.2}$$

Отже, без переписування старого коду інтенсивність спадає експоненційно: старий код «дозріває», а новий майже не вносить вразливостей пам'яті.

Приклад. Для $\tau = 2{,}5$ року через 5 років $V/V_0 = e^{-2} \approx 0{,}135$ (зниження на 86 % за незмінного обсягу коду на C/C++), для $\tau = 4{,}1$ — $e^{-1{,}22} \approx 0{,}30$ (рис. 5.1). Для Android (частка 0,76 у 2019 р.) решта вразливостей зі сталою інтенсивністю M дає $M/V_0 \approx 0{,}32$, і через 5 років модельна частка становить $0{,}135/(0{,}135 + 0{,}32) \approx 0{,}30$ для $\tau = 2{,}5$ і $0{,}48$ для $\tau = 4{,}1$; фактичні 24 % ще кращі, що Google пояснює паралельним посиленням коду на C++. Висновок для дорожньої карти: найбільший ефект дає переведення *нового* коду, а переписування старого виправдане вибірково (критерій (5.5)).

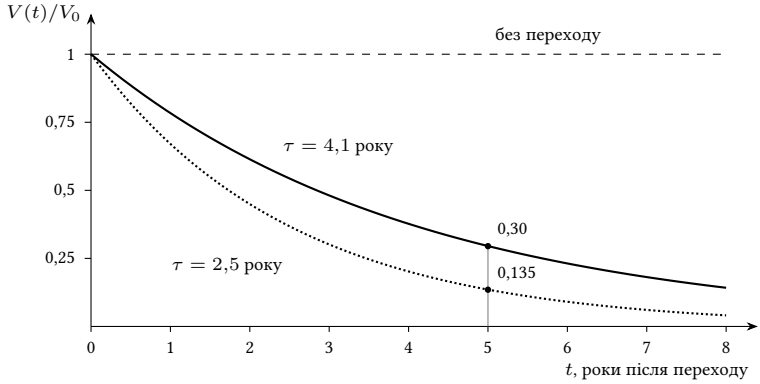


Рис. 5.1 — Спадання інтенсивності вразливостей пам'яті після переходу нового коду на мову з безпечною пам'яттю за моделлю (5.2) ($\varepsilon = 0{,}001$)

5.3 Безпечний розбір ворожого вводу

5.3.1 Підхід LangSec

Мова з безпечною пам'яттю не усуває помилок розуміння повідомлення. Сассаман, Паттерсон, Братус і Локасто сформулювали підхід **безпеки на основі теорії мов** (LangSec) [315]: множина допустимих повідомлень є *формальною мовою*, а код, що їх обробляє, — її *розпізнавачем*. Розпорошене по коду обробки розпізнавання називають **«рушничним» парсером** (shotgun parser): перевірки перемежовуються з обчисленнями, тож частина даних

використовується до перевірки інших, як поле довжини в Heartbleed (розділ 1). Принципи LangSec для розробника ПЗМБ:

1. *Явна граматика*: формат задається специфікацією (ABNF, ASN.1, опис Kaitai Struct), з якої генерують або за якою перевіряють розпізнавач.
2. *Мінімальна обчислювальна потужність*: за ієрархією Хомського бажано обмежитися регулярними або детермінованими контекстно-вільними мовами, для яких еквівалентність двох розпізнавачів розв'язна; для загальніших мов вона нерозв'язна, тож ME і захищений ним сервер можуть по-різному розуміти те саме повідомлення, що є основою ухилення від IDS і контрабанди запитів (розділ 8). Поля довжини формально виводять за межі контекстно-вільних мов, але безпечні, якщо обробляються в одному місці з лімітом.
3. *Повне розпізнавання перед обробкою*: обробник отримує лише типізований об'єкт, а повідомлення, що не належить мові, відкидається цілком.
4. *Однозначність і неподатливість*: кожен вхід має не більше одного розбору, а для автентифікованих форматів серіалізація й розбір взаємно обернені, тож два різні об'єкти не мають однакового подання.

Архітектуру показано на рис. 5.2; її поєднання з ізоляцією розбору в окремому процесі розглянуто в розділі 4.

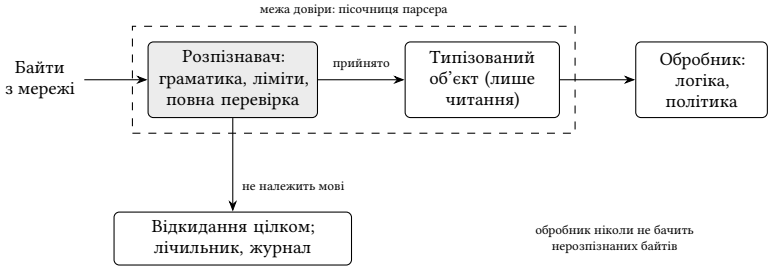


Рис. 5.2 — Архітектура «розпізнавач → обробник» за принципами LangSec (за [315])

5.3.2 Інструменти та приклад розпізнавача

Бібліотеки комбінаторів будують розпізнавач зі складених примітивів (прочитати u16, взяти n байтів, повторити): Hammer для C, nom для Rust. Шифл'є і Купрі показали на парсерах для VLC і Suricata, що nom дає розбір без копіювання з продуктивністю, порівнянною з ручним кодом на C [112]. *Декларативні генератори* (Kaitai Struct, Spicy у Zeek) відокремлюють специфікацію формату від реалізації. *Верифіковані парсери* дають найсильніші гарантії: EverParse [297] генерує з опису формату код на C з машинно пере-

віреним (у F*) доведенням безпеки пам'яті, коректності щодо специфікації й неподатливості; його застосовано до 293 типів повідомлень TLS 1.0–1.3, форматів Bitcoin і ASN.1 DER із продуктивністю, вищою за ручні парсери.

Лістинг 5.1 — ручний розпізнавач запису TLV на безпечному Rust: межі перевіряються до доступу до даних, арифметика зміщень перевірена (`checked_add`), звуження явне (`TryFrom`), значення повертається зрізом буфера, час життя якого контролює компілятор. На відміну від версії на C з розділу 1, пропущена перевірка тут дає помилку компіляції або контрольовану паніку, а не читання чужої пам'яті.

Лістинг 5.1 — Розпізнавач запису TLV на Rust без `unsafe`

```
/// Запис: тип (1 Б) | довжина (4 Б, big-endian) | значення
#[derive(Debug, PartialEq)]
pub enum ParseErr { Short, TooLong }
pub struct Tlv<'a> { pub typ: u8, pub val: &'a [u8] }

const MAX_VAL: usize = 16 * 1024;     // ліміт із вимог

/// Розпізнає запис з позиції off; повертає запис і
/// зміщення наступного. Межі перевіряє get(), а не індекс.
pub fn parse_at(buf: &[u8], off: usize)
    -> Result<(Tlv<'_>, usize), ParseErr>
{
    let h_end = off.checked_add(5).ok_or(ParseErr::Short)?;
    let hdr = buf.get(off..h_end).ok_or(ParseErr::Short)?;
    let lb = <[u8; 4]>::try_from(&hdr[1..])
        .map_err(|_| ParseErr::Short)?;
    let len = usize::try_from(u32::from_be_bytes(lb))
        .map_err(|_| ParseErr::TooLong)?;
    if len > MAX_VAL {
        return Err(ParseErr::TooLong); // до доступу до даних
    }
    let end = h_end.checked_add(len).ok_or(ParseErr::Short)?;
    let val = buf.get(h_end..end).ok_or(ParseErr::Short)?;
    Ok((Tlv { typ: hdr[0], val }, end))
}
```

Модуль парсера позначають `#![forbid(unsafe_code)]`, а обв'язку фазингу (розділ 13) будують над `parse_at`.

5.4 Цілочислова арифметика

5.4.1 Правила

У C беззнакова арифметика «загортається» за модулем 2^w без сигналу, знакове переповнення є невизначеною поведінкою (компілятор може вилучити перевірку, що на неї спирається), а неявні перетворення змінюють знак або обрізають значення. SEI CERT C містить відповідні правила INT30-C, INT31-C і INT32-C [322]. Настанови для мережевого коду:

- довжини й зміщення — лише беззнакові типи розміру (`size_t`, `usize`); поля з мережі розширюються до них явно;
- суму, що може переповнитися, не обчислюють до перевірки, а умову формулюють через віднімання від відомої межі (5.3);
- додавання й множення перевіряють вбудованими `__builtin_add_overflow`/`__builtin_mul_overflow` (GCC, Clang; у C23 — `ckd_add`/`ckd_mul`);
- звуження — лише після перевірки діапазону; у Rust — `TryFrom`, а не `as`;
- у Rust семантику обирають явно: `checked_*` (помилка), `saturating_*` (лічильники), `wrapping_*` (лише арифметика за модулем, як порядкові номери TCP).

5.4.2 Умова безпечного обчислення зміщення й розміру

Нехай n — розмір буфера, o — зміщення, ℓ — довжина фрагмента, c — кількість елементів розміру z; усі величини — беззнакові w-бітові, $M = 2^w - 1$. Доступ до $[o, o+\ell)$ і виділення $c \cdot z$ байтів безпечні тоді й лише тоді, коли

$$o \le n \ \wedge \ \ell \le n - o, \qquad z = 0 \ \vee \ c \le \lfloor M/z \rfloor \,. \tag{5.3}$$

Обидві умови обчислюються без переповнення, на відміну від наївної $o + \ell \le n$.

Приклад. Для 32-бітового `size_t`, кадру $n = 1500$ і заданих порушником $o =$ `0xFFFFFFF0`, $\ell =$ `0x20` маємо $(o + \ell) \bmod 2^{32} = 16 \le 1500$: наївна перевірка пропускає доступ майже на 4 ГіБ за межі, тоді як за (5.3) уже $o \le n$ хибне. Для $c = 2^{30} + 1$ записів по $z = 4$ байти добуток за модулем 2^{32} дорівнює 4, тож виділяється 4 байти під понад мільярд записів; умова $c \le \lfloor (2^{32} - 1)/4 \rfloor = 1\,073\,741\,823$ порушена, і запит відхиляється. Реалізацію наведено в лістингу 5.2.

Лістинг 5.2 — Перевірена арифметика зміщень і звуження на C

```
/* Безпечний діапазон: [off, off+len) у буфері розміру n;
   суму off+len не обчислюємо -- вона може загорнутися   */
static inline bool range_ok(size_t n, size_t off, size_t len)
{
    return off <= n && len <= n - off;
}

/* Кінець поля base+off+len з перевіркою перенесення;
   -1 -- переповнення або вихід за межі буфера n       */
int field_end(size_t n, size_t base, uint32_t off,
              uint32_t len, size_t *end)
{
    size_t t;
    if (__builtin_add_overflow(base, (size_t)off, &t) ||
```

```
        __builtin_add_overflow(t, (size_t)len, &t) || t > n)
        return -1;
    *end = t;
    return 0;
}
/* Звуження лише після перевірки діапазону (CERT INT31-C) */
static inline bool to_u16(uint32_t v, uint16_t *out)
{
    if (v > UINT16_MAX) return false;
    *out = (uint16_t)v; return true;
}
```

UBSan (розділ 13) виявляє загортання під час тестування, але не замінює явних перевірок.

5.5 Конкурентність, обробка помилок і керування ресурсами

5.5.1 Гонитви й атомарність

Площина даних обробляє пакети паралельно, а площина керування тим часом змінює правила й ключі. **Гонитва даних** — несинхронізований одночасний доступ двох потоків до пам'яті, з яких хоча б один записує; у C/C++ це невизначена поведінка. **TOCTOU** — логічна гонитва, за якої перевірка і використання розділені в часі: ME перевіряє пакет за однією версією правил, а дію бере з іншої; демон перевіряє файл конфігурації, а відкриває вже підмінений. Типові рішення: *розподіл стану за ядрами* (RSS за гешем 5-кортежу); *атомарна заміна конфігурації* — новий набір правил перевіряється окремо й публікується однією заміною вказівника, а стара версія звільняється після завершення читачів (RCU); *структури без блокувань* (`rte_ring` у DPDK) — лише перевірені бібліотечні; *дескриптори замість шляхів* (`openat`, `fstat`). Rust статично забороняє гонитви даних через типажі `Send` і `Sync`, але не логічні гонитви й взаємні блокування; для Go і C/C++ гонитви шукають детекторами (розділ 13).

5.5.2 Безпечна обробка помилок і ресурсів

З категорії A10:2025 [281] для ПЗМБ випливають такі правила.

1. *Fail-secure*: помилка розбору чи тайм-аут інспекції ведуть до відкидання пакета; fail-open — лише як явна вимога (розділ 14).
2. *Жодна помилка не ігнорується* (CERT ERR33-C [322]); `unwrap` на ворожому вході забороняє лінтер.
3. *Без розкриття внутрішньої інформації* (CWE-209): деталі — лише в журнал.
4. *Звільнення ресурсів на всіх шляхах*: `goto cleanup` у C, RAII/`Drop`, `defer` у Go; секрети затираються.

5. *Стратегія паніки*: `panic=abort` з перезапуском наглядачем або перехоплення на межі з'єднання; паніка видима в моніторингу.

Кожен ресурс, витрату якого контролює порушник, має ліміт, що перевіряється *до* виділення: розмір повідомлення (`MAX_VAL` у лістингу 5.1), кількість станів, буферизація фрагментів і потоків TCP, глибина рекурсії розбору; за досягнення ліміту діє явна політика витіснення. Проти атак на *алгоритмічну складність* застосовують ключовані геш-функції (SipHash у Rust) і рушії регулярних виразів з лінійним часом (розділи 7, 14).

5.6 Стандарти кодування й огляд коду

5.6.1 Стандарти й настанови

SEI CERT C/C++ [322] — правила й рекомендації за темами (INT, ARR, STR, MEM, ERR, CON) з оцінкою ризику кожного правила; найпридатніші для мережевого ПЗ загального призначення. *ISO/IEC TS 17961* формулює правила для C так, щоб їх порушення виявлялися аналізатором. *MISRA C:2023* обмежує мову підмножиною з передбачуваною поведінкою з документованими відхиленнями; доречний для прошивок і промислових шлюзів, для яких IEC 62443-4-1 [197] вимагає визначених правил кодування (для C++ — MISRA C++:2023, AUTOSAR C++14, JSF AV C++). Для Rust — Rust API Guidelines (типи-обгортки, `TryFrom`, документовані інваріанти `unsafe`) і правила Clippy проти `unwrap` і неперевіреного індексування; для Go — `go vet`, `gosec`, `govulncheck`. Стандарт корисний лише як *автоматично перевірювана вимога* — ворота якості конвеєра (розділ 3) з документованими відхиленнями.

5.6.2 Ефективність огляду коду

Огляд коду виявляє вади, недоступні аналізаторам (модель довіри, авторизація, стани протоколу). Кемерер і Полк на даних 371 і 246 програм процесу PSP показали, що швидкість огляду значуще впливає на його ефективність навіть з урахуванням кваліфікації: за швидкості не більше 200 рядків/год огляд коду виявляв понад половину дефектів, огляд проєкту — майже дві третини [221]. Залежність від швидкості r (рядків/год) опишемо моделлю пошуку з насиченням:

$$E(r) = 1 - e^{-k/r}, \qquad E_{\Sigma} = 1 - \prod_{i} \bigl(1 - E_i\bigr), \tag{5.4}$$

де k — інтенсивність виявлення (рядків/год), що залежить від кваліфікації й чек-листа; E_{Σ} — сукупна ефективність незалежних методів з ефективностями E_i (за кореляції методів — верхня межа).

Приклад. Калібрування $E(200) = 0{,}55$ дає $k = 200\ln(1/0{,}45) \approx 160$, звідки $E(100) \approx 0{,}80$, $E(400) \approx 0{,}33$. Для модуля 10 KLOC з 40 дефектами

огляд на швидкості 200 рядків/год (50 год) виявляє 22 дефекти, на швидкості 400 (25 год) — 13: далі проходять 27 дефектів замість 18, і їх пізнє виправлення (3.2) коштує більше за зекономлені години. Поєднання огляду (0,55), SAST (0,30) і фазингу (0,40) дає $E_{\Sigma} = 1 - 0{,}45 \cdot 0{,}70 \cdot 0{,}60 \approx 0{,}81$, що узгоджується з потребою комбінувати методи для DRE понад 85 % (3.7).

Чек-лист огляду випливає з табл. 5.1: чи перевірена кожна довжина за (5.3), хто звільняє кожен об'єкт, що відбувається на шляхах помилок, які ліміти мають ресурси. Українські дослідники на фрагментах коду C# показали, що гібридний підхід, у якому велика мовна модель підтверджує або спростовує звіти статичного аналізатора, перевищує за ефективністю кожен засіб окремо [61]; проте результат моделі — підказка рецензентові, а не заміна огляду (розділ 17).

5.7 Засоби посилення під час компіляції та виконання

5.7.1 Механізми

Для коду, що залишається на C/C++, засоби посилення (hardening) не усувають дефектів, але перетворюють частину з них на аварійне завершення замість керованої експлуатації. Механізми ОС (ASLR, DEP/NX) діють для всіх програм, решту вмикає компілятор (табл. 5.3, лістинг 5.3, за настановою OpenSSF [276]). `_FORTIFY_SOURCE=3` через `__builtin_dynamic_object_size` перевіряє в `memcpy`, `strcpy` тощо й розміри, відомі лише під час виконання. *Цілісність потоку керування* (CFI) буває програмною (Clang CFI, потребує LTO) і апаратною: Intel CET (IBT і тіньовий стек адрес повернення), ARM PAC (криптографічна автентифікація вказівників) і BTI. *ARM MTE* порівнює 4-бітовий тег кожних 16 байтів пам'яті з тегом у вказівнику: за випадкових тегів невідповідність виявляється з імовірністю $15/16 \approx 0{,}94$.

Таблиця 5.3 — Засоби посилення збирання C/C++ (за [276])

Засіб	Прапорці GCC/Clang	Ефект; обмеження
PIE (для ASLR)	`-fPIE -pie`	випадкова адреса образу; слабшає за витоку адрес
NX-стек	`-Wl,-z,noexecstack`	невиконуваний стек; не стосується JIT
Канарки стека	`-fstack-protector-strong`	лінійне переповнення стека до повернення; не захищає купу
Зіткнення стека	`-fstack-clash-protection`	перевірка великих виділень на стеку
Fortify	`-D_FORTIFY_SOURCE=3`	функції libc; розміри, які компілятор може обчислити статично або динамічно (рівень 3)
RELRO	`-Wl,-z,relro -Wl,-z,now`	GOT лише для читання; повільніший запуск
CFI	`-fcf-protection=full`, `-mbranch-protection=standard`	переходи й повернення; потрібна підтримка процесора й ОС
Бібліотека C++	`-D_GLIBCXX_ASSERTIONS`, режим libc++	межі контейнерів; лише бібліотечні операції
Ініціалізація	`-ftrivial-auto-var-init=zero`	змінні стека; не стосується купи

Лістинг 5.3 — Прапорці посиленого збирання мережевого демона (Makefile)

```
# Базовий набір за OpenSSF Compiler Options Hardening Guide
# (GCC >= 13 або Clang >= 16)
HARDEN_CFLAGS = -O2 -Wall -Wformat=2 -Wconversion \
  -Werror=format-security -Wimplicit-fallthrough \
  -U_FORTIFY_SOURCE -D_FORTIFY_SOURCE=3 \
  -fstack-protector-strong -fstack-clash-protection \
  -fstrict-flex-arrays=3 -ftrivial-auto-var-init=zero \
  -fPIE
# Апаратна цілісність потоку керування
ifeq ($(ARCH),x86_64)
  HARDEN_CFLAGS += -fcf-protection=full
else ifeq ($(ARCH),aarch64)
  HARDEN_CFLAGS += -mbranch-protection=standard
endif
HARDEN_LDFLAGS = -pie -Wl,-z,relro -Wl,-z,now \
  -Wl,-z,noexecstack -Wl,-z,nodlopen \
  -Wl,--as-needed -Wl,--no-copy-dt-needed-entries
# У конвеєрі CI артефакт перевіряють на наявність
# PIE, RELRO, NX, канарок (наприклад, checksec)
```

5.7.2 Межі засобів посилення

Більшість засобів посилення *імовірнісні* (ASLR, MTE) або охоплюють лише окремі класи переходів і не захищають від атак лише на дані, а також від логічних вад, впровадження команд і помилок авторизації. Вони мають ціну: продуктивність (CFI, синхронний MTE), сумісність (усі бібліотеки процесу

мають підтримувати CET чи BTI) і час запуску (BIND_NOW). Вони діють лише тоді, коли справді ввімкнені, тому властивості артефактів перевіряють у конвеєрі CI (розділ 10). Посилення — пізній рубіж ланцюга Секереша [334], що доповнює, але не замінює безпеку пам'яті й безпечний розбір.

5.8 Кейс: переписування парсерів протоколів Suricata мовою Rust

5.8.1 Контекст, рішення, уроки

IDS/IPS Suricata (розділ 8) розбирає десятки протоколів прикладного рівня з трафіку, який повністю контролює порушник, — класичне місце помилок пам'яті в C. Шифл'є (ANSSI) і Купрі показали, як вбудовувати в програму на C парсери на Rust з бібліотекою nom: ядро на C викликає функцію Rust через C ABI, отримує розпізнану структуру, а `unsafe` зосереджено на тонкій межі FFI [112]. Після експериментальних парсерів NFS і DNS у Suricata 4.0 (2017 р.) у випуску 5.0 (жовтень 2019 р.) Rust став обов'язковим, що прибрало паралельні реалізації на C, і з'явилися парсери RDP, SIP і SNMP на Rust [273].

Переваги: нові парсери не вносять помилок пам'яті там, де досяжність найвища (5.2); комбінатори наближають код до граматики протоколу; розбір без копіювання зберігає продуктивність. *Витрати*: залежність від інструментарію й пакетів Rust (розділ 10), дві мови, ручні інваріанти на межі FFI, навчання команди. *Уроки*: перехід виконується компонент за компонентом, починаючи з найдосяжніших; межа між мовами — вузька й типізована; фазинг залишається обов'язковим; обов'язковість нової мови прискорює перехід, прибираючи дублювання. Ті самі принципи демонструють rustls, sudo-rs і ntpd-rs ініціативи Prossimo (ISRG), а також Rust у ядрі Linux, який у грудні 2025 р. перестав вважатися експериментальним [137].

5.8.2 Кількісна оцінка доцільності переписування

Модель (5.1) дає критерій, коли переписування *наявного* компонента віку a зменшує ризик. Переписування L_m KLOC прибирає латентні вразливості пам'яті щільністю $\rho_m e^{-a/\tau}$ (крім залишкової частки ε), але вносить логічні вразливості нового коду щільністю ρ_l. З урахуванням середньої тяжкості s_m і s_l

$$\Delta R = L_m\left[\rho_m s_m\left(e^{-a/\tau}-\varepsilon\right)-\rho_l s_l\right] > 0 \iff a < a^* = \tau \ln \frac{1}{\varepsilon + \rho_l s_l/(\rho_m s_m)}. \tag{5.5}$$

Поріг не залежить від L_m: обсяг і досяжність компонента (r_k у (1.3)) задають лише *пріоритет* серед компонентів, що пройшли поріг.

Приклад. Нехай $\tau = 4{,}1$ року, $\varepsilon = 0{,}001$, $\rho_l/\rho_m = 0{,}3$. За однакової тяжкості $a^* = 4{,}1\ln(1/0{,}301) \approx 4{,}9$ року. У парсерах ПЗМБ помилка пам'яті зазвичай дає віддалене виконання коду, а логічна помилка розбору — переважно пропуск виявлення; за $s_m/s_l = 3$ маємо $a^* = 4{,}1\ln(1/0{,}101) \approx 9{,}4$

року (для $\tau = 2{,}5 - 5{,}7$ року). Парсери Suricata, створені на початку 2010-х рр. і переписані у 2017–2019 рр., мали вік 2–9 років, тобто здебільшого нижче порогу, а пріоритет справедливо отримали протоколи, доступні з Інтернету без автентифікації. Для стабільного коду віком понад 10 років раціональніші посилення, ізоляція й фазинг. Обмеження моделі: якщо старий код активно змінюють або відкривають до нього новий шлях досяжності, його вік «скидається».

Висновки до розділу

Вразливості ПЗМБ зосереджені в кількох відомих класах: помилки пам'яті в обробниках протоколів і цілочислові помилки як їхня першопричина, впровадження команд у площині менеджменту, неправильна обробка винятків, виснаження ресурсів. Найсильніший захід — перехід на мови з безпечною пам'яттю: помилки пам'яті становлять близько 70 % вразливостей коду на C/C++, а переведення нового коду Android на мови з безпечною пам'яттю (передусім Rust і Kotlin) знизило їхню частку з 76 до менш ніж 20 %. Модель експоненційного старіння вразливостей пояснює цей ефект і дає поріг віку для переписування наявного компонента. Гарантії Rust, Go і C++ мають межі (`unsafe`, FFI, логічні помилки, паніки), які слід відобразити в правилах кодування. LangSec вимагає явної граматики, мінімальної виразності вхідної мови й повного розпізнавання до обробки, а верифіковані генератори парсерів роблять ці властивості доказовими. Цілочислові перевірки формулюють без операцій, що можуть переповнитися, помилки обробляють за принципом fail-secure, ресурси обмежують. Огляд коду ефективний за обмеженої швидкості й у поєднанні з аналізом і фазингом; засоби посилення — обов'язковий, але ймовірнісний рубіж. Кейс Suricata показує реалістичність поступового переходу з вузькою межею FFI.

Питання для самоперевірки

1. Чому рейтинг CWE Top 25 2025 р. не повністю відображає пріоритети ПЗМБ? Які класи з табл. 5.1 найважливіші для площини даних, а які — для площини менеджменту?
2. Чому цілочислове переповнення рідко фігурує в CVE як кінцева слабкість, хоча часто є першопричиною? Що це означає для аналізу першопричин?
3. Розрізніть просторові й часові порушення безпеки пам'яті. На яких етапах ланцюга Секереша діють мова з безпечною пам'яттю і засоби посилення?

4. Наведіть дані про частку вразливостей пам'яті (Microsoft, Chromium, Android). Що має містити дорожня карта безпеки пам'яті VPN-шлюзу?
5. Поясніть власність, запозичення й час життя в Rust. Які межі мають гарантії мови?
6. Порівняйте Rust, Go, C++ з посиленою бібліотекою і CHERI (табл. 5.2). Що доцільне для площини даних, площини менеджменту і прошивки, яку неможливо переписати?
7. Виведіть з (5.1) середній час життя вразливості за відношенням щільностей для коду віку 5 років. Обчисліть за (5.2) частку вразливостей пам'яті через 3 роки після переходу, якщо початкова частка становила 0,7, а $\tau = 3$ роки.
8. Що таке «рушничний» парсер? Сформулюйте принципи LangSec і поясніть роль ієрархії Хомського в неоднозначності розбору між ME і захищеним сервером.
9. Порівняйте бібліотеки комбінаторів, декларативні генератори й верифіковані парсери. Що таке неподатливість формату?
10. Чому наївна перевірка $o + \ell \le n$ небезпечна? Перевірте за (5.3) доступ з $o = \texttt{0xFFFFFF00}$, $\ell = 512$, $n = 9000$ для 32-бітового `size_t`.
11. Як усунути гонитви й TOCTOU під час оновлення правил ME під навантаженням? Які правила обробки помилок випливають з категорії OWASP A10:2025?
12. За (5.4) оцініть ефективність огляду на швидкості 300 рядків/год і сукупну ефективність разом із SAST (0,25) і фазингом (0,45). Які обмеження має припущення незалежності?
13. Дослідницьке: розробіть методику калібрування моделі (5.5) на даних відкритого проєкту ПЗМБ (історія комітів, CVE, вік рядків за `git blame`) і перевірте гіпотезу експоненційного старіння вразливостей у коді парсерів протоколів.
14. Дослідницьке: сформулюйте задачу вибору порядку переписування компонентів ПЗМБ на мову з безпечною пам'яттю за обмеженого бюджету з урахуванням віку, досяжності, обсягу й вартості межі FFI і порівняйте оптимальний розв'язок із жадібною евристикою.

РОЗДІЛ 6

Розроблення криптографічних компонентів і реалізація захищених протоколів

Криптографічні компоненти — ядро VPN-шлюзів, TLS-проксі, засобів керування ключами й захищеного оновлення: помилка в одному порівнянні чи переході стану руйнує захист, що спирається на математично стійкі примітиви. Розділ розглядає не самі алгоритми, а *інженерію їх реалізації та використання*: вибір бібліотеки, помилки неправильного використання API, сталий час і побічні канали, випадкові числа й ключі, протоколи як кінцеві автомати, криптографічну гнучкість і постквантову міграцію, національні алгоритми й тестування. Він спирається на розділи 4 і 5; нормативні вимоги до засобів КЗІ наведено в розділі 2, архітектуру VPN — у розділі 9, формальну верифікацію — у розділі 12, фазинг — у розділі 13.

6.1 Вибір криптографічної бібліотеки

6.1.1 Принцип «не реалізуй криптографію сам» і його межі

Настанова «не реалізуй криптографію сам» (don't roll your own crypto) означає: розробник прикладного ПЗМБ не проєктує власних примітивів чи протоколів і не реалізує стандартних алгоритмів, якщо існує зріла супроводжувана реалізація. Обґрунтування — асиметрія витрат: функціонально правильний AES пишеться за дні, а стійкий до побічних каналів і крайових випадків — потребує років аналізу. Межі принципу: хтось має писати самі бібліотеки (для них — підрозділи 6.3 і 6.8); національні й постквантові алгоритми ще не мають такої бази перевірених реалізацій; а *використання* зрілої бібліотеки — окрема інженерна задача, де трапляється більшість помилок (підрозділ 6.2).

6.1.2 Критерії вибору

Критерії вибору бібліотеки для ПЗМБ: *супровід* (процес обробки вразливостей, участь в OSS-Fuzz); *верифікованість* (частка формально верифікованого коду, як у HACL* [352], безпечна пам'ять); *сертифікація* — модуль, валідований за FIPS 140-3 [268, 204]; *сталий час* і задокументована модель побічних каналів; *зручність API* — чи є безпечний шлях найкоротшим; *підтримка PQC* (ML-KEM, ML-DSA, SLH-DSA, гібридні групи TLS); *розширюваність* — додавання національних алгоритмів чи апаратних модулів без форку (табл. 6.1).

Таблиця 6.1 — Порівняння криптографічних бібліотек для ПЗМБ (стан на 2026 р.)

Бібліотека	Сильні сторони	FIPS і PQC	Обмеження
OpenSSL 3.x	архітектура провайдерів; повний набір протоколів; стабільний API	FIPS-провайдер; ML-KEM, ML-DSA, SLH-DSA і типова гібридна група з версії 3.5	великий обсяг коду (C); складний низькорівневий API
BoringSSL	чистка застарілого коду; використання в Chrome і Android	модуль BoringCrypto; гібридні групи TLS	без гарантій стабільності API для сторонніх
LibreSSL	спрощений код; API `libtls`	без FIPS; PQC з'являється пізніше	менша екосистема
libsodium	API, стійкий до неправильного використання; сталий час	без FIPS; орієнтація на класичні примітиви	не реалізує TLS, IPsec
rustls + aws-lc-rs	безпечна пам'ять протокольного рівня; простий API	модуль AWS-LC з валідацією FIPS; гібридна група TLS	лише TLS; примітиви на C/асемблері
Go `crypto`	стандартна бібліотека; мова з безпечною пам'яттю	власний модуль FIPS 140-3 і ML-KEM з Go 1.24	прив'язка до екосистеми Go

Для продукту з міжнародним і національним профілями вирішальною є архітектура розширення. В OpenSSL 3.x алгоритми надають **провайдери** (providers) — модулі, що реєструють реалізації за іменами й властивостями; застосунок запитує, наприклад, `EVP_MD_fetch(ctx, "SHA256", "fips=yes")`. Так підключають FIPS-провайдер, провайдер національних алгоритмів і апаратного модуля без змін коду протоколів (рис. 6.2). Практику налаштування TLS засобами OpenSSL розглянуто в [19].

6.2 Неправильне використання криптографічних API

6.2.1 Емпіричні дані

Більшість криптографічних вразливостей виникає не в бібліотеках, а в коді, що їх викликає. Егеле та співавтори [160] виявили, що 10 327 з 11 748 застосунків Android, які використовують криптографічні API (88 %), порушують щонайменше одне з шести базових правил: режим ECB, сталий IV у CBC, сталий ключ у коді, стала сіль чи замало ітерацій у PBKDF, стале зерно генератора. Наді та співавтори [260] на матеріалі Stack Overflow і опитування розробників Java показали, що причина — надто низькорівневий API: розробник має сам обрати алгоритм, режим, доповнення й генерування IV, не розуміючи наслідків. Акар і співавтори [63] в експерименті з 256 розробниками Python встановили, що спрощені API підвищують частку безпечних рішень лише за наявності документації з прикладами коду.

Окремий клас — вимкнена або неповна перевірка сертифікатів. Георгієв і співавтори [180] показали, що її зламано в численних небраузерних застосунках і SDK, здебільшого через заплутані параметри API: параметр, що очікує значення 2 для перевірки імені хоста, отримує логічне `true`, тобто 1. Для ПЗМБ це особливо небезпечно в каналах керування й оновлення (розділ 9).

6.2.2 Класи помилок і засоби запобігання

Узагальнення класів неправильного використання і відповідних заходів наведено в табл. 6.2; у CWE [136] їм відповідають, зокрема, CWE-327 (зламаний чи ризикований алгоритм), CWE-329 (непередбачуваний IV у CBC), CWE-295 (неправильна перевірка сертифіката), CWE-208 (спостережуване розходження в часі).

Таблиця 6.2 — Класи неправильного використання криптографічних API

Клас помилки	Наслідок	Запобігання
Режим ECB	однакові блоки відкритого тексту видно в шифротексті	лише AEAD (AES-GCM, ChaCha20-Poly1305)
Сталий IV / повтор nonce	для режиму лічильника — XOR відкритих текстів; для GCM — підробка тегів	лічильникові nonce; AES-GCM-SIV; API, що сам генерує nonce
Слабка KDF для пароля	офлайн-підбір пароля	Argon2id, scrypt або PBKDF2 з великою кількістю ітерацій і випадковою сіллю
Ключ у коді чи конфігурації	компрометація всіх екземплярів продукту	ключі на пристрої; сховище секретів; HSM/TPM
Вимкнена перевірка сертифіката	атака «людина посередині»	безпечні налаштування за замовчуванням; заборона на рівні CI
Порівняння MAC через `memcmp`	часовий витік, підбір тегу	функції сталого часу (лістинг 6.1)

Відповіддю стали **API, стійкі до неправильного використання** (misuse-resistant API): найпростіший виклик безпечний; незрозумілі розробнику параметри не виносяться в інтерфейс; nonce генерує бібліотека або він розширений до 192 біт (XChaCha20-Poly1305 у libsodium), щоб випадковий вибір був безпечним; ключ невіддільний від алгоритму і призначення (Google Tink). На рівні алгоритмів ту саму ідею реалізує AES-GCM-SIV [187]: повтор nonce розкриває лише рівність двох повідомлень. Статичні засоби (CogniCrypt, CryptoGuard, правила CodeQL і Semgrep) перевіряють виклики API на відповідність специфікаціям правильного використання і вбудовуються в конвеєр як ворота якості (розділ 12).

6.3 Реалізація зі сталим часом і побічні канали

6.3.1 Часові та кешеві канали

Побічний канал (side channel) — спостережувана характеристика обчислення (час, стан кешу, енергоспоживання, випромінювання), що залежить від секрету. Для мережевого ПЗ головні два канали, доступні віддаленому або сусідньому (у хмарі) порушнику. *Часовий*: тривалість залежить від секрету через розгалуження, ранній вихід із циклу чи інструкції зі змінною латентністю. *Кешевий*: адреса доступу залежить від секрету (табличні S-блоки, вікна піднесення до степеня), і порушник зі спільним кешем відновлює індекси методами Prime+Probe чи Flush+Reload.

Звідси **правила програмування зі сталим часом** (constant-time): (1) жодних розгалужень за секретом — лише маскові вибори; (2) жодних адрес, обчислених із секрету, — bitsliced-реалізація або апаратні інструкції; (3) жодних інструкцій зі змінною латентністю на секретних операндах; (4) кіль-

кість ітерацій залежить лише від публічних довжин. Розширення AES-NI, PCLMULQDQ, ARMv8 Crypto Extensions задовольняють правило (2) для AES і GHASH; для «Калини» bitsliced-реалізація з SIMD-інструкціями поєднує швидкість зі стійкістю до часових і кешевих атак [50]. Компілятор може повернути умовний перехід, тож критичні функції перевіряють у машинному коді.

Лістинг 6.1 — Порівняння тегів автентифікації зі сталим часом (C)

```
#include <stddef.h>
#include <stdint.h>

/* Повертає 1, якщо a == b, інакше 0. Час виконання
   залежить лише від len, а не від вмісту буферів
   чи позиції першої розбіжності (на відміну від memcmp) */
int ct_eq(const uint8_t *a, const uint8_t *b, size_t len)
{
    volatile uint8_t acc = 0;   /* volatile: не дати
                                   компілятору вийти рано */
    for (size_t i = 0; i < len; i++)
        acc |= (uint8_t)(a[i] ^ b[i]);
    /* acc == 0  ->  1; acc у [1, 255]  ->  0; без if */
    return (int)(1u & ((uint32_t)(acc - 1) >> 8));
}

/* Вибір без розгалуження: bit == 1 -> x, bit == 0 -> y */
uint32_t ct_select(uint32_t bit, uint32_t x, uint32_t y)
{
    uint32_t m = (uint32_t)0 - bit;   /* 0x0 або 0xFFFFFFFF */
    return (x & m) | (y & ~m);
}
```

У промисловому коді використовують готові функції (`CRYPTO_memcmp`, `sodium_memcmp`, крейт `subtle` у Rust); лістинг 6.1 лише ілюструє принцип.

6.3.2 Урок Lucky13

Атака Lucky Thirteen [65] показала, що сталий час має охоплювати *всю* обробку запису. У TLS до версії 1.2 з CBC діяла схема MAC-then-encrypt: після дешифрування знімалося доповнення і обчислювався HMAC над рештою. Кількість викликів функції стиснення SHA-1 залежала від довжини доповнення, тобто від відкритого тексту, і різниця в кілька тактів, усереднена за мільйонами сеансів, давала оракул відновлення відкритого тексту. Висновки: обирати конструкції, які легко реалізувати зі сталим часом (TLS 1.3 [304] залишив лише AEAD); не розкривати причину відмови (одна помилка `bad_record_mac`); вважати навіть мікросекундні відмінності експлуатованими.

6.3.3 Верифікація сталого часу

Формальний підхід: ct-verif [70] доводить для LLVM-представлення, що розгалуження й адреси пам'яті не залежать від секретів, зводячи задачу до перевірки добутку програми на саму себе (розділ 12). *Статистичний*: dudect [302] вимірює час для двох класів входів (сталий і випадковий) і перевіряє рівність середніх за t-критерієм Велча:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\dfrac{s_1^2}{N_1} + \dfrac{s_2^2}{N_2}}}, \tag{6.1}$$

де $\bar{x}_i$, s_i^2, N_i — вибіркове середнє, дисперсія і кількість вимірювань (у тактах процесора) для класу i. Перевищення порогу $|t| > 4{,}5$ (поріг, запозичений із методики оцінювання витоків TVLA) вважають свідченням витоку. *Приклад.* Для функції порівняння отримано $\bar{x}_1 = 1000{,}0$ і $\bar{x}_2 = 998{,}5$ тактів, $s_1 = s_2 = 20$ тактів, $N_1 = N_2 = 10^5$. Знаменник $\sqrt{2 \cdot 400/10^5} = 0{,}089$, тож $t \approx 16{,}8$: різниця у півтора такту, непомітна в одиничному вимірюванні, статистично достовірна.

Скільки вимірювань потрібно, щоб виявити витік величини δ за однакових дисперсій? З (6.1) за $N_1 = N_2 = N$ і $\bar{x}_1 - \bar{x}_2 = \delta$ отримуємо

$$N \;\geq\; 2\left(\frac{t_{\text{пор}}\,\sigma}{\delta}\right)^2. \tag{6.2}$$

Для $\sigma = 20$ тактів, $\delta = 0{,}5$ такту і $t_{\text{пор}} = 4{,}5$ маємо $N \geq 2 \cdot 180^2 \approx 6{,}5 \cdot 10^4$ вимірювань на клас. Отже, шум (σ) квадратично збільшує вартість тестування (тести запускають на ізольованих ядрах із фіксованою частотою), а від'ємний результат за обмеженого N лише обмежує витік зверху. Статистичні тести включають у нічні збирання, формальні застосовують до ядра криптографічних функцій.

Проти фізичних каналів (енергоспоживання, випромінювання), актуальних для вбудованих пристроїв і HSM, застосовують **маскування** (masking): секрет розділяється на $d + 1$ випадкових частин, і спостереження до d проміжних значень не дає про нього інформації ціною кратного зростання обчислень.

6.4 Випадкові числа та керування ключами

6.4.1 Генератори випадкових чисел

Стійкість ключів і nonce визначає криптографічно стійкий генератор псевдовипадкових чисел (CSPRNG). Правило для ПЗМБ: використовувати генератор ОС (`getrandom()` у Linux, `getentropy()` у BSD, `BCryptGenRandom` у

Windows) або обгортку бібліотеки над ним, не створюючи власних генераторів. Виклик `getrandom()` блокується до ініціалізації пулу ентропії, що усуває помилки «ключ до появи ентропії».

Масштаб наслідків показують два уроки. У 2008 р. виявилося, що зміна в пакеті OpenSSL дистрибутива Debian (CVE-2008-0166), зроблена для усунення попередження аналізатора пам'яті, залишила єдиним джерелом зерна ідентифікатор процесу: для кожного типу й довжини ключа існувало лише 32 768 ключів, і всі ключі SSH, TLS і OpenVPN, згенеровані за два роки, довелося відкликати. Генінгер і співавтори [192], обчисливши попарні НСД модулів RSA, зібраних скануванням інтернету, отримали закриті ключі RSA для 0,50 % хостів TLS, а через повтори випадкових значень — ключі DSA для 1,03 % хостів SSH. Більшість уражених хостів — маршрутизатори, міжмережеві екрани, VPN-концентратори, що генерували ключі під час першого завантаження з порожнім пулом ентропії. Звідси вимоги до прошивок: генерувати довготривалі ключі після накопичення ентропії; використовувати апаратне джерело (TRNG, TPM); засівати генератор на виробництві унікальним зерном; перевіряти відсутність повторів ключів у партії пристроїв.

6.4.2 Ліміти використання ключа і колізії nonce

Для AES-GCM повтор пари (ключ, nonce) катастрофічний (табл. 6.2). Якщо 96-бітні nonce обираються випадково, ймовірність хоча б однієї колізії серед q повідомлень за парадоксом днів народження

$$P_{\text{кол}} \;\approx\; 1 - e^{-q(q-1)/2^{n+1}} \;\approx\; \frac{q^2}{2^{n+1}}, \tag{6.3}$$

де n — довжина nonce у бітах. Для $n = 96$ і $q = 2^{32}$ маємо $P_{\text{кол}} \approx 2^{-33} \approx 1{,}2 \cdot 10^{-10}$; саме тому NIST SP 800-38D обмежує кількість шифрувань одним ключем із випадковими IV величиною 2^{32}. *Приклад для VPN-шлюзу*: тунель передає 10^6 пакетів/с, тож за добу $q = 8{,}64 \cdot 10^{10} \approx 2^{36{,}3}$, і $P_{\text{кол}} \approx (8{,}64 \cdot 10^{10})^2 / 2^{97} \approx 4{,}7 \cdot 10^{-8}$ — вище допустимого рівня $2^{-32} \approx 2{,}3 \cdot 10^{-10}$. Тому в протоколах передавання даних nonce формують із лічильника (ESP; у TLS 1.3 — номер запису XOR статичний IV), а ключ змінюють за обсягом даних: TLS 1.3 рекомендує оновлювати ключі AES-GCM не пізніше ніж через $2^{24{,}5}$ записів повного розміру [304], в IPsec ту саму роль відіграє час життя дочірніх SA за байтами (розділ 9).

6.4.3 Життєвий цикл ключа і апаратні модулі

Життєвий цикл ключа — генерування, реєстрація, розподіл, використання, відкликання, архівування і знищення — задає для кожного стану дозволені операції та ролі. Довготривалі ключі ПЗМБ (ідентичність шлюзу, підпис оновлень, кореневий ключ ЦС) зберігають в **апаратному модулі безпеки**

(HSM) або TPM 2.0 так, щоб закритий ключ не залишав модуля: застосунок звертається до нього через PKCS#11 (в OpenSSL 3.x — через провайдер). Модулі валідуються за FIPS 140-3 [268] (рівні 1–4, від програмного модуля до фізичного захисту з реагуванням на втручання), а в Україні засоби КЗІ, включно із системами керування ключами, проходять державну експертизу [34]. Типова помилка — тримати в HSM лише кореневий ключ, а ключі шлюзу — у незашифрованих файлах; які ключі потребують апаратного захисту, визначає модель загроз (розділ 4).

6.5 Реалізація протоколів як кінцевих автоматів

6.5.1 Вади кінцевих автоматів

Рукостискання є кінцевим автоматом: у кожному стані допустимі лише певні повідомлення. Реалізації з кількома версіями, наборами шифрів і режимами автентифікації будують *композитний* автомат, і прийняття повідомлення в недозволеному стані чи пропуск обов'язкового утворюють окремий клас вразливостей. Беурдуш і співавтори [87] виявили в реалізаціях TLS низку таких вад (атаки SMACK): *SKIP* — клієнт приймав пропуск повідомлень сервера (обміну ключами, підтвердження сертифіката), що дозволяло видати себе за сервер; *FREAK* — клієнти OpenSSL, Secure Transport і SChannel приймали експортний 512-бітний ключ RSA, хоча такий набір не узгоджувався. Де Ройтер і Полл [145] застосували **навчання автоматів** (active automata learning, LearnLib): автомат реалізації реконструюється як автомат Мілі за відповідями на послідовності повідомлень, після чого аналізуються зайві переходи; з дев'яти реалізацій TLS нові вади знайдено в трьох. Цей метод — фазинг станів протоколу (protocol state fuzzing) — застосовують до TLS, SSH, DTLS та IKE (розділ 13).

6.5.2 Проєктування явного автомата

Звідси принцип: автомат має бути *явним* — тип стану, одна функція переходу, таблиця дозволених пар «стан–повідомлення», а будь-яке інше повідомлення є фатальною помилкою. Так описано автомат клієнта TLS 1.3 в RFC 8446 [304] (рис. 6.1).

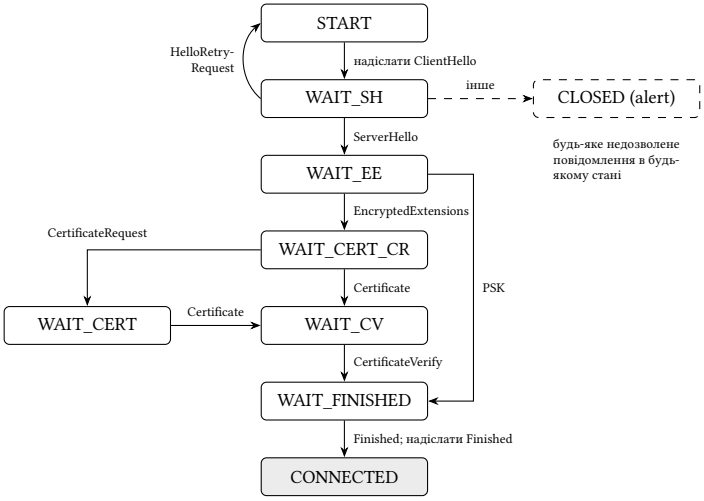


Рис. 6.1 — Спрощений автомат клієнта рукостискання TLS 1.3 (за [304])

Мови з розвиненою системою типів переносять перевірку переходів на етап компіляції за допомогою **типів стану** (typestate): кожен стан — окремий тип, перехід споживає старий стан і повертає новий, а недоступні в стані операції для його типу не існують. У лістингу 6.2 надіслати прикладні дані до завершення рукостискання чи повторно використати стан після переходу неможливо — код не компілюється. Перевірок під час виконання це не замінює, але усуває шляхи в обхід них.

Лістинг 6.2 — Модель станів клієнта на основі типів стану (Rust)

```
pub struct WaitSh;  pub struct WaitEe;
pub struct Connected { keys: TrafficKeys }

// Стан -- параметр типу; переходи споживають self
pub struct Client<S> { st: S, transcript: Transcript }

impl Client<WaitSh> {
    pub fn on_server_hello(self, m: &ServerHello)
        -> Result<Client<WaitEe>, Alert> {
        check_version_and_group(m)?;  // інакше -- alert
        let t = self.transcript.add(m);
        Ok(Client { st: WaitEe, transcript: t })
    }
}

impl Client<Connected> {
    // Метод існує лише для стану Connected:
    // виклик у WaitSh чи WaitEe -- помилка компіляції
    pub fn send_app_data(&mut self, d: &[u8]) -> Vec<u8> {
```

```
            seal_record(&mut self.st.keys, d)
        }
    }
```

6.6 Криптографічна гнучкість і постквантова міграція

6.6.1 Криптографічна гнучкість

Криптографічна гнучкість (crypto agility) — здатність замінити алгоритм, параметри чи реалізацію без зміни архітектури й протоколів. Складові: *інвентаризація* — криптографічний перелік компонентів (CBOM у CycloneDX [158]): які алгоритми, де і з якими ключами; *абстракція* — протоколи звертаються до алгоритмів через ідентифікатори й фабрики; *узгодження* з захистом від зниження версії; *політика* дозволених алгоритмів — підписана конфігурація, що оновлюється окремо від коду (рис. 6.2).

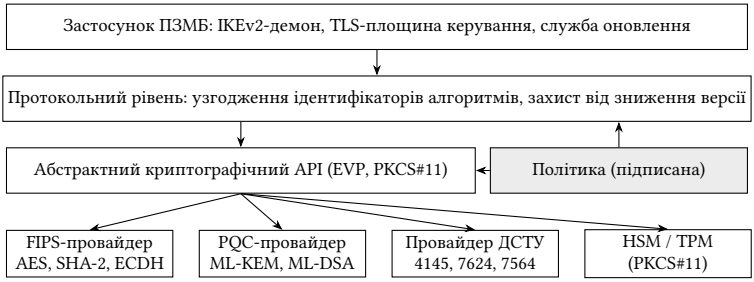


Рис. 6.2 — Архітектура криптографічної гнучкості ПЗМБ з підключуваними провайдерами

6.6.2 Постквантові стандарти та гібридний обмін ключами

NIST затвердив механізм інкапсуляції ключа ML-KEM [266], підписи ML-DSA [265] і геш-підписи без стану SLH-DSA [269]. Проєкт NIST IR 8547 [254] пропонує графік відмови від RSA і ECC: варіанти зі стійкістю 112 біт застарівають (deprecated) після 2030 р., а всі квантово-вразливі алгоритми заборонено (disallowed) після 2035 р.; документ досі є проєктом, але його дати вже слугують орієнтиром регуляторам.

На перехідний період застосовують **гібридний обмін ключами**: спільний секрет утворюється з результатів ECDH і ML-KEM, тож сеанс захищений, доки стійка хоча б одна складова. RFC 10024 [229] (серпень 2026 р.) стандартизував для TLS 1.3 групи X25519MLKEM768, SecP256r1MLKEM768 і SecP384r1MLKEM1024 з конкатенацією ключів і шифротекстів складових. Розгортання випередило стандартизацію: Chrome, Firefox, Cloudflare, OpenSSL 3.5, Go 1.24 і rustls пропонують X25519MLKEM768 за замовчуванням, захищаючи значну частку вебтрафіку від загрози «збирай зараз —

дешифруй потім». В IKEv2 [220] механізм багаторазового обміну ключами (RFC 9370) дозволяє після класичного обміну в IKE_SA_INIT виконати до семи додаткових в обмінах IKE_INTERMEDIATE; профіль ML-KEM для IKEv2 [219] на жовтень 2026 р. перебуває в черзі RFC Editor.

Головна інженерна проблема — розміри: постквантові ключі й підписи на один–два порядки більші за еліптичні (табл. 6.3).

Таблиця 6.3 — Розміри криптографічних об'єктів, байт (за [266, 265, 269])

Алгоритм	Відкритий ключ	Шифротекст / підпис	Рівень NIST
X25519	32	32	—
ECDSA P-256	64	64–72	—
RSA-3072	384	384	—
ML-KEM-768	1184	1088	3
ML-KEM-1024	1568	1568	5
X25519MLKEM768	1216	1120	гібрид
ML-DSA-44	1312	2420	2
ML-DSA-65	1952	3309	3
SLH-DSA-SHA2-128s	32	7856	1

Наслідки: ClientHello з гібридною групою перевищує типовий MTU і займає два сегменти TCP (частина проміжних пристроїв обробляла такі ClientHello некоректно — ризик і для DPI-систем, розділ 8); IKE_SA_INIT не фрагментується на рівні IKE, тому великі ключі передають в IKE_INTERMEDIATE, яке вже захищене і фрагментується; ланцюжок сертифікатів ML-DSA додає кілька кілобайтів. Накладні витрати рукостискання оцінюють моделлю

$$\Delta T_{\text{рук}} \;=\; \frac{8\,\Delta B}{R} \;+\; \Delta t_{\text{обч}} \;+\; k \cdot \mathrm{RTT}, \tag{6.4}$$

де ΔB — додатковий обсяг даних рукостискання, байт; R — пропускна здатність каналу, біт/с; $\Delta t_{\text{обч}}$ — додатковий час обчислень на обох сторонах; k — кількість додаткових обходів (наприклад, через HelloRetryRequest, IKE_INTERMEDIATE чи вичерпання початкового вікна TCP). *Приклад для TLS*: перехід з X25519 на X25519MLKEM768 дає $\Delta B = (1216 + 1120) - (32 + 32) = 2272$ байт; на каналі $R = 10$ Мбіт/с перша складова становить $1{,}8$ мс, обчислювальна складова ML-KEM-768 на сервері — десятки мікросекунд, $k = 0$. Затримка близько 2 мс несуттєва для окремого сеансу, але для шлюзу з тисячами нових з'єднань за секунду суттєвим стає трафік рукостискань. Продуктивність реалізацій ML-KEM і ML-DSA помітно залежить від бібліотеки: для Java Bouncy Castle стабільно випереджає вбудовану реалізацію JDK 25 [30], тож вибір варто підтверджувати власними вимірюваннями.

6.7 Національні криптографічні алгоритми в програмному забезпеченні

ПЗМБ для державних систем України має реалізовувати національні алгоритми (розділ 2): цифровий підпис ДСТУ 4145-2002 [13], блоковий шифр «Калина» ДСТУ 7624:2014 [14], функцію гешування «Купина» ДСТУ 7564:2014 [15] і потоковий шифр «Струмок» ДСТУ 8845:2019 [16]; теоретичні основи алгоритмів викладено в [6]. *Бібліотеки.* Реалізації ДСТУ 4145, «Калини» й «Купини» містить, зокрема, Bouncy Castle (Java, C#); для засобів КЗІ використовують реалізації вітчизняних розробників, що пройшли державну експертизу. Для продуктів на C/C++ раціональний шлях — окремий провайдер OpenSSL 3.x (рис. 6.2), що реєструє національні алгоритми під власними іменами та OID без форку бібліотеки. *Протоколи.* Національний профіль відображається в ідентифікаторах наборів шифрів TLS і перетворень IKEv2, які іноземні реалізації не підтримують, тому продукт для обох ринків узгоджує профіль явно і не допускає непомітного переходу між ними. *Сталий час.* Табличні реалізації «Калини», як і табличний AES, вразливі до кешевих атак; bitsliced-реалізація [50] усуває залежність адрес від ключа. *Тестові вектори і експертиза.* Контрольні приклади стандартів входять у модульні тести й самоперевірки під час запуску (аналог тестів відомої відповіді FIPS 140-3). Висновок державної експертизи засобу КЗІ [34] прив'язується до геш-значення програмного засобу, тож оновлення криптографічного модуля потребує планування повторного оцінювання — ще один аргумент за відокремлення модуля з чіткою межею.

6.8 Тестування криптографічних реалізацій

Криптографічна помилка зазвичай мовчки знижує стійкість, тому методи тестування комбінують. *Вектори відомої відповіді*: програма NIST CAVP і протокол ACVP перевіряють реалізації на векторах, що генерує сервер; проходження ACVP — передумова валідації за FIPS 140-3. *Вектори крайових випадків*: Project Wycheproof [294] містить тисячі векторів для відомих класів помилок — точки поза кривою, некоректне кодування підписів ASN.1, короткі nonce, модифіковані теги; кожен позначено як `valid`, `invalid` або `acceptable`. *Диференційне тестування* порівнює дві незалежні реалізації на однакових входах (лістинг 6.3) і в поєднанні з фазингом (Cryptofuzz в OSS-Fuzz, розділ 13) знаходить розбіжності там, де векторів немає. Доповнюють їх фазинг станів (підрозділ 6.5) і тести сталого часу (підрозділ 6.3).

Лістинг 6.3 — Диференційна перевірка AES-GCM на векторах Wycheproof (Python)

```
import json
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

def ref_open(key, iv, aad, ct_tag):
    try:
        return AESGCM(key).decrypt(iv, ct_tag, aad)
    except Exception:            # невірний тег, довжина
        return None

def check(path, our_open):
    """our_open -- обгортка над реалізацією, що тестується"""
    bad = 0
    for grp in json.load(open(path))["testGroups"]:
        for t in grp["tests"]:
            h = lambda f: bytes.fromhex(t[f])
            args = (h("key"), h("iv"), h("aad"),
                    h("ct") + h("tag"))
            ours, ref = our_open(*args), ref_open(*args)
            exp = h("msg") if t["result"] == "valid" else None
            if t["result"] == "acceptable":
                exp = ref        # допустима будь-яка поведінка
            if ours != exp or ours != ref:
                bad += 1
                print(t["tcId"], t["comment"])
    return bad
```

Найсильніші гарантії дає *формальна верифікація реалізацій*. HACL* [352] написано мовою F* з доведеннями безпеки пам'яті, функціональної коректності і сталого часу, а згенерований C-код вбудовано у Firefox (NSS) і ядро Linux; Fiat-Crypto синтезує верифіковану арифметику полів для BoringSSL (детально — розділ 12). Практичний висновок: обирати бібліотеки, де арифметика кривих і AEAD походять із верифікованих джерел, а власні тести зосереджувати на інтеграційному і протокольному рівнях.

6.9 Кейс: міграція VPN-шлюзу на гібридну постквантову криптографію

Український виробник VPN-шлюзу (кейс розділу 2) реалізує IPsec/IKEv2 для тунелів і TLS 1.3 для площини керування (вебінтерфейс, API, канал оновлень); типова інсталяція — 5000 тунелів IKEv2 з повторним узгодженням IKE SA щогодини. Замовники вимагають захисту від «збирай зараз — дешифруй потім» для трафіку з терміном конфіденційності понад 10 років. Етапи плану (рис. 6.3):

1. *Інвентаризація* (3 міс.): CBOM; пошук жорстко закодованих алгоритмів і фіксованих буферів під ключі (типова помилка — буфер 512 байт для KE-навантаження).

2. *Рефакторинг* (4 міс.): криптографічний модуль із провайдерами (рис. 6.2); OpenSSL 3.5 LTS; явний автомат IKEv2 з IKE_INTERMEDIATE і фрагментацією.
3. *Гібридний обмін ключами* (4 міс.): для TLS — X25519MLKEM768 [229] з резервом на X25519; для IKEv2 — ECDH в IKE_SA_INIT плюс ML-KEM-768 в IKE_INTERMEDIATE [219]; у національному профілі класична складова використовує національні алгоритми.
4. *Тестування* (паралельно): вектори Wycheproof і ACVP; сумісність зі strongSwan і libreswan; фазинг станів IKEv2; тести dudect; навантажувальні тести (розділ 14).
5. *Постквантова автентифікація*: ML-DSA спершу для підпису оновлень (через тривалий строк життя кореневого ключа), згодом — для сертифікатів шлюзів.

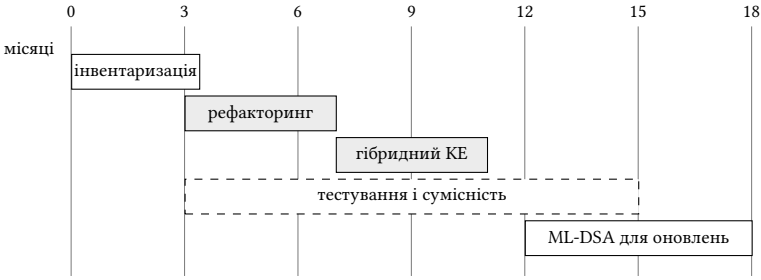


Рис. 6.3 — Етапи плану міграції VPN-шлюзу на гібридну PQC

Накладні витрати оцінимо за (6.4). Додатковий обмін з ML-KEM-768 передає 1184 байти від ініціатора і 1088 байтів від відповідача, тобто $\Delta B \approx 2272$ байти плюс заголовки (близько 2,4 кБ), і додає один обхід ($k = 1$). За RTT $= 40$ мс і $R = 10$ Мбіт/с: $\Delta T_{\text{рук}} \approx 1{,}9 + 0{,}1 + 40 \approx 42$ мс на встановлення IKE SA — прийнятно, бо повторне узгодження йде у фоні, а наявний тунель працює до заміни SA. Середнє навантаження: 5000 узгоджень на годину $\approx 1{,}4$ за секунду, тобто близько 3,4 кБ/с трафіку і менш як 1 мс процесорного часу на секунду. Критичним є піковий сценарій: після перезавантаження шлюз одночасно встановлює всі тунелі, і додаткові $5000 \cdot 2{,}4 \approx 12$ МБ та обхід мережі подовжують відновлення, тож план передбачає обмеження швидкості встановлення SA. Ризики в реєстрі: несумісність зі старими пристроями партнерів (класичний режим лише за явною політикою з журналюванням), зміни у фінальному RFC для IKEv2, повторна експертиза засобу КЗІ для національного профілю.

Висновки до розділу

Криптографічні компоненти ПЗМБ рідко ламаються через слабкість алгоритмів; типові причини — неправильне використання API, часові витоки, брак ентропії, вади кінцевих автоматів і негнучкість до заміни алгоритмів. Принцип «не реалізуй криптографію сам» зводить задачу до вибору зрілої бібліотеки, а архітектура провайдерів OpenSSL 3.x поєднує в одному продукті валідований модуль, постквантові й національні алгоритми та HSM. Оскільки більшість застосунків порушує базові правила використання API, стійкі до помилок API, AEAD і статичні перевірки в конвеєрі є необхідністю. Сталий час має охоплювати всю обробку секретів (урок Lucky13) і перевірятися формально та статистично; t-критерій Велча кількісно оцінює витік і потрібну кількість вимірювань. Ключі, згенеровані без ентропії, призвели до масової компрометації пристроїв, а парадокс днів народження обґрунтовує лічильникові nonce і ліміти ключів. Явні автомати з типами стану запобігають атакам класу SMACK, навчання автоматів їх тестує. Постквантова міграція триває: X25519MLKEM768 стандартизовано й увімкнено за замовчуванням, ML-KEM для IKEv2 завершує стандартизацію, NIST IR 8547 задає горизонт 2030–2035 рр.; головна проблема — розміри, що впливають на фрагментацію і пікове навантаження. Національні алгоритми потребують тих самих практик і планування державної експертизи.

Питання для самоперевірки

1. У чому полягає принцип «не реалізуй криптографію сам» і які його межі для розробників ПЗМБ?
2. Порівняйте бібліотеки табл. 6.1. Яку з них ви обрали б для VPN-шлюзу з національним і міжнародним профілями і чому?
3. Які причини неправильного використання криптографічних API встановили Егеле, Наді та Акар?
4. Чому вимкнена перевірка сертифікатів є системною вадою небраузерного ПЗ? Які заходи на рівні API і конвеєра їй запобігають?
5. Сформулюйте принципи API, стійких до неправильного використання. Як AES-GCM-SIV змінює наслідки повтору nonce?
6. Перелічіть правила програмування зі сталим часом. Чому лістинг 6.1 не використовує умовного оператора для формування результату?
7. Поясніть механізм атаки Lucky13 і які проєктні рішення TLS 1.3 усувають цей клас атак.
8. Обчисліть t-статистику (6.1) для $\bar{x}_1 - \bar{x}_2 = 0{,}3$ такту, $s = 15$ тактів, $N = 10^6$. Чи виявлено витік? Скільки вимірювань потрібно за (6.2) для $\delta = 0{,}1$ такту?

9. Які уроки для прошивок мережевих пристроїв випливають із випадку Debian OpenSSL і дослідження «Mining Your Ps and Qs»?
10. За (6.3) оцініть, через скільки пакетів із випадковими 96-бітними nonce ймовірність колізії перевищить 2^{-32}. Чому в протоколах передавання даних використовують лічильникові nonce?
11. Поясніть вади SKIP і FREAK. Як явний автомат (рис. 6.1) і типи стану (лістинг 6.2) запобігають таким вадам?
12. Які складові криптографічної гнучкості? Яка роль CBOM і підписаної політики?
13. За (6.4) оцініть накладні витрати гібридного рукостискання TLS із сертифікатами ML-DSA-65 (два сертифікати в ланцюжку і підпис CertificateVerify) на каналі 2 Мбіт/с.
14. Дослідницьке: запропонуйте методику навчання автомата реалізації IKEv2 з IKE_INTERMEDIATE; визначте алфавіт вхідних повідомлень, оракул еквівалентності і критерії небезпечних переходів.
15. Дослідницьке: сформулюйте задачу гібридного обміну ключами з класичною складовою на національних алгоритмах і постквантовою на ML-KEM; обґрунтуйте функцію комбінування секретів і план оцінювання продуктивності та стійкості до побічних каналів.

РОЗДІЛ 7

Розроблення засобів фільтрації та контролю мережевого трафіку

Міжмережевий екран (МЕ) — найпоширеніший клас ПЗМБ, на якому найнаочніше видно вимоги розділу 1: ворожий вхід на лінійній швидкості і бюджет у сотні тактів на пакет (1.1). Розглянуто архітектури МЕ, програмовані площини даних (eBPF/XDP, DPDK, P4), класифікацію пакетів, модель політики й аномалій правил, стійкість таблиці станів і тестування фільтрів. Безпечний розбір заголовків — у розділі 5, інспекція вмісту — у розділі 8, продуктивність і стійкість до DoS — у розділі 14.

7.1 Архітектури міжмережевих екранів

Настанова NIST SP 800-41 [316] розрізняє технології МЕ за тим, яку частину стану комунікації вони враховують. **Пакетний фільтр** ухвалює рішення для кожного пакета окремо за полями заголовків; він простий і швидкий, але не відрізняє відповіді на легітимний запит від непроханого пакета. **МЕ зі станом** (stateful inspection) веде **таблицю з'єднань**, перевіряє переходи автомата TCP і застосовує політику лише до першого пакета потоку. **Проксі прикладного рівня** завершує з'єднання клієнта і відкриває власне до сервера, тому бачить семантику запитів. **МЕ наступного покоління** (NGFW) додає ідентифікацію застосунків незалежно від порту (App-ID), прив'язку до користувача, TLS-інспекцію і функції IPS. **МЕ вебзастосунків** (WAF) спеціалізується на HTTP і захищає від класів атак OWASP Top 10 [281]. Порівняння наведено в табл. 7.1.

Таблиця 7.1 — Архітектури МЕ з погляду розробника

Архітектура	Стан	Основна перевага	Основний ризик реалізації
Пакетний фільтр	немає	мінімальний код, передбачуваний час	неповні перевірки фрагментів і заголовків
Зі станом	таблиця з'єднань	контекст потоку, проста політика	виснаження таблиці, помилки автомата TCP
Проксі	сеанс прикладного рівня	повна семантика протоколу	розбір складних протоколів у привілейованому процесі
NGFW	потік + застосунок + користувач	ідентифікація застосунків, TLS-інспекція	величезна поверхня атаки, площина менеджменту
WAF	запит HTTP	захист від атак на вебзастосунки	неоднозначність нормалізації, хибні спрацювання

Продуктивність NGFW тримається на розділенні *швидкого* й *повільного* шляхів (рис. 7.1). Перший пакет потоку проходить повільний шлях, і рішення записується в таблицю потоків; наступні пакети обробляються пошуком за 5-кортежем і кешованою дією. Коректність ME залежить від того, коли рішення можна кешувати (потік, що змінює застосунок посеред сеансу, має повертатися на повільний шлях), а виснаження повільного шляху новими потоками — природна мета DoS (підрозділ 7.5).

TLS-інспекція в NGFW (зокрема наскрізного прикладу, підрозділ 1.9) — окремий інженерний об'єкт: проксі завершує TLS клієнта сертифікатом власного ЦС і відкриває нове з'єднання до сервера, тож саме він відповідає за перевірку сертифіката сервера, відкликання, набір шифрів і версію протоколу. Вимірювання Дурумерика та ін. [157] показали, що майже всі досліджені засоби перехоплення знижували стійкість з'єднань, а частина не перевіряла належно сертифікатів сервера, відкриваючи шлях атаці «людина посередині». Тести проксі охоплюють некоректні, прострочені й відкликані сертифікати сервера та застарілі версії; ключ ЦС інспекції — критичний актив (розділ 6). Encrypted Client Hello (ECH) [305] шифрує ім'я сервера (SNI), і політики за SNI без перехоплення втрачають точність.

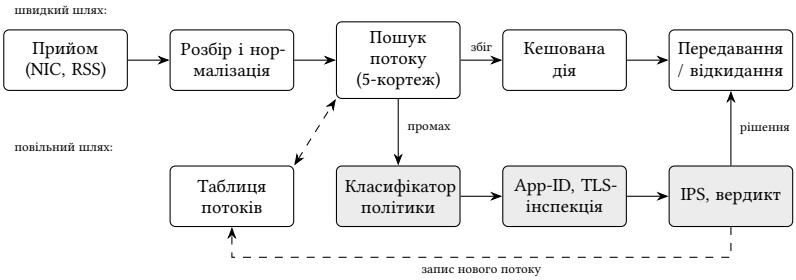


Рис. 7.1 — Конвеєр обробки пакета в NGFW: швидкий і повільний шляхи

7.2 Підсистеми ОС і програмовані площини даних

7.2.1 netfilter/nftables і pf

У Linux фільтрацію реалізує каркас netfilter: обробники реєструються на *хуках* шляху пакета (`ingress`, `prerouting`, `input`, `forward` тощо), а conntrack надає правилам стан з'єднання. nftables замінив iptables віртуальною машиною в ядрі, у яку компілюються правила, і *наборами* (sets) та словниками (maps) — геш-таблицями й деревами інтервалів, що замінюють тисячі однотипних правил одним пошуком; у BSD аналогічну роль відіграє pf. Для розробника ПЗМБ це перевірена площина даних; власний код зосере-

джується в площині керування, де виникають помилки трансляції політик (підрозділ 7.4).

7.2.2 eBPF і XDP

eBPF — механізм завантаження в ядро програм у байт-коді віртуальної машини (11 регістрів, стек 512 байтів), що обмінюються даними з простором користувача через *мапи*. **XDP** (eXpress Data Path) [196] виконує програму в драйвері мережевої карти *до* виділення `sk_buff` з вердиктом `XDP_DROP`, `XDP_PASS`, `XDP_TX` або `XDP_REDIRECT`, тож відкидання коштує десятки тактів.

Безпеку виконання гарантує **верифікатор** ядра: він символьно перебирає всі шляхи програми, відстежуючи тип і діапазон кожного регістра, і відхиляє програму з доступом до пакета без перевірки меж, читанням неініціалізованого стека, необмеженим циклом або понад ліміт складності. Сам верифікатор — складний код ядра, вади якого неодноразово ставали вразливостями підвищення привілеїв; тому досліджують формально обґрунтовані альтернативи на основі абстрактної інтерпретації (PREVAIL [181], використаний у eBPF для Windows) і верифікацію самого верифікатора (розділ 12). Переносність між версіями ядра забезпечує CO-RE (Compile Once — Run Everywhere) з описом типів ядра у форматі BTF.

7.2.3 DPDK і P4

DPDK переносить обробку в простір користувача: драйвер режиму опитування (poll mode) забирає пакети з черг карти без переривань і системних викликів. Це дає найвищу продуктивність на універсальних процесорах, але код обробки — звичайна програма на C без верифікатора, тож усі класи дефектів розділу 5 застосовні повною мірою. P4 [95] — мова опису конвеєра «розбирач — таблиці зіставлення-дії — збирач» для програмованих комутаторів і SmartNIC. Обмежена виразність (без циклів, фіксована кількість етапів) робить P4-програми придатними до вичерпної верифікації, але не до повноцінного conntrack. Порівняння технологій наведено в табл. 7.2.

Таблиця 7.2 — Технології площини даних для засобів фільтрації

Критерій	nftables	eBPF/XDP	DPDK	P4
Продуктивність	помірна, після `sk_buff`	висока, до `sk_buff`	найвища на ЦП, ядра опитування	лінійна швидкість апаратури
Безпека виконання	перевірений код ядра	верифікатор, обмежена модель	немає ізоляції, звичайний C	обмежена мова, без циклів
Гнучкість	декларативні правила	довільна логіка в межах верифікатора	повна	конвеєр таблиць, мало стану
Засоби верифікації	аналіз наборів правил	верифікатор, PREVAIL, тести `PROG_TEST_RUN`	SAST, фазинг, санітайзери	символьне виконання, перевірка моделей

Отже, масове відкидання доцільно виконувати в XDP або на P4, фільтрацію зі станом — у nftables, складну інспекцію — в окремих процесах із розділенням привілеїв (розділ 4).

7.3 Алгоритми класифікації пакетів

Класифікація пакетів — задача пошуку для пакета p першого (за пріоритетом) правила з N, кожне з яких задає умови на d полів заголовка: префікси адрес, діапазони портів, точні значення протоколу. Гупта і Маккеоун [189] систематизували підходи. *Лінійний пошук* переглядає правила по черзі; *дерева рішень* (HiCuts, HyperCuts) рекурсивно розрізають простір заголовків; *розбиття на кортежі* (Tuple Space Search, TSS [331]) групує правила за кортежем довжин префіксів $(l_1, \dots, l_d)$, у межах кортежу виконує точний геш-пошук за замаскованими полями, а кортежі перебирає; *TCAM* — асоціативна пам'ять, що порівнює ключ з усіма записами паралельно за один такт. Вартість у тактах на пакет для двох програмних методів:

$$C_{\text{лін}} = N\, c_{\text{п}}, \qquad C_{\text{TSS}} = m\,(c_{\text{х}} + c_{\text{м}}), \tag{7.1}$$

де $c_{\text{п}}$ — вартість перевірки одного правила; m — кількість різних кортежів ($m \ll N$); $c_{\text{х}}$ — вартість обчислення гешу й звертання до таблиці; $c_{\text{м}}$ — вартість маскування ключа. Пам'ять в обох методах лінійна, $O(N)$. Нехай $N = 10\,000$, $c_{\text{п}} = 6$ тактів: лінійний пошук потребує $60\,000$ тактів — у 300 разів більше за бюджет ≈ 200 тактів одного ядра для 10 Гбіт/с і пакетів по 64 байти (1.1). TSS з $m = 40$ і $c_{\text{х}} + c_{\text{м}} = 40$ тактів дає 1600 тактів — у межах бюджету восьми ядер. Тому класифікатор поєднують із кешем потоків (рис. 7.1): Open vSwitch кешує результати TSS у «мегапотоках» [289].

Для $d > 3$ полів, як показано в [189], задача в найгіршому випадку зводиться до локалізації точки серед N гіперпрямокутників, що має фунда-

ментальний компроміс

$$T = O(\log N),\ M = O(N^d) \quad \text{або} \quad T = O(\log^{d-1} N),\ M = O(N), \tag{7.2}$$

де T — час пошуку, M — обсяг пам'яті. Для $N = 10^4$ і $d = 5$ перший варіант потребує до 10^{20} комірок, другий — до $(\log_2 10^4)^4 \approx 13{,}3^4 \approx 3{,}1 \cdot 10^4$ звертань до пам'яті. Обидві межі непрактичні, тож евристичні алгоритми спираються на структуру реальних правил. TCAM обходить компроміс апаратно, але має обмежену ємність, а діапазон w-бітового поля в найгіршому випадку розгортається в $2w - 2$ префіксів: правило з довільними діапазонами обох портів займає до $(2 \cdot 16 - 2)^2 = 900$ записів.

Залежність часу класифікації від структури правил і трафіку створює загрозу *атак алгоритмічної складності*: порушник підбирає пакети, що примушують класифікатор працювати в найгіршому режимі. Так, атака Tuple Space Explosion [142] змушує кеш мегапотоків Open vSwitch породжувати велику кількість масок, і кожен пакет проходить лінійний перебір кортежів. Захист — обмеження кількості масок, ключовані геш-функції, квоти повільного шляху (розділ 14).

7.4 Політики та їхні аномалії

7.4.1 Формальна модель

Подамо правило як кортеж $R_i = (P_i, a_i)$, де $P_i \subseteq \Omega$ — множина пакетів (точок простору заголовків Ω), що задовольняють умови правила, $a_i \in \{\text{accept}, \text{deny}\}$ — дія. Політика з першим збігом — упорядкований список $R_1, \ldots, R_N$ з правилом за замовчуванням; рішення для пакета p визначає правило з найменшим номером, що його містить. Тоді множина пакетів, на які *фактично* впливає правило R_j, дорівнює

$$E_j = P_j \setminus \bigcup_{i<j} P_i, \tag{7.3}$$

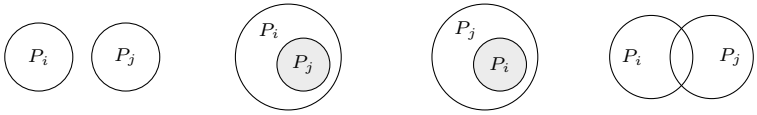
і правило ніколи не спрацьовує, якщо $E_j = \varnothing$. Аль-Шаер і Хамед [66] класифікували попарні аномалії правил R_i і R_j, $i < j$ (рис. 7.2):

Затінення (shadowing): $P_j \subseteq P_i$, $a_i \neq a_j$ — R_j ніколи не спрацьовує, намір адміністратора не виконується; найнебезпечніша аномалія.

Надлишковість (redundancy): $P_j \subseteq P_i$, $a_i = a_j$ — правило зайве.

Узагальнення (generalization): $P_i \subset P_j$, $a_i \neq a_j$ — верхнє правило задає виняток; часто навмисне, тому є попередженням.

Кореляція (correlation): $P_i \cap P_j \neq \varnothing$, жодна множина не є підмножиною іншої, $a_i \neq a_j$ — рішення для перетину залежить від порядку правил.



а) незалежність б) затінення / надлишковість в) узагальнення г) кореляція

Рис. 7.2 — Відношення множин пакетів двох правил R_i, R_j ($i < j$) і відповідні аномалії (за [66])

Числовий приклад. Нехай R_1: deny TCP з 10.0.0.0/8 на порт 22; R_2: accept TCP з 10.1.0.0/16 на порт 22. Оскільки 10.1.0.0/16 $\subset$ 10.0.0.0/8, за (7.3) $E_2 = \varnothing$ — затінення: адміністратор хотів дозволити SSH підмережі 10.1.0.0/16, але доступ заблоковано. Після перестановки правил маємо узагальнення, і намір виконується. Попарна перевірка $N = 2000$ правил — це $\approx 2 \cdot 10^6$ порівнянь, але правило, затінене *об'єднанням* кількох попередніх, виявляє лише обчислення (7.3).

7.4.2 Трансляція і верифікація політик

Площина керування транслює політику в термінах зон, груп об'єктів і застосунків у низькорівневі правила. Трансляція множить їхню кількість (50 джерел, 40 призначень і 10 служб дають 20 000 правил) і є джерелом дефектів: неправильний порядок, втрата винятків, різна семантика груп на різних платформах. Накопичення застарілих правил зменшують керування правилами впродовж життєвого циклу доступу — від запиту до автоматичного відкликання [48] — і контекстно орієнтовані політики нульової довіри [47].

Формальна верифікація кодує множини P_i булевими формулами над бітами заголовка, і запитання «чи існує пакет, який політика A дозволяє, а B забороняє» стає задачею виконуваності (SAT/SMT) або побудови двійкової діаграми рішень (BDD), у якій порожність E_j перевіряється за сталий час; контрприклад — готовий тест. Batfish [176] будує з конфігурацій модель площини даних мережі і перевіряє досяжність та ізоляцію до розгортання змін — як ворота якості CI/CD (розділ 3).

7.5 Таблиця станів і стійкість до виснаження

Таблиця з'єднань — ресурс, витрату якого контролює порушник: кожен новий потік, зокрема підроблений, займає запис до спливання тайм-ауту. Середню кількість записів дає закон Літтла:

$$\bar{n} = \lambda \cdot \bar{T}, \tag{7.4}$$

де λ — інтенсивність появи нових записів, $\bar{T}$ — середній час їх перебування в таблиці. Для легітимного навантаження $\lambda = 10^5$ з'єднань/с і $\bar{T} = 30$ с ма-

ємо $\bar{n} = 3 \cdot 10^6$ записів (близько 0,9 ГБ за 300 байтів на запис). SYN-повінь 10^6 пакетів/с із тайм-аутом напіввідкритого з'єднання 120 с (типове значення `nf_conntrack_tcp_timeout_syn_sent` у Linux) потребувала б $1{,}2 \cdot 10^8$ записів — таблиця переповнюється за секунди. Тайм-аут 5 с знижує потребу до $5 \cdot 10^6$, але не усуває залежності від λ.

Тому захист багаторівневий [159]: жорстка межа таблиці з витісненням насамперед напіввідкритих записів; адаптивні тайм-аути; квоти нових потоків на джерело і зону; **SYN cookies**, за яких сервер або SYN proxy кодує параметри з'єднання в початковому порядковому номері й не створює стану до підтвердження клієнтом. Вибір між fail-open і fail-closed за переповнення документують як архітектурне рішення (розділ 14).

7.6 Тестування коректності фільтрів

Помилка фільтра мовчазна, тому оракул задають явно. *Модульні тести правил* перевіряють вердикт на межах кожного правила — граничні адреси префікса, крайні порти, фрагменти, некоректні заголовки (розділ 11). *Диференційне тестування* подає однакові пакети на XDP-програму і модель політики (7.3) і повідомляє про розбіжності; з фазингом (розділ 13) воно знаходить неочевидні випадки. Для eBPF ядро надає команду `BPF_PROG_TEST_RUN`: програма виконується на поданому буфері без мережевої карти, а тест отримує код повернення.

XDP-програму, що відкидає UDP-пакети із забороненого порту джерела, наведено в лістингу 7.1. Перед кожним звертанням до заголовка перевіряється, що він лежить у межах `data_end`, інакше верифікатор відхилить програму; зміщення UDP залежить від `ihl`, тому перевіряється і його мінімум.

Лістинг 7.1 — XDP-програма з перевірками меж (C)

```
#include <linux/bpf.h>
#include <linux/if_ether.h>
#include <linux/in.h>
#include <linux/ip.h>
#include <linux/udp.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>
#define BLOCKED_SRC 11211   /* порт-джерело рефлексії */
SEC("xdp")
int drop_port(struct xdp_md *ctx)
{
    void *data = (void *)(long)ctx->data;
    void *end = (void *)(long)ctx->data_end;
    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > end) return XDP_PASS;
    if (eth->h_proto != bpf_htons(ETH_P_IP)) return XDP_PASS;
    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > end || ip->ihl < 5) return XDP_PASS;
    if (ip->protocol != IPPROTO_UDP) return XDP_PASS;
```

```
    struct udphdr *udp = (void *)ip + ip->ihl * 4;
    if ((void *)(udp + 1) > end) return XDP_PASS;
    return udp->source == bpf_htons(BLOCKED_SRC)
           ? XDP_DROP : XDP_PASS;
}
char LICENSE[] SEC("license") = "GPL";
```

Тест (лістинг 7.2) запускає закріплену програму через `bpftool prog run` на пакетах Scapy, зокрема з некоректним `ihl` і усіченим кадром.

Лістинг 7.2 — Тест вердиктів XDP-програми (Python, Scapy)

```
import re, subprocess
from scapy.all import Ether, IP, UDP, TCP
DROP, PASS = 1, 2                # коди XDP_DROP, XDP_PASS
def verdict(pkt: bytes) -> int:  # BPF_PROG_TEST_RUN
    out = subprocess.run(["bpftool", "prog", "run", "pinned",
        "/sys/fs/bpf/drop_port", "data_in", "-"], input=pkt,
        capture_output=True, check=True).stdout.decode()
    return int(re.search(r"Return value: (\d+)", out)[1])
E = Ether() / IP(dst="192.0.2.10")
for pkt, want in [(E / UDP(sport=11211), DROP),
    (E / UDP(dport=11211), PASS), (E / TCP(sport=11211), PASS),
    (Ether() / IP(ihl=4) / UDP(sport=11211), PASS),
    (bytes(E / UDP(sport=11211))[:38], PASS)]:      # усічений
    assert verdict(bytes(pkt)) == want, repr(pkt)
```

7.7 Кейс: XDP-фільтр для захисту сервісу оператора зв'язку

Публічні сервіси умовного українського регіонального оператора (DNS-резолвер, вебпортал абонентів) під час війни систематично зазнають DDoS-атак відбиттям і посиленням через відкриті сервери memcached, NTP і DNS [107]. Вимоги: відкидати трафік відбиття до стеку ядра за 10 Гбіт/с на інтерфейс; оновлювати заборонені порти й префікси без перезавантаження програми; за невизначеності передавати пакет до nftables із фільтром зі станом і квотами.

Архітектура. XDP-програма, розширена з лістингу 7.1, читає заборонені порти з мапи-масиву, префікси джерел — з мапи `LPM_TRIE`; агент площини керування оновлює мапи і збирає лічильники. *Верифікатор* відхилив першу версію через читання поля UDP до перевірки меж і цикл обходу заголовків IPv6 без сталої межі; завантаження у верифікатори кількох версій ядра стало кроком CI. *Тести.* Диференційне порівняння з моделлю політики на випадкових пакетах виявило хибне відкидання: у ненульових фрагментах IP на місці «заголовка UDP» лежать дані, і їхній збіг із забороненим портом відкидав легітимний трафік; такі фрагменти тепер передаються до nftables для обробки після збирання. *Вимірювання* за RFC 2544 [98] порівнюють частку відкинутих пакетів атаки і затримку легітимного трафіку. Урок: верифікатор гарантує безпеку виконання, але не коректність політики.

Висновки до розділу

Від пакетного фільтра до NGFW зростали контекст рішення ME і його поверхня атаки, а TLS-інспекція робить сам ME відповідальним за стійкість перехоплених з'єднань. Вибір площини даних (nftables, eBPF/XDP, DPDK, P4) — компроміс між продуктивністю, безпекою виконання і гнучкістю. Класифікація пакетів у найгіршому випадку має фундаментальний компроміс час–пам'ять (7.2), тому практичні алгоритми покладаються на структуру правил, що відкриває шлях атакам алгоритмічної складності. Модель політики (7.3) дає точні означення аномалій і зводить верифікацію до SAT/SMT або BDD, яку варто вбудовувати в конвеєр змін. Ємність таблиці станів оцінюється законом Літтла (7.4), а стійкість до виснаження потребує меж, адаптивних тайм-аутів, квот і SYN cookies. Верифікатор eBPF гарантує безпеку виконання, але коректність фільтра забезпечують лише тести з явним оракулом.

Питання для самоперевірки

1. Порівняйте архітектури ME за контекстом рішення та ризиками реалізації (табл. 7.1).
2. Поясніть розділення швидкого і повільного шляхів (рис. 7.1). Коли кешоване рішення стає некоректним?
3. Які перевірки виконує верифікатор eBPF? Чому сам верифікатор є частиною поверхні атаки?
4. Порівняйте XDP, DPDK і P4 (табл. 7.2). Що ви оберете для першого рубежу захисту від DDoS?
5. За (7.1) обчисліть вартість лінійного пошуку і TSS для $N = 50\,000$, $m = 120$ і порівняйте з бюджетом (1.1) для 25 Гбіт/с.
6. Поясніть компроміс (7.2). Як TCAM обходить його і чому розгортання діапазонів портів збільшує кількість записів?
7. Дайте формальні означення затінення, надлишковості, узагальнення і кореляції. Чому попарного аналізу недостатньо?
8. Як BDD і SAT/SMT застосовують для верифікації політик? Що перевіряє Batfish до розгортання змін? Які ризики створює TLS-інспекція і як їх тестувати?
9. За (7.4) оцініть розмір таблиці станів для $\lambda = 5 \cdot 10^4$ з'єднань/с, $\bar{T} = 60$ с і SYN-повені $2 \cdot 10^6$ пакетів/с з тайм-аутом 10 с.
10. Поясніть принцип SYN cookies і атаку на алгоритмічну складність класифікатора. Які заходи протидіють кожній?
11. Які перевірки меж у лістингу 7.1 вимагає верифікатор? Доповніть тест лістингу 7.2 випадками для фрагментів IP.

12. Дослідницьке: розробіть метод диференційного тестування, що порівнює XDP-програму з формальною моделлю політики (7.3), і оцініть покриття простору заголовків генерованими пакетами.
13. Дослідницьке: запропонуйте класифікатор із гарантованою верхньою межею часу для будь-якого трафіку і дослідіть його стійкість до атак типу Tuple Space Explosion.

РОЗДІЛ 8

Розроблення систем виявлення та запобігання вторгненням

Мережева система виявлення вторгнень (IDS) і її активний різновид — система запобігання вторгненням (IPS) — найскладніший щодо обробки вмісту клас ПЗМБ: вона розбирає десятки протоколів трафіку, контрольованого порушником, відтворює стан кожного з'єднання і зіставляє його з десятками тисяч правил у бюджеті часу (1.1). Розглянуто конвеєр NIDS/NIPS, рушії, зіставлення шаблонів, неоднозначності TCP/IP, правила як код, аналітичні компоненти й шифрований трафік, продуктивність і кейс за звітом CERT-UA. Модель загроз IDS-сенсора наскрізного прикладу (підрозділ 1.9) наведено в розділі 4, оцінювання ефективності виявлення — у розділі 15.

8.1 Архітектура NIDS/NIPS як програмного конвеєра

NIST SP 800-94 [317] розрізняє мережеві, безпроводові, поведінкові (NBA) і вузлові системи виявлення та запобігання вторгненням. З погляду розробника мережевий сенсор — конвеєр етапів (рис. 8.1) з власними інваріантами, ресурсами і класами дефектів.

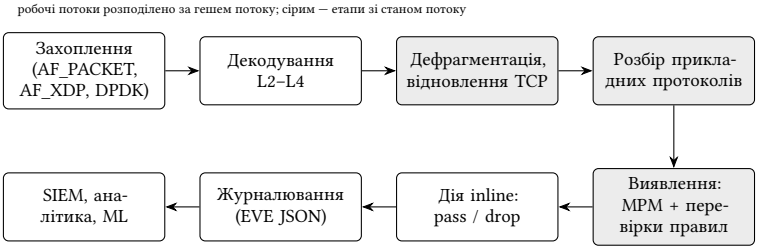


Рис. 8.1 — Конвеєр мережевої системи IDS/IPS

Декодування заголовків L2–L4 і тунелів — перша межа довіри з вимогами розділу 5. *Відновлення потоків* впорядковує сегменти TCP у байтовий потік кожного напрямку; цей етап має найбільший стан і є головною метою атак на виснаження пам'яті. *Розбір протоколів* виокремлює поля для правил (`http.uri`, `tls.sni`, `dns.query`), а *журналювання* формує сповіщення і метадані потоків — у Suricata записи EVE JSON [274].

IDS отримує копію трафіку (SPAN-порт, відгалужувач): її відмова не порушує зв'язку, а втрачений пакет означає «сліпе вікно». IPS стоїть у розри-

ві (`NFQUEUE`, пара інтерфейсів): її затримка додається до мережевої, а відмова зупиняє трафік. Звідси рішення про **fail-open** (пропускати без перевірки, зберігаючи доступність) або **fail-closed** (блокувати, зберігаючи захищеність), що перевіряють за методикою розділу 14. Латентність IPS задають перцентилем ($p_{99} \le 1$ мс), бо її визначають черги (підрозділ 8.7).

8.2 Snort 3, Suricata і Zeek

Snort Реша [307] задав мову правил — фактичний стандарт галузі; Snort 3 переписано на C++ з багатопотоковістю. Suricata (OISF, з 2010 р.) багатопотокова, сумісна з мовою правил Snort і розширює її полями протоколів. Zeek (колишній Bro) Паксона [286] перетворює трафік на семантичні події, а політику виявлення пише аналітик мовою сценаріїв зі станом. Їх часто поєднують: Suricata — для сигнатур і блокування, Zeek — для журналів протоколів і поведінкової аналітики (табл. 8.1).

Таблиця 8.1 — Порівняння рушіїв Snort 3, Suricata і Zeek

Критерій	Snort 3	Suricata	Zeek
Модель обробки	потоки пакетів, інспектори-модулі	робочі потоки за гешем потоку	рушій подій; кластер менеджера, проксі, робочих вузлів
Мова виявлення	правила Snort, конфігурація Lua	правила, сумісні зі Snort, + поля протоколів; Lua	сценарії Zeek зі станом
Розширюваність	плагіни C++	парсери на Rust/C, Lua, датасети	пакети Zeek, парсери Spicy
Основний вихід	сповіщення	EVE JSON: сповіщення, потоки, протоколи, файли	журнали протоколів (`conn`, `dns`, `ssl`)
Режим IPS	так	так (NFQ, AF_PACKET)	ні

8.3 Алгоритми зіставлення шаблонів

8.3.1 Автомат Ахо–Корасік

Правило містить літеральні шаблони (`content`) і регулярні вирази (`pcre`). Перевіряти N правил окремо неприйнятно, тому рушії спершу виконують **багатошаблонний пошук** (MPM) усіх літералів за один прохід. Алгоритм Ахо–Корасік [64] будує з шаблонів сумарної довжини m префіксне дерево з *функцією невдачі*, яка для кожного стану вказує на найдовший власний суфікс поточного префікса, що також є префіксом шаблону (рис. 8.2). Пошук у тексті довжини n виконує не більше $2n$ переходів:

$$T_{\mathrm{AC}} = O(n + m + z), \tag{8.1}$$

де $O(m)$ — побудова автомата, $O(n)$ — сканування, z — кількість входжень; час сканування не залежить від кількості шаблонів. *Приклад.* Для 10^4 шаблонів по 12 байтів ($m = 1{,}2 \cdot 10^5$) автомат має до $1{,}2 \cdot 10^5$ станів; повна таблиця переходів (256 символів по 4 байти) займе ≈ 123 МБ, що значно перевищує кеш процесора. Тому таблиці стискають, а швидкість визначають промахи кешу.

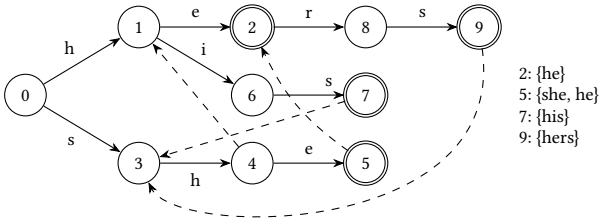


Рис. 8.2 — Автомат Ахо–Корасік для шаблонів {he, she, his, hers}; штрихові — нетривіальні переходи невдачі, решта веде в стан 0 (за [64])

8.3.2 Регулярні вирази й Hyperscan

NFA має $O(r)$ станів для виразу довжини r, але на кожному байті підтримує багато активних станів; DFA обробляє байт за один перехід, але має до 2^r станів. Ю та співавтори [348] показали, що типові для DPI конструкції (`.*A.*B`, обмежені повторення) спричиняють *вибух станів* при об'єднанні виразів в один DFA, і запропонували групувати вирази в кілька DFA. Рушії з поверненням (PCRE) мають ще й експоненційний час на окремих входах (ReDoS), тому для ПЗМБ потрібен рушій з лінійним часом або жорсткі ліміти перебору.

Hyperscan [341] декомпонує граф виразу на літерали, які шукаються SIMD-алгоритмами, і невеликі автомати, що запускаються лише біля знайдених літералів (прискорення Snort у 8,7 раза). Через пропрієтарну ліцензію нових версій Hyperscan (з 2024 р.) відкриті рушії переходять на відгалуження Vectorscan — приклад ризику ланцюга постачання (розділ 10).

8.3.3 Попередній фільтр «fast pattern»

Для кожного правила рушій обирає один літерал — **швидкий шаблон** (fast pattern; за замовчуванням найдовший або позначений ключовим словом `fast_pattern`) — і додає його в MPM. Повна перевірка правила виконується лише для кандидатів, чий швидкий шаблон знайдено. Середня вартість

інспекції L байтів

$$C = L\,c_{\text{MPM}} + \sum_{i=1}^{N} q_i\,c_i \;\approx\; L\,c_{\text{MPM}} + N\,\bar{q}\,\bar{c}, \tag{8.2}$$

де c_{MPM} — вартість сканування байта; q_i — імовірність появи швидкого шаблону правила i у даних; c_i — вартість повної перевірки правила. *Приклад* (умовні значення). Нехай $L = 1000$ байтів, $c_{\text{MPM}} = 2$ такти/байт, $N = 3 \cdot 10^4$, $\bar{c} = 2000$ тактів. За $\bar{q} = 10^{-4}$ маємо $C = 2000 + 6000 = 8000$ тактів; якщо частина правил має швидкий шаблон на кшталт `"GET "` і $\bar{q}$ зростає до 10^{-2}, то $C \approx 6{,}02 \cdot 10^5$ — у 75 разів більше. Тож якість швидкого шаблону — метрика якості правила.

8.4 Неоднозначність трафіку і нормалізація

Сенсор бачить пакети не так, як кінцевий вузол. Птачек і Ньюшем [296] описали наслідки цього розриву трьома класами атак: *вставлення* (сенсор приймає пакет, який вузол відкине), *ухилення* (навпаки) і *відмова в обслуговуванні* сенсора. Для розробника це *специфікація вимог* до відновлення потоків (табл. 8.2).

Таблиця 8.2 — Джерела неоднозначності та вимоги до сенсора

Неоднозначність	Причина розбіжності	Вимога до реалізації
Перекриття IP-фрагментів і сегментів TCP	стеки ОС по-різному обирають старі чи нові дані	збирання за політикою цільового вузла; подія-аномалія за розбіжного вмісту
TTL, недостатній до вузла	сенсор бачить пакет, вузол — ні	урахування топології; нормалізація TTL в IPS
Некоректні контрольні суми, опції	вузол відкидає, сенсор може прийняти	перевірки, еквівалентні стеку вузла
RST поза вікном	сенсор завершує потік передчасно	перевірка порядкових номерів у межах вікна
Пропуски через втрати	частину даних не побачено	явна позначка пропуску, лічильники

Нормалізація трафіку [190] застосовна в розриві: нормалізатор переписує потік, усуваючи неоднозначності (збирає фрагменти, відкидає суперечливі перекриття, вирівнює TTL), ціною можливого порушення наскрізної семантики протоколів. **Цільове відновлення** (target-based) залишає трафік незмінним, але збирає потік так, як це зробить конкретний вузол, за моделлю з інвентаризації або активного картографування; Suricata і Snort 3 задають політики дефрагментації й потоків для підмереж. Наслідки для розробника: диференційне тестування відновлення потоків з реальними

стеками ОС як оракулом (розділ 13) і ліміти стану на потік, вичерпання яких фіксується подією, а не мовчазним припиненням аналізу.

8.5 Розроблення й супровід правил як коду

8.5.1 Життєвий цикл і синтаксис правила

Набір правил — код, що виконується на кожному пакеті. Життєвий цикл правила: запит (звіт про загрозу, інцидент) → написання з метаданими (джерело, техніка MITRE ATT&CK [253]) → тести на PCAP → профілювання → пробна експлуатація лише зі сповіщенням → застосування, за потреби з `drop` → моніторинг **хибних спрацювань** (false positives; у посібнику синонім — *хибні тривоги*) → виведення з набору.

Правило Suricata складається з дії, заголовка і опцій у дужках [274]. Лістинг 8.1 показує фрагмент набору з кейсу (підрозділ 8.8): правила виявляють *порушення політики* і відомі індикатори.

Лістинг 8.1 — Фрагмент набору правил Suricata з метаданими

```
# Набір CERT-UA#4435: умовні індикатори (лише виявлення)
# 1. IEC-104 до RTU допускається лише з SCADA-серверів
alert tcp !$SCADA_SERVERS any -> $RTU_NET 2404 \
  (msg:"POLICY IEC-104 from non-SCADA host"; \
   flow:to_server,established; \
   threshold:type limit,track by_src,count 1,seconds 600; \
   classtype:policy-violation; sid:8000101; rev:3; \
   metadata:ref CERT-UA-4435, attack T0855;)
# 2. SSH з OT-сегмента в обхід вузлів-посередників
alert ssh $OT_NET any -> !$JUMP_HOSTS any \
  (msg:"POLICY SSH from OT segment"; \
   flow:to_server,established; \
   classtype:policy-violation; sid:8000102; rev:2;)
# 3. Файл з гешем зі звіту, переданий через SMB
alert smb any any -> any any \
  (msg:"IOC file hash from CERT-UA#4435 over SMB"; \
   filesha256:certua-4435.sha256; \
   classtype:trojan-activity; sid:8000103; rev:1;)
```

Мовою Sigma описують правила для *журналів* (події ОС, записи EVE і Zeek): YAML-документ задає джерело, умови відбору і логічний вираз, а конвертери транслюють його в запит SIEM. Мережа бачить переміщення між сегментами, журнали — дії на вузлах.

8.5.2 Тестування правил і конвеєр CI

Для кожного правила потрібні *позитивний* PCAP (записаний на стенді легітимними інструментами), *негативний* PCAP з близьким легітимним трафіком і *регресійний* корпус чистого трафіку. Формат тестів задає проєкт suricata-verify: каталог із PCAP, правилами і файлом `test.yaml`, що перевіряє записи EVE (лістинг 8.2).

Лістинг 8.2 — Тест правила у форматі suricata-verify

```
# tests/certua-4435-iec104/test.yaml
# PCAP: IEC-104 з тестового вузла поза SCADA_SERVERS
args:
  - --set vars.address-groups.SCADA_SERVERS=[10.10.1.5]
  - --set vars.address-groups.RTU_NET=[10.20.0.0/16]
checks:
  - filter:            # рівно одне сповіщення (threshold)
      count: 1
      match:
        event_type: alert
        alert.signature_id: 8000101
# Негативний тест --- окремий каталог: count: 0
```

Конвеєр CI правил (розділ 10) виконує перевірку синтаксису (`suricata -T`), контроль `sid`, `rev` і метаданих, тести suricata-verify, регресійний прогін, профілювання з порогом вартості і підписування пакета правил.

8.5.3 Метрики якості і хибні тривоги

Якість правила характеризують чутливість, частка хибних спрацювань, вартість (q_i, c_i) і стійкість до змін індикатора (правило на геші файлу крихке, на порушенні політики — стійке). Нехай правило i хибно спрацьовує на легітимному потоці незалежно від інших з імовірністю p_i. Тоді

$$P_{\text{FA}} = 1 - \prod_{i=1}^{N} (1 - p_i) \approx 1 - e^{-N\bar{p}}, \qquad E_{\text{FA}} = F \cdot P_{\text{FA}}, \tag{8.3}$$

де P_{FA} — імовірність хоча б однієї хибної тривоги на потік, $\bar{p}$ — середнє p_i, F — кількість потоків за добу, E_{FA} — очікувана кількість хибних тривог. *Приклад.* Для $N = 3 \cdot 10^4$, $\bar{p} = 10^{-8}$ і $F = 5 \cdot 10^7$ маємо $N\bar{p} = 3 \cdot 10^{-4}$ і $E_{\text{FA}} \approx 1{,}5 \cdot 10^4$ на добу. За 10 атакувальних потоків на добу і виявлення 90 % частка істинних сповіщень — $9/(9 + 15\,000) \approx 0{,}06\,\%$. Це *помилка базової частоти*, яку Аксельссон [81] назвав головною перешкодою практичному виявленню вторгнень. Тому p_i знижують уточненням контексту, повтори обмежують (`threshold`), а блокують лише правилами з дуже малим p_i (метрики — у розділі 15).

8.6 Аналітичні компоненти і шифрований трафік

Для невідомих загроз сенсор доповнюють евристиками (сканування, маячки з регулярним інтервалом) і моделями машинного навчання над *потоковими ознаками* (обсяги в кожному напрямку, розподіли розмірів і інтервалів, поля рукостискання) із записів `flow` Suricata і `conn` Zeek [58]. Inline допустимі лише легкі моделі; важчі працюють поза смугою. Детектори на

кшталт Kitsune [251] навчаються онлайн без розмітки, але вразливі до отруєння. Зоммер і Паксон [327] пояснили, чому ML у виявленні вторгнень працює гірше, ніж в інших галузях: висока ціна помилки, базова частота (8.3), мінливість трафіку, семантичний розрив між аномалією й атакою. Звідси вимоги: моніторинг *дрейфу* ознак, підписування моделей (розділ 10), пояснюваність сповіщень (розділ 15).

Більшість потоків нині захищено TLS, тож вміст недоступний сигнатурам без TLS-інспекції (розділ 7). **Аналіз шифрованого трафіку** (encrypted traffic analysis, ETA) спирається на метадані: розміри й інтервали записів, параметри рукостискання і відбитки клієнта — JA3 (геш версії, шифрів, розширень і груп ClientHello) та JA4, стійкий до випадкового порядку розширень у сучасних браузерах. Андерсон і Макгрю [73] показали, що поєднання ознак рукостискання, потоку й контексту DNS/HTTP дає змогу виявляти шкідливий шифрований трафік без розшифрування; огляд методів і контрзаходів наведено в [285]. Простір ознак звужується: TLS 1.3 [304] шифрує сертифікат сервера, ECH [305] — ім'я сервера, QUIC [213] — більшу частину транспортних заголовків, тож правила за `tls.sni` чи полями сертифіката деградують. Звідси вимоги: обчислювати відбитки й ознаки потоку в декодері і перевіряти деградацію виявлення на корпусах з TLS 1.3, ECH і QUIC.

8.7 Продуктивність і масштабування

Етапи зі станом (рис. 8.1) вимагають, щоб пакети потоку в *обох* напрямках потрапляли в той самий робочий потік. Карта розподіляє пакети за чергами механізмом RSS (геш Тепліца від 5-кортежу) зі стандартним несиметричним ключем. **Симетричне гешування** використовує ключ із повторюваним 16-бітовим шаблоном, за якого перестановка адрес і портів не змінює гешу; альтернатива — програмний розподіл (`cluster_flow`, XDP-балансувальник). Залишкова проблема — *слонові потоки*: один великий потік не ділиться між ядрами.

Робочий потік з кільцевим буфером на K пакетів моделюють системою M/M/1/K з інтенсивностями надходження λ і обслуговування μ. Імовірність відкидання пакета і середній час перебування (для M/M/1) дорівнюють

$$P_{\text{втр}} = \frac{(1-\rho)\,\rho^{K}}{1-\rho^{K+1}}, \qquad W = \frac{1}{\mu-\lambda}, \qquad \rho = \frac{\lambda}{\mu}. \tag{8.4}$$

Приклад. Для $\mu = 10^6$ пакетів/с, $\lambda = 9\cdot 10^5$ ($\rho = 0{,}9$) і $K = 2048$ маємо $P_{\text{втр}} \approx 0{,}1\cdot 0{,}9^{2048} \approx 10^{-95}$ і $W = 10$ мкс; за $\rho = 0{,}99$ затримка зростає до 100 мкс. Якщо ж через слоновий потік $\lambda = 1{,}2\cdot 10^6$ ($\rho = 1{,}2$), то за великого K маємо $P_{\text{втр}} \to 1 - 1/\rho \approx 16{,}7\,\%$ — буфер лише відкладає втрати. Через сплески модель дає нижню межу (розділ 14). Втрати визначає найбільш за-

вантажена черга, тому потрібні лічильники втрат по чергах і обхід (bypass) слонових і шифрованих потоків після інспекції їх початку.

8.8 Кейс: набір правил за звітом CERT-UA#4435

У квітні 2022 р. CERT-UA оприлюднила звіт про відвернуту атаку групи UAC-0082 (Sandworm) на об'єкт енергетики України [106]: вимкнення підстанцій програмою INDUSTROYER2 за протоколом IEC 60870-5-104 (TCP/2404), знищення даних (CADDYWIPER через групові політики), переміщення ланцюгами SSH-тунелів; звіт містить геші файлів. Команда розроблення сенсора (розділ 4) мала за тиждень підготувати набір правил з доведеною якістю, віддаючи перевагу правилам *поведінки й політики*, стійким до перекомпіляції шкідливого ПЗ, над атомарними індикаторами (табл. 8.3, лістинги 8.1–8.2).

Таблиця 8.3 — Набір правил кейсу і способи валідації

Ознака зі звіту	Джерело	Правило	Тести
Команди IEC-104 до підстанцій	Suricata	IEC-104 до RTU лише з SCADA-серверів	симулятор IEC-104 з дозволеного й недозволеного вузла
Ланцюги SSH-тунелів	Suricata, Zeek	SSH з OT-сегмента поза вузлами-посередниками	SSH-клієнт на стенді; сеанси адміністраторів
Виконання через SMB/DCERPC, GPO	Zeek, Sigma	нові служби, завдання, зміни політик з нетипових джерел	журнали тестового домену
Геші файлів	Suricata	`filesha256` для SMB і HTTP	безпечні файли з підставленими гешами

Правило IEC-104 не потребує розбору протоколу: перелік вузлів телемеханіки стабільний, тож з'єднання з іншого вузла — порушення політики з дуже малим p_i у (8.3). *Валідація.* Тести suricata-verify проходять на двох версіях рушія. На добовому корпусі чистого трафіку пілотного об'єкта правило SSH спершу спрацьовувало на системі резервного копіювання, тож її вузли додано до `$JUMP_HOSTS`. За нуля спрацювань на F потоках верхня межа p_i з довірою 95 % дорівнює $\approx 3/F$; її підставляють у (8.3). Після двох тижнів без хибних спрацювань правило IEC-104 переведено в режим `drop`. *Урок*: модель поведінки порушника і знання топології об'єкта дають стійкіші правила, ніж атомарні індикатори, а тест і метрика — частина правила.

Висновки до розділу

Мережева IDS/IPS — конвеєр, етапи якого мають власні інваріанти й ліміти стану; режим IPS додає вимоги до латентності й рішення про fail-open чи fail-closed. Пошук Ахо–Корасік має лінійний час (8.1) незалежно

від кількості шаблонів, а якість швидких шаблонів визначає вартість інспекції (8.2). Неоднозначності TCP/IP — специфікація вимог до відновлення потоків. Правила розробляють як код з тестами, профілюванням і конвеєром CI, пам'ятаючи, що хибні тривоги масштабуються з кількістю правил і потоків (8.3). TLS 1.3, ECH і QUIC зміщують виявлення до метаданих і відбитків. Втрати пакетів визначає найбільш завантажена черга (8.4). Кейс CERT-UA#4435 показав перевагу правил поведінки й політики над атомарними індикаторами.

Питання для самоперевірки

1. Опишіть етапи конвеєра NIDS/NIPS (рис. 8.1). Які з них мають стан потоку?
2. Порівняйте режими IDS і IPS за розміщенням, латентністю й наслідками відмови. Як обґрунтувати вибір між fail-open і fail-closed?
3. Порівняйте Snort 3, Suricata і Zeek (табл. 8.1). Коли доцільне їх поєднання?
4. Побудуйте автомат Ахо–Корасік для шаблонів {abc, bc, cab} з функцією невдачі. Поясніть складність (8.1).
5. Чому об'єднання регулярних виразів в один DFA спричиняє вибух станів? Які рішення застосовує Hyperscan?
6. За (8.2) обчисліть вартість інспекції 1500 байтів для $N = 5 \cdot 10^4$ за $\bar{q} = 10^{-4}$ і $\bar{q} = 5 \cdot 10^{-3}$. Як обрати добрий швидкий шаблон?
7. Сформулюйте три класи атак Птачека–Ньюшема і поясніть, чому вони є вимогами до реалізації сенсора (табл. 8.2). Порівняйте нормалізацію і цільове відновлення потоків.
8. Опишіть життєвий цикл правила і кроки конвеєра CI правил. Які тести обов'язкові? Як правила Sigma доповнюють правила Suricata?
9. За (8.3) оцініть добову кількість хибних тривог для $N = 4 \cdot 10^4$, $\bar{p} = 5 \cdot 10^{-9}$, $F = 10^8$. Яким має бути $\bar{p}$, щоб їх було не більше 100?
10. Які вимоги до ML-компонентів випливають з аргументів Зоммера і Паксона?
11. Які ознаки доступні IDS у трафіку TLS 1.3 з ECH і в QUIC? Що таке відбитки JA3/JA4 і які їхні обмеження?
12. За (8.4) обчисліть затримку й імовірність втрат для $\mu = 8 \cdot 10^5$ пакетів/с, $K = 1024$, $\rho \in \{0{,}8;\ 1{,}1\}$. Чому симетричне гешування не розв'язує проблеми слонових потоків?
13. Дослідницьке: запропонуйте метод диференційного тестування відновлення TCP-потоків щодо кількох стеків ОС: оракул, генератор послідовностей, метрику покриття.

14. Дослідницьке: сформулюйте задачу вибору підмножини правил і режимів (сповіщення/блокування) за обмежень (8.2) і (8.3); запропонуйте алгоритм і оцінювання q_i, p_i за даними експлуатації.

РОЗДІЛ 9

Розроблення засобів захищених комунікацій і контролю доступу до мережі

VPN-шлюзи, засоби ZTNA і NAC стоять на межі довіри й обробляють водночас ворожий трафік, ключі та облікові дані, тому саме вони домінують серед експлуатованих вразливостей граничних пристроїв (розділ 1). Розглянуто інженерію VPN, розподіл функцій між ядром і простором користувача, накладні витрати інкапсуляції, ZTNA і NAC, площину менеджменту, безпечне оновлення, керування секретами і кейс WireGuard. Наскрізний приклад VPN-шлюзу описано в підрозділі 1.9; криптографічні бібліотеки й постквантову міграцію — у розділі 6, формальну верифікацію — у розділі 12.

9.1 Інженерія VPN: протоколи та реалізації

9.1.1 IPsec/IKEv2

Архітектура IPsec [223] розділяє протокол захисту даних ESP [222] і протокол узгодження ключів IKEv2 [220]. Типово *IKE-демон* у просторі користувача (charon у strongSwan, pluto у libreswan) розбирає IKE_SA_INIT та IKE_AUTH, перевіряє сертифікати X.509, виконує EAP і керує асоціаціями безпеки (SA), а *ядро* (у Linux — XFRM з базами SAD і SPD) лише шифрує й перевіряє пакети за встановленими SA. Складність (типи навантажень, фрагментація IKE, NAT-T [83]) зосереджена в демоні, а гнучкість IPsec породжує небезпечні конфігурації й атаки зниження версії (розділ 6); віддалений доступ на IKEv2 (FlexVPN) розглянуто в [53].

9.1.2 WireGuard

WireGuard [153] — тунель третього рівня у вигляді віртуального інтерфейсу ядра Linux з рукостисканням за один обхід за шаблоном IK фреймворку Noise: Curve25519, ChaCha20-Poly1305, BLAKE2s, HKDF. Ключові рішення:

- *криптографічна маршрутизація* — відкритому ключу партнера відповідає перелік дозволених адрес (AllowedIPs) для вихідних і вхідних пакетів, тож маршрутизація і політика доступу збігаються;
- *відсутність криптографічної гнучкості* — фіксований набір алгоритмів, без переговорів і атак зниження версії;
- *мовчання* — на неавтентифіковані пакети відповіді немає, обробка починається з перевірки MAC; від виснаження процесора захищають зашифровані cookie;

- *жодного виділення пам'яті у відповідь на вхідні пакети*; сеансові ключі оновлюються приблизно кожні 120 с.

Реалізація для Linux має менш ніж 4 тис. рядків без криптографічних примітивів [153]. Даулінг і Патерсон [155] показали, що рукостискання не доводиться безпечним у класичній моделі AKE через підтвердження ключа першим транспортним повідомленням, і довели стійкість модифікованого варіанта; Ліпп, Бланше й Бхаргаван [236] отримали механізоване доведення в CryptoVerif (розділ 12).

9.1.3 TLS- і QUIC-VPN, SSH

OpenVPN використовує TLS як канал керування і власний протокол даних через UDP або TCP у процесі простору користувача, а модуль розвантаження (DCO) переносить канал даних у ядро. Портальні *SSL-VPN* поєднують TLS-тунель із вебпорталом, і саме вебчастина дала більшість критичних вразливостей (підрозділ 1.4). *MASQUE* (RFC 9298, RFC 9484) проксує UDP та IP-пакети поверх HTTP/3 і QUIC [213], потребуючи повного стека HTTP/3. Розділення привілеїв *SSH* (розділ 4) є еталоном для VPN-демонів. Порівняння — у табл. 9.1 і на рис. 9.1.

Таблиця 9.1 — Порівняння інженерних властивостей VPN-технологій

Технологія	Обсяг коду	Поверхня атаки	Гнучкість	Крипто-гнучкість
IPsec/ IKEv2	великий демон + ESP у ядрі	розбір IKE, X.509, EAP до автентифікації	висока, сумісність виробників	переговори; ризик зниження версії
WireGuard	$<$ 4 тис. рядків у ядрі	перевірка MAC до розбору; мовчання	лише L3, статичні ключі	відсутня
OpenVPN (TLS)	процес + бібліотека TLS	стек TLS, X.509, плагіни	висока; UDP і TCP	набори TLS
SSL-VPN порталу	вебсервер, портал, клієнт	розбір HTTP до автентифікації	дуже висока	набори TLS
MASQUE (QUIC)	стек HTTP/3 і QUIC	розбір QUIC і HTTP/3	висока; маскування під HTTPS	набори TLS 1.3
SSH	середній, privsep	розбір SSH до автентифікації	переадресація, адміністрування	переговори SSH

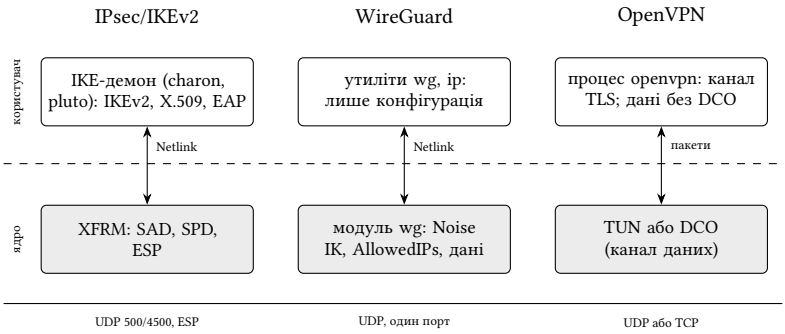


Рис. 9.1 — Розподіл компонентів VPN між простором користувача і ядром

9.2 Ядро чи простір користувача: продуктивність, ізоляція, накладні витрати

9.2.1 Компроміс розміщення

Розміщення в ядрі усуває копіювання пакетів і перемикання контексту: у вимірюваннях 2017 р. на гігабітному каналі WireGuard дав близько 1011 Мбіт/с, IPsec — 825–881 Мбіт/с, OpenVPN — 258 Мбіт/с [153]. Ціна — наслідки дефекту: помилка пам'яті в ядрі компрометує всю систему, а в демоні з монітором (підрозділ 4.4) — лише непривілейований процес. Звідси правило: *у ядро виносять лише простий шлях даних фіксованого формату,*

а складний розбір (X.509, EAP, TLS, HTTP) лишають у просторі користувача під розділенням привілеїв. Рукостискання WireGuard у ядрі складається з повідомлень фіксованої довжини, що спершу перевіряються за MAC. Альтернатива — реалізації мовами з безпечною пам'яттю в просторі користувача (wireguard-go, boringtun).

9.2.2 Модель накладних витрат інкапсуляції

Нехай внутрішній IP-пакет має довжину L байт, з яких h_{in} — заголовки внутрішніх IP і TCP; тунель додає H байт (зовнішні заголовки, заголовок протоколу тунелю, тег автентифікації) і доповнення $p(L)$; M — MTU зовнішнього каналу, h_{IP} — зовнішній IP-заголовок, h_{L2} — накладні витрати канального рівня на кадр (для Ethernet з преамбулою і міжкадровим інтервалом — 38 байт), C — швидкість лінії. Ефективна пропускна здатність для корисних даних

$$R_{\text{еф}} = C\,\frac{L - h_{\text{in}}}{L + H + p(L) + (n-1)\,h_{\text{IP}} + n\,h_{\text{L2}}}, \quad n = \left\lceil \frac{L + H + p(L) - h_{\text{IP}}}{M - h_{\text{IP}}} \right\rceil, \tag{9.1}$$

де n — кількість зовнішніх фрагментів (кожен наступний несе власний IP-заголовок). Для IPv4 WireGuard додає $H = 20 + 8 + 16 + 16 = 60$ байт (IP, UDP, заголовок транспортного повідомлення, тег Poly1305) і доповнює відкритий текст до кратного 16, *але не понад MTU тунелю*; ESP у тунельному режимі з AES-GCM і NAT-T — $H = 20 + 8 + 8 + 8 + 2 + 16 = 62$ байти (IP, UDP, SPI і номер, IV, хвостовик, ICV) з вирівнюванням до 4 байт [222].

Числовий приклад. $C = 1$ Гбіт/с, $M = 1500$, $h_{\text{in}} = 40$. Без тунелю $R = 1460/1538 \cdot C \approx 949$ Мбіт/с. WireGuard зі стандартним MTU тунелю 1420: пакет $L = 1420$ уже дорівнює MTU, тож за наведеним правилом $p = 0$, кадр $1420 + 60 + 38 = 1518$ байтів, $R = 1380/1518 \cdot C \approx 909$ Мбіт/с; ESP з NAT-T за $L = 1420$ має $p = 2$, кадр 1522 байти і $R \approx 907$ Мбіт/с. Якщо MTU тунелю помилково залишити 1500, пакет $1500 + 60 = 1560$ байтів фрагментується ($n = 2$): $R = 1460/1656 \cdot C \approx 882$ Мбіт/с, а ймовірність утрати пакета майже подвоюється. MTU 1420 вибрано із запасом для зовнішнього IPv6 ($H = 80$). Отже, обчислення MTU і MSS має бути автоматичним і тестованим; накладні витрати постквантового рукостискання оцінено формулою (6.4).

9.3 Засоби ZTNA і NAC

9.3.1 Архітектура за NIST SP 800-207

Архітектура нульової довіри [309] авторизує кожен запит до ресурсу за ідентичністю суб'єкта, станом пристрою і контекстом, а не за розташуванням у мережі. *Механізм політик* (PE) ухвалює рішення, *адміністратор полі-*

тик (PA) встановлює чи розриває сеанс, — разом це точка ухвалення рішень (PDP), — а *точка застосування політики* (PEP) пропускає трафік (рис. 9.2). Приклади реалізацій наведено в NIST SP 1800-35 [308], поетапне впровадження в гібридних інфраструктурах — в [7].

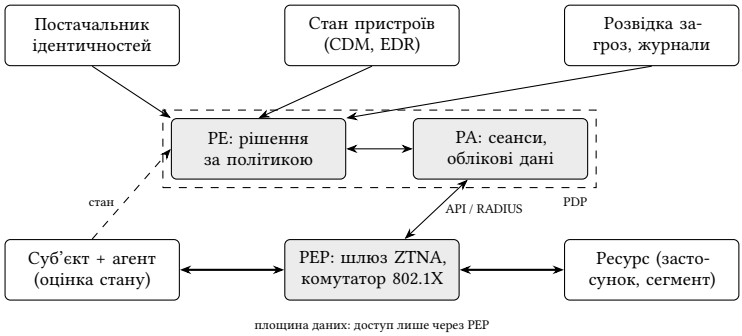


Рис. 9.2 — Логічні компоненти ZTNA/NAC (за [309])

9.3.2 Інженерні вимоги до компонентів

Агент на кінцевій точці збирає ознаки стану (posture): рівень виправлень ОС, шифрування диска, роботу EDR, сертифікат пристрою. Самозвіт агента скомпрометованої машини — лише твердження, тому сильніші схеми спираються на апаратну атестацію (TPM); сам агент — привілейоване ПЗ із вимогами підрозділів 9.4–9.5. *PEP* у ZTNA не відкриває ресурс у мережу: конектор біля ресурсу встановлює лише вихідні з'єднання до брокера. За недоступності PDP нові сеанси відхиляються (fail-closed, розділ 14). Безперервна оцінка вимагає повторної авторизації за зміни контексту; адаптивний вибір криптографічних параметрів каналу за динамічною оцінкою довіри запропоновано в [22].

9.3.3 NAC: IEEE 802.1X і RADIUS на боці сервера

У дротовому NAC комутатор є PEP, а RADIUS-сервер — PDP: після автентифікації EAP він повертає атрибути авторизації (VLAN, список доступу). Пристрої без прохача допускають за MAC-адресою в окремий сегмент. Для розробника сервера важливий урок *BlastRADIUS* (CVE-2024-3596) [184]: автентифікатор відповіді RADIUS — саморобна конструкція на MD5 зі спільним секретом, і порушник між клієнтом і сервером за допомогою колізії MD5 з вибраним префіксом за хвилини перетворює відмову на дозвіл. Уроки: (1) саморобні конструкції MAC неприпустимі; (2) необов'язковий захисний механізм (Message-Authenticator) не захищає, доки не перевіряється обов'язково й першим; (3) без версіонування застарілий примітив

живе десятиліттями; (4) відлунювані сервером атрибути несуть дані порушника. Виправлення: обов'язковий Message-Authenticator, RADIUS у TLS і RADIUS/1.1 (RFC 9765 [147]) без MD5.

9.4 Площина менеджменту як головне джерело вразливостей

9.4.1 Класи дефектів і принципи

Більшість критичних вразливостей граничних пристроїв (підрозділ 1.4) припадає не на криптографію чи шлях даних, а на *площину менеджменту* й портали: вебінтерфейси з розбором HTTP до автентифікації, REST API без авторизації кожної дії, обгортки CLI, що формують команди оболонки з введених даних. Вони працюють від root і часто доступні з Інтернету. Принципи, узгоджені з Secure by Design [114], переліком небезпечних практик [117] і настановами щодо граничних пристроїв [105]:

1. *мінімальна поверхня*: застарілі служби (Telnet, HTTP, SNMPv2c) вимкнені, вебінтерфейс — непривілейований процес, привілейовані дії — через монітор (розділ 4);
2. *автентифікація за замовчуванням*: жодних стандартних паролів, стійка до фішингу MFA адміністраторів;
3. *відмова від відкриття в Інтернет*: менеджмент за замовчуванням доступний лише на позасмуговому інтерфейсі;
4. *перевірювані вимоги* OWASP ASVS 5.0 [277] до вебінтерфейсу та API (табл. 9.2);
5. *журналювання адміністративних дій* з передаванням у SIEM і захистом від модифікації.

Таблиця 9.2 — Класи дефектів площини менеджменту і заходи

Клас (CWE)	Інженерний захід	ASVS 5.0, інше
Обхід автентифікації (287, 288)	перевірка в єдиному маршрутизаторі запитів; нові маршрути закриті за замовчуванням	V6, V8
Ін'єкція команд ОС (77, 78)	відмова від оболонки: `execve` з масивом аргументів, списки дозволених значень	V1
Обхід шляху, запис файлів (22)	канонізація, окремий каталог, процес без права запису в систему	V5
Відсутня авторизація API (862)	політика на кожну дію; тести матриці «роль × дія»	V4, V8
Стандартні облікові дані (1392, 798)	унікальний секрет пристрою, примусове налаштування	V6; CISA SbD
Експонування в Інтернет	менеджмент лише на OOB-інтерфейсі за замовчуванням	V13; [105]
Відсутність аудиту	журнал адміністративних дій, експорт у SIEM	V16

9.4.2 Модель частки вразливостей площини менеджменту

Нехай продукт має K площин (менеджмент, VPN, інше), N_i — кількість вразливостей площини i за період підтримки, $a_i \in [0,1]$ — частка інсталяцій, де площина досяжна для зовнішнього порушника. Кількість досяжних вразливостей і частка площини менеджменту m:

$$\Lambda = \sum_{i=1}^{K} N_i\, a_i, \qquad \varphi_m = \frac{N_m\, a_m}{\Lambda}. \tag{9.2}$$

Це модель-евристика з ілюстративними параметрами, окремий випадок (1.3), де a_i відіграє роль досяжності r_k. *Числовий приклад*: $N_m = 12$, $N_{\text{VPN}} = 6$, $N_{\text{інше}} = 2$. Якщо менеджмент відкрито в Інтернет усюди ($a_m = 1$), то $\Lambda = 20$, $\varphi_m = 0{,}6$. Якщо за замовчуванням він лише позасмуговий і його відкривають 5 % операторів ($a_m = 0{,}05$), $\Lambda = 8{,}6$ (на 57 % менше), $\varphi_m = 0{,}07$. Безпечне налаштування за замовчуванням змінює a_m без зміни коду; відповідальність за нього CISA покладає на виробника [114].

9.5 Безпечне оновлення прошивок і програмного забезпечення

9.5.1 Ланцюг довіри і схема A/B

Хто контролює оновлення, той контролює всі пристрої, тому механізм оновлення має забезпечувати автентичність образу, захист від відкату і відновлюваність (*захист*, *виявлення* і *відновлення* за NIST SP 800-193 [298]). Основа — *захищене завантаження*: незмінний код у ПЗП перевіряє підпис завантажувача, той — ядра, ядро — кореневої файлової системи (dm-verity); ключі підпису зберігаються в HSM поза конвеєром збирання (розділ 10).

Схема A/B тримає два системні розділи: образ записується в неактивний, завантажувач пробно запускає його і, якщо перевірка працездатності не підтвердить успіх, повертається до попереднього. *Захист від відкату* спирається на апаратний монотонний лічильник (eFuse, NV-індекс TPM, RPMB): образ із меншим номером не запускається. Лічильник збільшують лише *після фіксації* нового образу, інакше повернення A/B стане неможливим.

Нехай p_i — імовірність переривання встановлення, p_b — імовірність того, що новий образ не завантажиться, q — імовірність відмови механізму повернення, N — розмір парку. Імовірність втрати працездатності пристрою і очікувана кількість таких пристроїв

$$P^{(1)} = 1 - (1-p_i)(1-p_b), \qquad P^{(AB)} = q\,P^{(1)}, \qquad E = N\,P. \tag{9.3}$$

Числовий приклад. $p_i = 0{,}002$, $p_b = 0{,}001$, $q = 0{,}01$, $N = 10\,000$. Для оновлення «на місці» $P^{(1)} \approx 0{,}003$ і $E \approx 30$ пристроїв; для A/B $P^{(AB)} \approx 3 \cdot 10^{-5}$ і

$E \approx 0{,}3$. Систематичний дефект образу ($p_b \to 1$ для певної моделі) модель не покриває — проти нього застосовують поетапне розгортання з контрольною групою.

9.5.2 TUF, Uptane і SUIT

Підпис одним ключем не захищає від компрометації ключа, відкату чи «заморожування» метаданих. The Update Framework (TUF) [314] розділяє довіру між ролями *root* (офлайн), *targets* (що встановлювати), *snapshot* (стан репозиторію) і *timestamp* (свіжість); метадані мають строк дії й версії, а критичні ролі вимагають t підписів з n. За незалежної компрометації кожного ключа з імовірністю p імовірність підробки ролі

$$P_{\text{комп}}(t, n) = \sum_{k=t}^{n} \binom{n}{k} p^k (1-p)^{n-k}. \tag{9.4}$$

За $p = 0{,}05$ один ключ дає $0{,}05$, поріг 2 з $3 - 3 \cdot 0{,}05^2 \cdot 0{,}95 + 0{,}05^3 \approx 7{,}3 \cdot 10^{-3}$, поріг 3 з $5 - \approx 1{,}2 \cdot 10^{-3}$, тобто виграш у 7 і 43 рази. Модель справедлива лише за незалежності: ключі мають бути в різних HSM, у різних осіб і місцях. *Uptane* адаптує TUF для бортових блоків транспорту, а архітектура IETF SUIT (RFC 9019 [255]) описує ролі автора, сервера розповсюдження і пристрою та маніфест з ідентифікаторами класу пристрою, монотонним номером послідовності, гешем, розміром і підписом. Мінімальну перевірку на пристрої наведено в лістингу 9.1.

Лістинг 9.1 — Перевірка маніфесту й образу перед застосуванням (Python)

```
import hashlib, hmac, json
from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives.asymmetric.ed25519 \
    import Ed25519PublicKey

def check_update(pub, man, sig, img, counter, dev_class):
    """Повертає маніфест або піднімає ValueError."""
    try:  # 1. підпис перевіряється ДО розбору маніфесту
        Ed25519PublicKey.from_public_bytes(pub).verify(
            sig, man)
    except InvalidSignature:
        raise ValueError("недійсний підпис маніфесту")
    m = json.loads(man)
    if m.get("class") != dev_class:     # 2. інший пристрій
        raise ValueError("маніфест для іншого класу")
    if int(m["seq"]) <= counter:        # 3. захист від відкату
        raise ValueError("номер послідовності не зростає")
    if len(img) != int(m["size"]):      # 4. розмір і геш
        raise ValueError("невідповідність розміру")
    digest = hashlib.sha256(img).hexdigest()
    if not hmac.compare_digest(digest, m["sha256"]):
        raise ValueError("невідповідність гешу образу")
    return m  # лічильник збільшить завантажувач після фіксації
```

Номер береться з *підписаного* маніфесту, а версію перевіряє ще й завантажувач.

9.6 Керування конфігурацією і секретами

Закриті ключі шлюзу (IKE, WireGuard, TLS менеджменту) генеруються на пристрої і зберігаються в неекспортованому сховищі (TPM, HSM), а не у файлі конфігурації. *Експорт конфігурації* — типовий канал витоку: резервні копії шифруються ключем оператора, секрети виключаються, а імпорт перевіряє схему й підпис, як будь-який недовірений ввід.

9.7 Кейс: WireGuard як зразок мінімізації коду і безпеки за замовчуванням

Зіставимо WireGuard з типовим SSL-VPN граничного пристрою (підрозділ 1.4). У SSL-VPN перший ворожий пакет потрапляє до розбору HTTP і TLS у привілейованому процесі; у WireGuard — до перевірки MAC у полі фіксованого розташування. SSL-VPN підтримує паролі, SAML і плагіни і показує неавтентифікованим сторінку входу; WireGuard визнає лише ключі Curve25519, мовчить і має формально проаналізований протокол [155, 236]. Кожна відмінність зменшує один із множників моделі (1.3): обсяг S, досяжність r (MAC до розбору, мовчання) або частку вразливостей θ (немає розбору змінних структур). Ціна мінімалізму — керування користувачами, MFA і перевірку стану пристрою реалізують зовнішні компоненти, куди й переміщується поверхня атаки (підрозділ 9.4). Урок: *відокремити малий криптографічний шлях даних від складної, менш привілейованої площини менеджменту і не виставляти її в Інтернет.* Фіксований набір алгоритмів ускладнює постквантову міграцію: доступний лише PSK як додатковий секрет, тоді як IKEv2 підтримує гібридний обмін (розділ 6).

Висновки до розділу

Архітектура засобів захищених комунікацій визначається розподілом між простим шляхом даних фіксованого формату в ядрі і складним розбором у просторі користувача з розділенням привілеїв. WireGuard показує, що відмова від криптографічної гнучкості і мовчання дають поверхню атаки на порядки меншу, ніж у SSL-VPN. За моделлю (9.1) помилки MTU коштують кількох відсотків пропускної здатності і підвищують втрати. ZTNA і NAC переносять рішення з топології на ідентичність і стан пристрою, але додають агентів і PDP як нові цілі; BlastRADIUS засвідчує ціну саморобних конструкцій. Головне джерело вразливостей граничних пристроїв — площина менеджменту, а найдешевший захід — безпечні налаштування за замовчуванням (9.2). Безпечне оновлення поєднує підпис, захищене завантаження,

схему A/B, монотонний лічильник і розподіл довіри TUF/SUIT, обґрунтовані моделями (9.3) і (9.4).

Питання для самоперевірки

1. Як розподілено функції IPsec/IKEv2 між IKE-демоном і ядром? Які компоненти найризикованіші?
2. Поясніть криптографічну маршрутизацію WireGuard. Як вона поєднує маршрутизацію і політику доступу?
3. Які переваги й недоліки має відмова WireGuard від криптографічної гнучкості? Що показали формальні аналізи його рукостискання?
4. Порівняйте OpenVPN, SSL-VPN і MASQUE за поверхнею атаки.
5. Сформулюйте правило розміщення компонентів VPN між ядром і простором користувача.
6. Обчисліть за (9.1) ефективну пропускну здатність ESP без NAT-T для $L = 1420$ на каналі 1 Гбіт/с.
7. Назвіть компоненти PE, PA і PEP за NIST SP 800-207. Чому самозвіт агента про стан пристрою недостатній?
8. У чому суть BlastRADIUS і які чотири уроки він дає розробникам протоколів?
9. Які заходи протидіють класам дефектів площини менеджменту (табл. 9.2)?
10. Чому лічильник захисту від відкату збільшують лише після фіксації образу?
11. Як ролі TUF захищають від компрометації ключа і відкату? Коли справедлива модель (9.4)?
12. Дослідницьке: побудуйте модель вибору MTU тунелю за змішаного розподілу довжин пакетів і невідомого MTU шляху та методику її експериментальної перевірки.
13. Дослідницьке: запропонуйте метрику автоматизованого оцінювання безпечності налаштувань за замовчуванням площини менеджменту і пов’яжіть її з параметром a_m моделі (9.2).

РОЗДІЛ 10

Безпека ланцюга постачання програмного забезпечення та конвеєра CI/CD

Продукт мережевої безпеки складається здебільшого з чужого коду, тому його безпеку визначає не лише власний код (розділи 5, 6), а й цілісність усього шляху від розробника до пристрою клієнта — **ланцюга постачання ПЗ** (software supply chain). Розглянуто модель загроз ланцюга, SBOM і VEX, походження й цілісність збирань (SLSA, in-toto, Sigstore, відтворювані збирання), ризик залежностей, захист конвеєра CI/CD, метрики й кейс NGFW з наскрізних прикладів (підрозділ 1.9). Нормативні вимоги Регламенту (ЄС) 2024/2847 про кіберстійкість (CRA) наведено в розділі 2, доставлення оновлень на пристрій (TUF) — у розділі 9, обробку вразливостей компонентів — у розділі 16.

10.1 Модель загроз ланцюга постачання

10.1.1 Етапи і таксономії

Ланцюг постачання моделюють як граф етапів: *джерела*, *збирання*, *залежності* (кожна з власним ланцюгом) і *розповсюдження*. SLSA v1.2 [326] розрізняє загрози джерел (A — зловмисний виробник, B — несанкціонована зміна, C — компрометація системи керування кодом), збирання (D — зовнішні параметри, E — процес збирання), розповсюдження (F — публікація підробленого артефакту, G — компрометація каналу), споживання (H — вибір пакета-двійника, I — небезпечне використання), а також рекурсивні загрози залежностей (рис. 10.1). Ладіза та ін. [230] систематизували 107 векторів атак на відкриті ланцюги постачання у дерево атак і зіставили їх із 33 заходами захисту; формалізацію ланцюга подано також у [49]. В OWASP Top 10:2025 збоям ланцюга постачання відведено категорію A03 [281], організаційне керування ризиками (C-SCRM) описує NIST SP 800-161 Rev. 1 [96].

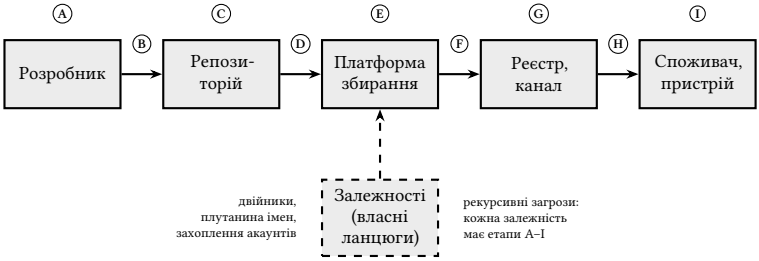


Рис. 10.1 — Ланцюг постачання ПЗ і точки атак A–I (за [326])

10.1.2 Резонансні інциденти

Інциденти 2020–2024 рр. (табл. 10.1) показали, що найнебезпечніші атаки обходять класичні засоби: шкідливий код підписано легітимним ключем і поширено штатним каналом оновлень. У SolarWinds Orion [121] бекдор SUNBURST вставлено під час збирання, тож репозиторій залишався чистим. У 3CX уперше зафіксовано каскад: троянізована програма на станції працівника привела до компрометації середовища збирання. Бекдор у xz-utils (CVE-2024-3094, CVSS 10,0) [264] — результат приблизно дворічної соціальної інженерії: зловмисник здобув права **супровідника** (maintainer), а об'єктний файл, замаскований під тестові дані, розпаковувався сценарієм збирання лише з *архіву випуску* (tarball), відсутнім у git; через libsystemd бекдор потрапляв у sshd і перехоплював автентифікацію SSH. Плутанина залежностей (2021) використала те, що менеджери пакетів за однакового імені обирали публічний пакет із вищою версією замість внутрішнього.

Таблиця 10.1 — Інциденти ланцюга постачання і контрзаходи

Інцидент	SLSA	Механізм	Контрзахід, що ускладнив би атаку
SolarWinds (2020)	E	вставка коду під час збирання; підпис легітимним сертифікатом	ізольоване ефемерне збирання (Build L3), незалежне відтворення
Плутанина залежностей (2021)	H	публічний пакет-двійник із вищою версією	приватні простори імен, один реєстр, гешовані lock-файли
3CX (2023)	E, D	каскад: троянізована програма на станції розробника → середовище збирання	відокремлення збирання від робочих станцій, походження, апаратні ключі
xz-utils (2024)	A, F	зловмисний супровідник; бекдор лише в архіві випуску	збирання з тегу git, порівняння архіву з git, огляд двома особами

Отже, контролювати слід *процес*, а не лише код — хто, з чого, де і як отримав артефакт; це дають походження (підрозділ 10.3) і SBOM.

10.2 Перелік компонентів ПЗ (SBOM) і VEX

10.2.1 Формати і мінімальні елементи

Перелік компонентів ПЗ (SBOM) — машинозчитуваний опис компонентів продукту, їхніх ідентифікаторів і залежностей. *CycloneDX* (OWASP; версія 1.6 стандартизована як ECMA-424 [158]) орієнтований на безпеку: VEX, сервіси, криптографічні активи (CBOM), моделі ШІ. *SPDX* (Linux Foundation; версія 2.2.1 — ISO/IEC 5962:2021 [208]) походить з ліцензійного аудиту; SPDX 3.0 (2024) має профілі Security, Build, AI. Компоненти ідентифікують через purl і CPE для зіставлення з базами вразливостей; граф `dependencies` має явно фіксувати й відсутність залежностей.

Мінімальні елементи NTIA 2021 р. переглянуто: після проєкту 2025 р. [120] CISA з NSA, FBI і 15 партнерськими агенціями 29.07.2026 оприлюднили остаточну редакцію [118]. Полів даних стало 17 замість семи: для компонента додано алгоритм і значення гешу та ліцензію, для самого SBOM — автора, *підпис автора*, формат, інструмент, *контекст генерації*, часову позначку й версію.

CRA (Додаток I, частина II, п. 1 [301]) вимагає складати SBOM у поширеному машинозчитуваному форматі *щонайменше для залежностей верхнього рівня* (розділ 2); SBOM входить до технічної документації й надається органам нагляду на запит, але публікувати його не обов'язково. SBOM підписують зовні, як атестацію до артефакту (підрозділ 10.3).

10.2.2 VEX

SBOM породжує шум: сканер повідомить про кожну CVE в кожному компоненті, хоча вразлива функція може бути не скомпільована чи недосяжна. **VEX** (Vulnerability Exploitability eXchange) — машинозчитуване твердження виробника про статус вразливості в продукті: *не уражено*, *уражено*, *виправлено*, *досліджується*; «не уражено» потребує обґрунтування (компонент чи вразливий код відсутні, код не на шляху виконання, не контрольований порушником, наявні вбудовані заходи). VEX виражають профілем CSAF 2.0 [135], у CycloneDX або OpenVEX; процес розглянуто в розділі 16.

10.2.3 Якість SBOM

Нехай T — компоненти, фактично наявні в артефакті (еталон з аудиту чи інструментованого збирання), S — компоненти в SBOM (збіг за purl і версією). Повнота виявлення (recall) і точність (precision) SBOM:

$$R_{\text{SBOM}} = \frac{|S \cap T|}{|T|}, \qquad P_{\text{SBOM}} = \frac{|S \cap T|}{|S|}, \qquad F_1 = \frac{2PR}{P+R}. \tag{10.1}$$

Числовий приклад. Прошивка містить $|T| = 420$ компонентів; інструмент аналізу образу видав $|S| = 390$ записів, з яких 372 збігаються з еталоном. Тоді $R = 372/420 \approx 0{,}886$, $P = 372/390 \approx 0{,}954$, $F_1 \approx 0{,}919$. Пропущені 48 компонентів — типово статично зібрані бібліотеки та **вбудовані копії коду** (vendoring) без метаданих пакета, а саме вони найчастіше містять давні вразливості. Тож SBOM генерують *під час збирання* з графа залежностей, а аналіз бінарних файлів використовують як незалежну перевірку повноти.

10.3 Походження і цілісність збирань

10.3.1 SLSA: трек збирання і трек джерел

Походження (provenance) — атестація про те, як отримано артефакт: геш результату, платформа, репозиторій і коміт, зовнішні параметри, залежності. *Трек збирання* SLSA [326]: Build L0 — гарантій немає; L1 — походження існує; L2 — керована платформа *підписує* походження; L3 — посилена платформа: збирання ізольовані, ключ підпису недоступний кроку користувача, тож походження не підробити зі сценарію збирання. Версія 1.2 (листопад 2025 р.) додала *трек джерел*: Source L1 — керування версіями; L2 — незмінна історія й атестації змін; L3 — примусові технічні контролі захищених гілок; L4 — огляд двома довіреними особами. Він адресує загрози A–C: SolarWinds закрила б Build L3 з незалежним відтворенням, а xz-utils — лише Source L4 разом зі збиранням із тегу.

10.3.2 in-toto і Sigstore

Рамка in-toto [336] задає підписаний власником проєкту *макет* (layout) — кроки ланцюга, уповноважених виконавців і очікувані матеріали та продукти; виконавці створюють підписані *метадані ланок*, а споживач перевіряє, що ланцюг виконано за макетом і результати кроків зчеплені гешами. Формат атестацій in-toto (subject + predicate) став основою походження SLSA.

Sigstore [263] усуває керування довгоживучими ключами: клієнт cosign генерує ефемерну пару, Fulcio після автентифікації OIDC (особи чи робочого процесу CI) видає короткоживучий сертифікат, а підпис фіксується в журналі прозорості Rekor. Перевіряють *ідентичність* («підписано процесом `release.yml` репозиторію X для тегу v4.2.0») і включення до журналу. Ключі образів прошивки, які перевіряє завантажувач, залишаються в HSM (розділ 9).

10.3.3 Відтворювані збирання

Збирання **відтворюване**, якщо з тих самих джерел, середовища й інструкцій будь-хто отримує побітово ідентичний артефакт [231]. Перешкоди — часові позначки, порядок файлів в архівах, шляхи збирання, локаль; їх усувають `SOURCE_DATE_EPOCH`, нормалізацією архівів і фіксованим образом середовища. Нехай p_b — імовірність непоміченої компрометації основної платформи, q_j — компрометації j-го незалежного відтворювача тим самим порушником. Підміна непомічена, лише коли скомпрометовано всіх:

$$P_{\text{непом}} = p_b \prod_{j=1}^{k} q_j. \tag{10.2}$$

За $p_b = 0{,}01$ і двох відтворювачів з $q = 0{,}05$ $P_{\text{непом}} = 0{,}01 \cdot 0{,}05^2 = 2{,}5 \cdot 10^{-5}$ — у 400 разів менше. Як і для порогових підписів (9.4), модель вимагає незалежності: відтворювачі мають використовувати різні платформи, облікові записи й постачальників образів. Відповідність загроз і контрзаходів підсумовано в табл. 10.2.

Таблиця 10.2 — Загрози ланцюга постачання і контрзаходи

Загроза (SLSA)	Контрзахід	Доказ для аудиту
A, B: зловмисна чи несанкціонована зміна	огляд двома особами, підписані коміти, захист гілок (Source L3–L4)	атестація джерел
C: компрометація системи коду	незмінна історія, MFA, журнал подій	атестація історії
D: зовнішні параметри	мінімальні параметри, фіксація в походженні	походження SLSA
E: процес збирання	ефемерні ізольовані виконавці (Build L3), відтворення	походження, звіт відтворення
F, G: публікація, канал	підпис Sigstore, журнал прозорості, TUF	запис Rekor, метадані TUF
H: вибір пакета	приватні простори імен, гешовані lock-файли, Scorecard	lock-файл, SBOM
Залежності (рекурсивно)	SCA, VEX, політика оновлень	SBOM, VEX

10.4 Керування залежностями та ризиком відкритих компонентів

10.4.1 Практики

Базові практики: *закріплення версій* і lock-файли з *гешами* (`Cargo.lock`, `go.sum`, `pip --require-hashes`); *приватне дзеркало* з карантином нових версій; заборона сценаріїв встановлення пакетів; *явні простори імен* внутрішніх пакетів; SCA з воротями за CVSS, EPSS і KEV [174, 123] (розділ 3). Нову залежність оцінюють OpenSSF Scorecard: близько двох десятків автоматичних перевірок — захист гілок, огляд коду, підписані випуски, фазинг, активність супровідників [349]. Високий бал не гарантує безпечності (xz-utils мав пристойні показники), але низький позначає занедбаний проєкт.

10.4.2 Кількісна модель ризику транзитивних залежностей

Нехай продукт транзитивно залежить від n компонентів, і за рік i-й компонент може бути скомпрометований незалежно з імовірністю p_i. Імовірність компрометації хоча б одного

$$P_{\text{комп}} = 1 - \prod_{i=1}^{n}(1-p_i) \;\approx\; 1 - e^{-\sum_i p_i} \;\approx\; \sum_i p_i \quad (\textstyle\sum_i p_i \ll 1). \tag{10.3}$$

Ціммерманн та ін. [351] показали для npm (2018 р.): встановлення середнього пакета неявно довіряє 79 стороннім пакетам і 39 супровідникам, а до 40 % пакетів залежать від коду з відомою вразливістю. Значення p_i невідомі, тож (10.3) — модель-евристика з ілюстративними параметрами. *Числовий приклад.* Для $n = 80$ і $p = 10^{-3}$ $P_{\text{комп}} = 1 - 0{,}999^{80} \approx 0{,}077$; для 300

унікальних компонентів — $1-0{,}999^{300} \approx 0{,}26$. Якщо 20 компонентів — проєкти з одним супровідником і $p = 5 \cdot 10^{-3}$, а решта 280 — з $p = 5 \cdot 10^{-4}$, то $\sum p_i = 0{,}1 + 0{,}14 = 0{,}24$ і $P \approx 0{,}21$: 7 % компонентів дають понад 40 % ризику, тож саме їх варто замінити, вбудувати з оглядом коду або ізолювати. Модель ігнорує кореляцію через спільних супровідників, тому є нижньою оцінкою.

10.5 Захист конвеєра CI/CD

Конвеєр CI/CD — привілейована система: він має доступ до коду, секретів підпису й публікації та виконує код із запитів на злиття. OWASP Top 10 CI/CD Security Risks [227] виокремлює, зокрема, недостатній контроль потоку змін, отруєння виконання конвеєра, гігієну облікових даних і неперевірену цілісність артефактів. Контрзаходи:

- *контроль потоку*: захист гілок, обов'язковий огляд без самозатвердження, підписані коміти й теги;
- *ізоляція*: ефемерні виконавці, окремі пули для запитів з форків (без секретів) і для випусків;
- *мінімальні привілеї*: токен за замовчуванням лише на читання, права — на рівні завдання;
- *OIDC-федерація*: замість довгоживучих секретів — короткоживучий токен ідентичності з умовою на репозиторій, гілку й процес;
- *сторонні дії* закріплюють гешем коміту, а не тегом, який можна перемістити; ключі підпису — у HSM/KMS;
- *інфраструктура як код*: сканування й тестування конфігурацій до розгортання [32].

Лістинг 10.1 ілюструє випусковий процес GitHub Actions; `<SHA>` — повний геш коміту дії.

Лістинг 10.1 — Випусковий робочий процес з SBOM і походженням (GitHub Actions)

```
name: release
on: {push: {tags: ["v*"]}}
permissions: {}             # за замовчуванням --- нічого
jobs:
  build:
    runs-on: ubuntu-24.04   # ефемерний виконавець
    permissions:
      contents: read
      id-token: write       # OIDC для Sigstore
      attestations: write
    steps:
      - uses: actions/checkout@<SHA>    # v4, закріплено
        with: {persist-credentials: false}
      - run: make reproducible          # детерміноване
```

```
      - uses: anchore/sbom-action@<SHA> # v0
        with: {format: cyclonedx-json,
               output-file: sbom.cdx.json}
      - uses: actions/attest-build-provenance@<SHA> # v2
        with: {subject-path: dist/ngfw.img}
      - uses: actions/attest-sbom@<SHA> # v2
        with: {subject-path: dist/ngfw.img,
               sbom-path: sbom.cdx.json}
```

Походження підписує платформа через OIDC, а споживач перевіряє репозиторій, процес, тег і геш образу. Такий процес відповідає Build L2: кроки `make` і генерація походження виконуються в одному завданні. Для L3 генерацію походження виносять у *повторно використовуваний робочий процес* (reusable workflow), ізольований від кроків збирання.

10.6 Метрики і зрілість ланцюга постачання

Покриття атестаціями для множини випущених артефактів A:

$$K_{\text{sig}} = \frac{|A_{\text{sig}}|}{|A|}, \qquad K_{\text{prov}} = \frac{|\{a \in A : \ \text{походження перевірене},\ L(a) \geq L^{*}\}|}{|A|}, \tag{10.4}$$

де A_{sig} — артефакти з перевіреним підписом, $L(a)$ — рівень SLSA, L^{*} — цільовий рівень. Для 46 артефактів випуску, з яких 41 підписаний, а 35 мають перевірене походження Build L3, $K_{\text{sig}} \approx 0{,}89$, $K_{\text{prov}} \approx 0{,}76$. Рахують лише *перевірені* споживачем атестації.

Свіжість залежностей Кокс та ін. [139] пропонують вимірювати відстань між використаною і найновішою версіями; у формі «бібліотеко-років» (libyears) запізнення підсумовують:

$$L_{\text{yr}} = \sum_{i=1}^{m} \max\bigl(0,\ t(v_i^{\text{new}}) - t(v_i^{\text{used}})\bigr), \tag{10.5}$$

де $t(\cdot)$ — дата випуску версії в роках. Для п’яти прямих залежностей із запізненнями 1,8; 0,3; 0; 2,5 і 0,9 року $L_{\text{yr}} = 5{,}5$, середнє 1,1 року, а одна залежність дає 45 % боргу. Метрика не враховує, чи містять новіші версії виправлення безпеки, тому її поєднують із MTTR і P_{SLA} за (3.8). Зрілість оцінюють за рівнями SLSA на треках джерел і збирання та за практиками PS.2–PS.3 і PW.4 SSDF [328] та SM-9/SM-10 IEC 62443-4-1 [197].

10.7 Кейс: захищений конвеєр для ПЗМБ українського виробника

Український виробник NGFW (наскрізний приклад; прошивка на Linux, 1 200 компонентів) виходить на ринок ЄС (розділ 2) і проходить державну експертизу у сфері ТЗІ [40]. Цілі: Source L3, Build L2 на першому етапі і

Build L3 після перенесення генерації походження в повторно використовуваний робочий процес; SBOM+VEX до кожного випуску; підпис і перевірка всіх артефактів (рис. 10.2).

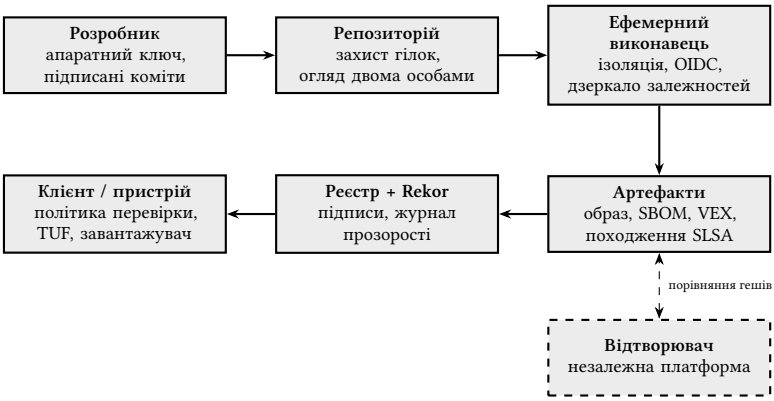


Рис. 10.2 — Архітектура захищеного конвеєра ПЗМБ

Джерела. Випускові гілки: огляд двома особами, підписані апаратними ключами коміти, CODEOWNERS для криптомодуля й парсерів. Сторонні проєкти беруть із підписаних тегів git, а архіви випусків порівнюють із тегом — урок xz-utils.

Залежності. Усі пакети — через внутрішнє дзеркало з 7-денним карантином і SCA; Scorecard обов'язковий для нових залежностей. За моделлю (10.3) аудит виявив 25 компонентів з одним супровідником ($p \approx 5 \cdot 10^{-3}$) і 1 175 інших ($p \approx 3 \cdot 10^{-4}$): $\sum p_i \approx 0{,}125 + 0{,}35 = 0{,}475$, $P_{\text{комп}} \approx 0{,}38$ на рік. Видалення 10, заміна 8 і вбудовування 7 таких пакетів з оглядом коду майже обнуляють внесок першої групи, а карантин (припущення: p удвічі менше) зменшує внесок другої до $0{,}18$, отже $P_{\text{комп}} \approx 0{,}16$; важливе ранжування заходів за внеском.

Збирання і випуск. Ефемерні виконавці в сегменті, недоступному з робочих станцій (урок 3CX), процес лістингу 10.1, незалежне відтворення (10.2). SBOM збирання звіряють з аналізом образу: мета $R_{\text{SBOM}} \geq 0{,}98$ за (10.1). Образ підписують ключем у HSM, атестації — через Sigstore; VEX (CSAF) публікують із бюлетенями PSIRT. Ті самі артефакти закривають CRA (SBOM, захищене розповсюдження оновлень — Додаток I, ч. II, пп. 1, 7) і держекспертизу; метрики керівництву — K_{prov}, $L_{\text{уг}}$ і P_{SLA}.

Висновки до розділу

Безпека ПЗМБ визначається цілісністю всього ланцюга постачання: інциденти SolarWinds, 3CX, xz-utils і плутанини залежностей показали, що порушник обирає найслабшу ланку — платформу збирання, станцію розробника, довіру до супровідника чи правило вибору пакета, — а легітимний підпис і штатний канал оновлень лише прискорюють поширення шкідливого коду. Таксономія SLSA (A–I) систематично зіставляє загрози з контрзаходами. SBOM (CycloneDX, SPDX) обов'язковий за CRA і уточнений мінімальними елементами CISA 2026 р.; його цінність визначають повнота й точність (10.1), а VEX відсікає неексплуатовані вразливості. Походження SLSA, in-toto, Sigstore і відтворювані збирання переводять довіру від окремих ключів і платформ до перевірюваних тверджень і незалежного відтворення (10.2). Ризик залежностей (10.3) зосереджений у малопідтримуваних компонентах, тож адресна мінімізація ефективніша за рівномірні заходи. Стан конвеєра вимірюють покриттям атестаціями (10.4), свіжістю залежностей (10.5) і MTTR. Відкриті проблеми — загроза A (зловмисний супровідник), кореляція ризиків, масштабована перевірка відтворюваності.

Питання для самоперевірки

1. Опишіть етапи ланцюга постачання ПЗ і загрози A–I за SLSA. Чому загрози залежностей є рекурсивними?
2. Порівняйте механізми інцидентів SolarWinds і 3CX. Чому підпис легітимним сертифікатом не захистив споживачів?
3. Які соціальні й технічні чинники уможливили бекдор у xz-utils? Чому аналіз репозиторію git його не виявляв?
4. Поясніть механізм плутанини залежностей і заходи протидії на рівні менеджера пакетів і реєстру.
5. Порівняйте CycloneDX і SPDX. Які поля додано в мінімальних елементах SBOM CISA 2026 р.?
6. Яку вимогу до SBOM містить CRA і чи зобов'язаний виробник публікувати SBOM?
7. Що таке VEX? Які статуси й обґрунтування він передбачає і як зменшує навантаження на PSIRT?
8. Обчисліть повноту, точність і F_1 SBOM за (10.1) для $|T| = 600$, $|S| = 570$, $|S \cap T| = 540$. Які компоненти найчастіше пропускаються?
9. Опишіть рівні треку збирання і треку джерел SLSA v1.2. Чому процес лістингу 10.1 відповідає Build L2, а не L3?
10. Поясніть безключовий підпис Sigstore: ролі cosign, Fulcio і Rekor. Що перевіряє споживач?

11. За (10.3) оцініть ризик для 150 залежностей з $p = 10^{-3}$ і покажіть, як зміниться результат після видалення 30 залежностей і карантину, що вдвічі знижує p.
12. Які ризики OWASP Top 10 CI/CD адресують OIDC-федерація, закріплення дій за гешем і ефемерні виконавці?
13. Дослідницьке: запропонуйте модель ризику залежностей, що враховує кореляцію через спільних супровідників (граф «пакет–супровідник»), і порівняйте її оцінки з моделлю (10.3) на реальному дереві залежностей ПЗМБ.
14. Дослідницьке: розробіть методику експериментального оцінювання повноти й точності SBOM для вбудованих прошивок мережевих пристроїв (статично зібрані компоненти й вбудовані копії коду) та метрики порівняння інструментів генерації.

РОЗДІЛ 11

Методологія тестування безпеки програмного забезпечення

Розділ відкриває блок технологій тестування і задає спільну методологічну рамку для розділів 12–15. Розглянуто процеси ISO/IEC/IEEE 29119, ризик-орієнтований розподіл зусиль, рівні й техніки проєктування тестів, проблему оракула, мутаційне тестування і тестування в Common Criteria. Трасованість вимог (розділ 4) тут є вхідними даними.

11.1 Тестування безпеки серед методів верифікації

Функціональне тестування перевіряє, що система *робить те, що повинна*. Тестування безпеки перевіряє переважно протилежне — що система *не робить того, чого не повинна*, за будь-якого, зокрема ворожого, входу: не приймає пакет з недійсним підписом, не пропускає трафік поза політикою. Такі вимоги негативні й квантифіковані за всіма входами, а тест — скінченна вибірка. Звідси асиметрія, сформульована Дейкстрою: тестування може показати наявність помилок, але не їхню відсутність [150]. Для безпеки вона гостріша, бо вхід обирає порушник: дефект, на який натрапляє один вхід із 2^{64}, байдужий для надійності, але критичний для безпеки (розділ 1).

Розрізняють **позитивне тестування** (заявлені функції безпеки працюють: дійсні облікові дані приймаються, тунель встановлюється) і **негативне тестування** (відмова на некоректних, граничних і зловмисних входах). Для ПЗМБ негативних тестів істотно більше; їхні специфікації виводять із випадків зловживання (розділ 4).

Мінімальний набір технік верифікації розробником визначає NISTIR 8397 [89]: моделювання загроз, автоматизоване тестування, статичне сканування, пошук жорстко закодованих секретів, захисти мови й компілятора, тести «чорної» скриньки і структурні тести, історичні (регресійні) тести, фазинг, вебсканери і перевірку включених компонентів. Порівняння методів наведено в табл. 11.1 (оцінки якісні).

Таблиця 11.1 — Методи верифікації безпеки ПЗМБ

Метод	Що виявляє	Стадія	Вартість / автомат.	Розділ
Моделювання загроз	вади проєктування, відсутні вимоги	проєктування	середня / низька	4
SAST, формальні методи	класи дефектів коду, порушення властивостей протоколу	реалізація	різна / висока	12
Модульні й протокольні тести	порушення специфікації, граничні випадки	реалізація, інтеграція	низька / висока	11.4
Санітайзери, фазинг	помилки пам'яті, збої розбору	реалізація, супровід	середня / висока	13
Тести стійкості	виснаження ресурсів, fail-open	система	висока / середня	14
Оцінювання виявлення	пропуски і хибні тривоги IDS	система	висока / середня	15
Аналіз складу ПЗ	вразливі залежності	збирання	низька / висока	10
Тест на проникнення	ланцюжки вразливостей, конфігурація	перед випуском	висока / низька	11.7

Жоден метод не є достатнім, тому ефективність виявлення (3.7) зростає від поєднання методів з різними «сліпими зонами».

11.2 ISO/IEC/IEEE 29119 і ризик-орієнтоване тестування

У серії ISO/IEC/IEEE 29119 частина 1 визначає поняття [210], частина 2 — процеси [211], частина 3 — документацію, частина 4 — техніки проєктування тестів [212]. Окремої «частини безпеки» немає: специфіку задають вимоги і вибір технік.

Частина 2 визначає процеси трьох рівнів (рис. 11.1): *організаційний*, *управління* і *динамічні процеси*; частина 3 — документи: план (обсяг, ризики, підхід, критерії входу й виходу), специфікації тестів, звіти.

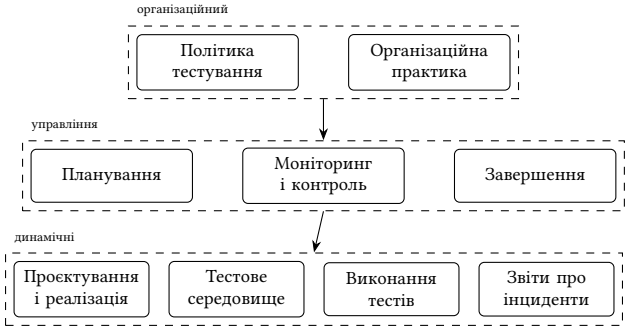


Рис. 11.1 — Рівні процесів тестування (за [211])

Центральна ідея серії — **ризик-орієнтоване тестування**: обсяг, глибина й порядок тестування визначаються ризиками, які переглядають за результатами тестів [171]. Для ПЗМБ ймовірність беруть з метрик поверхні атаки й історії вразливостей компонента (підрозділ 4.5), вплив — з моделі загроз.

Нехай продукт розбито на n областей; p_i і I_i — оцінки ймовірності вразливості й впливу в області i (шкала 1–5), E — бюджет (людино-дні), E_0 — мінімальний гарантований бюджет області. Тоді

$$R_i = p_i\, I_i, \qquad E_i = E_0 + \bigl(E - nE_0\bigr)\,\frac{R_i}{\sum_{j=1}^{n} R_j}. \tag{11.1}$$

Приклад. МЕ має чотири області: розбір заголовків ($p=4$, $I=5$, $R=20$), компіляція політики (3, 4, 12), вебінтерфейс керування (4, 4, 16), журналювання (2, 2, 4); $\sum R_j = 52$. За $E=40$ і $E_0=2$ маємо $E_i = 14{,}3;\ 9{,}4;\ 11{,}8;\ 4{,}5$ людино-дня. Модель лінійна, тому розподіл переглядають за фактичними знахідками.

11.3 Рівні тестування ПЗМБ

Модульне тестування найдешевше й найточніше локалізує дефект, тому основну масу негативних тестів розміщують тут. *Інтеграційне* перевіряє взаємодію компонентів (IKE-демон і IPsec ядра); типова вада — кожен компонент вважає, що валідацію виконує інший. *Системне* перевіряє продукт у середовищі розгортання, *приймальне* виконує замовник або лабораторія. *Регресійне* повторює тести після змін; до нього обов'язково додають тест на кожну виправлену вразливість.

Специфічний для ПЗМБ рівень — **тестування відповідності протоколу** (conformance testing) вимогам RFC (MUST, SHOULD) і **тестування взаємодії** (interoperability) з реалізаціями інших виробників; протокольні тести ETSI

специфікує мовою TTCN-3 з вердиктами `pass`, `fail`, `inconc`, а в сертифікації аналогічну роль відіграють оціночні дії cPP (розділ 2). Незалежність від мережі забезпечують **тестові дублери**: заглушки сокетів, фіктивні вузли-партнери з некоректним сценарієм обміну, мережеві простори імен, відтворення pcap. Співвідношення рівнів ілюструє **піраміда тестів** (рис. 11.2); фазинг проходить крізь усі рівні — від модульних обв'язок до системного фазингу протоколу.

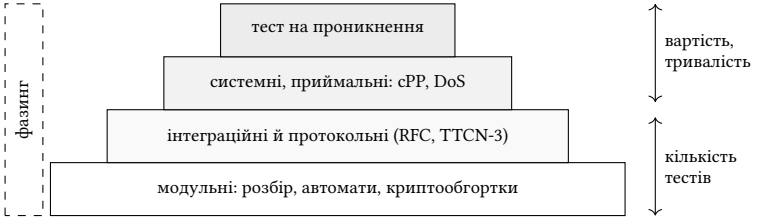


Рис. 11.2 — Піраміда тестів ПЗМБ

11.4 Техніки проєктування тестів

ISO/IEC/IEEE 29119-4 [212] поділяє техніки на засновані на специфікації, на структурі й на досвіді. *Класи еквівалентності* розбивають простір входів на класи з однаковою поведінкою; для поля версії протоколу — «підтримувана», «відома, але заборонена політикою», «невідома». Класи *недійсних* входів тестують поодинці, щоб одна помилка не маскувала іншу.

Аналіз граничних значень особливо важливий для мережевого коду, де межі задано полями довжини, лічильниками й зміщеннями: 0, мінімум, мінімум ± 1, «рівно до кінця буфера», «на байт за кінцем», максимум поля ($2^{16}-1$) і значення, що переповнює арифметику (розділ 5). Поле довжини загального заголовка корисного навантаження IKEv2 [220] включає 4-байтовий заголовок, тому граничні значення — 3, 4, довжина буфера і на одиницю більше (лістинг 11.1).

Лістинг 11.1 — Граничні значення поля довжини (Python, pytest)

```
import pytest
from ikev2.parse import parse_generic_hdr, ParseError
HDR = 4                       # довжина загального заголовка
def payload(length: int, body: int) -> bytes:
    # next payload, прапорці, довжина (big-endian), тіло
    return bytes(2) + length.to_bytes(2, "big") + bytes(body)
@pytest.mark.parametrize("length, body, ok", [
    (0, 0, False), (3, 0, False),   # менше за заголовок
    (4, 0, True), (5, 1, True),     # мінімум, мінімум + 1
    (64, 60, True),                 # рівно до кінця буфера
    (65, 60, False),                # на 1 байт за кінцем
```

```
    (65535, 65531, True),          # максимум поля
])
def test_length_bounds(length, body, ok):
    buf = payload(length, body)
    if not ok:
        with pytest.raises(ParseError):
            parse_generic_hdr(buf)
    else:
        assert len(parse_generic_hdr(buf).body) == length - HDR
```

Таблиці рішень застосовують для комбінацій умов: правило допуску ZTNA з чотирма умовами дає 2^4 стовпців-тестів. *Тестування на основі станів* спирається на модель автомата протоколу (підрозділ 6.5); критерії — усі стани, усі переходи, пари переходів. Для безпеки ключове покриття *недопустимих* переходів: кожна пара «стан $\times$ повідомлення», не передбачена специфікацією, має призводити до відмови. Повне покриття пар потребує $|S| \cdot |\Sigma|$ тестів, що для IKEv2 становить кілька десятків.

Комбінаторне тестування. Кун та співавтори [228] емпірично показали, що більшість збоїв спричиняє взаємодія одного-двох параметрів, а всі зафіксовані — не більш як шести. Тому будують **покривний масив** сили t: набір тестів, у якому кожна комбінація значень будь-яких t параметрів трапляється хоча б раз. Для k параметрів з v значеннями

$$N_{\text{повн}} = v^k, \qquad v^t \le N_t = O\big(v^t \log k\big), \tag{11.2}$$

де нижня межа очевидна, а логарифмічне зростання за k забезпечують жадібні алгоритми типу AETG [127]. *Приклад.* Конфігурацію VPN-шлюзу описано $k = 8$ параметрами по $v = 3$ значення: шифр ESP, група обміну, автентифікація, NAT-T, MTU, стек IP, DPD, перевірка відкликання. Повний перебір — $3^8 = 6561$ конфігурацій; покрити треба $\binom{8}{2} \cdot 3^2 = 252$ пари значень. Проста жадібна побудова дала 16 тестів (нижня межа 9), для $t = 3$ — 56 (межа 27). Недопустимі комбінації (ChaCha20 у режимі FIPS) задають обмеженнями генератора.

11.5 Проблема оракула і метаморфне тестування

Оракул тестування вирішує, чи правильний результат. Розрізняють специфіковані, похідні (з інших версій, властивостей), неявні (збій, звіт санітайзера) і людські оракули [84]. Фазинг обмежується неявним оракулом (розділ 13), але пропущений пакет чи відсутня тривога збою не дають.

Диференційне тестування порівнює реалізації між собою або з моделлю (для фільтрів і криптобібліотек — підрозділ 7.6 і розділ 6). **Метаморфне тестування** [110] перевіряє *відношення* між виходами на пов'язаних входах: якщо $x' = g(x)$, то $P(x)$ і $P(x')$ задовольняють відношення $\mathcal{R}$. Для ПЗМБ

природні відношення: перестановка сусідніх правил ME, що не перетинаються, або додавання затіненого правила не змінює рішень; інша сегментація потоку TCP чи фрагментація IP без перекриття не змінює тривог IDS після нормалізації (розділ 8); розбір → серіалізація → розбір дає ту саму структуру. Перевірку першого відношення бібліотекою Hypothesis наведено в лістингу 11.2.

Лістинг 11.2 — Метаморфна перевірка перестановки правил (Python)

```
from hypothesis import given, strategies as st
from fwmodel import rule_st, packet_st, evaluate, overlaps

@given(st.lists(rule_st(), min_size=2, max_size=12),
       st.lists(packet_st(), min_size=1, max_size=50),
       st.randoms())
def test_swap_disjoint_rules(rules, packets, rnd):
    perm = list(rules)
    for _ in range(len(perm)):
        i = rnd.randrange(len(perm) - 1)
        # міняємо лише сусідні правила, що не перетинаються
        if not overlaps(perm[i], perm[i + 1]):
            perm[i], perm[i + 1] = perm[i + 1], perm[i]
    for p in packets:
        # метаморфне відношення: вердикт не змінюється
        assert evaluate(rules, p) == evaluate(perm, p)
```

Метаморфне відношення — необхідна, але не достатня умова правильності (фільтр, що відкидає все, його задовольняє), тому його поєднують з тестами з явним оракулом.

11.6 Адекватність тестів: покриття і мутації

Структурні критерії адекватності [72] — покриття операторів, гілок, умов і **модифіковане покриття умов і рішень** (MC/DC): кожна атомарна умова має незалежно вплинути на рішення. В умові `auth_ok && (is_admin || local_tty)` лише MC/DC вимагає тесту, де саме `local_tty` змінює рішення (обхід через консоль недоступний віддаленому сеансу); для m умов достатньо $m+1$ тестів замість 2^m. Проте виконаний рядок не означає перевіреного результату: за контролю розміру набору кореляція покриття з виявленням дефектів слабка або помірна [199], а вразливості зосереджені в погано покритих гілках обробки помилок (розділ 5).

Мутаційне тестування [215] оцінює набір за здатністю виявляти штучні дефекти: мутант — копія програми з малою зміною (`<` на `<=`, видалення виклику); його *вбито*, якщо хоча б один тест падає; *еквівалентний* мутант семантично не відрізняється від оригіналу. **Мутаційний бал**

$$MS = \frac{K}{M - E_q}, \tag{11.3}$$

де M — кількість мутантів, K — убитих, E_q — еквівалентних. *Приклад.* Для парсера IKEv2 $M = 420$, $E_q = 20$, $K = 340$: $MS = 340/400 = 0{,}85$. Головні перешкоди — вартість і невирішуваність розпізнавання еквівалентних мутантів.

Загальні оператори погано відтворюють вразливості, тому Луаз та співавтори [238] запропонували *оператори, орієнтовані на безпеку*, що моделюють класи CWE. Для ПЗМБ доречні: видалення перевірки меж, зсув межі на одиницю, послаблення умови автентифікації (`&&` → `||`), ігнорування результату перевірки підпису, заміна порівняння сталого часу на `memcmp`, заміна відмови за замовчуванням дозволом. Окремий бал за безпековими мутантами показовіший: якщо з 60 таких мутантів вбито 39, то $MS_{\text{без}} = 0{,}65$ за загального $0{,}85$ — негативних тестів бракує там, де вони найпотрібніші.

11.7 Тестування в оцінюванні відповідності

У Common Criteria тестування регламентують вимоги гарантій ISO/IEC 15408-3 [202] і методологія ISO/IEC 18045 [203]. Клас *ATE* має сімейства ATE_COV (покриття), ATE_DPT (глибина), ATE_FUN (тести розробника) і ATE_IND (незалежне тестування оцінювачем). Сімейство *AVA_VAN* має п'ять рівнів аналізу вразливостей з тестуванням на проникнення; рівні 1–2 вимагають стійкості до порушника з базовим потенціалом, 3 — з розширеним базовим, 4 — з помірним, 5 — з високим. Для cPP мережевих пристроїв мінімум — ATE_IND.1 і AVA_VAN.1, а глибину задають оціночні дії (розділ 2).

Атакувальний потенціал за ISO/IEC 18045 — сума балів п'яти чинників (табл. 11.2):

$$AP = f_{\text{час}} + f_{\text{експ}} + f_{\text{знан}} + f_{\text{вікно}} + f_{\text{облад}}; \tag{11.4}$$

пороги: $AP \le 9$ — нижче базового, 10–13 — базовий, 14–19 — розширений базовий, 20–24 — помірний, ≥ 25 — високий. Вразливість, що потребує потенціалу понад заявлений рівень стійкості, вважають залишковою.

Таблиця 11.2 — Чинники атакувального потенціалу (скорочено за [203])

Чинник	Значення і бали
Витрачений час	$\leq$1 доба — 0; $\leq$1 тиждень — 1; $\leq$2 тижні — 2; $\leq$1 міс. — 4; $\leq$2 міс. — 7; ...; >6 міс. — 19
Кваліфікація	дилетант — 0; фахівець — 3; експерт — 6; кілька експертів — 8
Знання об'єкта	публічні — 0; обмежені — 3; чутливі — 7; критичні — 11
Вікно можливостей	необмежене — 0; легке — 1; помірне — 4; складне — 10
Обладнання	стандартне — 0; спеціалізоване — 4; замовне — 7; кілька замовних — 9

Приклад. Відсутня перевірка автентифікації в API порталу VPN: до тижня (1), фахівець (3), публічні знання, необмежене вікно, стандартне обладнання (по 0) — $AP = 4$, неприпустимо навіть для AVA_VAN.1. Часовий побічний канал у реалізації підпису: до двох місяців (7), експерт (6), обмежені знання (3), помірне вікно (4), спеціалізоване обладнання (4) — $AP = 24$, залишкова для AVA_VAN.3, але неприпустима для AVA_VAN.4.

NIST SP 800-115 [318] описує технічне оцінювання як планування, виконання і пост-обробку. Для площини менеджменту ПЗМБ (розділ 9) систематичним каталогом перевірок є OWASP WSTG [282] разом з ASVS [277]. Українські дослідники порівнюють методи тестування безпеки вебзастосунків [54] і показують, що інструменти автоматизованого тестування на проникнення слід комбінувати [56]; методику випробувань криптобібліотек запропоновано в [23]. Тест на проникнення — *один із методів*: він перевіряє гіпотези, не закриті дешевшими методами, а відсутність знахідок має найслабшу доказову силу.

11.8 Кейс: план тестування безпеки VPN-шлюзу

Розглянемо план тестування рівня випуску для наскрізного VPN-шлюзу (підрозділ 1.9: IKEv2/IPsec, вебінтерфейс і REST API, оновлення прошивки); план оформлюють за шаблоном ISO/IEC/IEEE 29119-3. *Цілі*: підтвердити SFR профілю NDcPP з PP-модулем VPN-шлюзу [293]; досягти повноти трасування (4.1) не нижче $0{,}95$ для загроз високого ризику; виявити вразливості площини менеджменту до зовнішнього оцінювання.

Розподіл зусиль. $E = 100$ людино-днів, $E_0 = 0$; оцінки p_i, I_i взято з моделі загроз і частки вразливостей площин (9.2); розрахунок за (11.1) ($\sum R_j = 86$) наведено в табл. 11.3.

Таблиця 11.3 — Розподіл зусиль тестування VPN-шлюзу за ризиком

Область	p	I	R	E_i	Основні техніки
Керування (веб, REST)	5	5	25	29,1	WSTG, ASVS, таблиці рішень доступу, DAST
Розбір IKEv2	4	5	20	23,3	граничні значення, фазинг, безпекові мутанти
Автомат IKEv2, ключі	3	5	15	17,4	недопустимі переходи, взаємодія
Селектори, політика ESP	3	4	12	14,0	попарне тестування, метаморфні відношення
Оновлення прошивки	2	5	10	11,6	негативні тести підпису, відкату, обриву
Журналювання й аудит	2	2	4	4,7	функціональні тести FAU

Рівні: модульні тести парсерів і автомата в CI на кожен запит на злиття (з мутацією змінених рядків); інтеграційні — на дублерах-партнерах; системні — з 16 конфігураціями попарного масиву замість 6561; тести взаємодії зі strongSwan і libreswan; стійкість — за розділом 14; завершальний тест на проникнення площини менеджменту.

Критерії входу: збирання відтворюване й підписане (розділ 10); модель загроз і матриця трасованості актуальні. *Критерії виходу*: усі тести SFR пройдено; немає відкритих дефектів критичного й високого рівнів; $MS_{\text{без}} \geq 0{,}9$ для парсерів і автомата; покриття гілок обробки помилок парсерів $\geq$ 90 %; фазинг протоколу без нових збоїв за узгоджений час (розділ 13); для кожної відомої залишкової вразливості AP (11.4) перевищує заявлений рівень AVA_VAN. Звіт про завершення є доказом для кейсу гарантування (розділ 17).

Висновки до розділу

Тестування безпеки перевіряє негативні вимоги, квантифіковані за всіма, зокрема ворожими, входами, тому впевненість досягається поєднанням методів з різними сліпими зонами. ISO/IEC/IEEE 29119 дає процесну рамку, а ризик-орієнтований розподіл зусиль (11.1) пов'язує план тестування з моделлю загроз. Ключові техніки проєктування — граничні значення полів довжини, покриття недопустимих переходів автомата і комбінаторне тестування, що за (11.2) зводить тисячі конфігурацій до десятків. Проблему оракула частково знімають диференційне й метаморфне тестування. Покриття коду — слабка міра адекватності; мутаційний бал (11.3), особливо за безпековими мутантами, точніше виявляє брак негативних тестів. У Common Criteria тестування регламентують класи ATE і AVA_VAN, а атакувальний потенціал (11.4) визначає допустимі залишкові вразливості.

Питання для самоперевірки

1. У чому асиметрія тестування безпеки порівняно з функціональним? Чому для ворожого входу вона гостріша?
2. Розрізніть позитивне й негативне тестування. Звідки виводять специфікації негативних тестів ПЗМБ?
3. Перелічіть техніки NISTIR 8397 і для кожної вкажіть класи дефектів ПЗМБ, які вона виявляє (табл. 11.1).
4. Опишіть частини ISO/IEC/IEEE 29119 і три рівні процесів тестування.
5. Розподіліть 60 людино-днів за (11.1) для IDS-сенсора з п'ятьма областями (оцінки оберіть самі) з $E_0 = 3$ і без нього.
6. Порівняйте рівні тестування ПЗМБ. Що таке тестування відповідності і взаємодії протоколу?
7. Визначте граничні значення для TLV-протоколу з 2-байтовим полем довжини, що не включає заголовок, і доповніть лістинг 11.1.
8. Чому важливе покриття недопустимих переходів автомата протоколу? Скільки тестів потрібно для автомата з 7 станами і 9 типами повідомлень?
9. Обчисліть за (11.2) розмір повного перебору і нижню межу попарного набору для 12 параметрів по 4 значення.
10. Сформулюйте три метаморфні відношення для NIDS. Чому метаморфне відношення не є достатньою умовою правильності?
11. Обчисліть (11.3) для $M = 250$, $E_q = 10$, $K = 192$. Запропонуйте п'ять безпекових операторів мутації для модуля перевірки сертифіката.
12. Обчисліть (11.4) для вразливості, що потребує місяця роботи експерта з обмеженими знаннями, легкого вікна і стандартного обладнання. Яким рівням AVA_VAN вона суперечить?
13. Дослідницьке: запропонуйте методику автоматичного виведення метаморфних відношень для реалізації IKEv2 з RFC 7296 і метрики їхньої здатності вбивати безпекові мутанти.
14. Дослідницьке: побудуйте модель адаптивного перерозподілу тестових зусиль зі спадною віддачею і байєсовим оновленням p_i за знахідками; запропонуйте експеримент для порівняння з лінійною моделлю (11.1).

РОЗДІЛ 12

Статичний аналіз і формальна верифікація програмного забезпечення мережевої безпеки

Тестування (розділ 11) досліджує скінченну вибірку виконань, а методи цього розділу міркують про *всі* виконання — ціною наближень або витрат на доведення. Розглянуто межі аналізу, SAST, економіку тріажу, символьне виконання, перевірку моделей, верифікацію протоколів і реалізацій та інтеграцію в конвеєр CI/CD. Розділ поглиблює згадані в розділах 5–9 EverParse, HACL*, верифікатор eBPF і аналізи WireGuard; гібридний фазинг — у розділі 13.

12.1 Теоретичні межі статичного аналізу

12.1.1 Теорема Райса і компроміс повноти та точності

За теоремою Райса [306] будь-яка нетривіальна семантична властивість програм алгоритмічно нерозв'язна. Питання «чи може парсер записати за межі буфера за деякого пакета» — саме така властивість. Практичний інструмент жертвує завершуваністю, **звуковістю** (soundness — жодна реальна вада не пропущена) або **повнотою в логічному сенсі** (completeness — жодне попередження не хибне).

Нехай B — множина місць, де властивість справді порушується, R — множина попереджень. Аналіз *звуковий*, якщо $B \subseteq R$, і *повний*, якщо $R \subseteq B$. Звукові аналізатори (абстрактна інтерпретація) надмірно наближають поведінку й видають **хибні спрацювання** (false positives, FP; у книзі також «хибні тривоги») — попередження без реальної вади; пошуковики дефектів (лінтери, більшість SAST, символьне виконання) недонаближають і пропускають вади, зате знахідки ймовірніше справжні. Оскільки в логіці «sound» означає «доводить лише істинне», у звітах слід явно вказувати, яку гарантію надає інструмент.

12.1.2 Класифікація аналізаторів

Аналізатори впорядковуються за чутливістю (до потоку керування, шляхів, контексту, полів) і силою гарантій (табл. 12.1). Вища чутливість зменшує частку FP, але підвищує вартість; верифікатори вимагають ще й людської праці над специфікацією і доведенням.

Таблиця 12.1 — Класи засобів статичного аналізу і верифікації

Клас	Техніка	Гарантія і FP	Приклади
Лінтери, шаблони	AST, синтаксичні шаблони	немає; FP низькі–середні	clang-tidy, Semgrep
Шляхочутливі	символьне виконання в межах функції	немає; FP середні	Clang Static Analyzer
Міжпроцедурний taint	граф потоку даних програми, запити	немає; FP середні	CodeQL
Абстрактна інтерпретація	надмірне наближення в абстрактних доменах	звукова для класу помилок; тривоги	Astrée, Frama-C/EVA, IKOS
Символьне виконання	SMT-розв'язання умов шляхів	знахідка — конкретний вхід; пропуски	KLEE
Обмежена перевірка моделей	SAT/SMT з розгорткою циклів	повна в межах розгортки	CBMC
Дедуктивна верифікація	доведення щодо специфікації	повна щодо специфікації	F*, Coq, Isabelle

Метод обирають не «найсильніший», а адекватний критичності компонента (рис. 12.1).

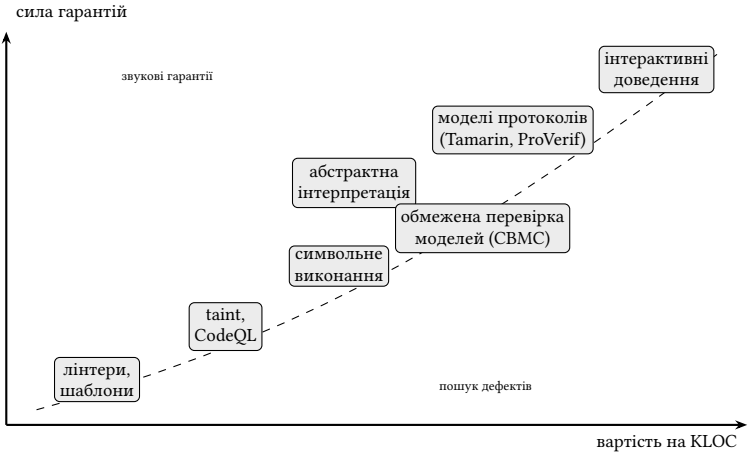


Рис. 12.1 — Спектр методів статичного аналізу і верифікації: вартість і сила гарантій (якісно)

12.2 Статичний аналіз безпеки коду

12.2.1 Taint-аналіз і абстрактна інтерпретація

Типова вразливість ПЗМБ — значення з ворожого пакета без перевірки потрапляє в чутливу операцію. **Taint-аналіз** формалізує це трійкою: *джерела*

(recv, кільцевий буфер DPDK), *стоки* (довжина в memcpy, індекс, розмір виділення, команда оболонки) і *санітайзери* (перевірка меж за (5.3), валідація за граматикою). Аналізатор шукає в графі потоку даних шлях від джерела до стоку в обхід санітайзерів (рис. 12.2), враховуючи поля структур і непрямі виклики через таблиці обробників протоколу.

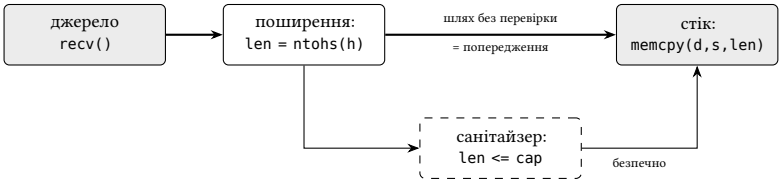


Рис. 12.2 — Модель taint-аналізу для мережевого вводу

Абстрактна інтерпретація [138] обчислює надмірні наближення значень в абстрактному домені (інтервали, октагони, многогранники), зв'язаному з конкретною семантикою парою Галуа; збіжність для циклів забезпечує оператор розширення (widening). Після читання `uint16_t len` інтервальний домен дає $len \in [0; 65\,535]$, після `if (len <= 256)` — $[0; 256]$, і запис у буфер на 256 байтів доведено безпечним для *всіх* пакетів. Непрямий інваріант (довжину перевірено в іншій функції через інше поле) домен не бачить — виникає хибне спрацювання. Звукові аналізатори Astrée, Frama-C/EVA, IKOS доречні для невеликих критичних модулів без динамічної пам'яті; той самий підхід лежить в основі PREVAIL (розділ 7).

12.2.2 Запитові та компіляторні аналізатори

Запитові аналізатори подають програму як базу фактів (AST, графи потоку керування й даних), а пошук дефектів — як запит. Лістинг 12.1 мовою CodeQL шукає шлях від буфера, заповненого recv, до довжини memcpy без обмежувального порівняння. Цінність підходу — *варіантний аналіз* (розділ 16): запит за однією знайденою вразливістю знаходить її аналоги в усій кодовій базі.

Лістинг 12.1 — Навчальний запит CodeQL: довжина з мережі в memcpy

```
/** @kind path-problem */
import cpp
import semmle.code.cpp.dataflow.new.TaintTracking
import semmle.code.cpp.controlflow.Guards
module Cfg implements DataFlow::ConfigSig {
  // джерело: буфер, заповнений recv()/recvfrom()
  predicate isSource(DataFlow::Node n) {
    exists(FunctionCall c | c.getTarget()
      .hasName(["recv", "recvfrom"]) and
      n.asDefiningArgument() = c.getArgument(1)) }
```

```
  // стік: аргумент довжини memcpy
  predicate isSink(DataFlow::Node n) {
    exists(FunctionCall c | c.getTarget().hasName("memcpy")
      and n.asExpr() = c.getArgument(2)) }
  // санітайзер: доступ, захищений перевіркою «менше»
  predicate isBarrier(DataFlow::Node n) {
    exists(GuardCondition g, Variable v |
      n.asExpr() = v.getAnAccess() and
      g.ensuresLt(v.getAnAccess(), _, _,
                  n.asExpr().getBasicBlock(), true)) } }
module Flow = TaintTracking::Global<Cfg>;
import Flow::PathGraph
from Flow::PathNode s, Flow::PathNode t
where Flow::flowPath(s, t)
select t.getNode(), s, t, "Довжина з мережі без перевірки меж"
```

Запит спрощено: санітайзер не перевіряє, *з чим* порівнюється довжина (порівняння з іншим ворожим полем не є перевіркою), і не враховує обгорток над `recv`. Уточнення санітайзерів зменшує хибні спрацювання, але може спричиняти пропуски, тому правила потребують власних тестів, як правила IDS (розділ 8). Semgrep швидший, але слабший у міжпроцедурних потоках.

12.2.3 Метрики якості й економіка тріажу

Якість аналізатора на розміченому наборі оцінюють **точністю** (precision), **повнотою виявлення** (recall) і їхнім гармонічним середнім:

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}, \tag{12.1}$$

де TP, FP — істинні й хибні спрацювання, FN — пропущені дефекти; частка хибних серед попереджень дорівнює $1 - P$. *Приклад.* За 80 відомих вад аналізатор видав 100 попереджень, з них 60 істинних: $P = 0{,}60$, $R = 0{,}75$, $F_1 = 0{,}667$. Українське порівняння SonarQube, Semgrep і CodeQL на OWASP Benchmark дало F_1 відповідно 49,8; 70,3 і 87,8 % [52], а LLM-верифікатор знахідок може зменшити частку FP [61]. Переносити такі цифри слід обережно: у синтетичних наборах на кожен випадок з вадою припадає випадок без неї [275], а у виробничому коді вади рідкісні, тож точність там нижча (ефект базової частоти, розділ 15).

Рішення вмикати правило в конвеєрі ґрунтується на очікуваній корисності потоку попереджень:

$$V = N\big[P\,(qL - c_f) - c_t\big], \qquad P^{*} = \frac{c_t}{qL - c_f}, \tag{12.2}$$

де N — кількість попереджень, c_t — вартість тріажу одного, c_f — вартість виправлення, q — імовірність, що дефект є експлуатованою вразливістю, L

— очікувані втрати від неї, P^* — точність беззбитковості. *Приклад.* $N = 400$, тріаж 0,5 год за 50 €/год ($c_t = 25$ €), $c_f = 100$ €, $q = 0{,}02$, $L = 10\,000$ €: $qL - c_f = 100$ €, $P^* = 0{,}25$. За $P = 0{,}30$ маємо $V = 2000$ €, за $P = 0{,}15$ — $V = -4000$ €. Для правил щодо ворожого вводу q і L вищі, а поріг P^* нижчий.

Вирішальною є довіра розробників: головні бар'єри — хибні спрацювання й попередження, що не пояснюють причини і способу виправлення [216]. Досвід Google [311]: показувати результати під час огляду коду, вимірювати *ефективні* хибні спрацювання (попередження, на які розробник не реагує) і вимикати перевірки, де їхня частка перевищує близько 10 %.

12.3 Символьне та конколічне виконання

Символьне виконання замінює вхід символьними змінними, на кожному розгалуженні роздвоює стан і накопичує *умову шляху*; SMT-розв'язувач (Z3, STP, Bitwuzla) перевіряє її виконуваність у теорії бітових векторів і будує вхід, що відтворює дефект. KLEE [104] виконує LLVM-біткод з моделлю середовища; на утилітах GNU Coreutils він досяг у середньому понад 90 % покриття рядків. **Конколічне виконання** (concolic) іде реальним входом, а символьні обмеження використовує, щоб заперечити одну умову й отримати вхід для сусіднього шляху, що спрощує взаємодію з бібліотеками.

Головна перешкода — **вибух шляхів**. Для b незалежних розгалужень, що залежать від входу, і циклу з k ітераціями

$$N_{\text{шл}} = 2^{b}, \qquad N_{\text{шл}}^{\text{цикл}} = \sum_{i=0}^{k} 2^{b_1 i} \approx 2^{b_1 k}, \tag{12.3}$$

де b_1 — кількість розгалужень у тілі циклу. *Приклад.* Заголовок з 20 незалежними прапорцями дає $2^{20} \approx 10^{6}$ шляхів — досяжно; до 16 TLV-опцій з трьома розгалуженнями на опцію — $2^{48} \approx 2{,}8 \cdot 10^{14}$, недосяжно. **Злиття станів** у точках сходження керування замінює 2^b станів одним з умовними виразами (`ite`), зводячи кількість станів до $O(b)$ ціною складніших формул; для парсерів обмежують розмір символьного входу. Українські дослідники пропонують спрямоване символьне виконання з метриками відстані до небезпечних ділянок для підтвердження попереджень статичного аналізу [25].

Для ПЗМБ символьне виконання найкорисніше на ізольованих парсерах, для *підтвердження* знахідок SAST і в гібридному фазингу (розділ 13).

12.4 Перевірка моделей і обмежена перевірка коду

Перевірка моделей вичерпно досліджує простір станів скінченної моделі щодо властивості в темпоральній логіці (LTL, CTL) і в разі порушення

видає контрприклад. SPIN (мова Promela) придатний для автоматів протоколів (розділ 6): моделюють сторони й канал із втратою і перестановкою повідомлень і перевіряють відсутність тупиків і недосяжність стану «тунель встановлено без автентифікації»; на рівні проєкту так само перевіряють специфікації TLA+ синхронізації кластера й A/B-оновлення (розділ 9).

Обмежена перевірка моделей (BMC) застосовується до коду: CBMC розгортає цикли до межі k, перекладає програму на C у формулу SAT/SMT і доводить, що в межах розгортки немає виходу за межі масиву, переповнення, недійсного вказівника чи порушення `assert`. Чонг та співавтори [113] описали чотирирічний досвід AWS: розробники пишуть *обв'язки доведення* (proof harness) з недетермінованими входами й передумовами, які перевіряються в CI; так верифіковано гіпервізори, криптографічний код і завантажувачі з передбачуваними витратами. Межа розгортки — явне припущення: якщо цикл розбору може мати понад k ітерацій, доведення доповнюють аргументом про обмеженість входу.

12.5 Формальна верифікація протоколів

У **символьній моделі** Долева–Яо [152] повідомлення — терми, криптографія ідеальна, а порушник контролює мережу: перехоплює, змінює, повторює й складає повідомлення з відомих термів у необмеженій кількості сеансів. Властивості — *секретність*, *автентифікація* як відповідність подій і *еквівалентність спостережень*. Задача загалом нерозв'язна, тож інструменти можуть не завершитися. **Обчислювальна модель** трактує повідомлення як рядки бітів, а порушника — як імовірнісний поліноміальний алгоритм; безпеку доводять послідовністю ігор зі зведенням до припущень про примітиви. Інструменти порівняно в табл. 12.2.

Таблиця 12.2 — Інструменти формального аналізу протоколів

Інструмент	Модель	Властивості	Автоматизація	Приклади
Tamarin [247]	символьна, переписування мультимножин	секретність, автентифікація, еквівалентність	автоматичний пошук та інтерактивні леми	TLS 1.3, 5G AKA
ProVerif [90]	символьна, прикладне π-числення	секретність, відповідність, еквівалентність	автоматичний; можливі хибні атаки, незавершення	TLS 1.3 (з CryptoVerif), навчальні моделі
CryptoVerif	обчислювальна, ігри	конкретна безпека з імовірностями	напівавтоматичний, з підказками	WireGuard [236], TLS 1.3

Кремерс та співавтори [140] побудували в Tamarin модель TLS 1.3 (проєкт 21), довели секретність ключів, взаємну автентифікацію й пряму секре-

тність і виявили неочевидну поведінку автентифікації клієнта після рукостискання, яку врахували до завершення RFC 8446 [304]. Басін та співавтори [85] формалізували 5G AKA і показали, що частина критичних цілей не досягається без припущень, відсутніх у стандарті. Спільний урок: формалізація виявляє передусім *неявні припущення* специфікацій, які розробник реалізації мусить знати й дотримувати.

У навчальній моделі ProVerif (лістинг 12.2) клієнт надсилає g^x, сервер відповідає підписаним g^y, клієнт шифрує секрет ключем g^{xy}.

Лістинг 12.2 — Навчальна модель обміну ключами в ProVerif

```
(* Навчальна модель: DH, відповідь сервера підписана *)
free c: channel.
type skey. type pkey. type G. type exponent.
const g: G [data].
fun exp(G, exponent): G.
equation forall x: exponent, y: exponent;
  exp(exp(g, x), y) = exp(exp(g, y), x).
fun pk(skey): pkey.
fun sign(G, skey): bitstring.
reduc forall m: G, k: skey; checksign(sign(m, k), pk(k)) = m.
fun senc(bitstring, G): bitstring.
reduc forall m: bitstring, k: G; sdec(senc(m, k), k) = m.
free payload: bitstring [private].
query attacker(payload).
let Client(pkS: pkey) =
  new x: exponent; out(c, exp(g, x));
  in(c, (gy: G, s: bitstring));
  if checksign(s, pkS) = gy then
  out(c, senc(payload, exp(gy, x))).
let Server(skS: skey) =
  in(c, gx: G); new y: exponent;
  out(c, (exp(g, y), sign(exp(g, y), skS))).
process new skS: skey; out(c, pk(skS));
  ( !Client(pk(skS)) | !Server(skS) )
```

Модель свідомо має ваду: підпис покриває лише g^y, а не пару (g^x, g^y) з ідентифікаторами сторін. Секретність `payload` зберігається (порушник не знає y), але запит відповідності «клієнт прийняв $g^y \Rightarrow$ сервер відповів саме на його g^x» має дати контрприклад із повторенням старої відповіді сервера; TLS 1.3 і IKEv2 підписують транскрипт.

12.6 Верифіковані реалізації

Доведення про модель протоколу нічого не каже про код, тому окремий напрям — реалізації з машинно перевіреними доведеннями. HACL* [352] (F* з екстракцією в C) гарантує безпеку пам'яті, функціональну коректність і сталий час; Fiat-Crypto [166] синтезує арифметику скінченних полів з доведенням у Coq; згенерований код використовується в BoringSSL. seL4 [225] доводить коректність мікроядра аж до бінарного коду (розділ 4).

Гарантію обмежує *ланцюг довіри*: доведено відповідність коду формальній специфікації, але не специфікації — наміру стандарту; довіряють перевіряльнику доведень, компілятору, моделі апаратури (спекулятивні канали поза моделлю сталого часу) і неверифікованій обгортці. Тому інтеграційні й протокольні тести лишаються обов'язковими (розділ 11).

Верифікатор eBPF (розділ 7) гарантує *безпеку ядра*, а не коректність фільтра, тому властивості на кшталт «жоден пакет із забороненого діапазону не отримує `XDP_PASS`» перевіряють окремо — символьним виконанням або BMC.

12.7 Інтеграція в конвеєр розроблення

Аналіз у конвеєрі має бути *інкрементальним*: на запит на злиття звітують лише про попередження в зміненому коді, а повний аналіз виконують щоночі. Накопичені попередження фіксують як **базову лінію**; ворота якості (розділ 3) блокують лише нові критичні знахідки, а придушення допускаються з обґрунтуванням і строком перегляду. Формат OASIS SARIF 2.1.0 з відбитками для зіставлення знахідок між запусками дає єдину базову лінію для кількох аналізаторів.

Навантаження на розробників характеризує **щільність попереджень**

$$D = \frac{W}{S}, \qquad T_{\Delta} = D_{\text{нов}}\,\Delta S\,t_t, \tag{12.4}$$

де W — кількість попереджень, S — обсяг коду в KLOC, $D_{\text{нов}}$ — щільність нових попереджень у зміні обсягом ΔS, t_t — час тріажу одного. За SATE IV [275] медіанна щільність — 1,5–6 попереджень на KLOC. *Приклад.* Прошивка VPN-шлюзу на 300 KLOC за $D = 4$ дає 1200 попереджень і за $t_t = 0{,}5$ год — 600 год тріажу, звідси потреба в базовій лінії; зміна на 2 KLOC дає 8 попереджень і $T_{\Delta} = 4$ год — прийнятно за точності, достатньої за (12.2).

Лінтери запускають на кожну фіксацію, CodeQL — на запит на злиття, обв'язки CBMC — на зміни парсерів і щоночі. Метрики: частка нових попереджень, виправлених до злиття, частка придушень і частка вразливостей з PSIRT (розділ 16), які аналізатор мав би виявити.

12.8 Кейс: SAST і обмежена перевірка моделі для парсера VPN-шлюзу

Постановка. Український виробник наскрізного VPN-шлюзу (підрозділ 1.9) готує продукт до вимог Регламенту (ЄС) 2024/2847 про кіберстійкість (CRA) [301]. Модуль розбору IKEv2 має ≈9 KLOC на C, з них парсер навантажень — ≈6 KLOC; парсер обробляє неавтентифіковані пакети першого обміну і є найкритичнішою частиною поверхні атаки. Варіанти: лише SAST

(CodeQL із правилами класу лістингу 12.1) або SAST разом з обв'язками CBMC.

Рішення. Для кожної функції розбору пишеться обв'язка: буфер до 128 байтів з недетермінованим вмістом, передумова коректного контексту, перевірка відсутності виходу за межі й переповнень. Межу розгортки обґрунтовують тим, що навантаження обробляються по одному. Обв'язки запускаються в CI на зміни парсера.

Оцінка доцільності. Чиста очікувана вигода додавання BMC

$$\Delta U = n_v\,(1 - e_S)\,e_M\,L - (C_0 + T\,C_m), \tag{12.5}$$

де n_v — очікувана кількість вразливостей парсера за горизонт T років (моделі розділу 3), e_S — частка, яку виявляє SAST, e_M — частка решти, яку виявляє CBMC, L — втрати від вразливості граничного пристрою, C_0, C_m — початкові й щорічні витрати. *Приклад.* $n_v = 2$, $e_S = 0{,}3$, $e_M = 0{,}5$, $L = 150\,000$ €, $C_0 = 18\,000$ € (три людино-місяці), $C_m = 4000$ €, $T = 3$: вигода $105\,000$ €, витрати $30\,000$ €, $\Delta U = 75\,000$ €. Беззбиткові втрати $L^* = 30\,000/0{,}7 \approx 42\,900$ € значно нижчі за типову вартість інциденту з граничним пристроєм (розділ 1), тож рішення стійке до похибок. Формула (12.5) — модель-евристика з ілюстративними параметрами; e_S, e_M уточнюють ретроспективно за знахідками фазингу (розділ 13) і PSIRT.

Висновки до розділу

Теорема Райса робить неминучим вибір між звуковістю і повнотою в логічному сенсі, тому методи утворюють спектр від лінтерів до машинно перевірених доведень, а метод має відповідати критичності компонента. Taint-аналіз і запитові аналізатори відповідають типовій вразливості ПЗМБ — ворожому значенню, що без перевірки досягає чутливої операції; абстрактна інтерпретація дає звукові гарантії для малих модулів. Неточні правила руйнують довіру розробників (12.2). Символьне виконання перетворює підозри на відтворювані входи, але обмежене вибухом шляхів; BMC дає доведення для C-коду з прийнятними витратами. Аналіз протоколів у Tamarin і ProVerif виявляє неявні припущення стандартів, а верифіковані реалізації переносять гарантії на код. Інкрементальний аналіз і базові лінії роблять ці методи частиною щоденного розроблення.

Питання для самоперевірки

1. Сформулюйте теорему Райса. Чому з неї випливає компроміс між звуковістю і повнотою в логічному сенсі?
2. Дайте означення звукового й повного аналізу через множини B і R.

3. Який клас аналізаторів (табл. 12.1) доцільний для парсера заголовка площини даних і чому?
4. Наведіть джерела, стоки й санітайзери taint-аналізу для VPN-шлюзу. Чому порівняння з іншим полем пакета не є санітайзером?
5. Як абстрактна інтерпретація доводить безпеку запису в буфер? Звідки хибні тривоги інтервального домену?
6. Які обмеження має запит лістингу 12.1 і як тестувати правила аналізатора?
7. За (12.1) обчисліть P, R, F_1 для 50 попереджень, з них 35 істинних, за 70 відомих вад. Чому результати на синтетичних наборах завищені?
8. За (12.2) знайдіть P^{*} для $c_t = 20$ €, $c_f = 150$ €, $q = 0{,}05$, $L = 20\,000$ €. Що таке ефективні хибні спрацювання?
9. Поясніть вибух шляхів за (12.3), переваги й вади злиття станів.
10. Чим BMC відрізняється від перевірки моделей у SPIN? Що є припущенням доведення CBMC?
11. Порівняйте символьну модель Долева–Яо з обчислювальною.
12. Яку ваду має модель лістингу 12.2 і як її виявити формально?
13. Які ланки ланцюга довіри верифікованої реалізації лишаються припущеннями?
14. Дослідницьке: запропонуйте методику порівняння SAST-інструментів на коді ПЗМБ з урахуванням базової частоти вад і вартості тріажу.
15. Дослідницьке: розробіть підхід до автоматичного виведення обв’язок BMC для парсерів протоколів з декларативних описів формату (розділ 5) й оцініть його за (12.5).

РОЗДІЛ 13

Динамічний аналіз і фазинг мережевого програмного забезпечення

Статичні методи (розділ 12) міркують про всі виконання ціною наближень; динамічний аналіз спостерігає реальні виконання й тому не дає хибних спрацювань щодо досяжності, але бачить лише збуджені шляхи. Фазинг автоматично генерує мільярди ворожих входів, а санітайзери перетворюють тихе пошкодження пам'яті на гучну аварію — неявний оракул (підрозділ 11.5). Для ПЗМБ, що розбирає ворожий трафік (розділ 5), це основний промисловий засіб пошуку вразливостей пам'яті. Розглянуто санітайзери, фазинг, керований покриттям, обв'язки, фазинг протоколів зі станом, статистичні моделі зупинки, оцінювання фазерів і тестування API.

13.1 Динамічний аналіз і санітайзери

Санітайзер — інструментація, яку компілятор вставляє в програму для перевірки інваріантів під час виконання. AddressSanitizer (ASan) [126] виявляє вихід за межі купи, стеку й глобальних об'єктів, використання після звільнення (use-after-free), подвійне звільнення і витоки. Механізм — **тіньова пам'ять**: кожні 8 байтів застосунку відображаються на 1 байт тіні, а навколо об'єктів розміщуються недоступні *червоні зони* (redzones); звільнена пам'ять потрапляє в карантин. Адресу тіні обчислюють так:

$$\mathrm{Sh}(a) = (a \gg 3) + \mathrm{Off}, \qquad \text{помилка} \iff k \neq 0 \ \wedge \ (a \bmod 8) + s - 1 \geq k, \tag{13.1}$$

де a — адреса доступу розміром $s \leq 8$ байтів, Off — зміщення тіні (на x86-64 Linux `0x7fff8000`), k — значення тіньового байта: 0 — усі 8 байтів доступні, $1 \ldots 7$ — доступні лише перші k, від'ємне — червона зона або звільнена пам'ять. *Приклад.* Буфер на 13 байтів займає 8-байтову гранулу з $k = 0$ і другу з $k = 5$; читання байта з індексом 13 дає $a \bmod 8 = 5$, $s = 1$: $5 + 1 - 1 \geq 5$ — ASan повідомляє про вихід за межі на один байт, який без інструментації непомітно прочитав би сусідні дані (сценарій Heartbleed, розділ 1).

Інші санітайзери доповнюють ASan (табл. 13.1). UndefinedBehaviorSanitizer (UBSan) ловить знакове переповнення, некоректні зсуви, неправильне вирівнювання — першопричини цілочислових вад (підрозділ 5.4). MemorySanitizer (MSan) виявляє використання неініціалізованої пам'яті, але вимагає інструментувати *весь* код, зокрема бібліотеки. ThreadSanitizer (TSan) виявляє гонитви даних між потоками. Valgrind не потребує перекомпіляції, але для фазингу надто повільний.

Таблиця 13.1 — Санітайзери LLVM/Clang (за документацією [126])

Засіб	Виявляє	Сповільнення	Пам'ять	Обмеження
ASan	межі, use-after-free, double free, витоки	≈2×	зростає, стек до 3×	несумісний з MSan і TSan
UBSan	невизначена поведінка: переповнення, зсуви	незначне	незначна	лише вибрані перевірки
MSan	неініціалізована пам'ять	≈3×	2–3×	потрібна повна інструментація
TSan	гонитви даних	5–15×	5–10×	лише виконані чергування

ASan, MSan і TSan взаємно несумісні, тож у CI потрібні окремі збирання; ASan+UBSan — типова пара для фазингу. Санітайзерні збирання не постачають у продукт: середовище виконання розширює поверхню атаки. Для «тихих» збирань використовують UBSan з пасткою (`-fsanitize-trap`), а на апаратурі з тегуванням пам'яті — HWASan і ARM MTE, придатні навіть для польової експлуатації (підрозділ 5.7).

13.2 Фазинг, керований покриттям

13.2.1 Визначення і класифікація

Фазинг (fuzzing) — автоматичне тестування великою кількістю згенерованих, зокрема некоректних, входів зі спостереженням за порушеннями оракула [333, 350]. Огляд Манеса та співавторів [241] класифікує фазери за обізнаністю про ціль: *чорна скринька*, *сіра* (полегшена інструментація — покриття гілок), *біла* (символьне виконання, розділ 12); за способом отримання входів: *мутаційні* (змінюють **початкові входи** (seeds) корпусу), *генераційні* (з моделі формату) і *граматичні*. Для бінарних протоколів ПЗМБ найпродуктивніший сірий мутаційний фазинг, підсилений знанням формату.

13.2.2 Цикл фазера і розклад входів корпусу

AFL популяризував **фазинг, керований покриттям** (coverage-guided greybox fuzzing): інструментація рахує переходи між базовими блоками в компактній карті, а вхід із новим переходом стає входом корпусу (рис. 13.1). AFL++ [172] об'єднав напрацювання досліджень (розклади потужності, CmpLog для магічних констант, власні мутатори). libFuzzer [126] працює в процесі: обв'язка викликається мільйони разів без перезапуску, що дає тисячі виконань за секунду.

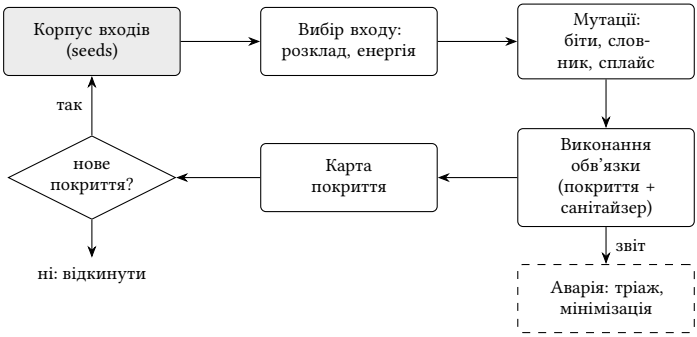


Рис. 13.1 — Цикл фазера, керованого покриттям

Бьоме, Фам і Ройчоудхурі [103] змоделювали фазинг як *марковський ланцюг*: стани — шляхи виконання, p_{ij} — імовірність, що мутація входу шляху i дасть шлях j. Більшість входів потрапляє в «високочастотні» шляхи (рання відмова за неправильним заголовком), тому AFLFast призначає входу корпусу *енергію* — кількість мутацій за раунд — за експоненційним розкладом:

$$p(i) = \min\left(\frac{\alpha(i)}{\beta} \cdot \frac{2^{s(i)}}{f(i)},\ M\right), \tag{13.2}$$

де $\alpha(i)$ — базова оцінка входу (час виконання, розмір, покриття), $\beta > 1$ — стала, $s(i)$ — кількість попередніх виборів входу, $f(i)$ — кількість згенерованих входів, що пройшли шляхом i, M — верхня межа. *Приклад.* За $\alpha = 100$, $\beta = 2$, $s = 3$ рідкісний шлях з $f = 4$ отримує $50 \cdot 8/4 = 100$ мутацій, а частий шлях з $f = 1000 - 50 \cdot 8/1000 = 0{,}4$, тобто практично жодної: зусилля зосереджуються в малодосліджених областях.

13.2.3 Знання формату

Випадкові мутації ламають контрольні суми й поля довжини структур «тип–довжина–значення». Засоби: *словники* констант (номери типів навантажень), автоматичне вилучення констант з порівнянь (CmpLog), *відключення контрольних сум* в обв'язці під прапорцем `FUZZING_BUILD_MODE_UNSAFE_FOR_PRODUCTION`, **структуро-обізнаний фазинг** (libprotobuf-mutator мутує дерево повідомлення, а обв'язка серіалізує його) і гібридні схеми з SMT-розв'язувачем (підрозділ 12.3).

13.3 Обв'язки і безперервний фазинг

Обв'язка (harness) — точка входу, що передає байти фазера тестованому коду. Принципи: *детермінізм* (без часу, випадковості, потоків), *відсутність I/O*, *швидкість* (мікросекунди на вхід), *скидання стану* (кожен виклик з чи-

стого контексту), *явні оракули* (`assert` інваріантів понад аварії). Обв'язку парсера заголовка IKEv2 [220] наведено в лістингу 13.1; оракул «розбір–кодування–порівняння» виявляє й логічні розбіжності.

Лістинг 13.1 — Обв'язка libFuzzer для парсера IKEv2 (C)

```
#include <stdint.h>
#include <string.h>
#include "ike_parse.h"   /* тестований парсер */

static uint8_t out[65535];

int LLVMFuzzerTestOneInput(const uint8_t *d, size_t n) {
  if (n > sizeof out) return 0;   /* межа UDP-датаграми */
  struct ike_ctx ctx;
  ike_ctx_init(&ctx);             /* свіжий стан щоразу */
  struct ike_msg m;
  if (ike_parse_msg(&ctx, d, n, &m) == IKE_OK) {
    size_t k = 0;
    /* оракул: повторне кодування заголовка (28 байтів)
       має збігатися з входом */
    if (ike_encode_hdr(&m, out, sizeof out, &k) != IKE_OK
        || k < 28 || memcmp(out, d, 28) != 0)
      __builtin_trap();
    ike_msg_free(&m);
  }
  ike_ctx_free(&ctx);             /* без витоків */
  return 0;                       /* без мережі й файлів */
}
```

Код Rust фазять через cargo-fuzz (той самий libFuzzer): у безпечному Rust фазинг знаходить паніки — для ПЗМБ це відмова в обслуговуванні, — а блоки `unsafe` перевіряє під ASan (розділ 5). Запуск із санітайзерами наведено в лістингу 13.2.

Лістинг 13.2 — Збирання і запуск з санітайзерами (shell)

```
# збирання: libFuzzer + ASan + UBSan, аварія на першій UB
clang -g -O1 -fno-omit-frame-pointer \
  -fsanitize=fuzzer,address,undefined \
  -fno-sanitize-recover=all \
  -DFUZZING_BUILD_MODE_UNSAFE_FOR_PRODUCTION \
  fuzz_ike.c ike_parse.c -o fuzz_ike
# кампанія: 4 процеси, словник, 8 год, тайм-аут 2 с
./fuzz_ike corpus/ seeds/ -dict=ikev2.dict -timeout=2 \
  -jobs=4 -workers=4 -max_total_time=28800
# мінімізація корпусу перед фіксацією в репозиторії
./fuzz_ike -merge=1 corpus_min/ corpus/
# регресія в CI: лише відтворення корпусу
./fuzz_ike -runs=0 corpus_min/ regressions/
```

OSS-Fuzz — безкоштовна служба безперервного фазингу відкритого ПЗ (зокрема OpenSSL, Wireshark, Suricata, strongSwan): проєкт надає скрипт

збирання обв'язок, а платформа фазить їх кількома фазерами з різними санітайзерами, дедуплікує аварії й перевіряє виправлення. CIFuzz кілька хвилин фазить ті самі обв'язки на кожен запит на злиття, ловлячи регресії, що реалізує практику SSDF PW.8 (розділ 3).

13.4 Фазинг мережевих протоколів зі станом

Сервер протоколу — реактивна система: відповідь залежить від *історії* повідомлень (IKE_SA_INIT перед IKE_AUTH), стан зберігається між запитами, взаємодія йде через сокети з тайм-аутами, а перезапуск процесу дорогий. Класичний фазер, що подає один буфер, застрягає в початковому стані; виклики — моделювання стану, генерація послідовностей і швидка взаємодія з ціллю.

AFLNet [290] поєднує *два рівні зворотного зв'язку* (рис. 13.2): покриття коду, як в AFL, і стан протоколу, який визначає за кодами відповідей сервера (наприклад, кодами FTP чи RTSP) і будує з них модель станів (IPSM). Фазер вибирає малодосліджений стан, бере послідовність, що до нього веде, зберігає префікс M_1 незмінним, мутує повідомлення M_2 і відтворює суфікс M_3.

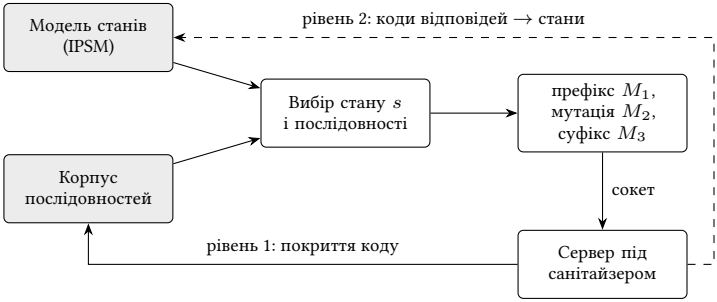


Рис. 13.2 — Дворівневий зворотний зв'язок AFLNet (за [290])

Коди відповідей не завжди відображають внутрішній стан, тому StateAFL визначає стан за знімками довгоживучих змінних пам'яті, а SGFuzz — за змінними, яким присвоюються іменовані константи (типові змінні стану в C-реалізаціях). Nyx-Net [320] розв'язує проблему швидкості: ціль виконується у віртуальній машині, гіпервізор робить *інкрементні знімки* після префікса повідомлень і відновлює їх замість перезапуску. Генераційні фазери на кшталт boofuzz описують протокол вручну. ChatAFL [248] використовує велику мовну модель, щоб вилучити з RFC граматики повідомлень і вивести фазер з плато покриття; на еталоні ProFuzzBench він

покрив на 47,6 % більше переходів станів, ніж AFLNet. Підходи порівняно в табл. 13.2.

Таблиця 13.2 — Підходи до фазингу протоколів зі станом

Підхід	Модель стану	Сильні сторони	Обмеження
boofuzz	ручна модель блоків і послідовностей	без інструментації; закриті цілі	трудомісткість; чорна скринька
AFLNet	коди відповідей	простий; відкритий код	неповна картина стану; повільний перезапуск
StateAFL, SGFuzz	змінні пам'яті	внутрішній стан без розмітки	евристики вибору змінних
Nyx-Net	знімки ВМ	висока швидкість; без скидання вручну	потрібен гіпервізор і агент
ChatAFL	граматики з RFC через LLM	вихід з плато; сильний корпус	вартість і недетермінізм LLM

Десокетизація (перехоплення `socket`/`recv`) підміняє мережевий I/O читанням з буфера; розробник власного продукту ефективніше відокремлює розбір і автомат станів від транспорту архітектурно (розділ 4). Фазинг доповнює вивчення автоматів станів (розділ 6): навчання виявляє зайві переходи, фазинг — вади пам'яті в кожному стані.

Для вбудованих прошивок граничних пристроїв, яких не вдається зібрати під хост, застосовують **перенесення в емулятор** (rehosting): прошивку виконують в емуляторі (QEMU) з моделями периферії й апаратних переривань, щоб запускати фазинг, санітайзери й налагодження без фізичного пристрою. Систематизація [170] показує, що головні труднощі — моделювання периферії та оцінювання точності емуляції: розбіжність з обладнанням дає як пропущені, так і хибні знахідки. Виробник, на відміну від стороннього дослідника, має вихідний код і документацію апаратури, тож може будувати моделі периферії разом із прошивкою й підтримувати їх у CI.

13.5 Статистичні моделі фазингу

Коли зупинити кампанію? STADS [101] трактує фазинг як *вибірку видів* в екології: кожен вхід — спостереження, «видами» є покриті гілки, шляхи чи унікальні аварії. Імовірність, що наступний вхід відкриє новий вид, оцінюється за Гудом–Тюрінгом, а загальну кількість видів — оцінювачем Chao1:

$$\hat{p}_{\text{new}} = \frac{f_1}{n}, \qquad \hat{S} = S_{\text{obs}} + \frac{f_1^2}{2 f_2}, \tag{13.3}$$

де n — кількість згенерованих входів, f_1 і f_2 — кількість видів, що спостерігалися рівно один і два рази, S_{obs} — кількість відкритих видів. *Приклад.* Після $n = 4 \cdot 10^8$ входів парсер має $S_{\text{obs}} = 1820$ покритих гілок, з них $f_1 = 4$,

$f_2 = 8$: $\hat{p}_{\text{new}} = 10^{-8}$ і $\hat{S} = 1820 + 16/16 = 1821$, тобто досягнута частка досяжних гілок $\approx 99{,}9$ %. За 2500 вх./с очікуваний час до наступної нової гілки ≈ 11 год. Величина $\hat{p}_{\text{new}}$ оцінює *залишковий ризик*; критерій зупинки — $\hat{p}_{\text{new}} < \varepsilon$ на кількох незалежних прогонах, але він стосується лише простору, досяжного *цим* фазером з *цією* обв'язкою.

Бьоме і Фальк [102] установили емпіричний закон: щоб знайти лінійно більше *нових* вразливостей за той самий час, потрібно експоненційно більше обчислень:

$$V(m) \approx V_0 + b \log_2 \frac{m}{m_0} \quad \Longrightarrow \quad m_k = m_0 \, 2^{k/b}, \tag{13.4}$$

де $V(m)$ — кількість знайдених вразливостей (видів) за фіксований час на m машинах, b — приріст на кожне подвоєння, m_k — машини, потрібні для k додаткових знахідок. *Приклад.* Команда за добу на $m_0 = 4$ ядрах знаходить $V_0 = 10$ аварій; за $b = 1$ три додаткові потребують $4 \cdot 2^3 = 32$ ядер, десять — 4096. Отже, ефективніше вкладати в нові обв'язки, словники й структуро-обізнані мутатори (вони змінюють криву, а не рухаються по ній). З урахуванням моделі витрат на програми винагород за вразливості (bug bounty) [21] раціонально, щоб власний фазинг знімав «дешевий» шар, а зовнішні дослідники — частину «дорогого».

13.6 Оцінювання фазерів

Кліс та співавтори [224] проаналізували оцінювання у 32 статтях і знайшли проблеми в кожній: один прогін за високої дисперсії, короткі кампанії, неконтрольовані початкові входи, підрахунок «унікальних аварій» за гешем стеку. Рекомендації: багато незалежних прогонів, кампанії порядку 24 год, критерій Манна–Уітні та розмір ефекту $\hat{A}_{12}$ Варги–Делані, оцінка за справжніми вадами:

$$\hat{A}_{12} = \frac{1}{n_2} \left(\frac{R_1}{n_1} - \frac{n_1 + 1}{2} \right), \tag{13.5}$$

де R_1 — сума рангів прогонів фазера A в об'єднаній упорядкованій вибірці, n_1, n_2 — кількість прогонів A і B; $\hat{A}_{12}$ оцінює ймовірність, що випадковий прогін A дасть більше покриття, ніж прогін B (0,5 — без різниці; за Варгою–Делані 0,56, 0,64, 0,71 — межі малого, середнього й великого ефекту). *Приклад.* $n_1 = n_2 = 10$, $R_1 = 140$: $\hat{A}_{12} = (14 - 5{,}5)/10 = 0{,}85$ — великий ефект; якщо p-значення Манна–Уітні менше 0,05, перевагу A можна стверджувати.

Еталони: Magma [191] вбудовує в реальні бібліотеки відомі вади з індикаторами досяжності й спрацювання («справжня істина»); FuzzBench порівнює фазери на реальних програмах; ProFuzzBench містить мережеві сервери (FTP, SMTP, RTSP, DTLS) для оцінювання фазерів станів.

13.7 DAST і тестування API площини менеджменту

Площина менеджменту граничних пристроїв — головне джерело критичних вразливостей (підрозділ 9.4). Її тестують динамічним аналізом застосунків (DAST) [282, 55]: OWASP ZAP у CI сканує тестовий екземпляр продукту за специфікацією OpenAPI. Фазинг REST API зі станом реалізує RESTler [79]: він виводить зі специфікації залежності «виробник–споживач» (ідентифікатор, створений `POST`, використовується в `DELETE`); оракули — коди 5xx, доступ до ресурсу після видалення, «запит без автентифікації не змінює конфігурацію».

13.8 Кейс: фазинг парсера IKEv2 VPN-шлюзу

Продовжимо кейс розділів 11 і 12: виробник наскрізного VPN-шлюзу (підрозділ 1.9) фазить модуль розбору IKEv2 (≈9 KLOC на C, з них парсер навантажень — ≈6 KLOC).

Налаштування. Розбір і автомат IKE відокремлено від сокетів; написано три обв'язки: розбір одного повідомлення (лістинг 13.1), розбір навантажень за типами і послідовність IKE_SA_INIT→IKE_AUTH (перевірки ICV і підписів у збиранні для фазингу вимкнено і покрито окремо векторами Wycheproof, розділ 6). Початковий корпус — 300 повідомлень з pcap сумісності зі strongSwan; словник — типи навантажень RFC 7296; збирання ASan+UBSan і окремо MSan.

Метрики. За тиждень на 16 ядрах ($\approx 3{\cdot}10^9$ входів) покриття гілок модуля зросло з 41 до 87 %; непокриті — переважно обробка помилок пам'яті й рідкісні режими EAP. За (13.3) для обв'язки навантажень $f_1 = 3$, $n = 1{,}1 \cdot 10^9$: $\hat{p}_{\text{new}} \approx 2{,}7 \cdot 10^{-9}$, тому кампанію переведено в режим безперервного фазингу з меншим пріоритетом, а зусилля — на обв'язку послідовностей, де $\hat{p}_{\text{new}}$ на два порядки вищий.

Тріаж і дедуплікація. Отримано 214 аварійних входів. Дедуплікація за трьома верхніми кадрами стеку дала 9 груп, мінімізація — 6 дефектів: два читання за межами буфера в розборі навантаження сертифіката (CWE-125), знакове переповнення під час підсумовування довжин (CWE-190), використання після звільнення при повторному IKE_SA_INIT з тим самим SPI (знайдено лише обв'язкою послідовностей), витік пам'яті на шляху помилки і порушення оракула кодування. Дефекти класифіковано за CVSS і передано процесу PSIRT (розділ 16).

Регресія. Мінімізований корпус (близько 2,1 тис. входів) і всі аварійні входи зберігаються в репозиторії; CIFuzz-подібний крок відтворює їх і 10 хв фазить змінені модулі на кожен запит на злиття (лістинг 13.2); обв'язки й корпус є доказом верифікації для CC (ATE, AVA_VAN) і Регламенту (ЄС) 2024/2847 про кіберстійкість (CRA). Урок: найнебезпечніший дефект (вико-

ристання після звільнення) недосяжний для фазингу одиничних повідомлень — без моделювання стану $\hat{p}_{\text{new}}$ був би оманливо низьким.

Висновки до розділу

Санітайзери перетворюють невидимі пошкодження пам'яті, невизначену поведінку й гонитви на відтворювані звіти ціною сповільнення від двократного до п'ятнадцятикратного і потребують окремих збирань у CI. Фазинг, керований покриттям, — промисловий стандарт пошуку вразливостей пам'яті в парсерах ПЗМБ; його ефективність визначають розклад входів корпусу, знання формату і якість обв'язки. Безперервний фазинг робить корпус регресійним активом. Протоколи зі станом вимагають моделювання стану й знімків, а прошивки — перенесення в емулятор. Модель STADS дає критерій зупинки й оцінку залишкового ризику, а закон експоненційної вартості пояснює, чому нові обв'язки цінніші за додаткові машини. Порівняння фазерів потребує багаторазових прогонів і статистичних критеріїв.

Питання для самоперевірки

1. Поясніть механізм тіньової пам'яті й червоних зон ASan. Обчисліть за (13.1), чи буде помилкою 4-байтове читання за зміщенням 10 у буфері на 13 байтів.
2. Порівняйте ASan, UBSan, MSan і TSan (табл. 13.1). Чому їх не можна поєднати в одному збиранні і чому санітайзерні збирання не постачають у продукт?
3. Класифікуйте фазери за обізнаністю про ціль і способом отримання входів. Який клас доцільний для бінарного протоколу ПЗМБ і чому?
4. Опишіть цикл фазера, керованого покриттям (рис. 13.1). Що вважається «цікавим» входом?
5. Поясніть марковську модель AFLFast. Розрахуйте енергію за (13.2) для входу корпусу з $s = 5$, $f = 64$, $\alpha = 100$, $\beta = 2$.
6. Які засоби дають змогу фазеру долати контрольні суми, поля довжини й магічні константи?
7. Сформулюйте принципи побудови обв'язки. Що порушилося б у лістингу 13.1, якби контекст був глобальним?
8. Як OSS-Fuzz і CIFuzz вбудовують фазинг у життєвий цикл розроблення? Яку роль відіграє регресійний корпус?
9. Чому фазинг серверів протоколів зі станом складніший за фазинг парсерів? Поясніть дворівневий зворотний зв'язок AFLNet (рис. 13.2).

10. Порівняйте StateAFL, SGFuzz, Nyx-Net і ChatAFL за способом моделювання стану і обмеженнями (табл. 13.2).
11. Обчисліть $\hat{p}_{\text{new}}$ і $\hat{S}$ за (13.3) для $n = 5 \cdot 10^7$, $S_{\text{obs}} = 900$, $f_1 = 10$, $f_2 = 5$. Чи достатньо цього для зупинки кампанії?
12. За (13.4) з $b = 1{,}5$ оцініть, у скільки разів треба збільшити обчислення, щоб знайти ще 6 нових вразливостей. Які дії ефективніші за масштабування?
13. Які вимоги Кліса та співавторів до оцінювання фазерів? Обчисліть $\hat{A}_{12}$ за (13.5) для $n_1 = n_2 = 20$, $R_1 = 500$ і інтерпретуйте.
14. Дослідницьке: запропонуйте модель стану для фазингу IKEv2, що поєднує коди сповіщень і внутрішні змінні автомата, та методику її порівняння з AFLNet за вимогами підрозділу 13.6.
15. Дослідницьке: розробіть критерій зупинки кампанії фазингу ПЗМБ, що поєднує оцінку $\hat{p}_{\text{new}}$ STADS, закон експоненційної вартості і ризик-орієнтований бюджет тестування (розділ 11), і спосіб його перевірки на даних OSS-Fuzz.

РОЗДІЛ 14

Тестування продуктивності, стійкості та відмовостійкості засобів мережевої безпеки

Засіб мережевої безпеки, що не встигає обробляти трафік, або пропускає його без інспекції, або відкидає легітимні з'єднання; в обох випадках порушується одна з властивостей безпеки. Тому продуктивність і поведінка ПЗМБ за перевантажень і відмов — предмет тестування безпеки. Розділ розглядає методології бенчмаркінгу RFC 2544, RFC 3511 і RFC 9411, стенди, моделі черг і масштабування, стійкість до DoS, відмовостійкість і хаос-інженерію, а також кейс порівняльного тестування NGFW. Він спирається на бюджет тактів на пакет (розділ 1), вимоги fail-secure/fail-open (розділ 4), захист таблиці станів (розділ 7) і модель черги сенсора (розділ 8).

14.1 Продуктивність як властивість безпеки

Зв'язок продуктивності з безпекою проявляється в чотирьох механізмах. *Вимкнення функцій безпеки заради швидкості*: якщо паспортна пропускна здатність досягається лише без інспекції TLS, IPS чи антивіруса, під тиском користувачів вимикають саме їх. *Fail-open під навантаженням*: IPS у розриві лінії за переповнення черг пропускає трафік без перевірки (розділ 8), і порушник, що вміє створити навантаження, отримує «вікно» неінспектованого трафіку. *Відмова в обслуговуванні самого засобу*: таблиця з'єднань, рукостискання TLS-проксі, пам'ять для збирання потоків — ресурси, витрату яких контролює порушник. *Деградація виявлення*: сенсор, що втрачає пакети, пропускає атаки, хоча формально «працює».

Звідси вимога RFC 9411 [82]: тестувати засіб у конфігурації реального розгортання з *увімкненими* функціями безпеки, а невмикання рекомендованої функції зазначати у звіті. Вимоги до поведінки за перевантаження формулюють як вимоги безпеки з критеріями приймання (розділи 4, 11).

14.2 Методології бенчмаркінгу

14.2.1 RFC 2544: базові вимірювання площини даних

RFC 2544 [98] визначає методику для пристроїв міжмережевої взаємодії. **Пропускна здатність** — найбільша швидкість без втрат кадрів; її шукають серією випробувань (зазвичай двійковим пошуком) тривалістю щонайменше 60 с, остаточне — повної тривалості. **Затримку** вимірюють на знайденій пропускній здатності в потоці не коротшому за 120 с, повторюючи щонайменше 20 разів. **Частку втрат кадрів** визначають, починаючи зі 100 % лі-

нійної швидкості і знижуючи її кроками не більше 10 %, доки два випробування поспіль не пройдуть без втрат. Тест **кадрів підряд** (back-to-back, уточнений у RFC 9004 [256]) шукає найдовшу пачку без втрат і характеризує буферизацію. Вимірюють також час відновлення після перевантаження. Стандартні розміри кадрів — від 64 до 1518 байтів.

Верхню межу задає *максимальна кадрова швидкість*. Кадр довжини L займає в лінії ще 8 байтів преамбули й 12 байтів міжкадрового інтервалу, тож

$$R_{\max} = \frac{C}{8\,(L + 20)}, \qquad t_{\text{пак}} = \frac{1}{R_{\max}}, \tag{14.1}$$

де $R_{\max}$ — кадрів за секунду; C — швидкість інтерфейсу, біт/с; L — довжина кадру з контрольною сумою, байтів; $t_{\text{пак}}$ — бюджет часу на пакет, як у (1.1). Для 10 Гбіт/с і $L = 64$ маємо $R_{\max} = 10^{10}/672 \approx 14{,}88$ млн кадрів/с, для $L = 1518$ — $10^{10}/12\,304 \approx 0{,}813$ млн, у 18,3 раза менше. Для 100 Гбіт/с бюджет на 64-байтовий кадр становить 6,72 нс, або ≈ 20 тактів ядра на 3 ГГц. Тож «20 Гбіт/с» без розміру кадру нічого не означає.

RFC 2544 призначено для ізольованих лабораторій (RFC 6815 [97]), а потоки UDP без стану не навантажують ні таблицю з'єднань, ні рушій інспекції.

14.2.2 RFC 3511 і RFC 9411

RFC 3511 [193] разом із термінологією RFC 2647 [262] адаптував бенчмаркінг до ME: ємність одночасних TCP-з'єднань, швидкість їх установлення, установлення з'єднань під SYN-повінню, швидкість транзакцій HTTP. RFC 9411 [82] (2023), підготовлений учасниками галузевої організації NetSecOPEN [261], скасовує RFC 3511 і охоплює NGFW та NGIPS у режимі розриву лінії. Його ключові положення:

- для NGFW рекомендовано ввімкнути інспекцію TLS, IDS/IPS, антивірус, антишпигунський і антибот-захист, захист від ухилення, ідентифікацію застосунків; конфігурацію публікують із результатами;
- передумова — перевірка ефективності безпеки на наборі щонайменше 500 CVE з CVSS 7–10, не старших 10 років;
- профіль навантаження з фаз ініціалізації, нарощування, утримання (не менше 300 с), спадання і збирання; KPI вимірюють лише у фазі утримання;
- критерії валідності: невдалих транзакцій і з'єднань, розірваних неочікуваним RST, менше 0,001 %; відхилення швидкості у фазі утримання менше 5 %;
- рекомендовані набори шифрів TLS 1.2 і TLS 1.3 [304] (для TLS 1.3 DUT працює як проксі); методику тестів HTTPS можна застосовувати до QUIC [213] і HTTP/3.

Основні метрики подано в табл. 14.1.

Таблиця 14.1 — Метрики продуктивності засобів мережевої безпеки

Метрика	Джерело	Зміст і умови вимірювання
Пропускна здатність, втрати, затримка	RFC 2544	кадри 64–1518 байтів, нуль втрат
Кадри підряд	RFC 2544, 9004	найдовша пачка без втрат; оцінка буфера
Інспектована пропускна здатність	RFC 9411	перевірений і дозволений трафік; суміш застосунків, HTTP/HTTPS з об’єктами 1–256 КБ
З’єднання за секунду (CPS)	RFC 3511, 9411	успішні TCP- або QUIC-з’єднання; об’єкти 1–64 КБ
Одночасні з’єднання	RFC 2647, 9411	ємність таблиці станів
Транзакції і рукостискання TLS за секунду	RFC 9411	повні відповіді з кодом 200; 1-RTT, за потреби 0-RTT
TTFB, TTLB	RFC 9411	мінімум, середнє, максимум, мс
Відновлення і скидання	RFC 2544	час повернення до норми

14.3 Генерація навантаження і тестові стенди

14.3.1 Генератори і профілі

Генератори бувають *апаратні* комерційні (апаратні мітки часу, емуляція мільйонів клієнтів зі станом) і *програмні* на DPDK (TRex, Pktgen-DPDK). MoonGen [162] зі сценаріями Lua насичує 10 Гбіт/с мінімальними кадрами з одного ядра і точно вимірює затримку апаратними мітками часу адаптерів. Відтворення PCAP дає реалістичний вміст, але не реакцію стека на поведінку DUT, тож TLS-проксі тестують емульованими клієнтами і серверами. Для потоків RFC 2544 адреси беруть з діапазону 198.18.0.0/15, зарезервованого для бенчмаркінгу.

Реалістичний профіль задає суміш застосунків, розміри об’єктів, частку шифрованого трафіку і шифри, кількість адрес клієнтів, частку нових з’єднань. Для TLS 1.3 і QUIC вартість визначає насамперед рукостискання, тож короткі сеанси навантажують інший ресурс, ніж довгі потоки.

14.3.2 Стенд, калібрування і відтворюваність

Схему стенда наведено на рис. 14.1; RFC 9411 радить емулювати маршрутизацію в обладнанні тестування замість зовнішніх комутаторів, а моніторинг DUT вести через окремий інтерфейс.

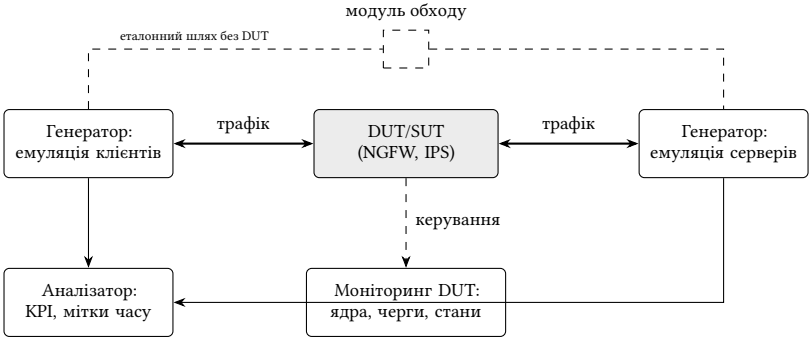


Рис. 14.1 — Схема стенда для тестування продуктивності (за [82])

Еталонний тест (без DUT) підтверджує, що генератор перевищує очікувану продуктивність DUT, а власна затримка стенда на порядок менша за вимірювану. Відтворюваність забезпечують фіксацією версій прошивки, сигнатур і конфігурації DUT як артефактів конвеєра (розділ 10), профілями навантаження як кодом і повторенням кожного вимірювання 3–5 разів з наведенням розкиду.

14.4 Моделі продуктивності

14.4.1 Черги і «коліно» латентності

Модель M/M/1 (8.4) дає $W = 1/(\mu - \lambda)$. Проте час обслуговування в ПЗМБ не експоненційний: пересилання пакета майже детерміноване, а інспекція TLS чи антивірусна перевірка мають дуже мінливу тривалість. Загальніший результат для M/G/1 — формула Поллачека–Хінчина:

$$W = \frac{1}{\mu} + \frac{\rho\,(1 + C_s^2)}{2\mu\,(1 - \rho)}, \qquad \rho = \frac{\lambda}{\mu} < 1, \tag{14.2}$$

де λ і μ — інтенсивності надходження й обслуговування; C_s — коефіцієнт варіації часу обслуговування ($C_s = 1$ дає M/M/1, $C_s = 0$ — M/D/1). Кількість запитів у системі дає закон Літтла (7.4): $\bar{n} = \lambda W$.

Приклад. Конвеєр інспекції HTTPS обслуговує $\mu = 10^4$ запитів/с (100 мкс на запит). За $\rho = 0{,}9$ модель M/M/1 дає $W = 1$ мс; якщо через різнорідність об'єктів $C_s^2 = 4$, то $W = 100 + 0{,}9 \cdot 5/(2 \cdot 10^4 \cdot 0{,}1) \approx 2{,}35$ мс. Нормована затримка $W\mu$ для M/M/1 перевищує п'ятикратну вже за $\rho > 0{,}8$ — це «коліно» (рис. 14.2); мінливість обслуговування шкодить так само, як завантаження, тож тест з однорідними об'єктами переоцінює запас, а звітувати слід перцентилі (p99, p99,9).

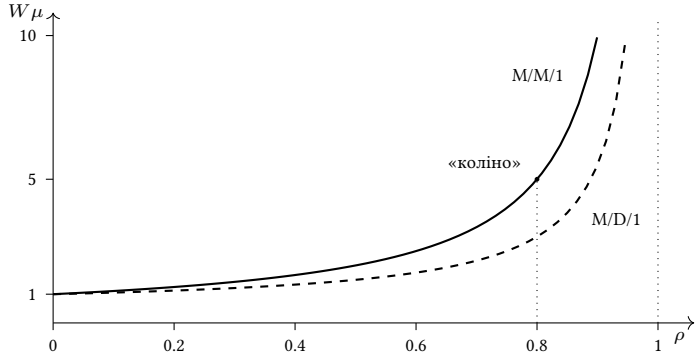


Рис. 14.2 — Нормована середня затримка залежно від завантаження

14.4.2 Масштабування багатоядерних конвеєрів

Межу прискорення розпаралеленого конвеєра дає закон Амдала [71], а з урахуванням витрат на узгодження — універсальний закон масштабованості (USL) Гюнтера [188]:

$$S(n) = \frac{1}{(1-p) + p/n}, \qquad C(n) = \frac{n}{1 + \sigma\,(n-1) + \kappa\, n\,(n-1)}, \tag{14.3}$$

де n — кількість ядер; p — частка роботи, що розпаралелюється; σ — коефіцієнт конкуренції за спільний ресурс; κ — коефіцієнт когерентності (обмін між ядрами, синхронізація спільних таблиць). Якщо в конвеєрі IPS послідовна частка (глобальне блокування журналу, спільна таблиця потоків) становить $1 - p = 0{,}05$, то для 16 ядер $S \approx 9{,}1$, для 64 — $\approx 15{,}4$, а межа за $n \to \infty$ — лише 20. USL з $\sigma = 0{,}05$, $\kappa = 0{,}001$ дає $C(16) \approx 8{,}0$, $C(32) \approx 9{,}0$, $C(64) \approx 7{,}8$: після оптимуму $n^{*} = \sqrt{(1-\sigma)/\kappa} \approx 31$ додавання ядер *знижує* продуктивність. Тому вимірюють криву масштабування за кількістю робочих потоків і апроксимують її (14.3): великий κ вказує на спільні структури даних, які варто поділити за ядрами.

14.5 Стійкість до DoS і стрес-тестування

Мета стрес-тестування — встановити *режим відмови*: за якого навантаження і як засіб деградує і чи повертається до норми.

Виснаження таблиці станів. Модель (7.4) і заходи захисту (SYN cookies, SYN-проксі [159]) розглянуто в розділі 7. Тест додає до легітимного профілю (50–70 % номіналу) потік напіввідкритих з'єднань зростаючої інтенсивності й вимірює частку успішних легітимних транзакцій, TTFB і заповнення

таблиці. Критерій приймання: «за потоку напіввідкритих з'єднань інтенсивністю X частка успішних легітимних транзакцій не нижча за Y %».

Атаки алгоритмічної складності. Кросбі і Воллак [141] показали, що входи, підібрані під відому геш-функцію, перетворюють геш-таблицю із середнім часом $O(1)$ на список з $O(n)$, а вставлення n елементів — на роботу $O(n^2)$; те саме стосується класифікаторів пакетів (розділ 7) і регулярних виразів з поверненням (ReDoS, розділ 8). Захист — ключовані геш-функції з секретним ключем (SipHash [80]), структури з гарантованим найгіршим випадком, рушії регулярних виразів з лінійним часом. Тестування: порівняння часу обробки типового і найгіршого входу, фазинг з цільовою функцією «такти на вхід» (розділ 13); ознака дефекту — надлінійне зростання часу.

Сплески і повільні з'єднання. Тест на сплески подає пачки на лінійній швидкості поверх середнього навантаження (зв'язок із тестом кадрів підряд). Повільні з'єднання з частковими запитами надовго займають ресурси проксі, обробника TLS чи WAF за мізерного трафіку; модель їх упередження запропоновано у [36]. Тест вимірює, скільки таких з'єднань засіб утримує до відмови легітимним клієнтам. Досвід захисту українських інтернет-ресурсів від DDoS під час війни [24] вимагає розраховувати засоби фільтрації на тривале навантаження.

Захист керувального трафіку. Площина керування і менеджменту (розділ 9) не конкурує з площиною даних завдяки пріоритетним чергам, обмеженню трафіку до пристрою (control plane policing) і виділеному інтерфейсу. Тест перевіряє, що під повним навантаженням адміністратор входить на пристрій, а сусідство в кластері не розривається.

14.6 Відмовостійкість

14.6.1 Fail-open, fail-closed і обхід

Поведінку за відмови (розділ 4) реалізують на кількох рівнях. Апаратні *модулі обходу* (bypass) з'єднують вхідний і вихідний порти реле за втрати живлення — fail-open на фізичному рівні; програмний обхід пропускає потоки без інспекції за переповнення черг чи збою процесу. Fail-closed блокує трафік за недоступності інспекції; для ME на межі довіри це правильний вибір, для IPS у критичному каналі — явне рішення з обов'язковою сигналізацією. Тест перевіряє і режим, і його *спостережуваність*: подію в SIEM, обсяг трафіку без інспекції.

14.6.2 Кластери і синхронізація станів

Кластери будують за схемами *active/passive* (адресу шлюзу переносить, наприклад, VRRP [234]) і *active/active* (кожен вузол має витримувати навантаження іншого). Для засобів зі станом ключова *синхронізація таблиці з'єднань*, а для VPN — ще й асоціацій безпеки й лічильників захисту від по-

вторення (розділ 9). Синхронізація з інтервалом Δ втрачає з'єднання, створені за останній інтервал: за 10^5 CPS і $\Delta = 0{,}2$ с це до $2{\cdot}10^4$ сеансів на кожне перемикання. Методи резервування сервісів систематизовано у [62].

Доступність n незалежних вузлів доступністю a і її уточнення для пари active/passive з неідеальним перемиканням:

$$A_n = 1 - (1-a)^n, \qquad U_{\text{НА}} \approx (1-a)^2 + (1-c)(1-a) + \nu\, t_{\text{sw}}, \tag{14.4}$$

де $U_{\text{НА}}$ — недоступність пари; c — покриття (ймовірність успішного перемикання); ν — частота відмов основного вузла; t_{sw} — час перемикання. *Приклад.* За $a = 0{,}999$ (8,76 год простою на рік) ідеальна пара має $A_2 = 1 - 10^{-6}$, близько 32 с простою на рік. За $c = 0{,}99$, $\nu = 4$ на рік і $t_{\text{sw}} = 3$ с маємо $U_{\text{НА}} \approx 10^{-6} + 10^{-5} + 12/(3{,}15{\cdot}10^7) \approx 1{,}14{\cdot}10^{-5}$, тобто ≈ 6 хв на рік: домінує неуспішне перемикання. Отже, вимірювати слід c і t_{sw}. Формула припускає незалежність вузлів, яку порушує спільна прошивка; вимоги неперервності узгоджують з ISO 22301 [200]. Матрицю тестів наведено в табл. 14.2.

Таблиця 14.2 — Тести відмовостійкості засобу мережевої безпеки

Сценарій відмови	Що вимірюють	Очікувана поведінка
Втрата живлення вузла	t_{sw}, втрачені сеанси	перемикання; сеанси збережено
Збій процесу інспекції	режим, трафік без інспекції	заданий режим, подія в SIEM, перезапуск
Розрив каналу синхронізації	розбіжність таблиць, «розщеплений мозок»	один активний вузол, сигналізація
Переповнення таблиці станів	відкидання нових з'єднань	наявні сеанси збережено, ліміти на джерело
Оновлення прошивки в кластері	простій, сумісність синхронізації	поетапне оновлення без розриву сеансів

14.6.3 Хаос-інженерія і тестування деградації

Хаос-інженерія (chaos engineering) — експерименти, що підтверджують здатність системи витримувати збурення в робочих умовах [86]: гіпотеза про *усталений стан*, реалістичні збурення, автоматизація і мінімальний «радіус ураження». Для засобів безпеки усталений стан включає *безпекові інваріанти*: частку інспектованого трафіку, затримку доставки подій у SIEM, відсутність трафіку в обхід політики. Експеримент «зупинити процес інспекції на одному вузлі» перевіряє водночас перемикання, режим відмови і сигналізацію; починають його на стенді чи цифровому двійнику (розділ 15). Тестування деградації перевіряє задокументовану послідовність, у якій засіб жертвує функціями під навантаженням (спершу журналювання потоків, але ніколи — політика доступу).

14.7 Кейс: порівняльне тестування NGFW з увімкненими функціями безпеки

14.7.1 Постановка і план

Організація обирає між двома NGFW класу 20 Гбіт/с (A і B; пор. наскрізний NGFW, підрозділ 1.9) для межі мережі з переважно шифрованим трафіком. План за RFC 9411 (стенд рис. 14.1): (1) еталонний тест стенда; (2) перевірка ефективності безпеки на наборі CVE; (3) однакова конфігурація: ідентифікація застосунків, IPS, антивірус, антишпигунський захист, інспекція TLS 1.3 для всього HTTPS; (4) RFC 2544 для порівняння з паспортом; (5) пропускна здатність HTTP і HTTPS з об'єктами 64 КБ, CPS з об'єктами 1 КБ, одночасні з'єднання, TTFB за 50 % навантаження; (6) перемикання HA під навантаженням. Фаза утримання — 300 с, кожне вимірювання — тричі.

14.7.2 Результати й аналіз

Результати в табл. 14.3 **умовні (ілюстративні)** і не стосуються конкретних продуктів; вони відтворюють типові закономірності.

Таблиця 14.3 — Умовні результати порівняльного тестування NGFW

Метрика (профіль)	NGFW A	NGFW B
Пропускна здатність, Гбіт/с (RFC 2544, 1518 байтів, без інспекції)	20,0	18,0
Інспектована HTTP, Гбіт/с (64 КБ, усі функції)	9,4	7,9
Інспектована HTTPS, Гбіт/с (64 КБ, інспекція TLS 1.3)	3,1	4,6
CPS HTTP / HTTPS (1 КБ), тис./с	120 / 18	95 / 31
Одночасні з'єднання, млн	4,0	3,2
Середня TTFB HTTPS за 50 % навантаження, мс	2,8	1,9
Час перемикання HA, с / розірвані сеанси, %	2,1 / 3,5	0,6 / 0,2

Падіння продуктивності з інспекцією TLS відносно паспорта становить для A $1 - 3{,}1/20 = 84{,}5\,\%$, для B — $\approx 74{,}4\,\%$: рейтинг за паспортом і за реальним профілем протилежний. Для суміші з часткою e шифрованого трафіку ефективна пропускна здатність — зважене гармонічне середнє:

$$T_{\text{еф}} = \left(\frac{1-e}{T_{\text{HTTP}}} + \frac{e}{T_{\text{HTTPS}}} \right)^{-1}, \tag{14.5}$$

де T_{HTTP}, T_{HTTPS} — інспектована пропускна здатність для відкритого і шифрованого трафіку. За $e = 0{,}8$: для A $T_{\text{еф}} = (0{,}2/9{,}4 + 0{,}8/3{,}1)^{-1} \approx 3{,}6$ Гбіт/с, для B — $(0{,}2/7{,}9 + 0{,}8/4{,}6)^{-1} \approx 5{,}0$ Гбіт/с, що в 3,6–5,6 раза нижче за паспорт. Для каналу 10 Гбіт/с з інспекцією TLS знадобиться потужніша модель, кластер active/active або вибіркова інспекція як прийнятий ризик. Низька CPS HTTPS у A вказує на вузьке місце в рукостисканнях, які за (14.2) першими досягнуть «коліна» в години пік. Менші t_{sw} і втрати сеансів B зменшують

$U_{\text{НА}}$ за (14.4). Рішення — B із запасом $T_{\text{еф}}$ не менше 30 % над піком і повторенням тестів після оновлень прошивки й сигнатур. Від постачальників вимагають результатів за RFC 9411 з описом конфігурації (зокрема звітів NetSecOPEN [261]).

Висновки до розділу

Продуктивність засобу мережевої безпеки є властивістю безпеки: брак запасу спонукає вимикати функції захисту, перевантаження переводить засіб в обхід або відмову в обслуговуванні. RFC 2544 задає базові вимірювання площини даних з межею (14.1), але засоби зі станом потребують методології RFC 9411: увімкнених функцій безпеки, реалістичних профілів, фази утримання й калібрування стенда. Формула (14.2) пояснює «коліно» латентності, закони Амдала і Гюнтера (14.3) — межі масштабування. Стрес-тестування визначає режими відмови. Відмовостійкість оцінюють через явний режим відмови, покриття й час перемикання (14.4), а хаос-інженерія перевіряє безпекові інваріанти. Кейс показує, що рейтинг за паспортом і за профілем з інспекцією TLS може бути протилежним, а ефективну пропускну здатність суміші слід оцінювати за (14.5).

Питання для самоперевірки

1. Наведіть чотири механізми, через які недостатня продуктивність засобу порушує властивості безпеки.
2. Які метрики визначає RFC 2544? Які мінімальні тривалості й кількості повторень установлено для пропускної здатності й затримки?
3. Обчисліть за (14.1) максимальну кадрову швидкість і бюджет часу на пакет для 25 Гбіт/с і кадрів 64 та 512 байтів.
4. Чому RFC 2544 недостатній для NGFW і чому його не застосовують у робочих мережах?
5. Які функції безпеки RFC 9411 рекомендує вмикати для NGFW? Що зазначають у звіті, якщо функцію не ввімкнено?
6. Опишіть фази профілю навантаження і критерії валідності за RFC 9411. Навіщо еталонний тест стенда?
7. Порівняйте апаратні й програмні генератори. Чому відтворення PCAP недостатнє для тестування TLS-проксі?
8. За (14.2) обчисліть затримку для $\mu = 5 \cdot 10^4$ запитів/с, $\rho = 0{,}85$ і $C_s^2 \in \{0; 1; 3\}$. Які висновки для проєктування тестів?
9. Чим USL відрізняється від закону Амдала (14.3)? Як за вимірюваннями виявити обмеження когерентністю?

10. Опишіть тест стійкості до виснаження таблиці станів і сформулюйте вимірюваний критерій приймання.
11. Що таке атака алгоритмічної складності? Які захисні механізми і методи тестування їй протидіють?
12. Порівняйте fail-open і fail-closed, апаратний і програмний обхід. Що, крім режиму, має перевіряти тест?
13. За (14.4) оцініть недоступність пари з $a = 0{,}9995$, $c = 0{,}98$, $\nu = 6$ на рік і $t_{\mathrm{sw}} = 1$ с. Який доданок домінує?
14. Дослідницьке: розробіть методику оцінювання c і t_{sw} кластера NGFW з довірчими інтервалами за малої кількості експериментів і модель втрачених сеансів залежно від інтервалу синхронізації та CPS.
15. Дослідницьке: сформулюйте задачу автоматизованого пошуку входів, що максимізують вартість обробки в конвеєрі IPS (фазинг ресурсоємності): метрики, оракул надлінійного зростання, протокол порівняння з тестами RFC 9411.

РОЗДІЛ 15

Оцінювання ефективності виявлення, тестові середовища та кіберполігони

Твердження «система виявляє 99 % атак» нічого не означає без відповіді на запитання: яких атак, на якому трафіку, за якої базової частоти, з якою часткою хибних тривог і невизначеністю, чи відтворюваний результат. Розділ розглядає метрики якості виявлення і вибір порогу, набори даних та їхні вади, пастки ML-IDS, змагальну стійкість, кіберполігони, емуляцію противника і покриття технік MITRE ATT&CK, статистику й відтворюваність, а також кейс порівняльного оцінювання двох NIDS (наскрізний IDS-сенсор, підрозділ 1.9). Сенсори, помилку базової частоти і ML-компоненти описано в розділі 8.

15.1 Метрики якості виявлення

15.1.1 Матриця помилок і похідні метрики

Детектор порівнює оцінку s кожного об'єкта (пакета, потоку, сеансу) з порогом θ. Зіставлення рішень з «істиною» дає **матрицю помилок**: TP і FN — виявлені й пропущені атакувальні об'єкти, FP — **хибні тривоги** (у книзі синонім — «хибне спрацювання»: сповіщення на легітимний об'єкт), TN — правильно пропущені легітимні. Похідні метрики: **повнота виявлення** (recall) $\mathrm{TPR} = TP/(TP+FN)$; частка хибних тривог $\mathrm{FPR} = FP/(FP+TN)$; **точність** (precision) $\mathrm{Prec} = TP/(TP+FP)$ — частка істинних серед сповіщень; $F_1 = 2\,\mathrm{Prec}\cdot\mathrm{TPR}/(\mathrm{Prec}+\mathrm{TPR})$. Спершу фіксують *одиницю оцінювання*: метрика «на потік» винагороджує гучні атаки, «на інцидент» — одне сповіщення за атаку.

Частка правильних рішень (accuracy) за дисбалансу класів марна, а F_1 ігнорує TN. Збалансованою метрикою є коефіцієнт кореляції Метьюза

$$\mathrm{MCC} = \frac{TP\cdot TN - FP\cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}, \qquad (15.1)$$

що набуває значень від -1 до 1 (0 — випадкове вгадування) і високий лише за доброї класифікації обох класів. *Приклад.* Із 10^5 потоків атакувальних 100; $TP=95$, $FN=5$, $FP=999$, $TN=98\,901$. Accuracy — 98,996 %, але $\mathrm{Prec}=95/1094\approx 0{,}087$, $F_1\approx 0{,}16$, $\mathrm{MCC}\approx 0{,}29$.

15.1.2 Точність і базова частота

Залежність точності від базової частоти π (частки атакувальних об'єктів) випливає з теореми Баєса:

$$\mathrm{Prec}(\pi) = \frac{\mathrm{TPR} \cdot \pi}{\mathrm{TPR} \cdot \pi + \mathrm{FPR} \cdot (1 - \pi)}. \tag{15.2}$$

Приклад. За $\mathrm{TPR} = 0{,}95$ і $\mathrm{FPR} = 0{,}01$ маємо $\mathrm{Prec} = 0{,}91$ при $\pi = 0{,}1$ (типово для лабораторного набору) і лише $0{,}087$ при $\pi = 10^{-3}$ (як у попередньому прикладі). Це помилка базової частоти Аксельссона [81] (формула (8.3)); методологічний висновок: точність і F_1, отримані на наборі з чужою π, перераховують за (15.2) для цільової π.

15.1.3 ROC- і PR-криві

Зміна порогу θ дає криву ROC (TPR від FPR) і криву PR (точність від повноти виявлення) з площами ROC-AUC і AUPRC. ROC не залежить від π і тому оманлива за дисбалансу: робоча частина кривої IDS лежить при $\mathrm{FPR} < 10^{-3}$. Сайто і Ремсмаєр [312] показали, що за дисбалансу PR-крива інформативніша; її базова лінія дорівнює π, а не діагоналі. Рис. 15.1 зображує *ту саму* умовну систему в обох просторах при $\pi = 10^{-3}$: ROC-AUC $\approx 0{,}97$, тоді як у робочій точці $\mathrm{TPR} = 0{,}8$, $\mathrm{FPR} = 0{,}01$ точність становить лише $0{,}074$.

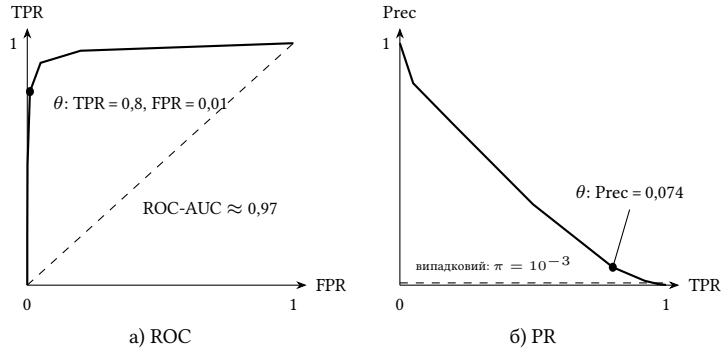


Рис. 15.1 — Одна й та сама умовна система в просторах ROC і PR при $\pi = 10^{-3}$

15.1.4 Вибір порогу за вартістю помилок

Пропуск атаки і хибна тривога мають різну ціну, а F_1 вважає їх рівноцінними. Очікувана вартість експлуатації детектора за період

$$C(\theta) = c_{FN} \cdot FN(\theta) + c_{FP} \cdot FP(\theta), \qquad \theta^{*} = \arg\min_{\theta} C(\theta), \tag{15.3}$$

де c_{FN} — середня ціна пропущеного атакувального об'єкта, c_{FP} — ціна тріажу хибної тривоги. У просторі ROC мінімум лежить у точці, де нахил дотичної дорівнює $c_{FP}(1-\pi)/(c_{FN}\pi)$; за малої π оптимум зсувається до дуже малих FPR. *Приклад* (умовні одиниці). За добу 10^6 потоків, із них 100 атакувальних. Поріг θ_1 (TPR $= 0{,}95$, FPR $= 10^{-2}$) дає $FN = 5$, $FP \approx 10^4$; θ_2 ($0{,}80$ і 10^{-3}) — $FN = 20$, $FP \approx 10^3$. За $c_{FN} = 500$, $c_{FP} = 1$: $C(\theta_1) = 12\,500$, $C(\theta_2) = 11\,000$ — кращий θ_2, хоча повнота нижча; за автоматизованого тріажу ($c_{FP} = 0{,}1$) $C(\theta_1) = 3500 < C(\theta_2) = 10\,100$. Звіт має містити криву й обґрунтовані c_{FN}, c_{FP}, π.

15.2 Набори даних і їхні вади

Основні публічні набори даних наведено в табл. 15.1. МакГ'ю [246] показав, що в оцінюванні DARPA 1998/1999 не обґрунтовано подібність синтетичного фону до реального трафіку і не виключено штучних відмінностей атак від фону, які детектор може «вивчити». Похідний KDD Cup 99 успадкував ці вади, а Таваллаї зі співавторами [335] знайшли, що близько 78 % і 75 % записів його навчальної і тестової частин дублюються.

Таблиця 15.1 — Публічні набори даних для оцінювання NIDS

Набір	Рік	Трафік і розмітка	Відомі вади
DARPA 1998/1999	1998–1999	синтетичний фон і атаки; PCAP	нереалістичний фон, артефакти, непрозора методика [246]
KDD Cup 99	1999	41 ознака з'єднань з DARPA 1998	вади DARPA; дублікати; застарілі атаки [335]
NSL-KDD	2009	KDD Cup 99 без дублікатів	походження KDD Cup 99
UNSW-NB15	2015	IXIA PerfectStorm, 9 сімейств атак, 49 ознак [257]	синтетичні атаки і фон; дисбаланс сімейств
CIC-IDS2017	2017	5 днів, профілі 25 користувачів, CICFlowMeter [323]	помилки потоків і розмітки [164, 233]
CSE-CIC-IDS2018	2018	та сама методика в хмарі, більший масштаб	той самий інструмент ознак

UNSW-NB15 і CIC-IDS2017 мали сучасніші атаки і повні PCAP, але постала проблема якості істини. Енгелен, Риммер і Йоосен [164] показали, що в CIC-IDS2017 багато записів спотворено помилками CICFlowMeter і розмітки (невдалі спроби атак позначено як атаки), і перерозмітили набір. Ланвен зі співавторами [233] показали, що виправлення розмітки суттєво змінює виміряну ефективність детекторів. Отже, частина опублікованих результатів характеризує артефакти набору; це стосується й українських робіт на цих наборах (наприклад, [57]), для яких природний наступний крок — перевірка на виправлених версіях і власних даних. Доказове оцінювання ПЗМБ потребує повних PCAP, відкритого інструмента ознак із версією,

журналу дій атакувальника як істини, документованих профілів фону і переліку відомих вад.

15.3 Методологічні пастки ML-IDS

Арп та співавтори [77] систематизували десять пасток машинного навчання в безпеці і виявили їх у кожній із 30 проаналізованих статей провідних конференцій. Для ML-IDS найістотніші: *упередженість вибірки*; *неточність міток*; *витік даних* (data snooping) — у навчання потрапляють майбутні записи, потоки того самого сеансу, статистики всього набору; *хибні кореляції* з артефактами (TTL, адреси стенда, час запису атак); *вибір параметрів* на тестовій вибірці; *невідповідні базові лінії й метрики*; *помилка базової частоти*; *лише лабораторна оцінка* і *невідповідна модель загроз*.

Випадкове перемішування дає моделі «бачити майбутнє»: потоки однієї атаки опиняються по обидва боки поділу. Реальний детектор навчається на минулому і працює в майбутньому, де трафік змінюється (**дрейф концепцій**: нові застосунки, протоколи, атаки), тож потрібне **часове розбиття** і оцінка деградації в часі. Вимоги до протоколу оцінювання ML-IDS:

1. Одиниця оцінювання, цільова π, ціни c_{FN}, c_{FP} і модель загроз фіксуються *до* експерименту.
2. Дані: виправлена версія набору або власні дані з журналом істини; дедуплікація; перевірка відсутності кореляції міток з ознаками стенда.
3. Розбиття за часом (для узагальнення — і за вузлами); статистики обробки не обчислюються на тестовій частині.
4. Гіперпараметри — лише на валідаційній частині; тестова використовується один раз.
5. Базові лінії: сигнатурна IDS і просте правило.
6. Метрики: PR-крива, точність при цільовій π, MCC, вартість (15.3), довірчі інтервали.
7. Повторні прогони з різними зернами і статистичне порівняння.
8. Оцінка дрейфу і змагальної стійкості.
9. Публікація коду, конфігурацій, версій і даних.

15.4 Змагальна стійкість і тестування ухилення

Стійкість до цілеспрямованих змін входу треба тестувати, а не припускати. Таксономія NIST AI 100-2 E2025 [340] класифікує атаки на машинне навчання за етапом життєвого циклу, метою, знаннями і можливостями противника. Для ML-IDS основні класи — *ухилення* (evasion) і *отруєння* (poisoning) даних, особливо за онлайн-донавчання (як у Kitsune [251]). База

знань MITRE ATLAS за зразком ATT&CK описує техніки противника щодо систем ШІ (розділ 4).

На відміну від комп'ютерного зору, змагальний приклад у просторі ознак потоку може не мати відповідника в просторі задачі: ознаки взаємозалежні, протокол накладає обмеження, а атака має зберегти функцію. Апруццезе та співавтори [75] запропонували модель реалістичних змагальних атак на NIDS з урахуванням доступу, знань противника і змін трафіку, що зберігають мету атаки. Вимоги до тестування стійкості: явна модель загроз з *допустимою множиною змін*, обмеженою протоколом і функціональністю атаки; метрика — залежність TPR від «бюджету» змін і знань противника; окрема перевірка ухилення на рівні мережі (розділ 8) і моделі; для онлайн-навчання — контроль джерел донавчання і моніторинг дрейфу як ознаки отруєння; підтвердження рішень ML-детектора про блокування іншими ознаками. Тести виконують на ізольованому стенді; результат — властивість детектора, а не опис способів ухилення.

15.5 Тестові середовища і кіберполігони

15.5.1 Емуляція і цифрові двійники

У *тестовому середовищі* трафік породжують живі вузли, а сценарії відтворювані. Mininet створює на одному вузлі мережу віртуальних хостів на просторах імен Linux; GNS3 і EVE-NG запускають образи мережевих ОС у віртуальних машинах; containerlab описує топологію в YAML, роблячи стенд кодом (розділ 10). **Цифровий двійник мережі** — синхронізована з оригіналом модель конкретної мережі (топологія, конфігурації, сервіси, профілі трафіку) для оцінювання правил чи моделі без ризику для неї. Найслабша ланка — *фоновий трафік*: PCAP не реагує на детектор, генератори навантаження (розділ 14) статистично бідні, агенти-імітатори користувачів реалістичніші, але дорожчі.

15.5.2 Кіберполігони

Кіберполігон (cyber range) — керована інфраструктура для моделювання мереж, сценаріїв атаки й захисту, навчання і тестування. Ямін, Катт і Гкіулос [347] систематизували їхні сценарії (навчання, змагання, тестування), функції (моделювання середовища, генерація трафіку, емуляція противника, моніторинг, оцінювання), інструменти й архітектуру. Для оцінювання NIDS (рис. 15.2) ключова функція — *журнал істини*: кожна дія емулятора противника фіксується з міткою часу, вузлами і техніками ATT&CK, що дає розмітку без помилок, описаних у підрозділі 15.2.



Рис. 15.2 — Архітектура кіберполігону для оцінювання NIDS (за [347])

У Європі це, зокрема, федеративний кіберполігон проєкту ECHO («Горизонт 2020») і щорічні навчання Locked Shields Центру передового досвіду НАТО з кібероборони (CCDCOE) за участі представників України. В Україні навчальний кіберполігон Київського столичного університету імені Бориса Грінченка [1] слугує віртуальною лабораторією для практичних навичок. Для розробника ПЗМБ кіберполігон — ще й стенд регресійного системного тестування виявлення (розділ 11).

15.6 Емуляція противника і покриття ATT&CK

Засоби **автоматизованої емуляції атак** (Breach and Attack Simulation, BAS) виконують безпечні тестові дії, зіставлені з техніками MITRE ATT&CK [253], і перевіряють появу сповіщень. Бібліотеки *атомарних тестів* (наприклад, Atomic Red Team) задають для техніки процедури з мінімальними побічними ефектами; це тестування на основі моделі з оракулом-сповіщенням. Одна процедура не представляє всіх варіантів техніки, тож «покрита техніка» означає лише «виявлено перевірені процедури». Покриття доцільно зважувати за моделлю загроз і рахувати частково:

$$\mathrm{Cov} = \frac{\sum_{t \in T} w_t \, k_t / n_t}{\sum_{t \in T} w_t}, \tag{15.4}$$

де T — техніки з моделі загроз; w_t — вага (поширеність в угрупованнях, що атакують галузь, критичність етапу); n_t і k_t — кількості виконаних і виявлених процедур техніки t. *Приклад.* Для NIDS обрано п'ять мережево спостережуваних технік: T1046 (сканування сервісів, $w = 1$, виявлено $3/3$), T1110 (підбір облікових даних, $w = 3$, $4/5$), T1021 (віддалені сервіси,

$w = 3$, $2/4$), T1071 (протоколи прикладного рівня для керування, $w = 2$, $1/3$), T1048 (ексфільтрація альтернативним протоколом, $w = 1$, $0/2$). Тоді $\mathrm{Cov} = (1 + 2{,}4 + 1{,}5 + 0{,}67 + 0)/10 \approx 0{,}56$, тоді як бінарний підрахунок «техніку виявлено хоч раз» дає $4/5 = 80\,\%$. Формула (15.4) не враховує хибних тривог, тому покриття подається разом із точністю і вартістю (15.3).

Програма MITRE ATT&CK Evaluations [252] публікує результати емуляції однакових сценаріїв проти продуктів учасників. Вона не ранжує учасників, проте постачальники обирають вигідні агрегати («100 % виявлення»), а конфігурації і сценарій можуть не відповідати умовам замовника. Тому корисні сирі дані за кроками, перераховані за власними вагами (15.4) з урахуванням хибних тривог.

15.7 Відтворюваність і статистика

Метрика на скінченній вибірці — оцінка з невизначеністю. Для частки $\hat{p} = k/n$ інтервал Вальда $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/n}$ погано працює поблизу 0 і 1, де й лежать TPR і FPR детекторів. Інтервал Вілсона [345]

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \tag{15.5}$$

($z = 1{,}96$ для довіри 95 %) не виходить за $[0, 1]$ і має близьке до номінального покриття навіть для малих n. *Приклад.* Для $k = 95$, $n = 100$ інтервал Вальда $[0{,}907;\ 0{,}993]$, Вілсона — $[0{,}888;\ 0{,}978]$, несиметричний і ширший донизу. Для $k = n = 20$ інтервал Вальда вироджується в точку $\{1\}$, а Вілсона дає $[0{,}839;\ 1]$: двадцять успішних тестів не доводять TPR понад 84 %.

Результати залежать від зерен, версій правил і втрат пакетів (розділ 14), тож потрібні повторні прогони (для ML — не менше 10 зерен, для стендових сценаріїв — 3–5 повторень з незалежним фоном) і порівняння розподілів критерієм Манна–Уітні з розміром ефекту $\hat{A}_{12}$ (13.5), як у методології Кліса та співавторів (розділ 13). Парне порівняння двох детекторів на тих самих об'єктах виконують критерієм Мак-Немара за розбіжними рішеннями.

Відтворюваність забезпечують артефакти: код, конфігурації, версії, топологія стенда, зерна, журнали істини; для робіт з оцінювання IDS нормою мають бути значки перевірки артефактів ACM (Artifacts Available, Evaluated, Results Reproduced).

15.8 Кейс: порівняльне оцінювання сигнатурної та ML-NIDS

Постановка. Оператор регіональної енергомережі обирає між сигнатурним сенсором (Suricata з комерційними правилами і правилами політик, розділ 8) і ML-сенсором на потокових ознаках (постачальник заявляє $F_1 = 0{,}99$ на CIC-IDS2017). Протокол будується за вимогами підрозділу 15.3.

Стенд і дані. Цифровий двійник сегмента підстанцій і офісної мережі розгорнуто на кіберполігоні (рис. 15.2). Фон — агенти користувачів і телемеханіка за профілями оператора, 14 діб; емулятор противника виконує 40 процедур 10 технік ATT&CK з моделі загроз; журнал істини дає розмітку. Заяву постачальника перевірено на виправленій версії CIC-IDS2017 [164].

Розбиття, пороги, статистика. ML-модель навчається на 1–7 добах, валідується на 8–9-й, тестується на 10–14 добах із частково новими процедурами; для контролю виконано й випадкове розбиття. Одиниця — потік; у тесті 2000 атакувальних і $2 \cdot 10^6$ легітимних потоків ($\pi = 10^{-3}$). Оператор задав $c_{FN} = 20$, $c_{FP} = 1$; поріг ML обрано за (15.3) на валідаційній частині. ML — 10 прогонів з різними зернами (у таблиці — медіана), стенд — 3 повторення з різним фоном, інтервали — за (15.5), порівняння — критерій Мак-Немара (табл. 15.2).

Таблиця 15.2 — Результати кейсу (ілюстративні умовні значення)

Метрика	Сигнатурна	ML, випадкове розбиття	ML, часове, поріг за замовч.	ML, часове, поріг за (15.3)
TPR [95 % ДІ]	0,62 [0,60; 0,64]	0,99 [0,98; 0,99]	0,81 [0,79; 0,83]	0,70 [0,68; 0,72]
FPR	$3 \cdot 10^{-5}$	10^{-3}	$4 \cdot 10^{-3}$	$2 \cdot 10^{-4}$
Prec при $\pi = 10^{-3}$	0,95	0,50	0,17	0,78
F_1	0,75	0,66	0,28	0,74
MCC	0,77	0,70	0,37	0,74
Вартість C	15 260	2400	15 600	12 400

Аналіз. Оптимізм випадкового розбиття зникає за часового: нові процедури виявляються гірше, FPR зростає через дрейф. Поріг постачальника за замовчуванням дає точність 0,17 — п'ять хибних тривог на кожну істинну; поріг за вартістю знижує FPR у 20 разів ціною частини повноти. Сигнатурна IDS має найкращі F_1 і MCC, але більшу вартість, ніж ML з обґрунтованим порогом: метрики дають різних «переможців», тож вирішує метрика замовника (15.3). Виявлені потоки детекторів мало перетинаються (сигнатури — відомі процедури, ML — аномалії обсягу), тому рекомендовано ешелонування: сигнатурна IDS блокує за правилами з малим FPR, ML постачає сповіщення для тріажу. Заява $F_1 = 0{,}99$ на виправленому CIC-IDS2017 не підтвердилася (пор. [233]). Покриття (15.4) — 0,58 і 0,66, разом 0,79. Артефакти збережено.

Висновки до розділу

Оцінювання виявлення вимагає явно заданих одиниці оцінювання, базової частоти і цін помилок. За дисбалансу класів accuracy і ROC-AUC оманливі; інформативні PR-крива, точність при цільовій базовій частоті, MCC і очіку-

вана вартість, що визначає поріг. Вади наборів даних від DARPA 1998/1999 до CIC-IDS2017 роблять частину опублікованих результатів характеристикою артефактів. Пастки ML-IDS усуваються протоколом із вимогами, зафіксованими до експерименту, зокрема часовим розбиттям. Змагальну стійкість оцінюють у реалістичній моделі загроз. Кіберполігони дають відтворювані сценарії з журналом істини, емуляція противника — зважене покриття технік ATT&CK. Результати супроводжуються інтервалами Вілсона, повторними прогонами й артефактами. Кейс показав, що вибір між сигнатурною і ML-NIDS залежить від метрики замовника, а раціональним рішенням часто є їх ешелонування.

Питання для самоперевірки

1. Як вибір одиниці оцінювання (пакет, потік, інцидент) впливає на TPR?
2. Обчисліть accuracy, точність, F_1 і MCC (15.1) для $TP = 40$, $FN = 10$, $FP = 200$, $TN = 99\,750$. Яка метрика найінформативніша?
3. Виведіть (15.2) з теореми Баєса. Як зміниться точність при $\mathrm{TPR} = 0{,}9$, $\mathrm{FPR} = 10^{-3}$ і переході від $\pi = 0{,}2$ до $\pi = 10^{-4}$?
4. Чому ROC-крива оманлива за дисбалансу класів? Яку базову лінію має PR-крива?
5. Для умов прикладу до (15.3) знайдіть, за якого відношення c_{FN}/c_{FP} пороги θ_1 і θ_2 рівноцінні.
6. У чому критика МакГ'ю щодо DARPA 1998/1999? Що успадкували KDD Cup 99 і NSL-KDD?
7. Які вади CIC-IDS2017 виявлено [164, 233] і як вони впливають на результати?
8. Наведіть пастки ML за Арпом та співавторами і приклади витоку даних та хибної кореляції в ML-IDS.
9. Чому випадкове розбиття завищує оцінки ML-IDS? Як оцінювати дрейф концепцій?
10. Чим специфічні змагальні приклади в мережевому домені? Що таке допустима множина змін?
11. Порівняйте Mininet, GNS3/EVE-NG і containerlab. Яку роль відіграє журнал істини на кіберполігоні?
12. Обчисліть покриття (15.4), якщо в прикладі виявлено обидві процедури T1048. Чому покриття подають разом із точністю?
13. Обчисліть інтервали Вілсона (15.5) і Вальда для 48 виявлень з 50 атак.

14. Дослідницьке: розробіть методику побудови на кіберполігоні набору даних для оцінювання NIDS, що усуває вади табл. 15.1, і метрики якості самого набору (реалістичність фону, повнота розмітки).
15. Дослідницьке: сформулюйте модель загроз і протокол оцінювання змагальної стійкості потокової ML-IDS з обмеженнями простору задачі та метрику стійкості як функцію бюджету змін і знань противника.

РОЗДІЛ 16

Управління вразливостями, розкриття та супровід програмного забезпечення мережевої безпеки

Випуском продукту інженерія ПЗМБ не завершується: для граничного пристрою, що роками працює на межі довіри, саме супровід визначає реальний рівень захищеності. Розділ відкриває блок про супровід і гарантування та розглядає управління вразливостями з позиції виробника: процеси за ISO/IEC 30111 і 29147, PSIRT, координоване розкриття, ідентифікацію й пріоритизацію, обов'язки за Регламентом (ЄС) 2024/2847 про кіберстійкість (CRA), виправлення граничних пристроїв, аналіз варіантів, моделі вікна вразливості й оптимального випуску. Строки звітування наведено в розділі 2, безпечне оновлення — у розділі 9, SBOM і VEX — у розділі 10.

16.1 Процес обробки вразливостей виробником

16.1.1 ISO/IEC 30111 і ISO/IEC 29147

ISO/IEC 30111 [207] визначає **обробку вразливостей** (vulnerability handling) — внутрішній процес виробника: підготовка, приймання, перевірка, розроблення виправлення, випуск і післявипускна діяльність. ISO/IEC 29147 [206] регламентує **розкриття** (disclosure) — зовнішній інтерфейс: приймання звітів, комунікацію з дослідниками, публікацію бюлетенів. Життєвий цикл вразливості в цих процесах показано на рис. 16.1.

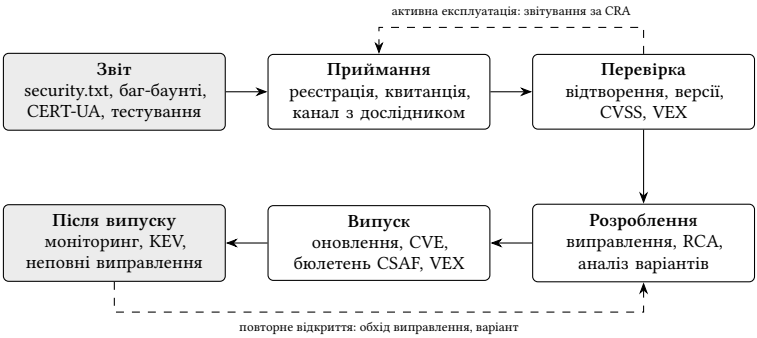


Рис. 16.1 — Життєвий цикл вразливості в процесі виробника (за [207, 206])

16.1.2 PSIRT і точка входу

Процес реалізує **команда реагування на інциденти безпеки продуктів** (PSIRT), що, на відміну від CSIRT, відповідає за безпеку *продуктів* у клієнтів. FIRST PSIRT Services Framework v1.1 [175] визначає шість сфер послуг: керування екосистемою зацікавлених сторін, виявлення вразливостей (зокрема моніторинг сторонніх компонентів за SBOM), сортування й аналіз, усунення, розкриття і навчання. Для виробника ПЗМБ критичні цілодобова досяжність (строки CRA відлічуються в годинах) і формалізоване право ухвалювати рішення про позаплановий випуск.

Точкою входу є файл `/.well-known/security.txt` (RFC 9116 [178]) з обов'язковими полями `Contact` і `Expires`; його доцільно публікувати і на сайті виробника, і на вебінтерфейсі пристрою. Контактну адресу прямо вимагає частина II Додатка I CRA [301].

16.2 Координоване розкриття вразливостей

16.2.1 Моделі розкриття

Повне розкриття — негайна публікація подробиць, що дає перевагу порушникам; *відповідальне* — повідомлення лише виробнику, що залежить від його доброї волі. **Координоване розкриття** (CVD) — компроміс: дослідник, виробник і за потреби координатор (CERT/CC, національна CSIRT) узгоджують строк ембарго, після якого інформацію публікують незалежно від стану виправлення. Настанова CERT/CC [194] описує ролі і фази CVD, наголошуючи, що мета — мінімізувати сукупну шкоду. Найвідоміша формалізація строків — політика Google Project Zero «90+30» [186]: 90 днів на виправлення, публікація подробиць через 30 днів після нього, а для вразливостей, що вже експлуатуються, — 7 днів. Арора, Теланг і Сю [76] показали, що строк ембарго є інструментом впливу на виробника (підрозділ 16.7).

16.2.2 Багатосторонні випадки

Найскладніші — вразливості протоколів, що уражають десятки виробників. Атака HTTP/2 Rapid Reset (CVE-2023-44487) використовувала можливість клієнта скасовувати потоки; Google, Cloudflare і AWS скоординовано оприлюднили відомості 10.10.2023, після чого виправлення мали випустити всі постачальники реалізацій HTTP/2. Такі випадки вимагають координатора, спільного ембарго і заяв VEX кожного виробника.

16.2.3 Баг-баунті і український контекст

Програми **баг-баунті** доповнюють CVD винагородою (модель витрат — у розділі 13 [21]); обов'язковий елемент політики — *безпечна гавань* (safe harbor): відмова від переслідування дослідника, що діяв у межах правил. В Україні Мінцифри 2020 р. провело баг-баунті застосунку «Дія» на копії

продуктивного середовища. Постанова КМУ № 1580 [12] (розділ 2) запровадила порядок пошуку вразливостей у державних ІКС і об'єктах КІІ, де координатором розкриття є CERT-UA [107]. Отже, звіт може надійти через CERT-UA чи власника системи.

16.3 Ідентифікація та оцінювання вразливостей

16.3.1 CVE, EUVD і CWE

Ідентифікатор CVE присвоює **орган нумерації** (CNA) — виробник, дослідницька організація чи координатор у межах своєї сфери; виробнику ПЗМБ доцільно стати CNA, щоб контролювати якість опису і строки публікації. У квітні 2025 р. контракт MITRE на супровід програми CVE ледь не завершився без продовження; CISA в останній момент продовжило його на 11 місяців, а на початку 2026 р. фінансування програми переведено в захищену статтю бюджету [100]. Паралельно 13.05.2025 запущено Європейську базу вразливостей EUVD, яку веде ENISA на підставі NIS2 [169]; з листопада 2025 р. ENISA є кореневою організацією (Root) програми CVE [165]. Бюлетень виробника на ринку ЄС має містити обидва ідентифікатори. Класифікація за CWE [136] описує першопричину, а не симптом; для аналізу варіантів (підрозділ 16.6) потрібна точна базова слабкість (CWE-787, CWE-288), а не узагальнена категорія.

16.3.2 CVSS 4.0, EPSS, SSVC, KEV і VEX

CVSS 4.0 [173] має групи метрик *базових* (зокрема нова метрика вимог до атаки AT), *загроз* (зрілість експлойту), *середовища* і *додаткових* (без впливу на оцінку). Номенклатура вказує враховані групи: CVSS-B, CVSS-BT, CVSS-BE, CVSS-BTE; виробник публікує CVSS-B, а пріоритет оператора визначає CVSS-BTE. Спрінг та співавтори [330] наголосили, що CVSS вимірює технічну тяжкість, а не ризик, а арифметика над порядковою шкалою некоректна. Тому CVSS доповнюють іншими системами (табл. 16.1).

Таблиця 16.1 — Системи оцінювання і пріоритизації вразливостей

Система	Що оцінює	Результат	Обмеження
CVSS 4.0	технічну тяжкість з урахуванням загроз і середовища	оцінка 0–10, вектор	не ризик; порядкова шкала
EPSS v4	імовірність спостереження експлуатації за 30 днів	імовірність, перцентиль	лише опубліковані CVE; без урахування середовища
SSVC	рішення для конкретної сторони	Track, Track*, Attend, Act	якісні точки рішення; калібрування
CISA KEV	факт активної експлуатації	так / ні, строк	запізнення; неповнота
VEX	чи уражений конкретний продукт	статус і обґрунтування	довіра до заяви виробника

EPSS [174, 214] щодня оцінює ймовірність спостереження спроб експлуатації кожної опублікованої CVE протягом 30 днів (версія 4 — з 17.03.2025). Денисюк [9] показав, що навіть логістична регресія за ознаками CVSS, CWE і виробника, навчена на мітках KEV, дає високу повноту виявлення за дисбалансу класів, але успадковує неповноту KEV. SSVC у версії CISA [124] замінює числову оцінку деревом рішень за точками «стан експлуатації», «технічний вплив», «автоматизованість», «поширеність у місії», «суспільне благополуччя». KEV [123] фіксує факт експлуатації. VEX повідомляє статус продукту (`not_affected`, `affected`, `fixed`, `under_investigation`); статус «не уражено» потребує обґрунтування, перелік яких наведено в підрозділі 10.2. Фрагмент CSAF VEX [135] наведено в лістингу 16.1.

Лістинг 16.1 — Фрагмент CSAF VEX для двох ліній продукту

```
"vulnerabilities": [{
  "cve": "CVE-2026-NNNNN",
  "ids": [{"system_name": "EUVD",
           "text": "EUVD-2026-NNNNN"}],
  "product_status": {
    "fixed": ["GW-7.4.2-h1", "GW-7.2.10"],
    "known_affected": ["GW-7.2.9"],
    "known_not_affected": ["GW-LITE-3.1"]
  },
  "flags": [{
    "label": "component_not_present",
    "product_ids": ["GW-LITE-3.1"]
  }],
  "remediations": [{
    "category": "vendor_fix",
    "details": "Оновити до 7.2.10",
    "product_ids": ["GW-7.2.9"]
  }]
}]
```

16.3.3 Комбінований пріоритет

Практичні схеми поєднують імовірність експлуатації і вплив:

$$R_i = \tilde{p}_i \cdot \frac{S_i}{10}, \qquad \tilde{p}_i = \begin{cases} 1, & i \in \text{KEV}, \\ p_i^{\text{EPSS}}, & \text{інакше}, \end{cases} \tag{16.1}$$

де S_i — оцінка CVSS-BE для середовища оператора, p_i^{EPSS} — імовірність за EPSS. *Числовий приклад.* Вразливість A: $S = 9{,}3$, $p = 0{,}02$, $R_A = 0{,}019$; B: $S = 7{,}5$, $p = 0{,}60$, $R_B = 0{,}45$; C: $S = 6{,}5$, у KEV, $R_C = 0{,}65$. Ранжування C, B, A протилежне ранжуванню за CVSS. Формула (16.1) — модель-евристика: добуток порядкової оцінки й імовірності не має строгої інтерпретації [330], EPSS оцінює експлуатацію «будь-де», а для щойно виявленої вразливості власного продукту не існує (тоді PSIRT спирається на CVSS-BT і досяжність, формула (1.3)). Для критичних рішень R_i — лише вхід до дерева SSVC.

16.4 Обов'язки виробника за CRA протягом періоду підтримки

Перелік вимог частини II Додатка I CRA наведено в розділі 2; розглянемо інженерні наслідки. *Період підтримки*, як правило, становить не менше п'яти років; кожне оновлення безпеки залишається доступним щонайменше 10 років після випуску або до кінця періоду підтримки. Для ПЗМБ це обмежує кількість одночасно підтримуваних гілок прошивки. Оновлення безпеки поширюються *без зволікання і безоплатно*, за можливості — окремо від функціональних. Вразливість у сторонньому компоненті виробник повідомляє його супровідникові (maintainer). З 11.09.2026 активно експлуатовані вразливості й серйозні інциденти повідомляються через єдину платформу звітування ENISA в строки (2.1); обов'язок стосується й продуктів, випущених до 11.12.2027. PSIRT потрібні критерії класифікації «активно експлуатована», готові шаблони звітів і журнал рішень для ринкового нагляду.

16.5 Виправлення для граничних пристроїв і вікно вразливості

16.5.1 Гарячі виправлення і тимчасові заходи

Граничні пристрої не можна перезавантажувати будь-коли, а вразливості площини менеджменту експлуатуються масово за дні (розділ 1). Тому PSIRT має три рівні реагування: *тимчасовий захід* (mitigation) — зміна конфігурації, правило фільтрації чи сигнатура IDS; *гаряче виправлення* (hotfix) — мінімальна зміна для підтримуваних гілок зі скороченим, але обов'язковим регресійним набором; *плановий випуск* з повним тестуванням. Доставка має відповідати розділу 9 (підпис, захист від відкату, A/B). Тимчасовий захід перевіряють так само ретельно, як виправлення.

Випадок Ivanti Connect Secure 2024 р. показав, що після компрометації оновлення недостатньо. Директива CISA ED 24-01 [122] вимагала перевір-

ки цілісності інструментом виробника, а спільна настанова агенцій [115] зафіксувала, що інструменти перевірки цілісності в низці випадків не виявили компрометації, а присутність порушника могла зберігатися навіть після скидання до заводських налаштувань. Висновок для виробника: перевірка цілісності має спиратися на апаратний корінь довіри і вимірюване завантаження, а відновлення — перевстановлювати образ із довіреного носія.

16.5.2 Модель вікна вразливості

Фрей та співавтори [179] описали **вікно вразливості** як інтервал між появою експлойту і встановленням виправлення. Нехай t_e — початок масової експлуатації, t_r — випуск виправлення, t_i — середній час його встановлення (від моменту розкриття), λ — інтенсивність атак на експонований пристрій. За пуассонівського потоку атак імовірність компрометації

$$P_c = 1 - e^{-\lambda T}, \qquad T = \max\bigl(0,\; t_r + t_i - t_e\bigr). \tag{16.2}$$

Якщо T_m діб діє тимчасовий захід з ефективністю m, показник степеня дорівнює $\lambda\bigl[(1-m)T_m + (T - T_m)\bigr]$ (рис. 16.2).

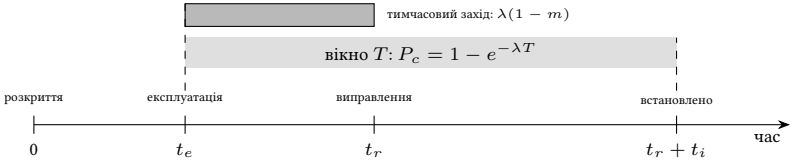


Рис. 16.2 — Вікно вразливості граничного пристрою (за [179])

Час t_e швидко скорочується: за аналізом Mandiant вразливостей, експлуатованих 2023 р., середній час до експлуатації зменшився з 63 днів у 2018–2019 рр. до 5 днів, а 70 % вразливостей експлуатовано як вразливості нульового дня [109].

Числовий приклад. $\lambda = 0{,}02$ доби$^{-1}$, $t_r = 0$, $t_i = 21$ доба, $t_e = 5$ діб: $T = 16$, $P_c = 1 - e^{-0{,}32} \approx 0{,}27$. Тимчасовий захід з $m = 0{,}8$ протягом перших 7 діб вікна дає показник $0{,}02 \cdot (1{,}4 + 9) = 0{,}208$ і $P_c \approx 0{,}19$. Автоматичне оновлення з $t_i = 4$ дає $T = 0$, але для вразливості нульового дня $t_e < 0$ і вікно існує за будь-якої швидкості встановлення. Отже, виробник впливає на P_c і через t_i, і через готовність тимчасових заходів.

16.6 Аналіз першопричин і варіантів

Аналіз першопричин (RCA) відповідає на три питання: яка слабкість (CWE) спричинила дефект; чому його не виявили засоби верифікації (розділи 11–13); де ще в коді є та сама слабкість. Останнє — предмет **аналізу**

варіантів (variant analysis): за виправленим дефектом формулюють запит (наприклад, CodeQL, розділ 12), що шукає подібні шаблони в усій кодовій базі. Кожне виправлення супроводжує регресійний тест і поповнення корпусу фазингу (розділ 13).

За оглядом Google, 17 із 41 вразливості нульового дня, експлуатованої 2022 р., були варіантами раніше повідомлених, зокрема через неповні виправлення [332]. Показовий кейс — Ivanti EPMM: після виправлення експлуатованого обходу автентифікації CVE-2023-35078 знайдено CVE-2023-35082, що виникала з того самого надто дозвільного фільтра доступу і фактично обходила виправлення; згодом її також внесено до KEV. Виправлення має усувати клас шляхів (нормалізація, централізована перевірка доступу), а не конкретний URL. Якість виправлень характеризують *часткою неповних виправлень* і *ефективним* часом усунення:

$$f_{\text{неп}} = \frac{N_{\text{обх}}}{N_{\text{випр}}}, \qquad \text{MTTR}^{*} = \frac{1}{n}\sum_{i=1}^{n}\bigl(\Delta_i + \delta_i\bigr), \tag{16.3}$$

де $N_{\text{випр}}$ — виправлені за період вразливості, $N_{\text{обх}}$ — ті з них, для яких згодом знайдено обхід або варіант з тією самою першопричиною; Δ_i — час до першого виправлення (як у (3.8)), δ_i — додатковий час до повного виправлення. *Числовий приклад.* З 40 виправлених за рік вразливостей високого і критичного рівнів для 3 знайдено обхід: $f_{\text{неп}} = 0{,}075$. Для трьох критичних вразливостей з $\Delta = 5$, 9 і 12 діб $\text{MTTR} = 8{,}7$ доби; якщо друге виправлення неповне, а повне вийшло ще через 20 діб, $\text{MTTR}^{*} = 15{,}3$ доби.

16.7 Оптимальний час випуску виправлення

Арора, Теланг і Сю [76] показали, що виробник зазвичай випускає виправлення пізніше за соціальний оптимум, бо враховує лише частину збитків клієнтів, а скорочення захищеного періоду змушує його поспішати. Спрощена модель: нехай τ — тривалість розроблення виправлення, витрати на прискорення спадають як K/τ, збитки клієнтів за добу до випуску $h = N\lambda D$ (N — уражені пристрої, λ — інтенсивність успішних атак, D — збиток від компрометації), а виробник враховує частку $\alpha \le 1$ цих збитків. Тоді

$$J(\tau) = \frac{K}{\tau} + \alpha h \tau, \qquad \tau^{*} = \sqrt{\frac{K}{\alpha h}}. \tag{16.4}$$

Числовий приклад. $K = 420\,000$ євро·доба (випуск за 7 діб коштує додатково 60 тис. євро), $N = 5000$, $D = 40\,000$ євро, до розкриття $\lambda = 10^{-5}$ доби$^{-1}$, тож $h = 2000$ євро/добу. Соціальний оптимум ($\alpha = 1$): $\tau^{*} = \sqrt{210} \approx 14{,}5$ доби; виробник з $\alpha = 0{,}25$ обирає $\tau^{*} = \sqrt{840} \approx 29$ діб. За початку експлуатації ($\lambda = 10^{-3}$) навіть за $\alpha = 0{,}25$ $\tau^{*} \approx 2{,}9$ доби. Тому фіксоване ембарго і

CRA (що збільшує α) скорочують час випуску. Обмеження: K, λ, D важко оцінити, а поспіх підвищує $f_{\text{неп}}$, чого (16.4) не враховує.

16.8 Кейс: критична вразливість у VPN-шлюзі українського виробника

Модельний випадок: український виробник наскрізного VPN-шлюзу (підрозділ 1.9) для ринку ЄС і державних ІКС підтримує гілки прошивки 7.2 і 7.4 та полегшену лінію GW-LITE без вебпорталу. Дослідник, що діяв за порядком ПКМУ № 1580, виявив обхід автентифікації у вебпорталі SSL-VPN (CWE-288) і повідомив CERT-UA. Хронологію наведено в табл. 16.2 (Дk — доба від отримання звіту).

Таблиця 16.2 — Хронологія обробки вразливості

Час	Подія	Рішення, артефакт
Д0, 10:00	звіт від CERT-UA захищеним каналом	квитанція за 2 год; VEX `under_investigation`
Д0, 18:00	відтворено; уражено 7.2 і 7.4	CVSS-B 9.3; ембарго до Д30
Д2	аналіз варіантів: той самий шаблон в API менеджменту	обсяг виправлення розширено
Д4, 14:00	CERT-UA: ознаки експлуатації	момент t_0 за CRA; ембарго скасовано
Д5, 10:00	раннє попередження ENISA	через 20 год після t_0
Д5, 16:00	тимчасовий захід: правило, сигнатура IDS	сповіщення клієнтів і CERT-UA
Д6	гаряче виправлення 7.4.2-h1 і 7.2.10	CVE, EUVD, бюлетень CSAF, VEX
Д7, 10:00	повідомлення про вразливість (72 год)	оцінка впливу, індикатори
Д19	остаточний звіт ENISA	14 діб після t_f (Д5)
Д36	публікація технічних подробиць	30 діб після виправлення

Ключові рішення. (1) EPSS для невідомої вразливості відсутній, тому критичність визначено за CVSS-B і досяжністю; за SSVC до Д4 — Attend (плановий випуск у межах ембарго). (2) Аналіз варіантів виявив ту саму слабкість в API менеджменту; перевірку доступу централізовано. (3) З ознаками експлуатації рішення змінилося на Act: за (16.4) τ^* становить дві-три доби, тому замість планового випуску на Д18 гаряче виправлення вийшло на Д6. (4) Для GW-LITE опубліковано `known_not_affected` з обґрунтуванням `component_not_present` (лістинг 16.1). (5) Обидва перші строки CRA дотримано із запасом 4 год завдяки цілодобовому чергуванню; строк остаточного звіту за (2.1) відлічується від t_f — моменту, коли став доступним *перший* коригувальний або пом'якшувальний захід (тимчасовий захід Д5), а не гаряче виправлення Д6, тому звіт подано на Д19. (6) Клієнтам з ознаками компрометації рекомендовано перевстановлення образу з довіреного носія [115].

Ефект рішень за (16.2): від Д4 ($t_e = 0$), з $\lambda = 0{,}03$ доби$^{-1}$ і $t_i = 5$ діб (автоматичне оновлення в більшості клієнтів) $T = 2 + 5 = 7$ діб, з них 6 під заходом з $m = 0{,}7$: показник $0{,}03 \cdot (1 + 0{,}3 \cdot 6) = 0{,}084$, $P_c \approx 0{,}08$. За плановим випуском на Д18 без заходу $T = 19$ і $P_c = 1 - e^{-0{,}57} \approx 0{,}43$. Для $\approx$1200 експонованих інсталяцій (підрозділ 1.9) це 97 проти 521 очікуваного скомпрометованого пристрою (параметри ілюстративні).

Висновки до розділу

Управління вразливостями продовжує безпечний життєвий цикл після випуску і для ПЗМБ вирішальне, бо граничні пристрої експлуатуються масово, а час до експлуатації скоротився до днів. Процес виробника описують ISO/IEC 30111 і 29147, реалізує PSIRT. Координоване розкриття з фіксованим ембарго стимулює виробника, що обґрунтовано моделлю Арори–Теланга–Сю. Пріоритизація поєднує CVSS 4.0, EPSS, KEV, SSVC і VEX; показник (16.1) — евристика, а не міра ризику. CRA перетворює ці практики на обов'язки й вимагає цілодобової готовності PSIRT. Модель (16.2) показує, що ймовірність компрометації зменшують не лише швидкий випуск, а й тимчасові заходи та скорочення часу встановлення; метрики (16.3) і аналіз варіантів протидіють неповним виправленням.

Питання для самоперевірки

1. Порівняйте предмет ISO/IEC 30111 і ISO/IEC 29147. Які фази обробки вразливості визначає ISO/IEC 30111?
2. Чим PSIRT відрізняється від CSIRT? Назвіть сфери послуг FIRST PSIRT Services Framework.
3. Які поля security.txt (RFC 9116) обов'язкові і чому файл доцільно розміщувати на самому граничному пристрої?
4. Порівняйте повне, відповідальне і координоване розкриття. Як працює політика «90+30» і навіщо окремий строк для вразливостей, що вже експлуатуються?
5. Які особливості багатосторонніх випадків розкриття ілюструє HTTP/2 Rapid Reset?
6. Яку роль відіграють CERT-UA і порядок ПКМУ № 1580 у розкритті вразливостей ПЗМБ у державних ІКС?
7. Що таке CNA? Як криза фінансування програми CVE 2025 р. вплинула на роль ENISA і EUVD?
8. Опишіть групи метрик CVSS 4.0 і номенклатуру CVSS-B/BT/BE/BTE. Чому CVSS не є мірою ризику?

9. Порівняйте EPSS, SSVC і KEV за вхідними даними, результатом і обмеженнями (табл. 16.1).
10. Які статуси й обґрунтування передбачає VEX? Поясніть зміст лістингу 16.1.
11. Обчисліть пріоритет (16.1) для вразливості з CVSS-BE 8,1 і EPSS 0,15. Як зміниться ранжування, якщо вона потрапить до KEV?
12. Які обов'язки CRA щодо періоду підтримки й оновлень безпеки впливають на кількість підтримуваних гілок прошивки?
13. Чому після компрометації граничного пристрою оновлення недостатньо? Які вимоги до перевірки цілісності випливають з кейсу Ivanti?
14. Дослідницьке: розширте модель (16.4), врахувавши залежність частки неповних виправлень $f_{\text{неп}}$ від τ; дослідіть зміну оптимального часу випуску й запропонуйте оцінювання параметрів за історичними даними PSIRT.
15. Дослідницьке: запропонуйте модель прогнозування ймовірності експлуатації ще не опублікованої вразливості власного продукту ПЗМБ (аналог EPSS для PSIRT) з урахуванням досяжності, класу CWE та історії експлуатації граничних пристроїв і схему її валідації.

РОЗДІЛ 17

Гарантування безпеки, штучний інтелект у розробленні й тестуванні та напрями наукових досліджень

Кожен метод попередніх розділів породжує *свідчення*, але жоден окремо не відповідає на запитання оцінювача чи регулятора: чому продукту можна довіряти? Заключний розділ розглядає кейси гарантування, організацію робіт розробника для сертифікації, зведену систему метрик, вплив штучного інтелекту (ШІ) на розроблення й тестування, методологію досліджень і відкриті проблеми для дисертацій. Схеми оцінювання відповідності описано в розділі 2.

17.1 Кейси гарантування

17.1.1 Структура і нотації

Кейс гарантування (assurance case) — обґрунтована, перевірювана сукупність аргументів і свідчень на підтримку твердження про властивість системи. ISO/IEC/IEEE 15026-2:2022 [209] визначає його складники: *твердження* (claim) з умовами застосовності, *аргументи*, *свідчення* (evidence) і *обґрунтування* кроків міркування, а також припущення. Кейс робить явним, чому свідчень достатньо.

Найпоширеніша графічна нотація — **GSN** (Goal Structuring Notation) за Community Standard версії 3 [182]: цілі (прямокутники), стратегії (паралелограми), розв'язки-свідчення (кола), контекст (прямокутник із заокругленими кутами), припущення й обґрунтування (овали з позначками A і J); зв'язки «підтримується» (зафарбована стрілка) і «в контексті» (контурна). Нотація **CAE** (Claims–Arguments–Evidence) Блумфілда і Неткачової [91] пропонує типові блоки аргументації: декомпозицію, підстановку, конкретизацію, обчислення.

17.1.2 Свідчення з конвеєра і кількісна впевненість

Конвеєр сам є джерелом свідчень (розділи 10–13); кожне свідчення прив'язують до гешу артефакту.

Байєсовий підхід трактує свідчення E_i як спостереження з відношенням правдоподібності щодо твердження G:

$$O(G \mid E_1, \dots, E_n) = O(G) \prod_{i=1}^{n} \Lambda_i, \qquad \Lambda_i = \frac{P(E_i \mid G)}{P(E_i \mid \neg G)}, \qquad P = \frac{O}{1+O}, \tag{17.1}$$

де $O(G) = P(G)/(1 - P(G))$ — апріорні шанси, Λ_i — доказова сила свідчення: у скільки разів імовірніше отримати «чистий» результат перевірки, якщо твердження істинне. Добуток коректний лише за *умовної незалежності* свідчень. *Приклад.* За $P(G) = 0{,}3$ ($O = 0{,}43$), фазингу без нових аварій ($\Lambda_1 = 3$), доведень BMC ($\Lambda_2 = 4$) і тесту на проникнення без знахідок ($\Lambda_3 = 1{,}3$) маємо $O = 0{,}43 \cdot 3 \cdot 4 \cdot 1{,}3 \approx 6{,}7$ і $P \approx 0{,}87$. Якщо фазинг і BMC мають спільні хибні припущення про контекст обв'язки, їхня сукупна сила менша за добуток: другий аргумент може додати значно менше впевненості, ніж обіцяє наївна незалежність [237]. Значення Λ_i ілюстративні (модель-евристика); їх калібрують за знахідками PSIRT (розділ 16).

Альтернатива — *елімінативна індукція* (бейконівська впевненість) Гудінафа, Вайнстока і Кляйна [185]: для твердження перелічують *сумніви* (defeaters) — причини, з яких воно може бути хибним, — і показують, які з них усунуто. Впевненість записують як $m|n$ (усунуто m з n).

17.2 Організація робіт розробника для сертифікації

17.2.1 Від цілей безпеки до артефактів

Підготовку до оцінювання за CC/EUCC [201, 132], до державної експертизи (розділ 2) і до вимог Регламенту (ЄС) 2024/2847 про кіберстійкість (CRA) [301] доцільно вести як *побічний продукт* розроблення: модель загроз (розділ 4) $\rightarrow$ цілі безпеки й SFR у завданні безпеки (ST) $\rightarrow$ проєктна документація ADV $\rightarrow$ свідчення життєвого циклу ALC (керування конфігурацією, постачання, обробка вад ALC_FLR) $\rightarrow$ тестова документація ATE $\rightarrow$ аналіз вразливостей AVA (розділ 11). Одне джерело обслуговує кілька схем (табл. 17.1), що знижує вартість відповідності (2.5).

Таблиця 17.1 — Артефакти процесу розроблення як свідчення для схем оцінювання

Артефакт процесу	CC / EUCC	Держекспертиза КЗІ/ТЗІ	CRA
Модель загроз, трасування	ST: проблема безпеки, цілі, SFR	ТЗ, обґрунтування функцій захисту	оцінка ризиків
Архітектура, специфікація API	ADV_FSP, ADV_TDS	пояснювальна записка	опис проєкту
Конфігурація, SLSA	ALC_CMC, ALC_CMS, ALC_DEL	комплектність, версії	цілісність оновлень, SBOM
Тести, мутації, покриття	ATE_COV, ATE_FUN	програма й методика випробувань	звіти тестування
SAST, BMC, фазинг	AVA_VAN, ATE_DPT	результати випробувань, вихідні тексти	відсутність відомих вразливостей
PSIRT, VEX	ALC_FLR, обробка вразливостей	порядок супроводу	обробка вразливостей, звітування

Для засобів КЗІ експертиза перевіряє національні алгоритми (розділ 6) за вихідними текстами, тому відтворюване збирання (розділ 10) прямо скорочує роботу експертів.

17.2.2 Безперервне гарантування

Сертифікат засвідчує конкретну версію, а граничний пристрій оновлюється щомісяця (розділ 16). Тому EUCC [132] розрізняє безперервність гарантій для незначних змін, переоцінювання і повторну сертифікацію (сертифікат діє не більше п’яти років). **Безперервне гарантування** (continuous assurance) означає, що кейс живе в репозиторії поруч із кодом, а конвеєр після кожної зміни визначає інвалідовані свідчення і перевиробляє їх (рис. 17.1).



Рис. 17.1 — Цикл безперервного гарантування ПЗМБ

Порівняємо повну повторну сертифікацію k значних випусків з інкрементальною:

$$C_{\text{інк}} = C_0 + \sum_{j=1}^{k} \bigl(\rho_j\, C_0 + c_a\bigr), \qquad C_{\text{пов}} = (k+1)\, C_0, \qquad \rho^{*} = 1 - \frac{c_a}{C_0}, \tag{17.2}$$

де C_0 — вартість первинного оцінювання, $\rho_j \in [0;1]$ — частка свідчень, інвалідованих j-м випуском, c_a — вартість аналізу впливу, ρ^* — поріг, вище якого інкрементальний шлях невигідний. *Приклад.* $C_0 = 200$ тис. євро, $k = 4$, $c_a = 15$ тис. євро, $\rho_j = 0{,}2$: $C_{\text{інк}} = 200 + 4 \cdot (40 + 15) = 420$ тис. проти $C_{\text{пов}} = 1000$ тис. євро; $\rho^* = 0{,}925$. Важіль — зменшення ρ_j архітектурою: за ізоляції парсера зміна вебінтерфейсу не інвалідує свідчень щодо IKEv2.

17.3 Метрики і програма безпеки продукту

Програма безпеки продукту поєднує процеси розділу 3, PSIRT і кейси гарантування в керовану систему з цільовими значеннями метрик (табл. 17.2).

Таблиця 17.2 — Зведена система метрик програми безпеки ПЗМБ

Метрика	Формула	Орієнтовна ціль
Повнота трасування	(4.1)	$\geq 0{,}95$ для загроз високого ризику
Безпековий мутаційний бал	(11.3)	$\geq 0{,}9$ для парсерів і автоматів
Поріг точності SAST	(12.2)	правило в конвеєрі лише за $P \geq P^*$
Імовірність нової поведінки	(13.3)	$\hat{p}_{\text{new}} < \varepsilon$ на кількох прогонах
Покриття атестаціями	(10.4)	$K_{\text{prov}} = 1$ для випусків
Повнота SBOM	(10.1)	$R_{\text{SBOM}} \geq 0{,}98$
MTTR, P_{SLA}	(3.8)	$P_{\text{SLA}} \geq 0{,}95$ для критичних
Компрометація у вікні	(16.2)	гаряче виправлення для KEV
Розрив відповідності	(2.3)	$G = 0$ для обов'язкових вимог
Впевненість у твердженні	(17.1)	не нижче цільової для кейсу

Метрики свідчень піддаються маніпуляції (закон Гудгарта): покриття можна підняти тестами без оракулів. Тому кожну доповнюють *метрикою результату* — часткою вразливостей з PSIRT, які мала б виявити відповідна перевірка; саме вона калібрує Λ_i у (17.1).

17.4 Штучний інтелект у розробленні

Асистенти програмування на основі великих мовних моделей (LLM) стали буденністю, але емпіричні дослідження фіксують їхні вади. Пірс та співавтори [287] згенерували GitHub Copilot 1689 програм для сценаріїв CWE Top 25; близько 40 % виявилися вразливими. Перрі та співавтори [288] показали, що учасники з ШІ-асистентом писали менш безпечний код, ніж контрольна група, і водночас частіше вважали його безпечним. Тому згенерований код для ПЗМБ — *недовірений внесок* (розділ 5).

Ризик для ланцюга постачання — **галюциновані залежності**: модель рекомендує неіснуючий пакет, а порушник реєструє пакет із таким ім'ям (slopsquatting, за аналогією з typosquatting, розділ 10). Спраклен та співав-

тори [329] отримали частку галюцинованих пакетів щонайменше 5,2 % для комерційних і 21,7 % для відкритих моделей. Якщо рекомендовано m пакетів і кожен незалежно є галюцинацією з імовірністю h, то

$$P_h = 1 - (1 - h)^m. \tag{17.3}$$

Приклад. Для $h = 0{,}052$, $m = 8$: $P_h = 1 - 0{,}948^8 \approx 0{,}35$; для $h = 0{,}217$ — $P_h \approx 0{,}86$. Отже, перевірка існування, віку й походження кожного нового пакета через внутрішнє дзеркало обов'язкова; незалежність — спрощення, бо галюцинації повторюються.

NIST SP 800-218A [93] доповнює SSDF практиками для розроблення систем з генеративним ШІ. З ризиків OWASP Top 10 for LLM Applications 2025 [280] для конвеєра ПЗМБ найважливіші впровадження інструкцій (LLM01), неналежна обробка виходу (LLM05) і надмірна агентність (LLM06). Українські автори систематизують загрози LLM [3] і ризики розроблення ПЗ на їх основі [27]. Агент із правом змінювати код перетворює будь-який прочитаний недовірений текст на потенційну інструкцію. Правила для команди ПЗМБ: згенерований код проходить ті самі ворота SAST, тестів і огляду (для парсерів і криптографії — другий рецензент); нові залежності — лише через дзеркало; вихідні тексти засобів КЗІ, ключі й звіти про невиправлені вразливості не передаються зовнішнім службам; агенти мають токени з мінімальними правами, без права злиття, а їхні дії журналюються.

17.5 Штучний інтелект у тестуванні

LLM-керований фазинг протоколів (ChatAFL, розділ 13) вилучає граматики з RFC і виводить фазер з плато покриття; OSS-Fuzz-Gen генерує LLM-обв'язки, що збільшують покриття і вже знаходять нові вразливості (зокрема в OpenSSL). Методи машинного навчання для автоматизації тестування на проникнення оглянуто в [17].

Найповніший публічний експеримент — фінал DARPA AI Cyber Challenge (AIxCC) у серпні 2025 р. [144]: автономні системи кібермислення аналізували понад 54 млн рядків коду, виявили 54 штучно внесені вразливості і виправили 43, а також знайшли 18 реальних невідомих вразливостей; середній час виправлення — 45 хв.

Автоматичне виправлення (APR) може бути *правдоподібним, але хибним* — маскувати симптом або вносити новий дефект. Тому LLM-виправлення для ПЗМБ перевіряють відтворенням аварійного входу, регресійним корпусом фазингу, обв'язками ВМС і оглядом людиною.

17.6 Методологія наукових досліджень

Дослідження в інженерії ПЗМБ здебільшого конструктивні. Структура: проблема з кількісною мірою; гіпотеза у фальсифіковній формі («фазер A

досягає більшого покриття станів, ніж B, за 24 год»); дизайн — еталонні цілі, базові методи, кількість повторів, критерій; аналіз загроз валідності за Волін та співавторами [346] (рис. 17.2); артефакти для відтворення.

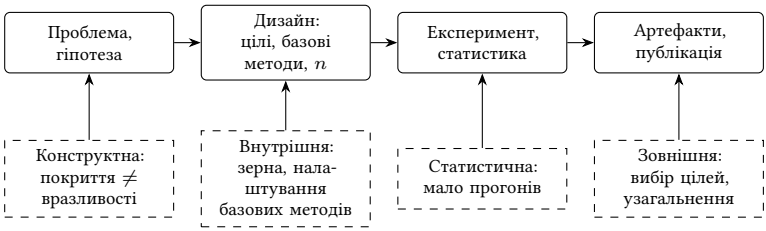


Рис. 17.2 — Етапи експериментального дослідження і загрози валідності (за [346])

Конструктна валідність питає, чи вимірюється потрібне (покриття лише замінює знайдені вразливості), *внутрішня* — чи не пояснюється ефект іншим чинником, *зовнішня* — чи узагальнюються результати, *статистична* — чи досить повторів. Для порівняння двох фазерів критерієм Манна–Уітні загальний обсяг вибірки за Нетером [270] виражається через бажаний розмір ефекту A_{12} (13.5):

$$N = \frac{\left(z_{1-\alpha/2} + z_{1-\beta}\right)^2}{12\,c\,(1-c)\left(A_{12} - \tfrac{1}{2}\right)^2}, \qquad n_A = cN, \quad n_B = (1-c)N, \tag{17.4}$$

де α — рівень значущості, $1-\beta$ — потужність, c — частка прогонів фазера A. *Приклад.* Для $\alpha = 0{,}05$, $1-\beta = 0{,}8$ (z-значення 1,96 і 0,84), $c = 0{,}5$ і великого ефекту $A_{12} = 0{,}71$: $N = 7{,}85/(3 \cdot 0{,}0441) \approx 59{,}3$, тобто по 30 прогонів — як у рекомендації Кліса та співавторів [224]. Для середнього ефекту $A_{12} = 0{,}64$ потрібно $N \approx 134$; десять прогонів на фазер дають потужність 0,8 лише для $A_{12} \geq 0{,}86$, тож «відсутність різниці» в таких статтях малоінформативна.

Артефакти (код, корпуси, конфігурації, зерна, контейнери) публікують з постійним ідентифікатором (розділ 15). Етичні засади досліджень ІКТ (звіт Menlo) для ПЗМБ означають координоване розкриття (розділ 16), експерименти лише на власних стендах або з дозволу власника і мінімізацію персональних даних. Провідні майданчики — USENIX Security, IEEE S&P, ACM CCS, NDSS, ICSE; в Україні — фахові видання, зокрема «Кібербезпека: освіта, наука, техніка» [20].

17.7 Відкриті проблеми і напрями дисертаційних досліджень

Спільна риса відкритих проблем — розрив між методом на еталоні і гарантією для продукту (табл. 17.3).

Таблиця 17.3 — Напрями дисертаційних досліджень у розробленні й тестуванні ПЗМБ

Напрям і проблема	Можливий метод	Очікуваний результат; розділи
Верифікований розбір на лінійній швидкості	генерація парсерів з DSL у C/eBPF/P4 з доведенням еквівалентності	парсер з доведенням без втрати пропускної здатності; 5, 7, 12
Залишковий ризик після фазингу	оцінки з урахуванням адаптивності фазера	калібровані верхні межі ризику; 13
Безперервна сертифікація за CRA і EUCC	машинно-оброблювані кейси, аналіз впливу змін	модель ρ_j (17.2), прототип; 2, 10
Калібрування доказової сили	оцінювання Λ_i за даними PSIRT	емпіричні Λ_i для (17.1); 11, 16
Стійкі ML-IDS	гарантована стійкість за протокольних обмежень	детектор із доведеною межею; 8, 15
Перевірювані LLM-виправлення	LLM разом з BMC і диференційним тестуванням	виправлення з машинно перевіреним свідченням; 12, 16
PQC-міграція граничних пристроїв	гібридні схеми, перевірка сталого часу	профіль міграції для обмежених платформ; 6, 9
Виявлення загроз у шифрованому трафіку (TLS 1.3, ECH, QUIC)	ознаки метаданих і відбитків з оцінюванням на часовому розбитті	детектор без розшифрування з відомою $\mathrm{Prec}(\pi)$; 8, 15
Безпечні площини менеджменту	типізовані API, фазинг зі станом	усунення класу обходу автентифікації; 9, 13
Метрики ланцюга постачання	зв'язок Scorecard, SLSA, libyears з інцидентами	валідовані предиктори ризику; 10
Національні алгоритми у верифікованих бібліотеках	синтез і доведення реалізацій ДСТУ 7624, 4145, 8845	верифікована бібліотека КЗІ; 6, 12

17.8 Приклад: кейс гарантування для парсера IKEv2

Завершимо наскрізний кейс розділів 11–13 і 16: виробник VPN-шлюзу (підрозділ 1.9) будує кейс гарантування для модуля розбору IKEv2 ($\approx$9 KLOC на C) як частину матеріалів для оцінювання за EUCC з AVA_VAN.3 і технічної документації CRA (рис. 17.3).

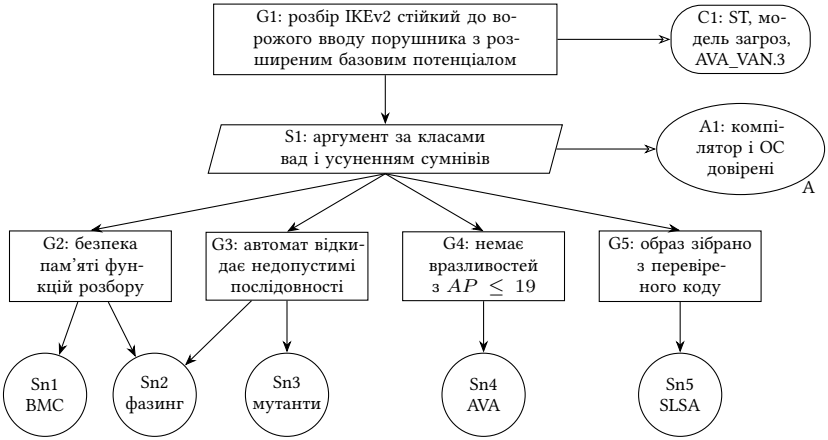


Рис. 17.3 — Фрагмент кейсу гарантування для парсера IKEv2 у нотації GSN (за [182])

Свідчення. Sn1 — обв'язки CBMC для функцій розбору (розділ 12); Sn2 — фазинг трьома обв'язками з $\hat{p}_{\text{new}} \approx 2{,}7 \cdot 10^{-9}$ для обв'язки навантажень і шістьма виправленими дефектами (розділ 13); Sn3 — $MS_{\text{без}} \geq 0{,}9$ для парсера й автомата (розділ 11); Sn4 — аналіз вразливостей AVA_VAN і тест на проникнення з оцінками атакувального потенціалу (11.4); Sn5 — атестації SLSA, що пов'язують Sn1–Sn4 з гешем образу. Усі свідчення перевиробляються за циклом рис. 17.1.

Кількісна впевненість. За (17.1) з апріорною $P(G1) = 0{,}3$ (історія вразливостей граничних пристроїв, розділ 1) і Λ: BMC — 4, фазинг — 3, мутанти — 1,5, тест на проникнення — 1,3 — за незалежності $O \approx 10{,}0$, $P \approx 0{,}91$. Проте BMC і фазинг спираються на спільний контекст обв'язки, тож консервативно їхню спільну силу взято 8 замість 12: $O \approx 6{,}7$, $P \approx 0{,}87$ — нижче цільових 0,9. Додано *різнорідне* свідчення — диференційне тестування розбору зі strongSwan ($\Lambda = 1{,}5$): $O \approx 10{,}0$, $P \approx 0{,}91$. Висновок найчутливіший до Λ фазингу; її калібрують за даними PSIRT.

Елімінативна оцінка. З 14 сумнівів усунуто 11 ($11|14$). Залишкові — 13 % непокритих гілок у рідкісних режимах EAP, вимкнена в збиранні для фазингу перевірка ICV і довіра до компілятора (A1) — внесено до звіту як обмеження і до плану тестування наступного випуску.

Висновки до розділу

Кейс гарантування перетворює розрізнені результати перевірок на структуровану аргументацію в нотаціях GSN чи CAE. Підготовка до CC/EUCC, державної експертизи й CRA ефективна, коли артефакти є побічним продуктом процесу, а безперервне гарантування узгоджує сертифікацію з ча-

стими оновленнями. ШІ підвищує продуктивність, але згенерований код, галюциновані залежності й агенти з надмірними правами створюють ризики, які усувають ворота конвеєра; LLM-виправлення потребують незалежної перевірки. Дослідження вимагають фальсифіковних гіпотез, обґрунтованої кількості повторів, аналізу загроз валідності й відкритих артефактів.

Загальний підсумок курсу. Безпека ПЗМБ — не властивість окремого засобу, а результат інженерного процесу, у якому вимоги, архітектура, мова реалізації, конвеєр, різнорідна верифікація і супровід породжують свідчення, а кейс гарантування зводить їх у перевірюване обґрунтування довіри. Засіб захисту на межі довіри має бути перевірений щонайменше так само ретельно, як системи, які він захищає; методи, що роблять таку перевірку дешевшою, повнішою й доказовою, — головне завдання досліджень у галузі.

Питання для самоперевірки

1. Що таке кейс гарантування? Назвіть його складники за ISO/IEC/IEEE 15026-2.
2. Порівняйте нотації GSN і CAE. Які блоки аргументації пропонує CAE?
3. Поясніть оновлення впевненості (17.1). Чому припущення про незалежність свідчень критичне?
4. Чим бейконівська впевненість відрізняється від імовірнісної? Що таке сумнів (defeater)?
5. Як артефакти процесу відображаються на класи ADV, ALC, ATE, AVA і на вимоги держекспертизи (табл. 17.1)?
6. Що таке безперервне гарантування? Коли інкрементальне підтримання сертифіката вигідне (17.2)?
7. Чому метрики свідчень піддаються маніпуляції? Навіщо прив'язувати свідчення до версії артефакту?
8. Що встановили Пірс та співавтори і Перрі та співавтори щодо коду з ШІ-асистентами?
9. Що таке slopsquatting? Обчисліть P_h (17.3) для $h = 0{,}1$, $m = 5$.
10. Які ризики OWASP Top 10 for LLM 2025 найважливіші для агентів у конвеєрі і які правила їх обмежують?
11. Які результати показав фінал AIxCC і чому LLM-виправлення потребують незалежної перевірки?
12. Скільки прогонів потрібно для ефекту $A_{12} = 0{,}64$ за (17.4)? Назвіть загрози валідності порівняння фазерів.

13. Дослідницьке: розробіть метод калібрування доказової сили Λ_i свідчень (BMC, фазинг, тест на проникнення) за даними PSIRT і відкритих баз вразливостей; оцініть вплив залежності свідчень на підсумкову впевненість.
14. Дослідницьке: оберіть напрям з табл. 17.3, сформулюйте фальсифіковну гіпотезу, експериментальний дизайн з обґрунтуванням кількості прогонів, аналіз загроз валідності й план публікації артефактів.

Список використаних джерел

[1] Бурячок В. Л., Шевченко С. М., Складанний П. М. Віртуальна лабораторія для моделювання процесів в інформаційній та кібербезпеці як засіб формування практичних навичок студентів. *Кібербезпека: освіта, наука, техніка.* 2018. № 2(2). С. 98–104. DOI: 10.28925/2663-4023.2018.2.98104.

[2] Гавриляк В. Р., Опірський І. Р. Аналіз використання штучного інтелекту в DevSecOps та кібербезпеці. *Сучасний захист інформації.* 2024. № 4(60). DOI: 10.31673/2409-7292.2024.040008.

[3] Гайдур Г., Власенко В., Петрова О. Захист великих мовних моделей: ризики, загрози та підходи до безпеки. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 1, № 29. С. 664–675. DOI: 10.28925/2663-4023.2025.29.918. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/918.

[4] Гнатюк С., Побережна З., Скуратівський А. Метод інтегрування вимог кібербезпеки в життєвий цикл розроблення програмного забезпечення. *Кібербезпека: освіта, наука, техніка.* 2026. Т. 4, № 32. С. 947–962. DOI: 10.28925/2663-4023.2026.32.1184. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1184.

[5] Гнатюк С., Сидоренко В., Скуратівський А. Модель управління вимогами кібербезпеки при впровадженні програмного забезпечення. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 4, № 28. С. 715–726. DOI: 10.28925/2663-4023.2025.28.841. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/841.

[6] Горбенко І. Д., Горбенко Ю. І. Прикладна криптологія: теорія, практика, застосування: монографія. Харків: Форт, 2012. 878 с.

[7] Городицький О., Опірський І. Покроковий підхід до імплементації Zero Trust у гібридних архітектурах корпоративної безпеки. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 3, № 31. С. 346–366. DOI: 10.28925/2663-4023.2025.31.1029. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1029.

[8] Гречко В., Бабенко Т., Мирутенко Л. Безпечне програмне забезпечення, що розробляє рекомендації. *Кібербезпека: освіта, наука, техніка.* 2019. Т. 2, № 6. С. 82–93. DOI: 10.28925/2663-4023.2019.6.8293. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/113.

[9] Денисюк В. Прогнозування експлуатації CVE на основі відкритих даних NVD та KEV для ризико-орієнтованої пріоритизації. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 2, № 30. С. 428–444. DOI: 10.28925/2663-4023.2025.30.906.

[10] Державна служба спеціального зв'язку та захисту інформації України: офіційний вебсайт. URL: https://cip.gov.ua/.

[11] Деякі питання захисту інформаційних, електронних комунікаційних, інформаційно-комунікаційних, технологічних систем: поста-

нова Кабінету Міністрів України від 18.06.2025 № 712. URL: https://zakon.rada.gov.ua/laws/show/712-2025-п.

[12] Деякі питання пошуку та виявлення потенційних вразливостей в інформаційно-комунікаційних системах: постанова Кабінету Міністрів України від 03.12.2025 № 1580. URL: https://zakon.rada.gov.ua/laws/show/1580-2025-п.

[13] ДСТУ 4145-2002. Інформаційні технології. Криптографічний захист інформації. Цифровий підпис, що ґрунтується на еліптичних кривих. Формування та перевіряння. Київ: Держспоживстандарт України, 2003.

[14] ДСТУ 7624:2014. Інформаційні технології. Криптографічний захист інформації. Алгоритм симетричного блокового перетворення. Київ: Мінекономрозвитку України, 2015.

[15] ДСТУ 7564:2014. Інформаційні технології. Криптографічний захист інформації. Функція гешування. Київ: Мінекономрозвитку України, 2015.

[16] ДСТУ 8845:2019. Інформаційні технології. Криптографічний захист інформації. Алгоритм симетричного потокового перетворення. Київ: ДП «УкрНДНЦ», 2019.

[17] Журавчак А., Піскозуб А. Аналіз методів машинного навчання для автоматизації тестування на проникнення. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 3, № 27. С. 54–62. DOI: 10.28925/2663-4023.2025.27.711. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/711.

[18] Загальні вимоги до кіберзахисту об'єктів критичної інфраструктури: затв. постановою Кабінету Міністрів України від 19.06.2019 № 518. URL: https://zakon.rada.gov.ua/laws/show/518-2019-п.

[19] Ільєнко А., Ільєнко С., Прокопенко О., Кравчук І. Практичні підходи організації захищеної передачі даних по протоколу TLS засобами OpenSSL. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 2, № 22. С. 122–133. DOI: 10.28925/2663-4023.2023.22.122133. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/532.

[20] Кібербезпека: освіта, наука, техніка: науковий журнал / Київський столичний університет імені Бориса Грінченка. URL: https://csecurity.kubg.edu.ua/.

[21] Кіпчук Ф., Соколов В. Модель розрахунку витрат на баг-баунті програми тестування вразливостей безпеки. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 2, № 22. С. 68–83. DOI: 10.28925/2663-4023.2023.22.6883. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/531.

[22] Костюк Ю., Складанний П., Мазур Н., Рзаєва С., Гнатченко Д., Гончаренко І. Формальна модель адаптивного вибору криптографічних параметрів захисту каналів у корпоративних комп'ютерних мережах на основі динамічної оцінки довіри. *Кібербезпека: освіта, наука, техніка.* 2026. Т. 4, № 32. С. 20–44. DOI: 10.28925/2663-4023.2026.32.1111. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1111.

[23] Котух Є., Марухненко О., Халімов Г., Коробчинський М. Розробка методики випробувань бібліотеки криптографічних перетворювань на прикла-

ді криптосистеми MST3 на основі узагальнених Сузукі 2-груп. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 2, № 22. С. 113–121. DOI: 10.28925/2663-4023.2023.22.113121.

[24] Лактіонов І., Кміть А., Опірський І., Гарасимчук О. Дослідження інструментів захисту інтернет-ресурсів від DDoS-атак під час кібервійни. *Кібербезпека: освіта, наука, техніка.* 2022. Т. 1, № 17. С. 91–111. DOI: 10.28925/2663-4023.2022.17.91111. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/387.

[25] Лаптєв О., Гапон А., Ткачов А. Метод захисту програмного забезпечення на основі гібридного аналізу коду. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 1, № 29. С. 139–151. DOI: 10.28925/2663-4023.2025.29.871. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/871.

[26] Лещенко Б. С. Комплексна модель інтеграції безпеки у життєвий цикл розробки для хмарних середовищ. *Сучасний захист інформації.* 2025. № 2(62). DOI: 10.31673/2409-7292.2025.024887.

[27] Ляшкевич М. Ю., Ляшкевич В. Я., Шувар Р. Я. Безпека та інші ризики, пов'язані з розробкою програмного забезпечення на основі великих мовних моделей. *Український журнал інформаційних технологій.* 2025. Т. 7, № 1. С. 86–96. DOI: 10.23939/ujit2025.01.086.

[28] НД ТЗІ 1.1-002-99. Загальні положення щодо захисту інформації в комп'ютерних системах від несанкціонованого доступу. Київ: ДСТСЗІ СБ України, 1999. URL: https://cip.gov.ua/ua/news/normativni-dokumenti-z-pitan-tzi.

[29] НД ТЗІ 2.5-004-99. Критерії оцінки захищеності інформації в комп'ютерних системах від несанкціонованого доступу. Київ: ДСТСЗІ СБ України, 1999.

[30] Олексійчук Ю. Емпіричне дослідження продуктивності постквантових криптографічних алгоритмів ML-KEM, ML-DSA та SLH-DSA у різних версіях JDK. *Кібербезпека: освіта, наука, техніка.* 2026. Т. 1, № 33. С. 165–176. DOI: 10.28925/2663-4023.2026.33.1210. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1210.

[31] Особливості деструктивних кібератак Sandworm у відношенні українських провайдерів (CERT-UA#7627). *CERT-UA.* 2023. URL: https://cert.gov.ua/article/6123309.

[32] Пархоменко І., Савонік М. Запобігання загрозам хибних конфігурацій AWS інфраструктури як коду на основі тестування. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 1, № 29. С. 236–251. DOI: 10.28925/2663-4023.2025.29.887. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/887.

[33] Плєхов О., Гринкевич Г., Василенко В. Розширена модель загроз STRIDE для офлайн-застосунків аварійного зв'язку на основі BLE mesh-мереж. *Кібербезпека: освіта, наука, техніка.* 2026. Т. 2, № 34. С. 726–743. DOI: 10.28925/2663-4023.2026.34.1368. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1368.

[34] Положення про державну експертизу в сфері криптографічного захисту інформації: затв. наказом Адміністрації Державної служби спеціального зв'язку та захисту інформації України від 23.06.2008 № 100 (у редакції наказу від 24.04.2026 № 302). Зареєстровано в Міністерстві юстиції України 16.07.2008 за № 651/15342. URL: https://zakon.rada.gov.ua/laws/show/z0651-08.

[35] Полотай О., Брич Т., Кухарська Н., Ящук В., Ткаченко А. Інтеграція систем виявлення вторгнень у структуру корпоративної мережі: підходи, виклики та ефективність реагування на інциденти. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 1, № 29. С. 405–418. DOI: 10.28925/2663-4023.2025.29.889. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/889.

[36] Поночовний П. Модель упередження низькошвидкісних HTTP DDoS атак на кінцевого користувача. *Кібербезпека: освіта, наука, техніка.* 2024. Т. 2, № 26. С. 291–304. DOI: 10.28925/2663-4023.2024.26.695. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/695.

[37] Правила забезпечення захисту інформації в інформаційних, електронних комунікаційних та інформаційно-комунікаційних системах: затв. постановою Кабінету Міністрів України від 29.03.2006 № 373. URL: https://zakon.rada.gov.ua/laws/show/373-2006-п.

[38] Про внесення змін до деяких законів України щодо захисту інформації та кіберзахисту державних інформаційних ресурсів, об'єктів критичної інформаційної інфраструктури: Закон України від 27.03.2025 № 4336-IX. URL: https://zakon.rada.gov.ua/laws/show/4336-20.

[39] Про внесення змін до постанови Кабінету Міністрів України від 29 березня 2006 р. № 373: постанова Кабінету Міністрів України від 26.11.2025 № 1531. URL: https://zakon.rada.gov.ua/laws/show/1531-2025-п.

[40] Про затвердження Положення про державну експертизу у сфері технічного захисту інформації: наказ Адміністрації Державної служби спеціального зв'язку та захисту інформації України від 16.05.2007 № 93 (зі змінами, внесеними наказом від 17.04.2026 № 287). Зареєстровано в Міністерстві юстиції України 16.07.2007 за № 820/14087. URL: https://zakon.rada.gov.ua/laws/show/z0820-07.

[41] Про затвердження Технічного регламенту засобів криптографічного захисту інформації: постанова Кабінету Міністрів України від 21.10.2020 № 991. URL: https://zakon.rada.gov.ua/laws/show/991-2020-п.

[42] Про захист інформації в інформаційно-комунікаційних системах: Закон України від 05.07.1994 № 80/94-ВР. URL: https://zakon.rada.gov.ua/laws/show/80/94-вр.

[43] Про захист персональних даних: Закон України від 01.06.2010 № 2297-VI. URL: https://zakon.rada.gov.ua/laws/show/2297-17.

[44] Про критичну інфраструктуру: Закон України від 16.11.2021 № 1882-IX. URL: https://zakon.rada.gov.ua/laws/show/1882-20.

[45] Про основні засади забезпечення кібербезпеки України: Закон України від 05.10.2017 № 2163-VIII (редакція від 19.10.2025). URL: https://zakon.rada.gov.ua/laws/show/2163-19.

[46] Про технічні регламенти та оцінку відповідності: Закон України від 15.01.2015 № 124-VIII. URL: https://zakon.rada.gov.ua/laws/show/124-19.

[47] Сиротинський Р. Контекстно орієнтований підхід організації мережевих політик безпеки в архітектурі нульової довіри. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 3, № 31. С. 198–216. DOI: 10.28925/2663-4023.2025.31.1013. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1013.

[48] Сиротинський Р., Тишик І. Життєвий цикл доступу до мережі: структурований підхід до управління правилами брандмауера в середовищі нульової довіри. *Кібербезпека: освіта, наука, техніка.* 2026. Т. 4, № 32. С. 419–430. DOI: 10.28925/2663-4023.2026.32.1194. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1194.

[49] Скіп А., Баришев Ю. Аналіз безпеки ланцюгів постачання програмного забезпечення. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 1, № 29. С. 263–273. DOI: 10.28925/2663-4023.2025.29.820. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/820.

[50] Совин Я., Хома В. Програмна bitsliced-імплементація шифру «Калина», орієнтована на використання SIMD-інструкцій мікропроцесорів з архітектурою x86-64. *Кібербезпека: освіта, наука, техніка.* 2020. Т. 3, № 7. С. 131–152. DOI: 10.28925/2663-4023.2020.7.131152. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/140.

[51] Спасітєлєва С., Чичкань І., Шевченко С., Жданова Ю. Розробка безпечних контейнерних застосунків з мікросервісною архітектурою. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 1, № 21. С. 193–210. DOI: 10.28925/2663-4023.2023.21.193210.

[52] Спис Д., Ільєнко А. Систематизація та класифікація сучасних методів статичного аналізу вебдодатків. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 2, № 30. С. 157–179. DOI: 10.28925/2663-4023.2025.30.957. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/957.

[53] Тишик І. Вибір технології віддаленого доступу для ефективної організації захисту мережевих з'єднань. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 3, № 19. С. 34–45. DOI: 10.28925/2663-4023.2023.19.3445. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/437.

[54] Толкачова А., Піскозуб А. Методи для тестування безпеки веб-застосунків. *Кібербезпека: освіта, наука, техніка.* 2024. Т. 2, № 26. С. 115–122. DOI: 10.28925/2663-4023.2024.26.668.

[55] Трофименко О., Дика А., Лобода Ю. Аналіз інструментів тестування вебзастосунків. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 4, № 20. С. 62–71. DOI: 10.28925/2663-4023.2023.20.6271. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/467.

[56] Хома В., Партика А., Сабодашко Д. Систематизація та порівняльний аналіз інструментів автоматизованого тестування на проникнення веб-додатків. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 3, № 31. С. 717–726. DOI: 10.28925/2663-4023.2025.31.1068.

[57] Чичкарьов Є., Зінченко О., Бондарчук А., Асєєва Л. Метод вибору ознак для системи виявлення вторгнень з використанням ансамблевого підходу та нечіткої логіки. *Кібербезпека: освіта, наука, техніка.* 2023. № 1(21). С. 234–251. DOI: 10.28925/2663-4023.2023.21.234251.

[58] Шляхова А., Шевчук О., Оніщенко В. Виявлення кібератак у мережевому трафіку на основі алгоритмів машинного навчання. *Кібербезпека: освіта, наука, техніка.* 2026. Т. 1, № 33. С. 301–312. DOI: 10.28925/2663-4023.2026.33.1158. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1158.

[59] Шпак М., Курченко О., Щебланін Ю. Зменшення рівня вразливості garbage collection за допомогою arena allocator. *Кібербезпека: освіта, наука, техніка.* 2026. Т. 4, № 32. С. 550–564. DOI: 10.28925/2663-4023.2026.32.1096. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1096.

[60] Юдіна Д. Метод розрахунку рівня кіберзахисту об'єктів критичної інформаційної інфраструктури. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 2, № 30. С. 473–487. DOI: 10.28925/2663-4023.2025.30.986. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/986.

[61] Ярема О., Загородна Н. Порівняльний аналіз ефективності виявлення вразливостей програмного коду з допомогою LLM та SAST. *Кібербезпека: освіта, наука, техніка.* 2026. Т. 4, № 32. С. 878–891. DOI: 10.28925/2663-4023.2026.32.1165. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1165.

[62] Яскевич В., Клочко О. Методи підвищення відмовостійкості інтернет-сервісів. *Кібербезпека: освіта, наука, техніка.* 2019. № 3. С. 104–111. DOI: 10.28925/2663-4023.2019.3.104111. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/67.

[63] Acar Y., Backes M., Fahl S., Garfinkel S., Kim D., Mazurek M. L., Stransky C. Comparing the Usability of Cryptographic APIs. *2017 IEEE Symposium on Security and Privacy (SP).* San Jose: IEEE, 2017. P. 154–171. DOI: 10.1109/SP.2017.52.

[64] Aho A. V., Corasick M. J. Efficient String Matching: An Aid to Bibliographic Search. *Communications of the ACM.* 1975. Vol. 18, no. 6. P. 333–340. DOI: 10.1145/360825.360855.

[65] Al Fardan N. J., Paterson K. G. Lucky Thirteen: Breaking the TLS and DTLS Record Protocols. *2013 IEEE Symposium on Security and Privacy (SP).* Berkeley: IEEE, 2013. P. 526–540. DOI: 10.1109/SP.2013.42.

[66] Al-Shaer E. S., Hamed H. H. Modeling and Management of Firewall Policies. *IEEE Transactions on Network and Service Management.* 2004. Vol. 1, no. 1. P. 2–10. DOI: 10.1109/TNSM.2004.4623689.

[67] Alexopoulos N., Brack M., Wagner J. P., Grube T., Mühlhäuser M. How Long Do Vulnerabilities Live in the Code? A Large-Scale Empirical Measurement Study on

FOSS Vulnerability Lifetimes. *Proceedings of the 31st USENIX Security Symposium*. Boston: USENIX Association, 2022. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/alexopoulos.

[68] Alhazmi O. H., Malaiya Y. K. Application of Vulnerability Discovery Models to Major Operating Systems. *IEEE Transactions on Reliability*. 2008. Vol. 57, No. 1. P. 14–22. DOI: 10.1109/TR.2008.916872.

[69] Alhazmi O. H., Malaiya Y. K. Modeling the Vulnerability Discovery Process. *16th IEEE International Symposium on Software Reliability Engineering (ISSRE '05)*. Chicago: IEEE, 2005. P. 129–138. DOI: 10.1109/ISSRE.2005.30.

[70] Almeida J. B., Barbosa M., Barthe G., Dupressoir F., Emmi M. Verifying Constant-Time Implementations. *25th USENIX Security Symposium (USENIX Security 16)*. Austin: USENIX Association, 2016. P. 53–70. URL: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/almeida.

[71] Amdahl G. M. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. *Proceedings of the AFIPS Spring Joint Computer Conference*. New York: ACM, 1967. P. 483–485. DOI: 10.1145/1465482.1465560.

[72] Ammann P., Offutt J. Introduction to Software Testing. 2nd ed. Cambridge: Cambridge University Press, 2016. 364 p. DOI: 10.1017/9781316771273.

[73] Anderson B., McGrew D. Identifying Encrypted Malware Traffic with Contextual Flow Data. *Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security (AISec '16)*. New York: ACM, 2016. P. 35–46. DOI: 10.1145/2996758.2996768.

[74] Anderson R. Security Engineering: A Guide to Building Dependable Distributed Systems. 3rd ed. Indianapolis: Wiley, 2020. 1232 p. URL: https://www.cl.cam.ac.uk/~rja14/book.html.

[75] Apruzzese G., Andreolini M., Ferretti L., Marchetti M., Colajanni M. Modeling Realistic Adversarial Attacks against Network Intrusion Detection Systems. *Digital Threats: Research and Practice*. 2022. Vol. 3, no. 3. P. 1–19. DOI: 10.1145/3469659.

[76] Arora A., Telang R., Xu H. Optimal Policy for Software Vulnerability Disclosure. *Management Science*. 2008. Vol. 54, No. 4. P. 642–656. DOI: 10.1287/mnsc.1070.0771.

[77] Arp D., Quiring E., Pendlebury F., Warnecke A., Pierazzi F., Wressnegger C., Cavallaro L., Rieck K. Dos and Don'ts of Machine Learning in Computer Security. *Proceedings of the 31st USENIX Security Symposium*. Boston: USENIX Association, 2022. P. 3971–3988. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/arp.

[78] Arrangement on the Recognition of Common Criteria Certificates in the field of IT Security. 02.07.2014. URL: https://www.commoncriteriaportal.org/ccra/.

[79] Atlidakis V., Godefroid P., Polishchuk M. RESTler: Stateful REST API Fuzzing. *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. Montreal: IEEE, 2019. P. 748–758. DOI: 10.1109/ICSE.2019.00083.

[80] Aumasson J.-P., Bernstein D. J. SipHash: A Fast Short-Input PRF. *Progress in Cryptology — INDOCRYPT 2012*. Lecture Notes in Computer Science, vol. 7668. Berlin; Heidelberg: Springer, 2012. P. 489–508. DOI: 10.1007/978-3-642-34931-7_28.

[81] Axelsson S. The Base-Rate Fallacy and the Difficulty of Intrusion Detection. *ACM Transactions on Information and System Security*. 2000. Vol. 3, no. 3. P. 186–205. DOI: 10.1145/357830.357849.

[82] Balarajah B., Rossenhoevel C., Monkman B. Benchmarking Methodology for Network Security Device Performance. RFC 9411. IETF, 2023. DOI: 10.17487/RFC9411. URL: https://www.rfc-editor.org/rfc/rfc9411.

[83] Barker E., Dang Q., Frankel S., Scarfone K., Wouters P. Guide to IPsec VPNs. NIST SP 800-77, Rev. 1. Gaithersburg, MD, 2020. DOI: 10.6028/NIST.SP.800-77r1. URL: https://csrc.nist.gov/pubs/sp/800/77/r1/final.

[84] Barr E. T., Harman M., McMinn P., Shahbaz M., Yoo S. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*. 2015. Vol. 41, no. 5. P. 507–525. DOI: 10.1109/TSE.2014.2372785.

[85] Basin D., Dreier J., Hirschi L., Radomirović S., Sasse R., Stettler V. A Formal Analysis of 5G Authentication. *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*. New York: ACM, 2018. P. 1383–1396. DOI: 10.1145/3243734.3243846.

[86] Basiri A., Behnam N., de Rooij R., Hochstein L., Kosewski L., Reynolds J., Rosenthal C. Chaos Engineering. *IEEE Software*. 2016. Vol. 33, no. 3. P. 35–41. DOI: 10.1109/MS.2016.60.

[87] Beurdouche B., Bhargavan K., Delignat-Lavaud A., Fournet C., Kohlweiss M., Pironti A., Strub P.-Y., Zinzindohoue J. K. A Messy State of the Union: Taming the Composite State Machines of TLS. *2015 IEEE Symposium on Security and Privacy (SP)*. San Jose: IEEE, 2015. P. 535–552. DOI: 10.1109/SP.2015.39.

[88] Black Duck. Building Security In Maturity Model (BSIMM16): Report. Burlington, MA: Black Duck Software, 2026. URL: https://www.blackduck.com/resources/analyst-reports/bsimm.html.

[89] Black P. E., Guttman B., Okun V. Guidelines on Minimum Standards for Developer Verification of Software. NISTIR 8397. Gaithersburg, MD, 2021. DOI: 10.6028/NIST.IR.8397. URL: https://csrc.nist.gov/pubs/ir/8397/final.

[90] Blanchet B. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*. 2016. Vol. 1, no. 1–2. P. 1–135. DOI: 10.1561/3300000004.

[91] Bloomfield R., Netkachova K. Building Blocks for Assurance Cases. *2014 IEEE International Symposium on Software Reliability Engineering Workshops*. Naples: IEEE, 2014. P. 186–191. DOI: 10.1109/ISSREW.2014.72.

[92] Boehm B., Basili V. R. Software Defect Reduction Top 10 List. *Computer*. 2001. Vol. 34, No. 1. P. 135–137. DOI: 10.1109/2.962984.

[93] Booth H., Souppaya M., Vassilev A., Ogata M., Stanley M., Scarfone K. Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile. NIST SP 800-218A. Gaithersburg, MD, 2024. DOI: 10.6028/NIST.SP.800-218A. URL: https://csrc.nist.gov/pubs/sp/800/218/a/final.

[94] Bossavit L. The Leprechauns of Software Engineering: How Folklore Turns into Fact and What to Do About It. Leanpub, 2015 (rev. 2021). URL: https://leanpub.com/leprechauns.

[95] Bosshart P., Daly D., Gibb G., Izzard M., McKeown N., Rexford J., Schlesinger C., Talayco D., Vahdat A., Varghese G., Walker D. P4: Programming Protocol-Independent Packet Processors. *ACM SIGCOMM Computer Communication Review*. 2014. Vol. 44, no. 3. P. 87–95. DOI: 10.1145/2656877.2656890.

[96] Boyens J., Smith A., Bartol N., Winkler K., Holbrook A., Fallon M. Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations. NIST SP 800-161, Rev. 1. Gaithersburg, MD, 2022. DOI: 10.6028/NIST.SP.800-161r1. URL: https://csrc.nist.gov/pubs/sp/800/161/r1/final.

[97] Bradner S., Dubray K., McQuaid J., Morton A. Applicability Statement for RFC 2544: Use on Production Networks Considered Harmful. RFC 6815. IETF, 2012. DOI: 10.17487/RFC6815. URL: https://www.rfc-editor.org/rfc/rfc6815.

[98] Bradner S., McQuaid J. Benchmarking Methodology for Network Interconnect Devices. RFC 2544. IETF, 1999. DOI: 10.17487/RFC2544. URL: https://www.rfc-editor.org/rfc/rfc2544.

[99] Braiterman Z., Shostack A., Marcil J., de Vries S., Michlin I., Wuyts K., Hurlbut R., Schoenfield B. S. E., Scott F., Coles M., Romeo C., Miller A., Tarandach I., Douglen A., French M. Threat Modeling Manifesto. 2020. URL: https://www.threatmodelingmanifesto.org/.

[100] Brumfield C. CVE program funding secured, easing fears of repeat crisis. *CSO Online*. 09.03.2026. URL: https://www.csoonline.com/article/4142600/cve-program-funding-secured-easing-fears-of-repeat-crisis.html.

[101] Böhme M. STADS: Software Testing as Species Discovery. *ACM Transactions on Software Engineering and Methodology*. 2018. Vol. 27, no. 2. P. 1–52. DOI: 10.1145/3210309.

[102] Böhme M., Falk B. Fuzzing: On the Exponential Cost of Vulnerability Discovery. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. New York: ACM, 2020. P. 713–724. DOI: 10.1145/3368089.3409729.

[103] Böhme M., Pham V.-T., Roychoudhury A. Coverage-based Greybox Fuzzing as Markov Chain. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. New York: ACM, 2016. P. 1032–1043. DOI: 10.1145/2976749.2978428.

[104] Cadar C., Dunbar D., Engler D. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI '08)*. San Diego: USENIX Association, 2008. P. 209–224. URL: https://www.usenix.org/legacy/events/osdi08/tech/full_papers/cadar/cadar.pdf.

[105] Canadian Centre for Cyber Security et al. Security Considerations for Edge Devices. Joint guidance, February 2025. URL: https://media.defense.gov/2025/Feb/03/2003636950/-1/-1/1/SECURITY-CONSIDERATIONS-FOR-EDGE-DEVICES.PDF.

[106] CERT-UA. Кібератака групи Sandworm (UAC-0082) на об'єкти енергетики України з використанням шкідливих програм INDUSTROYER2 та CADDYWIPER (CERT-UA#4435). 12.04.2022. URL: https://cert.gov.ua/article/39518.

[107] CERT-UA: Урядова команда реагування на комп'ютерні надзвичайні події України. URL: https://cert.gov.ua/.

[108] Charrier C., Sadowski J., Lecigne C., Stolyarov V. Hello 0-Days, My Old Friend: A 2024 Zero-Day Exploitation Analysis. *Google Threat Intelligence Group.* 29.04.2025. URL: https://cloud.google.com/blog/topics/threat-intelligence/2024-zero-day-trends/.

[109] Charrier C., Weiner R. How Low Can You Go? An Analysis of 2023 Time-to-Exploit Trends. Mandiant (Google Cloud). 15.10.2024. URL: https://cloud.google.com/blog/topics/threat-intelligence/time-to-exploit-trends-2023.

[110] Chen T. Y., Kuo F.-C., Liu H., Poon P.-L., Towey D., Tse T. H., Zhou Z. Q. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Computing Surveys.* 2018. Vol. 51, no. 1. Art. 4. P. 1–27. DOI: 10.1145/3143561.

[111] Chiara P. G. The Cyber Resilience Act: the EU Commission's proposal for a horizontal regulation on cybersecurity for products with digital elements. An introduction. *International Cybersecurity Law Review.* 2022. Vol. 3, No. 2. P. 255–272. DOI: 10.1365/s43439-022-00067-6.

[112] Chifflier P., Couprie G. Writing Parsers Like it is 2017. *2017 IEEE Security and Privacy Workshops (SPW).* San Jose: IEEE, 2017. P. 80–92. DOI: 10.1109/SPW.2017.39.

[113] Chong N., Cook B., Kallas K., Khazem K., Monteiro F. R., Schwartz-Narbonne D., Tasiran S., Tautschnig M., Tuttle M. R. Code-Level Model Checking in the Software Development Workflow. *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '20).* New York: ACM, 2020. P. 11–20. DOI: 10.1145/3377813.3381347.

[114] CISA et al. Secure by Design: Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Secure by Design Software. Washington: CISA, 2023. URL: https://www.cisa.gov/securebydesign.

[115] CISA, FBI, ASD's ACSC, NCSC-UK et al. Threat Actors Exploit Multiple Vulnerabilities in Ivanti Connect Secure and Policy Secure Gateways. Cybersecurity Advisory AA24-060B. 29.02.2024. URL: https://www.cisa.gov/news-events/cybersecurity-advisories/aa24-060b.

[116] CISA, FBI, MS-ISAC, ASD's ACSC. #StopRansomware: LockBit 3.0 Ransomware Affiliates Exploit CVE 2023-4966 Citrix Bleed Vulnerability. Cybersecurity Advisory AA23-325A. 21.11.2023. URL: https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-325a.

[117] CISA, FBI. Product Security Bad Practices. Version 2. Washington: CISA, 17.01.2025. URL: https://www.cisa.gov/resources-tools/resources/product-security-bad-practices.

[118] CISA, NSA, FBI et al. 2026 Minimum Elements for a Software Bill of Materials (SBOM). Washington: CISA, 29.07.2026. URL: https://www.cisa.gov/sites/default/files/2026-07/2026_cisa_sbom_minimum_elements_508c.pdf.

[119] CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ. The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously. Joint guidance. 06.12.2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[120] CISA. 2025 Minimum Elements for a Software Bill of Materials (SBOM). Washington: CISA, 2025. URL: https://www.cisa.gov/resources-tools/resources/2025-minimum-elements-software-bill-materials-sbom.

[121] CISA. Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations. Cybersecurity Advisory AA20-352A. 17.12.2020 (rev. 15.04.2021). URL: https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a.

[122] CISA. ED 24-01: Mitigate Ivanti Connect Secure and Ivanti Policy Secure Vulnerabilities. Emergency Directive. 19.01.2024. URL: https://www.cisa.gov/news-events/directives/ed-24-01-mitigate-ivanti-connect-secure-and-ivanti-policy-secure-vulnerabilities.

[123] CISA. Known Exploited Vulnerabilities Catalog. Cybersecurity and Infrastructure Security Agency. URL: https://www.cisa.gov/known-exploited-vulnerabilities-catalog.

[124] CISA. Stakeholder-Specific Vulnerability Categorization Guide. Washington: CISA, November 2022. URL: https://www.cisa.gov/resources-tools/resources/stakeholder-specific-vulnerability-categorization-ssvc.

[125] Cisco Talos. ArcaneDoor — New espionage-focused campaign found targeting perimeter network devices. 24.04.2024. URL: https://blog.talosintelligence.com/arcanedoor-new-espionage-focused-campaign-found-targeting-perimeter-network-devices/.

[126] Clang Documentation: AddressSanitizer, MemorySanitizer, ThreadSanitizer, UndefinedBehaviorSanitizer, libFuzzer. LLVM Project. URL: https://clang.llvm.org/docs/.

[127] Cohen D. M., Dalal S. R., Fredman M. L., Patton G. C. The AETG System: An Approach to Testing Based on Combinatorial Design. *IEEE Transactions on Software Engineering*. 1997. Vol. 23, no. 7. P. 437–444. DOI: 10.1109/32.605761.

[128] collaborative Protection Profile for Network Devices. Version 4.0. Network Device iTC, 25.11.2025. URL: https://www.commoncriteriaportal.org/pps/.

[129] Commission Delegated Regulation (EU) 2022/30 of 29 October 2021 supplementing Directive 2014/53/EU with regard to the application of the essential requirements

referred to in Article 3(3), points (d), (e) and (f), of that Directive. *Official Journal of the EU*. L 7. P. 6–10. URL: https://eur-lex.europa.eu/eli/reg_del/2022/30/oj.

[130] Commission Implementing Decision (EU) 2025/138 of 28 January 2025 amending Implementing Decision (EU) 2022/2191 as regards harmonised standards in support of the essential requirements of Directive 2014/53/EU that relate to cybersecurity. *Official Journal of the EU*. L series, 30.01.2025. URL: https://eur-lex.europa.eu/eli/dec_impl/2025/138/oj.

[131] Commission Implementing Regulation (EU) 2024/2690 of 17 October 2024 laying down rules for the application of Directive (EU) 2022/2555 as regards technical and methodological requirements of cybersecurity risk-management measures. *Official Journal of the EU*. L series, 18.10.2024. URL: https://eur-lex.europa.eu/eli/reg_impl/2024/2690/oj.

[132] Commission Implementing Regulation (EU) 2024/482 of 31 January 2024 laying down rules for the application of Regulation (EU) 2019/881 as regards the adoption of the European Common Criteria-based cybersecurity certification scheme (EUCC). *Official Journal of the EU*. L series, 07.02.2024. URL: https://eur-lex.europa.eu/eli/reg_impl/2024/482/oj.

[133] Commission Implementing Regulation (EU) 2025/2392 of 28 November 2025 on the technical description of the categories of important and critical products with digital elements pursuant to Regulation (EU) 2024/2847. *Official Journal of the EU*. L series, 01.12.2025. URL: https://eur-lex.europa.eu/eli/reg_impl/2025/2392/oj.

[134] Common Attack Pattern Enumeration and Classification (CAPEC). The MITRE Corporation. URL: https://capec.mitre.org/.

[135] Common Security Advisory Framework Version 2.0. OASIS Standard. Ed. by L. Rock, S. Hagen, T. Schmidt. OASIS Open, 18.11.2022. URL: https://docs.oasis-open.org/csaf/csaf/v2.0/os/csaf-v2.0-os.html.

[136] Common Weakness Enumeration (CWE). The MITRE Corporation. URL: https://cwe.mitre.org/.

[137] Corbet J. The (successful) end of the kernel Rust experiment. *LWN.net*. 10.12.2025. URL: https://lwn.net/Articles/1049831/.

[138] Cousot P., Cousot R. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '77)*. New York: ACM, 1977. P. 238–252. DOI: 10.1145/512950.512973.

[139] Cox J., Bouwers E., van Eekelen M., Visser J. Measuring Dependency Freshness in Software Systems. *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering (ICSE)*. Florence: IEEE, 2015. Vol. 2. P. 109–118. DOI: 10.1109/ICSE.2015.140.

[140] Cremers C., Horvat M., Hoyland J., Scott S., van der Merwe T. A Comprehensive Symbolic Analysis of TLS 1.3. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. New York: ACM, 2017. P. 1773–1788. DOI: 10.1145/3133956.3134063.

[141] Crosby S. A., Wallach D. S. Denial of Service via Algorithmic Complexity Attacks. *Proceedings of the 12th USENIX Security Symposium*. Washington, DC: USENIX Association, 2003. URL: https://www.usenix.org/conference/12th-usenix-security-symposium/denial-service-algorithmic-complexity-attacks.

[142] Csikor L., Divakaran D. M., Kang M. S., Kőrösi A., Sonkoly B., Haja D., Pezaros D. P., Schmid S., Rétvári G. Tuple Space Explosion: A Denial-of-Service Attack against a Software Packet Classifier. *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '19)*. New York: ACM, 2019. P. 292–304. DOI: 10.1145/3359989.3365431.

[143] CWE Top 25 Most Dangerous Software Weaknesses. The MITRE Corporation. URL: https://cwe.mitre.org/top25/.

[144] DARPA. AI Cyber Challenge Marks Pivotal Inflection Point for Cyber Defense. 08.08.2025. URL: https://www.darpa.mil/news/2025/aixcc-results.

[145] de Ruiter J., Poll E. Protocol State Fuzzing of TLS Implementations. *24th USENIX Security Symposium (USENIX Security 15)*. Washington: USENIX Association, 2015. P. 193–206. URL: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/de-ruiter.

[146] DeBellis D., Storer K., Lewis A., Good B., Villalba D., Maxwell E., Castillo K., Irvine M., Harvey N. Accelerate State of DevOps 2024 Report. DORA, Google, 2024. URL: https://dora.dev/research/2024/dora-report/.

[147] DeKok A. RADIUS/1.1: Leveraging Application-Layer Protocol Negotiation (ALPN) to Remove MD5. RFC 9765. IETF, 2025. DOI: 10.17487/RFC9765. URL: https://www.rfc-editor.org/rfc/rfc9765.

[148] Deng M., Wuyts K., Scandariato R., Preneel B., Joosen W. A Privacy Threat Analysis Framework: Supporting the Elicitation and Fulfillment of Privacy Requirements. *Requirements Engineering*. 2011. Vol. 16, No. 1. P. 3–32. DOI: 10.1007/s00766-010-0115-7.

[149] Dewri R., Poolsappasit N., Ray I., Whitley D. Optimal Security Hardening Using Multi-objective Optimization on Attack Tree Models of Networks. *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS'07)*. New York: ACM, 2007. P. 204–213. DOI: 10.1145/1315245.1315272.

[150] Dijkstra E. W. The Humble Programmer. *Communications of the ACM*. 1972. Vol. 15, no. 10. P. 859–866. DOI: 10.1145/355604.361591.

[151] Directive (EU) 2022/2555 of the European Parliament and of the Council of 14 December 2022 on measures for a high common level of cybersecurity across the Union (NIS 2 Directive). *Official Journal of the EU*. L 333. P. 80–152. URL: https://eur-lex.europa.eu/eli/dir/2022/2555/oj.

[152] Dolev D., Yao A. C. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*. 1983. Vol. 29, no. 2. P. 198–208. DOI: 10.1109/TIT.1983.1056650.

[153] Donenfeld J. A. WireGuard: Next Generation Kernel Network Tunnel. *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS 2017)*. San Diego: Internet Society, 2017. DOI: 10.14722/ndss.2017.23160.

[154] Dowd M., McDonald J., Schuh J. The Art of Software Security Assessment: Identifying and Preventing Software Vulnerabilities. Upper Saddle River: Addison-Wesley, 2006. 1200 p.

[155] Dowling B., Paterson K. G. A Cryptographic Analysis of the WireGuard Protocol. *Applied Cryptography and Network Security (ACNS 2018)*. Lecture Notes in Computer Science, vol. 10892. Cham: Springer, 2018. P. 3–21. DOI: 10.1007/978-3-319-93387-0_1.

[156] Durumeric Z., Li F., Kasten J., Amann J., Beekman J., Payer M., Weaver N., Adrian D., Paxson V., Bailey M., Halderman J. A. The Matter of Heartbleed. *Proceedings of the 2014 Conference on Internet Measurement Conference (IMC '14)*. New York: ACM, 2014. P. 475–488. DOI: 10.1145/2663716.2663755.

[157] Durumeric Z., Ma Z., Springall D., Barnes R., Sullivan N., Bursztein E., Bailey M., Halderman J. A., Paxson V. The Security Impact of HTTPS Interception. *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS 2017)*. San Diego: Internet Society, 2017. DOI: 10.14722/ndss.2017.23456.

[158] ECMA-424. CycloneDX Bill of Materials Specification. Geneva: Ecma International, 2024. URL: https://ecma-international.org/publications-and-standards/standards/ecma-424/.

[159] Eddy W. TCP SYN Flooding Attacks and Common Mitigations. RFC 4987. IETF, 2007. DOI: 10.17487/RFC4987. URL: https://www.rfc-editor.org/rfc/rfc4987.

[160] Egele M., Brumley D., Fratantonio Y., Kruegel C. An Empirical Study of Cryptographic Misuse in Android Applications. *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS'13)*. New York: ACM, 2013. P. 73–84. DOI: 10.1145/2508859.2516693.

[161] Eick S. G., Loader C. R., Long M. D., Votta L. G., Vander Wiel S. Estimating Software Fault Content Before Coding. *Proceedings of the 14th International Conference on Software Engineering (ICSE '92)*. New York: ACM, 1992. P. 59–65. DOI: 10.1145/143062.143090.

[162] Emmerich P., Gallenmüller S., Raumer D., Wohlfart F., Carle G. MoonGen: A Scriptable High-Speed Packet Generator. *Proceedings of the 2015 Internet Measurement Conference (IMC '15)*. New York: ACM, 2015. P. 275–287. DOI: 10.1145/2815675.2815692.

[163] EN 18031-1:2024. Common security requirements for radio equipment — Part 1: Internet connected radio equipment. Brussels: CEN-CENELEC, 2024.

[164] Engelen G., Rimmer V., Joosen W. Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study. *2021 IEEE Security and Privacy Workshops (SPW)*. San Francisco: IEEE, 2021. P. 7–12. DOI: 10.1109/SPW53761.2021.00009.

[165] ENISA. Stepping up our role in Vulnerability Management: ENISA Becomes CVE Root. 20.11.2025. URL: https://enisa.europa.eu/news/stepping-up-our-role-in-vulnerability-management-enisa-becomes-cve-root.

[166] Erbsen A., Philipoom J., Gross J., Sloan R., Chlipala A. Simple High-Level Code for Cryptographic Arithmetic — With Proofs, Without Compromises. *2019 IEEE*

Symposium on Security and Privacy (SP). San Francisco: IEEE, 2019. P. 1202–1219. DOI: 10.1109/SP.2019.00005.

[167] ETSI GS NFV 002 V1.2.1. Network Functions Virtualisation (NFV); Architectural Framework. Sophia Antipolis: ETSI, 2014. URL: https://www.etsi.org/deliver/etsi_gs/NFV/001_099/002/01.02.01_60/gs_NFV002v010201p.pdf.

[168] ETSI launches approval process for 17 European Standards supporting the Cyber Resilience Act: press release. Sophia Antipolis: ETSI, 13.08.2026. URL: https://www.etsi.org/newsroom/press-releases/etsi-launches-approval-process-for-17-european-standards-supporting-the-cyber-resilience-act/.

[169] European Commission. EU launches a European vulnerability database to boost its digital security. 13.05.2025. URL: https://digital-strategy.ec.europa.eu/en/news/eu-launches-european-vulnerability-database-boost-its-digital-security.

[170] Fasano A., Ballo T., Muench M., Leek T., Bulekov A., Dolan-Gavitt B., Egele M., Francillon A., Lu L., Gregory N., Balzarotti D., Robertson W. SoK: Enabling Security Analyses of Embedded Systems via Rehosting. *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security (ASIA CCS '21)*. New York: ACM, 2021. P. 687–701. DOI: 10.1145/3433210.3453093.

[171] Felderer M., Schieferdecker I. A taxonomy of risk-based testing. *International Journal on Software Tools for Technology Transfer*. 2014. Vol. 16, no. 5. P. 559–568. DOI: 10.1007/s10009-014-0332-3.

[172] Fioraldi A., Maier D., Eißfeldt H., Heuse M. AFL++: Combining Incremental Steps of Fuzzing Research. *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, 2020. URL: https://www.usenix.org/conference/woot20/presentation/fioraldi.

[173] FIRST. Common Vulnerability Scoring System Version 4.0: Specification Document. Forum of Incident Response and Security Teams, 2023. URL: https://www.first.org/cvss/v4.0/specification-document.

[174] FIRST. Exploit Prediction Scoring System (EPSS). Forum of Incident Response and Security Teams. URL: https://www.first.org/epss/.

[175] FIRST. PSIRT Services Framework. Version 1.1. Forum of Incident Response and Security Teams. URL: https://www.first.org/standards/frameworks/psirts/psirt_services_framework_v1.1.

[176] Fogel A., Fung S., Pedrosa L., Walraed-Sullivan M., Govindan R., Mahajan R., Millstein T. A General Approach to Network Configuration Analysis. *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI '15)*. USENIX Association, 2015. P. 469–483.

[177] Fortinet PSIRT. Out-of-bound Write in sslvpnd (FG-IR-24-015, CVE-2024-21762). 08.02.2024. URL: https://fortiguard.fortinet.com/psirt/FG-IR-24-015.

[178] Foudil E., Shafranovich Y. A File Format to Aid in Security Vulnerability Disclosure. RFC 9116. IETF, 2022. DOI: 10.17487/RFC9116. URL: https://www.rfc-editor.org/rfc/rfc9116.

[179] Frei S., May M., Fiedler U., Plattner B. Large-Scale Vulnerability Analysis. *Proceedings of the 2006 SIGCOMM Workshop on Large-Scale Attack Defense (LSAD '06)*. New York: ACM, 2006. P. 131–138. DOI: 10.1145/1162666.1162671.

[180] Georgiev M., Iyengar S., Jana S., Anubhai R., Boneh D., Shmatikov V. The Most Dangerous Code in the World: Validating SSL Certificates in Non-Browser Software. *Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS'12)*. New York: ACM, 2012. P. 38–49. DOI: 10.1145/2382196.2382204.

[181] Gershuni E., Amit N., Gurfinkel A., Narodytska N., Navas J. A., Rinetzky N., Ryzhyk L., Sagiv M. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '19)*. New York: ACM, 2019. P. 1069–1084. DOI: 10.1145/3314221.3314590.

[182] Goal Structuring Notation Community Standard. Version 3. SCSC-141C. Assurance Case Working Group, Safety-Critical Systems Club, 2021. URL: https://scsc.uk/scsc-141C.

[183] Goel A. L., Okumoto K. Time-Dependent Error-Detection Rate Model for Software Reliability and Other Performance Measures. *IEEE Transactions on Reliability*. 1979. Vol. R-28, No. 3. P. 206–211. DOI: 10.1109/TR.1979.5220566.

[184] Goldberg S., Haller M., Heninger N., Milano M., Shumow D., Stevens M., Suhl A. RADIUS/UDP Considered Harmful. *Proceedings of the 33rd USENIX Security Symposium*. Philadelphia: USENIX Association, 2024. P. 7429–7446. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/goldberg.

[185] Goodenough J. B., Weinstock C. B., Klein A. Z. Eliminative Induction: A Basis for Arguing System Confidence. *2013 35th International Conference on Software Engineering (ICSE)*. San Francisco: IEEE, 2013. P. 1161–1164. DOI: 10.1109/ICSE.2013.6606668.

[186] Google Project Zero. Vulnerability Disclosure Policy. URL: https://projectzero.google/vulnerability-disclosure-policy.html.

[187] Gueron S., Langley A., Lindell Y. AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption. RFC 8452. IRTF, 2019. DOI: 10.17487/RFC8452. URL: https://www.rfc-editor.org/rfc/rfc8452.

[188] Gunther N. J. Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services. Berlin; Heidelberg: Springer, 2007. DOI: 10.1007/978-3-540-31010-5.

[189] Gupta P., McKeown N. Algorithms for Packet Classification. *IEEE Network*. 2001. Vol. 15, no. 2. P. 24–32. DOI: 10.1109/65.912717.

[190] Handley M., Paxson V., Kreibich C. Network Intrusion Detection: Evasion, Traffic Normalization, and End-to-End Protocol Semantics. *Proceedings of the 10th USENIX Security Symposium*. Washington, D.C.: USENIX Association, 2001. URL: https://www.usenix.org/conference/10th-usenix-security-symposium/network-intrusion-detection-evasion-traffic-normalization.

[191] Hazimeh A., Herrera A., Payer M. Magma: A Ground-Truth Fuzzing Benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*. 2020. Vol. 4, no. 3. P. 1–29. DOI: 10.1145/3428334.

[192] Heninger N., Durumeric Z., Wustrow E., Halderman J. A. Mining Your Ps and Qs: Detection of Widespread Weak Keys in Network Devices. *21st USENIX Security Symposium (USENIX Security 12)*. Bellevue: USENIX Association, 2012. P. 205–220. URL: https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/heninger.

[193] Hickman B., Newman D., Tadjudin S., Martin T. Benchmarking Methodology for Firewall Performance. RFC 3511. IETF, 2003. DOI: 10.17487/RFC3511. URL: https://www.rfc-editor.org/rfc/rfc3511.

[194] Householder A. D., Wassermann G., Manion A., King C. The CERT Guide to Coordinated Vulnerability Disclosure. Special Report CMU/SEI-2017-SR-022. Pittsburgh: Software Engineering Institute, Carnegie Mellon University, 2017. URL: https://vuls.cert.org/confluence/display/CVD.

[195] Howard M., Lipner S. The Security Development Lifecycle. Redmond: Microsoft Press, 2006. 352 p.

[196] Høiland-Jørgensen T., Brouer J. D., Borkmann D., Fastabend J., Herbert T., Ahern D., Miller D. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '18)*. New York: ACM, 2018. P. 54–66. DOI: 10.1145/3281411.3281443.

[197] IEC 62443-4-1:2018. Security for industrial automation and control systems — Part 4-1: Secure product development lifecycle requirements. Geneva: IEC, 2018.

[198] IEC 62443-4-2:2019. Security for industrial automation and control systems — Part 4-2: Technical security requirements for IACS components. Geneva: IEC, 2019.

[199] Inozemtseva L., Holmes R. Coverage Is Not Strongly Correlated with Test Suite Effectiveness. *Proceedings of the 36th International Conference on Software Engineering (ICSE '14)*. New York: ACM, 2014. P. 435–445. DOI: 10.1145/2568225.2568271.

[200] ISO 22301:2019. Security and resilience — Business continuity management systems — Requirements. Geneva: ISO, 2019. URL: https://www.iso.org/standard/75106.html.

[201] ISO/IEC 15408-1:2022. Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Part 1: Introduction and general model. Geneva: ISO, 2022. URL: https://www.iso.org/standard/72891.html.

[202] ISO/IEC 15408-3:2022. Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Part 3: Security assurance components. Geneva: ISO, 2022.

[203] ISO/IEC 18045:2022. Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Methodology for IT security evaluation. Geneva: ISO, 2022.

[204] ISO/IEC 19790:2012. Information technology — Security techniques — Security requirements for cryptographic modules. Geneva: ISO, 2012. URL: https://www.iso.org/standard/52906.html.

[205] ISO/IEC 27034-1:2011. Information technology — Security techniques — Application security — Part 1: Overview and concepts. Geneva: ISO, 2011. URL: https://www.iso.org/standard/44378.html.

[206] ISO/IEC 29147:2018. Information technology — Security techniques — Vulnerability disclosure. Geneva: ISO, 2018. URL: https://www.iso.org/standard/72311.html.

[207] ISO/IEC 30111:2019. Information technology — Security techniques — Vulnerability handling processes. Geneva: ISO, 2019. URL: https://www.iso.org/standard/69725.html.

[208] ISO/IEC 5962:2021. Information technology — SPDX® Specification V2.2.1. Geneva: ISO, 2021.

[209] ISO/IEC/IEEE 15026-2:2022. Systems and software engineering — Systems and software assurance — Part 2: Assurance case. Geneva: ISO, 2022. URL: https://www.iso.org/standard/80625.html.

[210] ISO/IEC/IEEE 29119-1:2022. Software and systems engineering — Software testing — Part 1: General concepts. Geneva: ISO, 2022.

[211] ISO/IEC/IEEE 29119-2:2021. Software and systems engineering — Software testing — Part 2: Test processes. Geneva: ISO, 2021.

[212] ISO/IEC/IEEE 29119-4:2021. Software and systems engineering — Software testing — Part 4: Test techniques. Geneva: ISO, 2021.

[213] Iyengar J. (Ed.), Thomson M. (Ed.). QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000. IETF, 2021. DOI: 10.17487/RFC9000. URL: https://www.rfc-editor.org/rfc/rfc9000.

[214] Jacobs J., Romanosky S., Edwards B., Adjerid I., Roytman M. Exploit Prediction Scoring System (EPSS). *Digital Threats: Research and Practice*. 2021. Vol. 2, No. 3. P. 1–17. DOI: 10.1145/3436242.

[215] Jia Y., Harman M. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*. 2011. Vol. 37, no. 5. P. 649–678. DOI: 10.1109/TSE.2010.62.

[216] Johnson B., Song Y., Murphy-Hill E., Bowdidge R. Why Don't Software Developers Use Static Analysis Tools to Find Bugs? *2013 35th International Conference on Software Engineering (ICSE)*. San Francisco: IEEE, 2013. P. 672–681. DOI: 10.1109/ICSE.2013.6606613.

[217] Jones C., Bonsignour O. The Economics of Software Quality. Upper Saddle River: Addison-Wesley, 2011.

[218] Jung R., Jourdan J.-H., Krebbers R., Dreyer D. RustBelt: Securing the Foundations of the Rust Programming Language. *Proceedings of the ACM on Programming Languages*. 2018. Vol. 2, Issue POPL. Article 66. P. 1–34. DOI: 10.1145/3158154.

[219] Kampanakis P. Post-quantum Key Exchange with ML-KEM in the Internet Key Exchange Protocol Version 2 (IKEv2). Internet-Draft draft-ietf-ipsecme-ikev2-mlkem-09. IETF, 2026. URL: https://datatracker.ietf.org/doc/draft-ietf-ipsecme-ikev2-mlkem/.

[220] Kaufman C., Hoffman P., Nir Y., Eronen P., Kivinen T. Internet Key Exchange Protocol Version 2 (IKEv2). RFC 7296. IETF, 2014. DOI: 10.17487/RFC7296. URL: https://www.rfc-editor.org/rfc/rfc7296.

[221] Kemerer C. F., Paulk M. C. The Impact of Design and Code Reviews on Software Quality: An Empirical Study Based on PSP Data. *IEEE Transactions on Software Engineering*. 2009. Vol. 35, No. 4. P. 534–550. DOI: 10.1109/TSE.2009.27.

[222] Kent S. IP Encapsulating Security Payload (ESP). RFC 4303. IETF, 2005. DOI: 10.17487/RFC4303. URL: https://www.rfc-editor.org/rfc/rfc4303.

[223] Kent S., Seo K. Security Architecture for the Internet Protocol. RFC 4301. IETF, 2005. DOI: 10.17487/RFC4301. URL: https://www.rfc-editor.org/rfc/rfc4301.

[224] Klees G., Ruef A., Cooper B., Wei S., Hicks M. Evaluating Fuzz Testing. *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*. New York: ACM, 2018. P. 2123–2138. DOI: 10.1145/3243734.3243804.

[225] Klein G., Elphinstone K., Heiser G., Andronick J., Cock D., Derrin P., Elkaduwe D., Engelhardt K., Kolanski R., Norrish M., Sewell T., Tuch H., Winwood S. seL4: Formal Verification of an OS Kernel. *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP'09)*. New York: ACM, 2009. P. 207–220. DOI: 10.1145/1629575.1629596.

[226] Kordy B., Piètre-Cambacédès L., Schweitzer P. DAG-based Attack and Defense Modeling: Don't Miss the Forest for the Attack Trees. *Computer Science Review*. 2014. Vol. 13–14. P. 1–38. DOI: 10.1016/j.cosrev.2014.07.001.

[227] Krivelevich D., Gil O., Segal O. et al. OWASP Top 10 CI/CD Security Risks. OWASP Foundation, 2022. URL: https://owasp.org/www-project-top-10-ci-cd-security-risks/.

[228] Kuhn D. R., Wallace D. R., Gallo A. M. Software Fault Interactions and Implications for Software Testing. *IEEE Transactions on Software Engineering*. 2004. Vol. 30, no. 6. P. 418–421. DOI: 10.1109/TSE.2004.24.

[229] Kwiatkowski K., Kampanakis P., Westerbaan B. E., Stebila D. Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3. RFC 10024. IETF, 2026. DOI: 10.17487/RFC10024. URL: https://www.rfc-editor.org/info/rfc10024.

[230] Ladisa P., Plate H., Martinez M., Barais O. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. *2023 IEEE Symposium on Security and Privacy (SP)*. San Francisco: IEEE, 2023. P. 1509–1526. DOI: 10.1109/SP46215.2023.10179304.

[231] Lamb C., Zacchiroli S. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software*. 2022. Vol. 39, no. 2. P. 62–70. DOI: 10.1109/MS.2021.3073045.

[232] Landlock: unprivileged access control. *The Linux Kernel Documentation: Userspace API*. URL: https://docs.kernel.org/userspace-api/landlock.html.

[233] Lanvin M., Gimenez P.-F., Han Y., Majorczyk F., Mé L., Totel É. Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes. *Risks and Security of Internet and Systems (CRiSIS 2022)*. Lecture Notes in Computer Science, vol. 13857. Cham: Springer, 2023. P. 18–33. DOI: 10.1007/978-3-031-31108-6_2.

[234] Lindem A., Dogra A. Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6. RFC 9568. IETF, 2024. DOI: 10.17487/RFC9568. URL: https://www.rfc-editor.org/rfc/rfc9568.

[235] Lipner S. The Trustworthy Computing Security Development Lifecycle. *20th Annual Computer Security Applications Conference (ACSAC 2004)*. Tucson: IEEE, 2004. P. 2–13. DOI: 10.1109/CSAC.2004.41.

[236] Lipp B., Blanchet B., Bhargavan K. A Mechanised Cryptographic Proof of the WireGuard Virtual Private Network Protocol. *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. Stockholm: IEEE, 2019. P. 231–246. DOI: 10.1109/EuroSP.2019.00026.

[237] Littlewood B., Wright D. The Use of Multilegged Arguments to Increase Confidence in Safety Claims for Software-Based Systems: A Study Based on a BBN Analysis of an Idealized Example. *IEEE Transactions on Software Engineering*. 2007. Vol. 33, No. 5. P. 347–365. DOI: 10.1109/TSE.2007.1002.

[238] Loise T., Devroey X., Perrouin G., Papadakis M., Heymans P. Towards Security-Aware Mutation Testing. *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. Tokyo: IEEE, 2017. P. 97–102. DOI: 10.1109/ICSTW.2017.24.

[239] M-Trends 2025: Data, Insights, and Recommendations From the Frontlines. Mandiant (Google Cloud), 23.04.2025. URL: https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2025/.

[240] Manadhata P. K., Wing J. M. An Attack Surface Metric. *IEEE Transactions on Software Engineering*. 2011. Vol. 37, No. 3. P. 371–386. DOI: 10.1109/TSE.2010.60.

[241] Manès V. J. M., Han H., Han C., Cha S. K., Egele M., Schwartz E. J., Woo M. The Art, Science, and Engineering of Fuzzing: A Survey. *IEEE Transactions on Software Engineering*. 2021. Vol. 47, no. 11. P. 2312–2331. DOI: 10.1109/TSE.2019.2946563.

[242] Massacci F., Nguyen V. H. An Empirical Methodology to Evaluate Vulnerability Discovery Models. *IEEE Transactions on Software Engineering*. 2014. Vol. 40, No. 12. P. 1147–1162. DOI: 10.1109/TSE.2014.2354037.

[243] McCabe T. J. A Complexity Measure. *IEEE Transactions on Software Engineering*. 1976. Vol. SE-2, No. 4. P. 308–320. DOI: 10.1109/TSE.1976.233837.

[244] McConnell S. Code Complete: A Practical Handbook of Software Construction. 2nd ed. Redmond: Microsoft Press, 2004. 960 p.

[245] McGraw G. Software Security: Building Security In. Boston: Addison-Wesley, 2006. 448 p.

[246] McHugh J. Testing Intrusion Detection Systems: A Critique of the 1998 and 1999 DARPA Intrusion Detection System Evaluations as Performed by Lincoln

Laboratory. *ACM Transactions on Information and System Security*. 2000. Vol. 3, no. 4. P. 262–294. DOI: 10.1145/382912.382923.

[247] Meier S., Schmidt B., Cremers C., Basin D. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. *Computer Aided Verification (CAV 2013)*. Lecture Notes in Computer Science, vol. 8044. Berlin, Heidelberg: Springer, 2013. P. 696–701. DOI: 10.1007/978-3-642-39799-8_48.

[248] Meng R., Mirchev M., Böhme M., Roychoudhury A. Large Language Model guided Protocol Fuzzing. *Network and Distributed System Security Symposium (NDSS 2024)*. San Diego: Internet Society, 2024. DOI: 10.14722/ndss.2024.24556.

[249] Microsoft Security Development Lifecycle (SDL): Practices. Microsoft. URL: https://www.microsoft.com/en-us/securityengineering/sdl/practices.

[250] Microsoft Security Response Center. A proactive approach to more secure code. *MSRC Blog*. 16.07.2019. URL: https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/.

[251] Mirsky Y., Doitshman T., Elovici Y., Shabtai A. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. *Network and Distributed System Security Symposium (NDSS 2018)*. San Diego: Internet Society, 2018. DOI: 10.14722/ndss.2018.23204.

[252] MITRE ATT&CK® Evaluations. The MITRE Corporation. URL: https://evals.mitre.org/.

[253] MITRE ATT&CK®: Adversarial Tactics, Techniques, and Common Knowledge (Enterprise and Mobile matrices). The MITRE Corporation. URL: https://attack.mitre.org/.

[254] Moody D., Perlner R., Regenscheid A., Robinson A., Cooper D. Transition to Post-Quantum Cryptography Standards. NIST IR 8547 (Initial Public Draft). Gaithersburg, MD: NIST, 2024. DOI: 10.6028/NIST.IR.8547.ipd. URL: https://csrc.nist.gov/pubs/ir/8547/ipd.

[255] Moran B., Tschofenig H., Brown D., Meriac M. A Firmware Update Architecture for Internet of Things. RFC 9019. IETF, 2021. DOI: 10.17487/RFC9019. URL: https://www.rfc-editor.org/rfc/rfc9019.

[256] Morton A. Updates for the Back-to-Back Frame Benchmark in RFC 2544. RFC 9004. IETF, 2021. DOI: 10.17487/RFC9004. URL: https://www.rfc-editor.org/rfc/rfc9004.

[257] Moustafa N., Slay J. UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set). *2015 Military Communications and Information Systems Conference (MilCIS)*. Canberra: IEEE, 2015. P. 1–6. DOI: 10.1109/MilCIS.2015.7348942.

[258] Musa J. D. A Theory of Software Reliability and Its Application. *IEEE Transactions on Software Engineering*. 1975. Vol. SE-1, No. 3. P. 312–327. DOI: 10.1109/TSE.1975.6312856.

[259] Musa J. D., Okumoto K. A Logarithmic Poisson Execution Time Model for Software Reliability Measurement. *Proceedings of the 7th International Conference on Software Engineering (ICSE '84)*. Orlando: IEEE Computer Society, 1984. P. 230–238.

[260] Nadi S., Krüger S., Mezini M., Bodden E. Jumping Through Hoops: Why Do Java Developers Struggle with Cryptography APIs? *Proceedings of the 38th International Conference on Software Engineering (ICSE'16)*. New York: ACM, 2016. P. 935–946. DOI: 10.1145/2884781.2884790.

[261] NetSecOPEN: Performance Testing and Certification of Network Security Products. URL: https://www.netsecopen.org/.

[262] Newman D. Benchmarking Terminology for Firewall Performance. RFC 2647. IETF, 1999. DOI: 10.17487/RFC2647. URL: https://www.rfc-editor.org/rfc/rfc2647.

[263] Newman Z., Meyers J. S., Torres-Arias S. Sigstore: Software Signing for Everybody. *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. New York: ACM, 2022. P. 2353–2367. DOI: 10.1145/3548606.3560596.

[264] NIST NVD. CVE-2024-3094: Malicious code in upstream tarballs of xz (liblzma). Published 29.03.2024. URL: https://nvd.nist.gov/vuln/detail/CVE-2024-3094.

[265] NIST. Module-Lattice-Based Digital Signature Standard. FIPS 204. Gaithersburg, MD, 2024. DOI: 10.6028/NIST.FIPS.204. URL: https://csrc.nist.gov/pubs/fips/204/final.

[266] NIST. Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203. Gaithersburg, MD, 2024. DOI: 10.6028/NIST.FIPS.203. URL: https://csrc.nist.gov/pubs/fips/203/final.

[267] NIST. Secure Software Development Framework (SSDF) Version 1.2: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST SP 800-218 Rev. 1 (Initial Public Draft). Gaithersburg, MD, 2025. URL: https://csrc.nist.gov/pubs/sp/800/218/r1/ipd.

[268] NIST. Security Requirements for Cryptographic Modules. FIPS 140-3. Gaithersburg, MD, 2019. DOI: 10.6028/NIST.FIPS.140-3. URL: https://csrc.nist.gov/pubs/fips/140-3/final.

[269] NIST. Stateless Hash-Based Digital Signature Standard. FIPS 205. Gaithersburg, MD: NIST, 2024. DOI: 10.6028/NIST.FIPS.205. URL: https://csrc.nist.gov/pubs/fips/205/final.

[270] Noether G. E. Sample Size Determination for Some Common Nonparametric Tests. *Journal of the American Statistical Association*. 1987. Vol. 82, No. 398. P. 645–647. DOI: 10.1080/01621459.1987.10478478.

[271] NSA, CISA. Memory Safe Languages: Reducing Vulnerabilities in Modern Software Development. Cybersecurity Information Sheet. 24.06.2025. URL: https://www.cisa.gov/resources-tools/resources/memory-safe-languages-reducing-vulnerabilities-modern-software-development.

[272] Nygard M. Documenting Architecture Decisions. Cognitect Blog. 15.11.2011. URL: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions.

[273] OISF. Announcing Suricata 5.0.0. Open Information Security Foundation, 15.10.2019. URL: https://suricata.io/2019/10/15/announcing-suricata-5-0-0/.

[274] OISF. Suricata User Guide: Rules Format; EVE JSON Output; High Performance Configuration. Open Information Security Foundation. URL: https://docs.suricata.io/.

[275] Okun V., Delaitre A., Black P. E. Report on the Static Analysis Tool Exposition (SATE) IV. NIST Special Publication 500-297. Gaithersburg, MD: NIST, 2013. 46 p. DOI: 10.6028/NIST.SP.500-297.

[276] OpenSSF Best Practices Working Group. Compiler Options Hardening Guide for C and C++. Open Source Security Foundation, 2026. URL: https://best.openssf.org/Compiler-Hardening-Guides/Compiler-Options-Hardening-Guide-for-C-and-C++.html.

[277] OWASP Application Security Verification Standard (ASVS). Version 5.0.0. OWASP Foundation, 2025. URL: https://owasp.org/www-project-application-security-verification-standard/.

[278] OWASP pytm: A Pythonic framework for threat modeling. OWASP Foundation. URL: https://github.com/OWASP/pytm.

[279] OWASP Software Assurance Maturity Model (SAMM). Version 2. OWASP Foundation. URL: https://owaspsamm.org/.

[280] OWASP Top 10 for LLM Applications 2025. OWASP GenAI Security Project, 2024. URL: https://genai.owasp.org/llm-top-10/.

[281] OWASP Top 10:2025. OWASP Foundation, 2025. URL: https://owasp.org/Top10/2025/.

[282] OWASP Web Security Testing Guide. Version 4.2. OWASP Foundation, 2020. URL: https://owasp.org/www-project-web-security-testing-guide/.

[283] Ozment A., Schechter S. E. Milk or Wine: Does Software Security Improve with Age? *Proceedings of the 15th USENIX Security Symposium*. Vancouver: USENIX Association, 2006. URL: https://www.usenix.org/conference/15th-usenix-security-symposium/milk-or-wine-does-software-security-improve-age.

[284] Palo Alto Networks. CVE-2024-3400 PAN-OS: Arbitrary File Creation Leads to OS Command Injection Vulnerability in GlobalProtect. Security Advisory. 12.04.2024. URL: https://security.paloaltonetworks.com/CVE-2024-3400.

[285] Papadogiannaki E., Ioannidis S. A Survey on Encrypted Network Traffic Analysis Applications, Techniques, and Countermeasures. *ACM Computing Surveys*. 2021. Vol. 54, no. 6. P. 1–35. DOI: 10.1145/3457904.

[286] Paxson V. Bro: a System for Detecting Network Intruders in Real-Time. *Computer Networks*. 1999. Vol. 31, no. 23–24. P. 2435–2463. DOI: 10.1016/S1389-1286(99)00112-7.

[287] Pearce H., Ahmad B., Tan B., Dolan-Gavitt B., Karri R. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. *2022 IEEE Symposium on Security and Privacy (SP)*. San Francisco: IEEE, 2022. P. 754–768. DOI: 10.1109/SP46214.2022.9833571.

[288] Perry N., Srivastava M., Kumar D., Boneh D. Do Users Write More Insecure Code with AI Assistants? *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS'23)*. Copenhagen: ACM, 2023. P. 2785–2799. DOI: 10.1145/3576915.3623157.

[289] Pfaff B., Pettit J., Koponen T., Jackson E., Zhou A., Rajahalme J., Gross J., Wang A., Stringer J., Shelar P., Amidon K., Casado M. The Design and Implementation of Open vSwitch. *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI '15)*. USENIX Association, 2015. P. 117–130.

[290] Pham V.-T., Böhme M., Roychoudhury A. AFLNet: A Greybox Fuzzer for Network Protocols. *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. Porto: IEEE, 2020. P. 460–465. DOI: 10.1109/ICST46399.2020.00062.

[291] Piessens F. Software Security Knowledge Area. Version 1.0.1. *The Cyber Security Body of Knowledge (CyBOK)*. © Crown Copyright, The National Cyber Security Centre, 2021. URL: https://www.cybok.org/.

[292] pledge(2) — restrict system operations. *OpenBSD Manual Pages*. URL: https://man.openbsd.org/pledge.2.

[293] PP-Configuration for Network Devices, Intrusion Prevention Systems, Stateful Traffic Filter Firewalls, and Virtual Private Network Gateways. Version 2.0: Validation Report. NIAP, 2025. URL: https://commoncriteriaportal.org/nfs/ccpfiles/files/ppfiles/CFG_NDcPP-IPS-FW-VPNGW_v2.0_VR.pdf.

[294] Project Wycheproof: Test Vectors for Cryptographic Libraries. C2SP (Community Cryptography Specification Project). URL: https://github.com/C2SP/wycheproof.

[295] Provos N., Friedl M., Honeyman P. Preventing Privilege Escalation. *Proceedings of the 12th USENIX Security Symposium*. Washington, D.C.: USENIX Association, 2003. P. 231–242. URL: https://www.usenix.org/conference/12th-usenix-security-symposium/preventing-privilege-escalation.

[296] Ptacek T. H., Newsham T. N. Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection: Technical Report. Calgary: Secure Networks, Inc., 1998. URL: https://apps.dtic.mil/sti/pdfs/ADA391565.pdf.

[297] Ramananandro T., Delignat-Lavaud A., Fournet C., Swamy N., Chajed T., Kobeissi N., Protzenko J. EverParse: Verified Secure Zero-Copy Parsers for Authenticated Message Formats. *Proceedings of the 28th USENIX Security Symposium*. Santa Clara: USENIX Association, 2019. P. 1465–1482.

[298] Regenscheid A. Platform Firmware Resiliency Guidelines. NIST Special Publication 800-193. Gaithersburg, MD: NIST, 2018. DOI: 10.6028/NIST.SP.800-193. URL: https://csrc.nist.gov/pubs/sp/800/193/final.

[299] Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation). *Official Journal of the EU*. L 119. P. 1–88. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj.

[300] Regulation (EU) 2019/881 of the European Parliament and of the Council of 17 April 2019 on ENISA (the European Union Agency for Cybersecurity) and on information and communications technology cybersecurity certification (Cybersecurity Act). *Official Journal of the EU*. L 151. P. 15–69. URL: https://eur-lex.europa.eu/eli/reg/2019/881/oj.

[301] Regulation (EU) 2024/2847 of the European Parliament and of the Council of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act). *Official Journal of the EU*. L series, 20.11.2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[302] Reparaz O., Balasch J., Verbauwhede I. Dude, is my code constant time? *Design, Automation & Test in Europe Conference & Exhibition (DATE 2017)*. Lausanne: IEEE, 2017. P. 1697–1702. DOI: 10.23919/DATE.2017.7927267.

[303] Rescorla E. Is Finding Security Holes a Good Idea? *IEEE Security & Privacy*. 2005. Vol. 3, No. 1. P. 14–19. DOI: 10.1109/MSP.2005.17.

[304] Rescorla E. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. IETF, 2018. DOI: 10.17487/RFC8446. URL: https://www.rfc-editor.org/rfc/rfc8446.

[305] Rescorla E., Oku K., Sullivan N., Wood C. A. TLS Encrypted Client Hello. RFC 9849. IETF, 2026. DOI: 10.17487/RFC9849. URL: https://www.rfc-editor.org/rfc/rfc9849.

[306] Rice H. G. Classes of Recursively Enumerable Sets and Their Decision Problems. *Transactions of the American Mathematical Society*. 1953. Vol. 74, no. 2. P. 358–366. DOI: 10.1090/S0002-9947-1953-0053041-6.

[307] Roesch M. Snort — Lightweight Intrusion Detection for Networks. *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*. Seattle: USENIX Association, 1999. P. 229–238. URL: https://www.usenix.org/legacy/event/lisa99/roesch.html.

[308] Rose S., Borchert O., Kerman A., Souppaya M. et al. Implementing a Zero Trust Architecture: High-Level Document. NIST Special Publication 1800-35. Gaithersburg, MD: NIST, 2025. DOI: 10.6028/NIST.SP.1800-35. URL: https://csrc.nist.gov/pubs/sp/1800/35/final.

[309] Rose S., Borchert O., Mitchell S., Connelly S. Zero Trust Architecture. NIST Special Publication 800-207. Gaithersburg, MD, 2020. DOI: 10.6028/NIST.SP.800-207. URL: https://csrc.nist.gov/pubs/sp/800/207/final.

[310] Ross R., Winstead M., McEvilley M. Engineering Trustworthy Secure Systems. NIST SP 800-160 Vol. 1, Rev. 1. Gaithersburg, MD, 2022. DOI: 10.6028/NIST.SP.800-160v1r1. URL: https://csrc.nist.gov/pubs/sp/800/160/v1/r1/final.

[311] Sadowski C., Aftandilian E., Eagle A., Miller-Cushon L., Jaspan C. Lessons from Building Static Analysis Tools at Google. *Communications of the ACM*. 2018. Vol. 61, no. 4. P. 58–66. DOI: 10.1145/3188720.

[312] Saito T., Rehmsmeier M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*. 2015. Vol. 10, no. 3. Art. e0118432. DOI: 10.1371/journal.pone.0118432.

[313] Saltzer J. H., Schroeder M. D. The Protection of Information in Computer Systems. *Proceedings of the IEEE*. 1975. Vol. 63, No. 9. P. 1278–1308. DOI: 10.1109/PROC.1975.9939.

[314] Samuel J., Mathewson N., Cappos J., Dingledine R. Survivable Key Compromise in Software Update Systems. *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS '10)*. New York: ACM, 2010. P. 61–72. DOI: 10.1145/1866307.1866315.

[315] Sassaman L., Patterson M. L., Bratus S., Locasto M. E. Security Applications of Formal Language Theory. *IEEE Systems Journal*. 2013. Vol. 7, No. 3. P. 489–500. DOI: 10.1109/JSYST.2012.2222000.

[316] Scarfone K., Hoffman P. Guidelines on Firewalls and Firewall Policy. NIST SP 800-41, Rev. 1. Gaithersburg, MD, 2009. DOI: 10.6028/NIST.SP.800-41r1. URL: https://csrc.nist.gov/pubs/sp/800/41/r1/final.

[317] Scarfone K., Mell P. Guide to Intrusion Detection and Prevention Systems (IDPS). NIST SP 800-94. Gaithersburg, MD, 2007. DOI: 10.6028/NIST.SP.800-94. URL: https://csrc.nist.gov/pubs/sp/800/94/final.

[318] Scarfone K., Souppaya M., Cody A., Orebaugh A. Technical Guide to Information Security Testing and Assessment. NIST SP 800-115. Gaithersburg, MD, 2008. DOI: 10.6028/NIST.SP.800-115. URL: https://csrc.nist.gov/pubs/sp/800/115/final.

[319] Schneier B. Attack Trees. *Dr. Dobb's Journal*. 1999. Vol. 24, No. 12. P. 21–29.

[320] Schumilo S., Aschermann C., Jemmett A., Abbasi A., Holz T. Nyx-Net: Network Fuzzing with Incremental Snapshots. *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22)*. New York: ACM, 2022. P. 166–180. DOI: 10.1145/3492321.3519591.

[321] Seccomp BPF (SECure COMPuting with filters). *The Linux Kernel Documentation: Userspace API*. URL: https://docs.kernel.org/userspace-api/seccomp_filter.html.

[322] SEI CERT C Coding Standard: Rules for Developing Safe, Reliable, and Secure Systems. 2016 Edition. Pittsburgh: Software Engineering Institute, Carnegie Mellon University, 2016. URL: https://resources.sei.cmu.edu/downloads/secure-coding/assets/sei-cert-c-coding-standard-2016-v01.pdf.

[323] Sharafaldin I., Habibi Lashkari A., Ghorbani A. A. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*. SciTePress, 2018. P. 108–116. DOI: 10.5220/0006639801080116.

[324] Shostack A. Threat Modeling: Designing for Security. Indianapolis: Wiley, 2014. 624 p.

[325] Sindre G., Opdahl A. L. Eliciting Security Requirements with Misuse Cases. *Requirements Engineering*. 2005. Vol. 10, No. 1. P. 34–44. DOI: 10.1007/s00766-004-0194-4.

[326] SLSA Specification v1.2 (Build Track, Source Track, Threats). Open Source Security Foundation, 2025. URL: https://slsa.dev/spec/v1.2/.

[327] Sommer R., Paxson V. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. *2010 IEEE Symposium on Security and Privacy*. Oakland: IEEE, 2010. P. 305–316. DOI: 10.1109/SP.2010.25.

[328] Souppaya M., Scarfone K., Dodson D. Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST SP 800-218. Gaithersburg, MD, 2022. DOI: 10.6028/NIST.SP.800-218. URL: https://csrc.nist.gov/pubs/sp/800/218/final.

[329] Spracklen J., Wijewickrama R., Sakib A. H. M. N., Maiti A., Viswanath B., Jadliwala M. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. *34th USENIX Security Symposium (USENIX Security 25)*. Seattle: USENIX Association, 2025. P. 3687–3706. URL: https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen.

[330] Spring J., Hatleback E., Householder A., Manion A., Shick D. Time to Change the CVSS? *IEEE Security & Privacy*. 2021. Vol. 19, No. 2. P. 74–78. DOI: 10.1109/MSEC.2020.3044475.

[331] Srinivasan V., Suri S., Varghese G. Packet Classification Using Tuple Space Search. *Proceedings of the ACM SIGCOMM '99 Conference*. New York: ACM, 1999. P. 135–146. DOI: 10.1145/316188.316216.

[332] Stone M. The Ups and Downs of 0-days: A Year in Review of 0-days Exploited In-the-Wild in 2022. Google Security Blog. 27.07.2023. URL: https://security.googleblog.com/2023/07/the-ups-and-downs-of-0-days-year-in.html.

[333] Sutton M., Greene A., Amini P. Fuzzing: Brute Force Vulnerability Discovery. Upper Saddle River: Addison-Wesley, 2007. 576 p.

[334] Szekeres L., Payer M., Wei T., Song D. SoK: Eternal War in Memory. *2013 IEEE Symposium on Security and Privacy (SP)*. Berkeley: IEEE, 2013. P. 48–62. DOI: 10.1109/SP.2013.13.

[335] Tavallaee M., Bagheri E., Lu W., Ghorbani A. A. A Detailed Analysis of the KDD CUP 99 Data Set. *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. Ottawa: IEEE, 2009. P. 1–6. DOI: 10.1109/CISDA.2009.5356528.

[336] Torres-Arias S., Afzali H., Kuppusamy T. K., Curtmola R., Cappos J. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. *Proceedings of the 28th USENIX Security Symposium*. Santa Clara: USENIX Association, 2019. P. 1393–1410. URL: https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

[337] UcedaVélez T., Morana M. M. Risk Centric Threat Modeling: Process for Attack Simulation and Threat Analysis. Hoboken: Wiley, 2015. 696 p.

[338] Vander Stoep J. Rust in Android: move fast and fix things. *Google Security Blog*. 13.11.2025. URL: https://security.googleblog.com/2025/11/rust-in-android-move-fast-fix-things.html.

[339] Vander Stoep J., Rebert A. Eliminating Memory Safety Vulnerabilities at the Source. *Google Security Blog*. 25.09.2024. URL: https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html.

[340] Vassilev A., Oprea A., Fordyce A., Anderson H., Davies X., Hamin M. Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. NIST AI 100-2 E2025. Gaithersburg, MD: NIST, 2025. DOI: 10.6028/NIST.AI.100-2e2025.

[341] Wang X., Hong Y., Chang H., Park K., Langdale G., Hu J., Zhu H. Hyperscan: A Fast Multi-pattern Regex Matcher for Modern CPUs. *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI '19)*. Boston: USENIX Association, 2019. URL: https://www.usenix.org/conference/nsdi19/presentation/wang-xiang.

[342] Watson R. N. M., Anderson J., Laurie B., Kennaway K. Capsicum: Practical Capabilities for UNIX. *Proceedings of the 19th USENIX Security Symposium*. Washington, D.C.: USENIX Association, 2010. URL: https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix.

[343] Watson R. N. M., Woodruff J., Neumann P. G., Moore S. W., Anderson J., Chisnall D., Dave N., Davis B., Gudka K., Laurie B., Murdoch S. J., Norton R., Roe M., Son S., Vadera M. CHERI: A Hybrid Capability-System Architecture for Scalable Software Compartmentalization. *2015 IEEE Symposium on Security and Privacy (SP)*. San Jose: IEEE, 2015. P. 20–37. DOI: 10.1109/SP.2015.9.

[344] Williams L. Secure Software Lifecycle Knowledge Area. Version 1.0.2. *The Cyber Security Body of Knowledge (CyBOK)*. © Crown Copyright, The National Cyber Security Centre, 2021. URL: https://www.cybok.org/knowledgebase1_1/.

[345] Wilson E. B. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*. 1927. Vol. 22, no. 158. P. 209–212. DOI: 10.1080/01621459.1927.10502953.

[346] Wohlin C., Runeson P., Höst M., Ohlsson M. C., Regnell B., Wesslén A. Experimentation in Software Engineering. Berlin, Heidelberg: Springer, 2012. DOI: 10.1007/978-3-642-29044-2.

[347] Yamin M. M., Katt B., Gkioulos V. Cyber Ranges and Security Testbeds: Scenarios, Functions, Tools and Architecture. *Computers & Security*. 2020. Vol. 88. Art. 101636. DOI: 10.1016/j.cose.2019.101636.

[348] Yu F., Chen Z., Diao Y., Lakshman T. V., Katz R. H. Fast and Memory-Efficient Regular Expression Matching for Deep Packet Inspection. *Proceedings of the 2006 ACM/IEEE Symposium on Architecture for Networking and Communications Systems (ANCS '06)*. New York: ACM, 2006. P. 93–102. DOI: 10.1145/1185347.1185360.

[349] Zahan N., Kanakiya P., Hambleton B., Shohan S., Williams L. OpenSSF Scorecard: On the Path Toward Ecosystem-Wide Automated Security Metrics. *IEEE Security & Privacy*. 2023. Vol. 21, no. 6. P. 76–88. DOI: 10.1109/MSEC.2023.3279773.

[350] Zeller A., Gopinath R., Böhme M., Fraser G., Holler C. The Fuzzing Book. Saarbrücken: CISPA Helmholtz Center for Information Security. URL: https://www.fuzzingbook.org/.

[351] Zimmermann M., Staicu C.-A., Tenny C., Pradel M. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. *Proceedings of the 28th USENIX Security Symposium*. Santa Clara: USENIX Association, 2019. P. 995–1010. URL: https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman.

[352] Zinzindohoué J. K., Bhargavan K., Protzenko J., Beurdouche B. HACL*: A Verified Modern Cryptographic Library. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS'17)*. New York: ACM, 2017. P. 1789–1806. DOI: 10.1145/3133956.3134043.

Додаток. Завдання для самостійної роботи

Завдання згруповано за тематичними блоками посібника. Для кожної теми зазначено зміст роботи та форму подання результату. Результати доцільно оформлювати як фрагменти дослідницької роботи магістранта: з постановкою задачі, обґрунтуванням методу, числовими результатами та висновками. Практичні завдання виконуються лише щодо власного коду, відкритих проєктів у межах їхніх правил або на ізольованому стенді. Багато завдань спираються на дослідницькі питання самоперевірки відповідних розділів.

Для виконання завдань достатньо відкритих засобів: Suricata, Zeek і nftables (блок II); Clang із санітайзерами, AFL++, libFuzzer, CodeQL або Semgrep, ProVerif чи Tamarin (блок III); генератори SBOM і засоби підпису артефактів із проєктів CycloneDX і Sigstore; відкриті набори даних, розглянуті в розділі 15, з урахуванням їхніх відомих вад. Рекомендований обсяг звіту — 10–20 с.; критерії оцінювання: коректність постановки задачі й методу (30 %), відтворюваність і повнота експерименту (30 %), обґрунтованість висновків і зв'язок з джерелами (25 %), якість оформлення (15 %).

Блок I. Засади

1. **Аналіз ПЗМБ як об'єкта розроблення** (розділ 1). Для обраного відкритого продукту (ME, IDS, VPN) описати площини даних, керування й менеджменту, поверхню атаки й модель порушника; оцінити очікувану кількість експлуатованих вразливостей за моделлю розділу. *Форма:* аналітична записка зі схемами й розрахунком.
2. **Реєстр нормативних вимог** (розділ 2). Для умовного українського виробника засобу мережевої безпеки визначити застосовні вимоги України та ЄС (зокрема клас продукту за CRA), обчислити індекс розриву відповідності. *Форма:* реєстр вимог і розрахунок.
3. **Оцінювання зрілості процесу розроблення** (розділ 3). Оцінити команду або відкритий проєкт за OWASP SAMM, побудувати цільовий профіль; за історією вразливостей проєкту підібрати параметри моделі виявлення вразливостей. *Форма:* звіт з оцінювання і дорожня карта.
4. **Модель загроз і безпечна архітектура** (розділ 4). Побудувати DFD з межами довіри, модель загроз STRIDE і кількісне дерево атак для обраного компонента; порівняти поверхню атаки двох варіантів архітектури. *Форма:* модель загроз і проєктне рішення.

Блок II. Технології розроблення

5. **Безпечний парсер протоколу** (розділ 5). Реалізувати розпізнавач формату повідомлення мовою з безпечною пам'яттю, обґрунтувати перевірки меж і цілочислової арифметики, налаштувати посилене збирання. *Форма:* код, тести й пояснювальна записка.
6. **Криптографічний компонент і план міграції** (розділ 6). Перевірити реалізацію порівняння зі сталим часом статистичним тестом, оцінити межі використання ключа AEAD, розробити план міграції обраного протоколу на гібридну PQC. *Форма:* звіт з вимірюваннями і план.
7. **Фільтр трафіку на eBPF/XDP** (розділ 7). Розробити й протестувати фільтр для захисту сервісу, проаналізувати аномалії набору правил і ємність таблиці станів. *Форма:* код, тести, вимірювання.
8. **Набір правил IDS як код** (розділ 8). Розробити правила за публічним звітом CERT-UA, регресійні тести на PCAP і конвеєр перевірки; оцінити очікувану кількість хибних тривог. *Форма:* репозиторій правил і звіт.
9. **Безпечне оновлення засобу захищених комунікацій** (розділ 9). Спроєктувати механізм оновлення з підписом, захистом від відкату й схемою A/B; оцінити ймовірність втрати працездатності. *Форма:* проєктне рішення і модель.
10. **Захищений конвеєр збирання** (розділ 10). Налаштувати конвеєр із генерацією SBOM, атестацією походження та підписом артефактів для відкритого проєкту; оцінити ризик транзитивних залежностей. *Форма:* конфігурація конвеєра і звіт.

Блок III. Технології тестування

11. **План тестування безпеки** (розділ 11). Розробити ризик-орієнтований план тестування засобу мережевої безпеки за ISO/IEC/IEEE 29119, комбінаторний набір конфігурацій і оцінку мутаційного балу. *Форма:* план тестування за шаблоном стандарту.
12. **Статичний аналіз і формальна модель** (розділ 12). Застосувати SAST до обраного парсера, оцінити точність і вартість тріажу; побудувати навчальну модель протоколу в ProVerif або Tamarin. *Форма:* звіт з метриками і модель.
13. **Фазинг мережевого компонента** (розділ 13). Написати обв'язку для парсера, провести серію прогонів із санітайзерами, оцінити залишкову ймовірність нових знахідок і порівняти два налаштування фазера статистично. *Форма:* обв'язка, корпус і звіт.
14. **Бенчмаркінг засобу безпеки** (розділ 14). Провести на стенді вимірювання пропускної здатності й затримки програмного ME або IDS за

методологією RFC 2544/RFC 9411, побудувати модель насичення й порівняти з вимірюваннями. *Форма:* протокол вимірювань і модель.

15. **Оцінювання ефективності виявлення** (розділ 15). Порівняти дві системи виявлення на відкритому наборі даних із часовим розбиттям, довірчими інтервалами й аналізом вад набору. *Форма:* протокол оцінювання й результати.

Блок IV. Супровід, гарантування та дослідження

16. **Процес обробки вразливостей** (розділ 16). Розробити політику розкриття, процедуру PSIRT і документ VEX для умовного продукту; змоделювати вікно вразливості й вибір часу випуску виправлення. *Форма:* комплект процедур і розрахунок.
17. **Кейс гарантування і наукове дослідження** (розділ 17). Побудувати кейс гарантування в нотації GSN з доказами з конвеєра для обраного компонента або виконати систематичний огляд за одним із перспективних напрямів як основу дисертаційного дослідження. *Форма:* кейс гарантування або огляд із постановкою наукової задачі.